

Aprende Python

Sergio Delgado Quintero

15 de abril de 2024

Core

1 Introducción **3**
- 1.1 Hablando con la máquina . . . 3
- 1.2 Algo de historia . . . 8
- 1.3 Python . . . 14

2 Entornos de desarrollo **25**
- 2.1 Thonny . . . 26
- 2.2 Contexto real . . . 30
- 2.3 VSCode . . . 39

3 Tipos de datos **49**
- 3.1 Datos . . . 50
- 3.2 Números . . . 63
- 3.3 Cadenas de texto . . . 78

4 Control de flujo **105**
- 4.1 Condicionales . . . 106
- 4.2 Bucles . . . 127

5 Estructuras de datos **145**
- 5.1 Listas . . . 146
- 5.2 Tuplas . . . 178
- 5.3 Diccionarios . . . 185
- 5.4 Conjuntos . . . 203
- 5.5 Ficheros . . . 214

6 Modularidad **225**
- 6.1 Funciones . . . 226
- 6.2 Objetos y Clases . . . 276
- 6.3 Excepciones . . . 333

6.4 Módulos 345

7 Procesamiento de texto **357**
7.1 re 358
7.2 string 369

8 Acceso a datos **373**
8.1 sqlite 374

9 Ciencia de datos **395**
9.1 jupyter 396
9.2 numpy 420
9.3 pandas 463
9.4 matplotlib 527

10 Scraping **569**
10.1 requests 570
10.2 beautifulsoup 578
10.3 selenium 593



Curso gratuito para aprender el lenguaje de programación **Python** con un enfoque **práctico**, incluyendo **ejercicios** y cobertura para distintos **niveles de conocimiento**.[1]

Este proyecto va de la mano con pycheck una herramienta que permite **trabajar todos los ejercicios propuestos** con casos de prueba incluidos y verificación de los resultados.

Licencia: Creative Commons Reconocimiento 4.0 Internacional: CC BY 4.0.

Consejo: «Programming is not about typing, it's about thinking.» – Rich Hickey

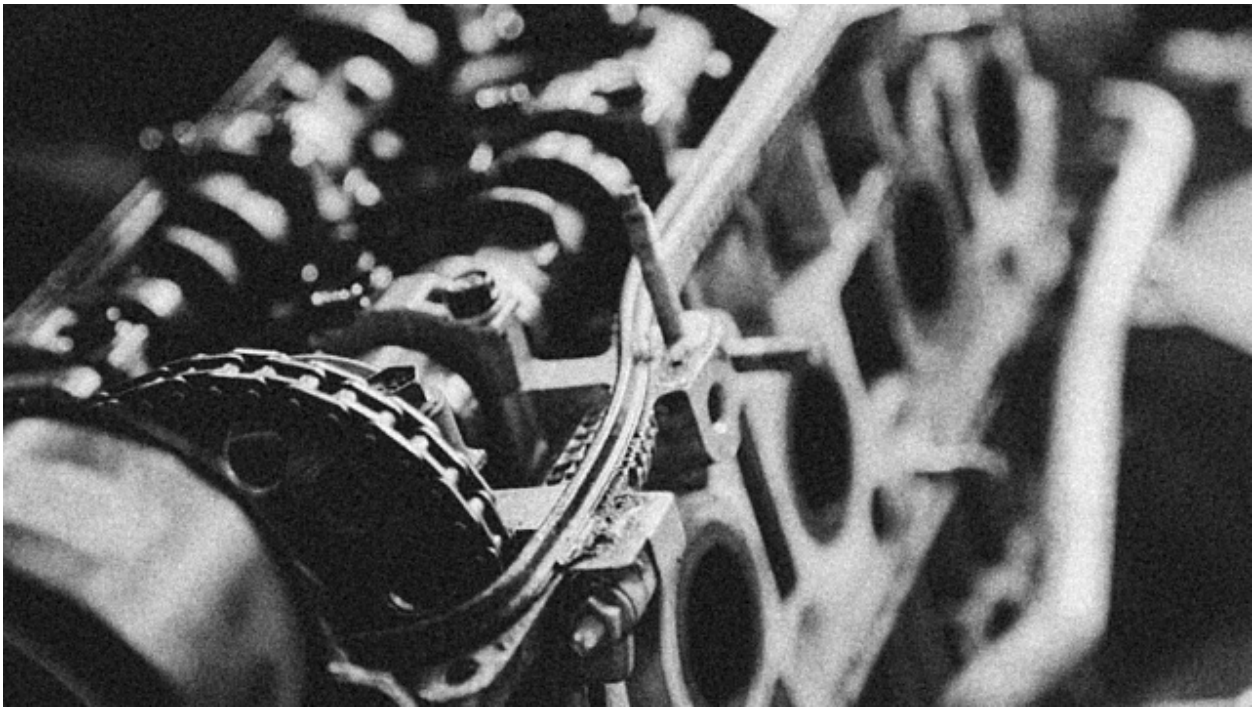
[1] En la foto de portada aparecen los Monty Python. Fuente: noticiascyl

CAPÍTULO 1

Introducción

Este capítulo es una introducción a la programación para conocer, desde un enfoque sencillo pero aclaratorio, los mecanismos que hay detrás de ello.

1.1 Hablando con la máquina



Los ordenadores son dispositivos complejos pero están diseñados para hacer una cosa bien: **ejecutar aquello que se les indica**. La cuestión es cómo indicar a un ordenador lo que queremos que ejecute. Esas indicaciones se llaman técnicamente **instrucciones** y se expresan en un lenguaje. Podríamos decir que **programar** consiste en escribir instrucciones para que sean ejecutadas por un ordenador. El lenguaje que utilizamos para ello se denomina **lenguaje de programación**.[1]

1.1.1 Código máquina

Pero aún seguimos con el problema de cómo hacer que un ordenador (o máquina) entienda el lenguaje de programación. A priori podríamos decir que un ordenador sólo entiende un lenguaje muy «simple» denominado código máquina. En este lenguaje se utilizan únicamente los símbolos **0** y **1** en representación de los *niveles de tensión* alto y bajo, que al fin y al cabo, son los estados que puede manejar un circuito digital. Hablamos de sistema binario. Si tuviéramos que escribir programas de ordenador en este formato sería una tarea ardua, pero afortunadamente se han ido creando con el tiempo lenguajes de programación intermedios que, posteriormente, son convertidos a código máquina.

Si intentamos visualizar un programa en código máquina, únicamente obtendríamos una secuencia de ceros y unos:

```
00001000 00000010 01111011 10101100 10010111 11011001 01000000 01100010
00110100 00010111 01101111 10111001 01010110 00110001 00101010 00011111
10000011 11001101 11110101 01001110 01010010 10100001 01101010 00001111
11101010 00100111 11000100 01110101 11011011 00010110 10011111 01010110
```

1.1.2 Ensamblador

El primer lenguaje de programación que encontramos en esta «escalada» es **ensamblador**. Veamos un ejemplo de código en ensamblador del típico programa que se escribe por primera vez, el *«Hello, World»*:

```
SYS_SALIDA equ 1

section .data
    msg db "Hello, World",0x0a
    len equ $ - msg ;longitud de msg

section .text
global _start ;para el linker
_start: ;marca la entrada
    mov eax, 4 ;llamada al sistema (sys_write)
```

(continué en la próxima página)

[1] Foto original por Garett Mizunaka en Unsplash.

(proviene de la página anterior)

```
    mov ebx, 1 ;descripción de archivo (stdout)
    mov ecx, msg ;msg a escribir
    mov edx, len ;longitud del mensaje
    int 0x80 ;llama al sistema de interrupciones

fin: mov eax, SYS_SALIDA ;llamada al sistema (sys_exit)
    int 0x80
```

Aunque resulte difícil de creer, lo «único» que hace este programa es mostrar en la pantalla de nuestro ordenador la frase «Hello, World», pero además teniendo en cuenta que sólo funcionará para una arquitectura x86.

1.1.3 C

Aunque el lenguaje ensamblador nos facilita un poco la tarea de desarrollar programas, sigue siendo bastante complicado ya que las instrucciones son muy específicas y no proporcionan una semántica entendible. Uno de los lenguajes que vino a suplir – en parte – estos obstáculos fue C. Considerado para muchas personas como un referente en cuanto a los lenguajes de programación, permite hacer uso de instrucciones más claras y potentes. El mismo ejemplo anterior del programa *«Hello, World»* se escribiría así en lenguaje *C*:

```
#include <stdio.h>

int main() {
    printf("Hello, World");
    return 0;
}
```

1.1.4 Python

Si seguimos «subiendo» en esta lista de lenguajes de programación, podemos llegar hasta Python. Se dice que es un lenguaje de *más alto nivel* en el sentido de que sus instrucciones son más entendibles por un humano. Veamos cómo se escribiría el programa *«Hello, World»* en el lenguaje de programación Python:

```
print('Hello, World')
```

¡Pues así de fácil! Hemos pasado de *código máquina* (ceros y unos) a *código Python* en el que se puede entender perfectamente lo que estamos indicando al ordenador. La pregunta que surge es: ¿cómo entiende una máquina lo que tiene que hacer si le pasamos un programa hecho en Python (o cualquier otro lenguaje de alto nivel)? La respuesta es un **compilador**.

1.1.5 Compiladores

Los compiladores son programas que convierten un lenguaje «cualquiera» en *código máquina*. Se pueden ver como traductores, permitiendo a la máquina interpretar lo que queremos hacer.

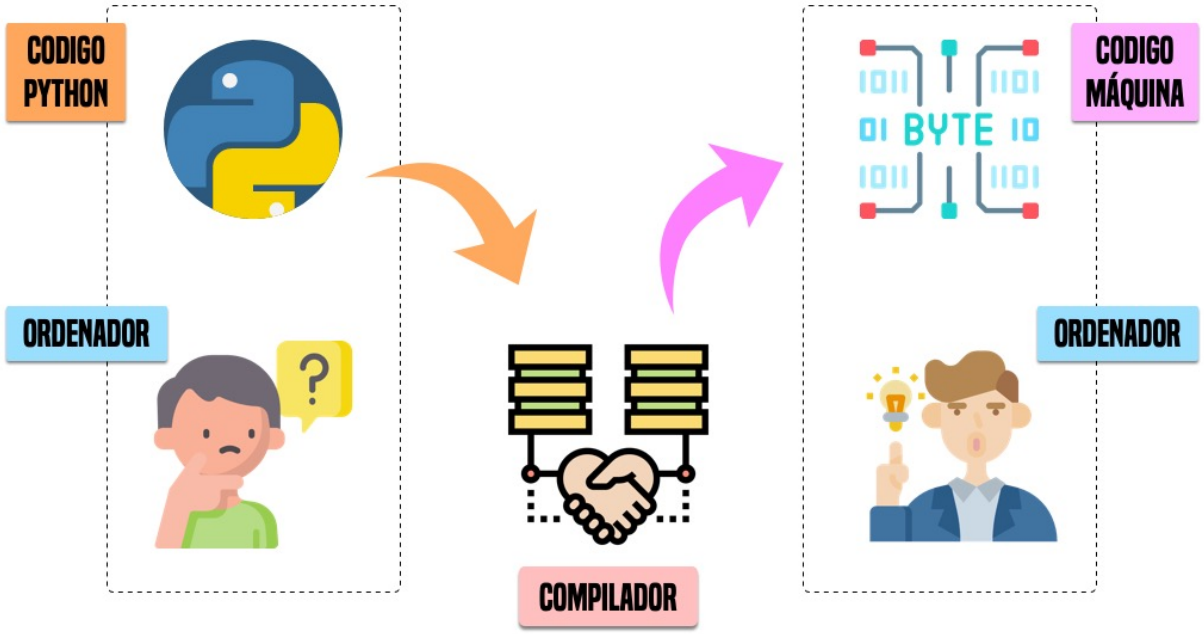


Figura 1: Esquema de funcionamiento de un compilador[2]

En el caso particular de Python el proceso de compilación genera un código intermedio denominado **bytecode**.

Si partimos del ejemplo anterior:

```
print('Hello, World')
```

el programa se compilaría[3] al siguiente «bytecode»:

```
0           0 RESUME                   0

1           2 PUSH_NULL
            4 LOAD_NAME                0 (print)
            6 LOAD_CONST               0 ('Hello, World')
            8 PRECALL                  1
           12 CALL                     1
           22 RETURN_VALUE
```

[2] Iconos originales por Flaticon.
[3] Véase más información sobre el intérprete de bytecode.

A continuación estas instrucciones básicas son ejecutadas por el intérprete de «bytecode» de Python (o máquina virtual):

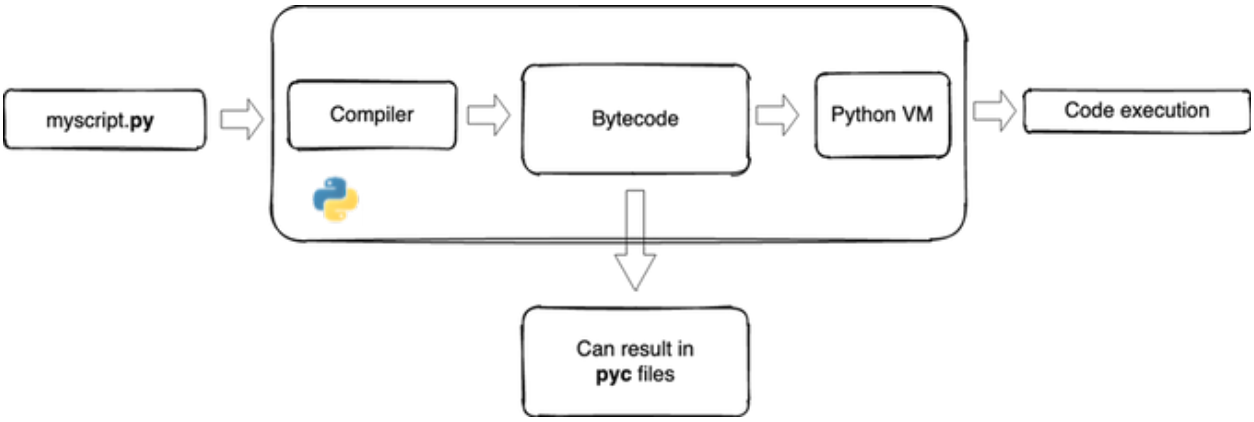


Figura 2: Modelo de ejecución de un programa Python[4]

Nota: Si queremos ver una diferencia entre un lenguaje compilado como C y un lenguaje «interpretado» como Python es que, aunque ambos realizan un proceso de traducción del código fuente, la compilación de C genera un código objeto que debe ser ejecutado en una segunda fase explícita, mientras que la compilación de Python genera un «bytecode» que se ejecuta (interpreta) de forma «transparente».

[4] Imagen extraída del artículo Python bytecode analysis.

1.2 Algo de historia



La historia de la programación está relacionada directamente con la aparición de los computadores, que ya desde el siglo XV tuvo sus inicios con la construcción de una máquina que realizaba operaciones básicas y raíces cuadradas (Gottfried Wilheml von Leibniz); aunque en realidad la primera gran influencia hacia la creación de los computadores fue la máquina diferencial para el cálculo de polinomios, proyecto no concluido de Charles Babbage (1793-1871) con el apoyo de Lady Ada Countess of Lovelace (1815-1852), primera persona que incursionó en la programación y de quien proviene el nombre del lenguaje de programación ADA creado por el DoD (Departamento de defensa de Estados Unidos) en la década de 1970.[1]

1.2.1 Hitos de la computación

La siguiente tabla es un resumen de los principales hitos en la historia de la computación:

Tabla 1: Hitos en la computación

Personaje	Aporte	Año
Gottfried Leibniz	Máquinas de **operaciones básicas**	XV
Charles Babbage	**Máquina diferencial** para el cálculo de polinomios	XVII

continué en la próxima página

[1] Foto original por Dario Veronesi en Unsplash.

Tabla 1 – proviene de la página anterior

Personaje	Aporte	Año
Ada Lovelace	Matemática, informática y escritora británica. **Primera programadora** de la historia por el desarrollo de algoritmos para la máquina analítica de Babbage	XVII
George Boole	Contribuyó al **algebra binaria** y a los sistemas de circuitos de computadora (álgebra booleana)	1854
Herman Hollerit	Creador de un **sistema para automatizar** la pesada tarea del censo	1890
Alan Turing	**Máquina de Turing** - una máquina capaz de resolver problemas - Aportes de Lógica Matemática - Computadora con tubos de vacío	1936
John Atanasoff	Primera **computadora digital** electrónica patentada: Atanasoff Berry Computer (ABC)	1942
Howard Aiken	En colaboración con IBM desarrolló el **Mark I**, una computadora electromecánica de 16 metros de largo y más de dos de alto que podía realizar las cuatro operaciones básicas y trabajar con información almacenada en forma de tablas	1944
Grace Hopper	Primera **programadora** que utilizó el Mark I	1945
John W. Mauchly	Junto a John Presper Eckert desarrolló una **computadora electrónica** completamente operacional a gran escala llamada Electronic Numerical Integrator And Computer (ENIAC)	1946
John Von Neumann	Propuso guardar en **memoria** no solo la **información**, sino también los **programas**, acelerando los procesos	1946

Luego los avances en las ciencias informáticas han sido muy acelerados, se reemplazaron los tubos de vacío por transistores en 1958 y en el mismo año, se sustituyeron por circuitos integrados, y en 1961 se miniaturizaron en **chips de silicio**. En 1971 apareció el primer microprocesador de Intel; y en 1973 el primer sistema operativo CP/M. El primer computador personal es comercializado por IBM en el año 1980.

[2] Fuente: Meatze.



Figura 3: Ada Lovelace: primera programadora de la historia[2]

1.2.2 De los computadores a la programación

De acuerdo a este breve viaje por la historia, la programación está vinculada a la aparición de los computadores, y los lenguajes tuvieron también su evolución. Inicialmente, como ya hemos visto, se programaba en **código binario**, es decir en cadenas de 0s y 1s, que es el lenguaje que entiende directamente el computador, tarea extremadamente difícil; luego se creó el **lenguaje ensamblador**, que aunque era lo mismo que programar en binario, al estar en letras era más fácil de recordar. Posteriormente aparecieron **lenguajes de alto nivel**, que en general, utilizan palabras en inglés, para dar las órdenes a seguir, para lo cual utilizan un proceso intermedio entre el lenguaje máquina y el nuevo código llamado código fuente, este proceso puede ser un compilador o un intérprete.

Un **compilador** lee todas las instrucciones y genera un resultado; un **intérprete** ejecuta y genera resultados línea a línea. En cualquier caso han aparecido nuevos lenguajes de programación, unos denominados estructurados y en la actualidad en cambio los lenguajes orientados a objetos y los lenguajes orientados a eventos.[3]

1.2.3 Cronología de lenguajes de programación

Desde la década de 1950 se han sucedido multitud de lenguajes de programación que cada vez incorporan más funcionalidades destinadas a cubrir las necesidades del desarrollo de aplicaciones. A continuación se muestra una tabla con la historia de los lenguajes de programación más destacados:

El **número** actual de lenguajes de programación depende de lo que se considere un *lenguaje de programación* y a *quién se pregunte*. Según TIOBE más de 250; según Wikipedia más de 700, según Language List más de 2500; y para una cifra muy alta podemos considerar a Online Historical Encyclopaedia of Programming Languages que se acerca a los **9000**.

1.2.4 Creadores de lenguajes de programación

El avance de la computación está íntimamente relacionado con el desarrollo de los lenguajes de programación. Sus creadores y creadoras juegan un *rol fundamental* en la historia tecnológica. Veamos algunas de estas personas:[4]

Tabla 2: Creadores de lenguajes de programación

Personaje	Aporte
Alan Cooper	Desarrollador de **Visual Basic**
Alan Kay	Pionero en programación orientada a objetos. Creador de **Smalltalk**

continué en la próxima página

[3] Fuente: Universidad Técnica del Norte.
[4] Fuente: Wikipedia.

Tabla 2 – proviene de la página anterior

Personaje	Aporte
Anders Hejlsberg	Desarrollador de **Turbo Pascal**, **Delphi** y **C#**
Bertrand Meyer	Inventor de **Eiffel**
Bill Joy	Inventor de **vi**. Autor de **BSD Unix**. Creador de **SunOS**, el cual se convirtió en **Solaris**
Bjarne Stroustrup	Desarrollador de **C++**
Dennis Ritchie	Inventor del **lenguaje C** y del Sistema Operativo **Unix**
Brian Kernighan	Coautor del primer libro de programación en lenguaje **C** con Dennis Ritchie y coautor de los lenguajes de programación **AWK** y **AMPL**
Edsger W. Dijkstra	Desarrolló las bases para la programación estructurada. Algoritmo de caminos mínimos
Grace Hopper	Desarrolladora de **Flow-Matic**, influenciando el lenguaje **COBOL**
Guido van Rossum	Creador de **Python**
James Gosling	Desarrollador de **Oak**. Precursor de **Java**
Joe Armstrong	Creador de **Erlang**
John Backus	Inventor de **Fortran**
John McCarthy	Inventor de **LISP**
John von Neumann	Creador del concepto de sistema operativo
Ken Thompson	Inventor de B. Desarrollador de **Go**. Coautor del sistema operativo Unix
Kenneth E. Iverson	Desarrollador de **APL**. Co-desarrollador de **J** junto a Roger Hui
Larry Wall	Creador de **Perl** y Perl 6
Martin Odersky	Creador de **Scala**. Previamente contribuyó en el diseño de Java
Mitchel Resnick	Creador del lenguaje visual **Scratch**
Nathaniel Rochester	Inventor del primer lenguaje en **ensamblador** simbólico (IBM 701)
Niklaus Wirth	Inventor de **Pascal**, **Modula** y **Oberon**
Robin Milner	Inventor de **ML**. Compartió crédito en el método Hindley–Milner de inferencia de tipo polimórfica
Seymour Papert	Pionero de la inteligencia artificial. Inventor del lenguaje de programación **Logo** en 1968
Stephen Wolfram	Creador de **Mathematica**
Yukihiro Matsumoto	Creador de **Ruby**

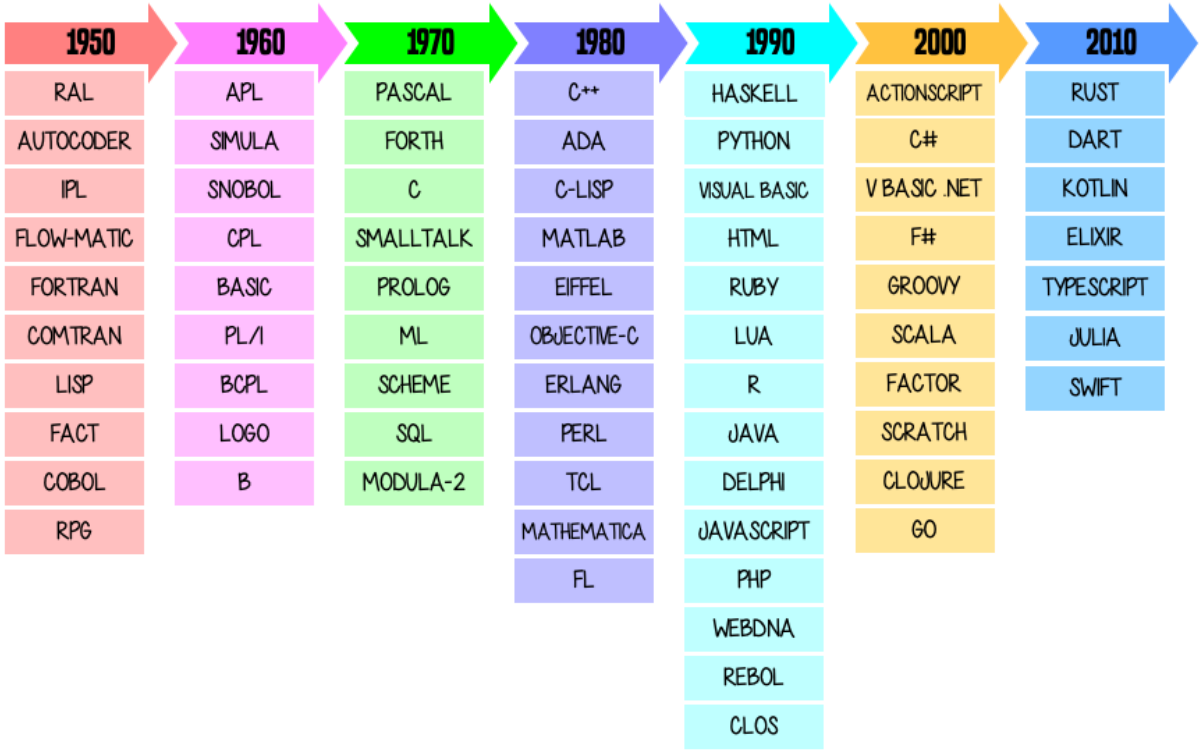


Figura 4: Cronología de los lenguajes de programación más destacados

1.3 Python



Python es un lenguaje de programación de *alto nivel* creado a finales de los 80/principios de los 90 por Guido van Rossum, holandés que trabajaba por aquella época en el *Centro para las Matemáticas y la Informática* de los Países Bajos. Sus instrucciones están muy cercanas al **lenguaje natural** en inglés y se hace hincapié en la **legibilidad** del código. Toma su nombre de los Monty Python, grupo humorista de los 60 que gustaban mucho a Guido. Python fue creado como sucesor del lenguaje `ABC`.[1]

1.3.1 Características del lenguaje

A partir de su definición de la Wikipedia:

- Python es un lenguaje de programación **interpretado** y **multiplataforma** cuya filosofía hace hincapié en una sintaxis que favorezca un **código legible**.
- Se trata de un lenguaje de programación **multiparadigma**, ya que soporta **orientación a objetos, programación imperativa** y, en menor medida, programación funcional.
- Añadiría, como característica destacada, que se trata de un lenguaje de **propósito general**.

[1] Foto original por Markéta Marcellová en Unsplash.

Ventajas

- Libre y gratuito (OpenSource).
- Fácil de leer, parecido a pseudocódigo.
- Aprendizaje relativamente fácil y rápido: claro, intuitivo….
- Alto nivel.
- Alta Productividad: simple y rápido.
- Tiende a producir un buen código: orden, limpieza, elegancia, flexibilidad, …
- Multiplataforma. Portable.
- Multiparadigma: programación imperativa, orientada a objetos, funcional, …
- Interactivo, modular, dinámico.
- Librerías extensivas («pilas incluídas»).
- Gran cantidad de librerías de terceros.
- Extensible (C++, C, …) y «embebible».
- Gran comunidad, amplio soporte.
- Interpretado.
- Tipado dinámico[5].
- Fuertemente tipado[6].

Desventajas

- Interpretado (velocidad de ejecución, multithread vs GIL, etc.).
- Consumo de memoria.
- Errores no detectables en tiempo de compilación.
- Desarrollo móvil.
- Documentación a veces dispersa e incompleta.
- Varios módulos para la misma funcionalidad.
- Librerías de terceros no siempre del todo maduras.

[5] Tipado dinámico significa que una variable puede cambiar de tipo durante el tiempo de vida de un programa. C es un lenguaje de tipado estático.

[6] Fuertemente tipado significa que, de manera nativa, no podemos operar con dos variables de tipos distintos, a menos que realice una conversión explícita. Javascript es un lenguaje débilmente tipado.

1.3.2 Uso de Python

Al ser un lenguaje de propósito general, podemos encontrar aplicaciones prácticamente en todos los campos científico-tecnológicos:

- Análisis de datos.
- Aplicaciones de escritorio.
- Bases de datos relacionales / NoSQL
- Buenas prácticas de programación / Patrones de diseño.
- Concurrencia.
- Criptomonedas / Blockchain.
- Desarrollo de aplicaciones multimedia.
- Desarrollo de juegos.
- Desarrollo en dispositivos embebidos.
- Desarrollo web.
- DevOps / Administración de sistemas / Scripts de automatización.
- Gráficos por ordenador.
- Inteligencia artificial.
- Internet de las cosas.
- Machine Learning.
- Programación de parsers / scrapers / crawlers.
- Programación de redes.
- Propósitos educativos.
- Prototipado de software.
- Seguridad.
- Tests automatizados.

De igual modo son muchas las empresas, instituciones y organismos que utilizan Python en su día a día para mejorar sus sistemas de información. Veamos algunas de las más relevantes:

Existen ránkings y estudios de mercado que sitúan a Python como uno de los lenguajes más *usados* y la vez, más *amados* dentro del mundo del desarrollo de software.

En el momento de la escritura de este documento, la última actualización del Índice TIOBE es de *noviembre de 2023* en el que Python ocupaba el **primer puesto de los lenguajes de programación más usados**, por delante de *C* y *C++*.



Figura 5: Grandes empresas y organismos que usan Python

Nov 2023	Nov 2022	Change	Programming Language	Ratings	Change
1	1		Python	14.16%	-3.02%
2	2		C	11.77%	-3.31%
3	4	^	C++	10.36%	-0.39%

Figura 6: Índice TIOBE Noviembre 2023

Igualmente en la encuesta a desarrolladores/as de Stack Overflow del 2023 Python ocupaba el **tercer puesto de los lenguajes de programación más populares**, sólo por detrás de *HTML/CSS* y *JavaScript*:



Figura 7: Encuesta Stack Overflow 2023

También podemos reseñar el informe anual que realiza GitHub sobre el uso de tecnologías en su plataforma. En la edición de 2023 del estado del opensource de GitHub, Python ocupaba el **segundo puesto de los lenguajes de programación más usados**, sólo por detrás de *JavaScript*:

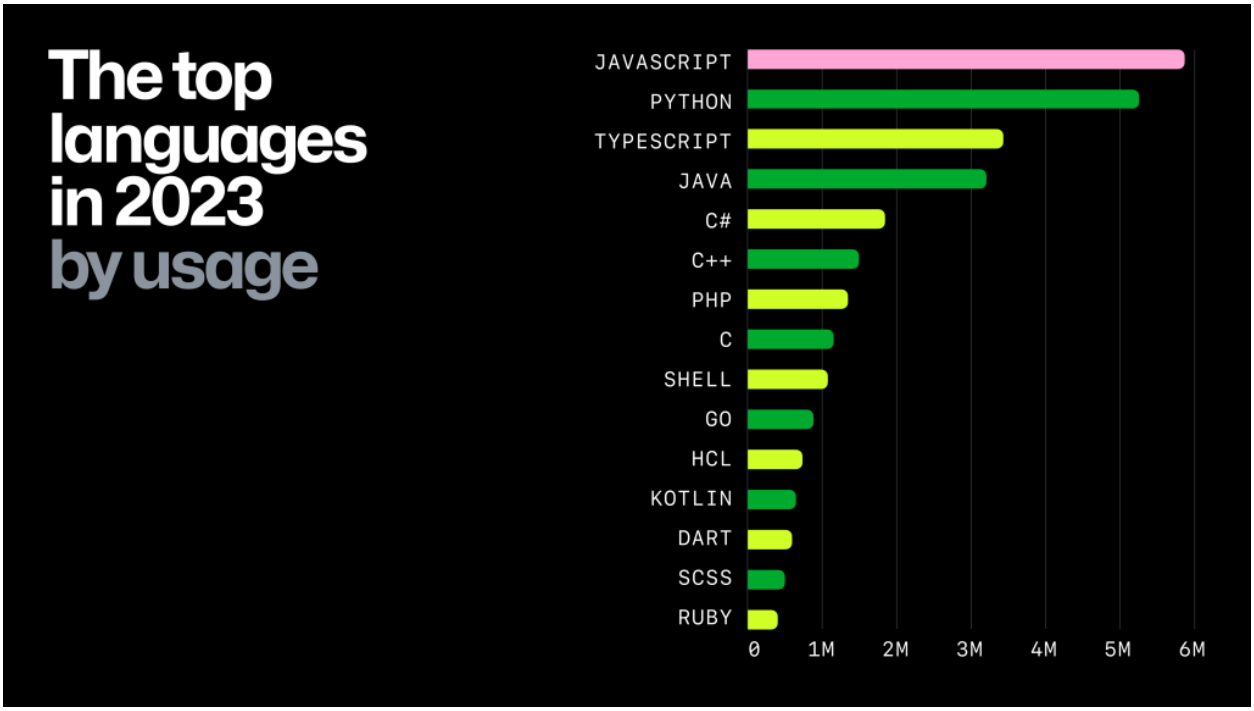


Figura 8: Informe GitHub 2023

1.3.3 Versiones de Python

En el momento de la escritura de este material, se muestra a continuación la evolución de las versiones mayores de Python a lo largo de la historia:[3]

Versión	Fecha de lanzamiento
Python 1.0	Enero 1994
Python 1.5	Diciembre 1997
Python 1.6	Septiembre 2000
Python 2.0	Octubre 2000
Python 2.1	Abril 2001
Python 2.2	Diciembre 2001
Python 2.3	Julio 2003
Python 2.4	Noviembre 2004
Python 2.5	Septiembre 2006
Python 2.6	Octubre 2008
Python 2.7	Julio 2010
Python 3.0	**Diciembre 2008**
Python 3.1	Junio 2009
Python 3.2	Febrero 2011
Python 3.3	Septiembre 2012
Python 3.4	Marzo 2014
Python 3.5	Septiembre 2015
Python 3.6	Diciembre 2016
Python 3.7	Junio 2018
Python 3.8	Octubre 2019
Python 3.9	Octubre 2020
Python 3.10	Octubre 2021
Python 3.11	Octubre 2022
Python 3.12	Octubre 2023
Python 3.13	*Octubre 2024 ?*

Un dato curioso, o directamente un «frikismo»: Desde Python 3.8, cada nueva versión estable sale a la luz en el mes de **Octubre**. En este escenario de Python *3.version* se cumplen las siguientes igualdades:

$$version = year - 2011$$
$$year = version + 2011$$

El cambio de **Python 2** a **Python 3** fue bastante «traumático» ya que se **perdió la compatibilidad** en muchas de las estructuras del lenguaje. Los «*core-developers*»[4], con *Guido van Rossum* a la cabeza, vieron la necesidad de aplicar estas modificaciones

[3] Fuente: python.org.

[4] Término que se refiere a los/las desarrolladores/as principales del lenguaje de programación.

en beneficio del rendimiento y expresividad del lenguaje de programación. Este cambio implicaba que el código escrito en Python 2 no funcionaría (de manera inmediata) en Python 3.

El pasado **1 de enero de 2020** finalizó oficialmente el **soporte a la versión 2.7** del lenguaje de programación Python. Es por ello que se recomienda lo siguiente:

- Si aún desarrollas aplicaciones escritas en Python 2, deberías migrar a Python 3.
- Si vas a desarrollar una nueva aplicación, deberías hacerlo directamente en Python 3.

Importante: Únete a **Python 3** y aprovecha todas sus ventajas.

1.3.4 CPython

Nivel avanzado

Existen múltiples **implementaciones** de Python según el lenguaje de programación que se ha usado para desarrollarlo. Veamos algunas de ellas:

Implementación	Lenguaje
CPython	C
Jython	Java
IronPython	C#
Brython	JavaScript
RustPython	Rust
MicroPython	C

Nota: Cuando hacemos referencia a Python hablamos (implícitamente) de CPython. Este manual versa exclusivamente sobre CPython.

1.3.5 Zen de Python

Existen una serie de *reglas* «filosóficas» que indican una manera de hacer y de pensar dentro del mundo **pitónico**[2] creadas por Tim Peters, llamadas el Zen de Python y que se pueden aplicar incluso más allá de la programación:

[2] Dícese de algo/alguien que sigue las convenciones de Python.

```
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

En su traducción de la Wikipedia:

- Bello es mejor que feo.
- Explícito es mejor que implícito.
- Simple es mejor que complejo.
- Complejo es mejor que complicado.
- Plano es mejor que anidado.
- Espaciado es mejor que denso.
- La legibilidad es importante.
- Los casos especiales no son lo suficientemente especiales como para romper las reglas.
- Sin embargo la practicidad le gana a la pureza.
- Los errores nunca deberían pasar silenciosamente.
- A menos que se silencien explícitamente.
- Frente a la ambigüedad, evitar la tentación de adivinar.
- Debería haber una, y preferiblemente solo una, manera obvia de hacerlo.
- A pesar de que esa manera no sea obvia a menos que seas Holandés.

- Ahora es mejor que nunca.
- A pesar de que nunca es muchas veces mejor que *ahora* mismo.
- Si la implementación es difícil de explicar, es una mala idea.
- Si la implementación es fácil de explicar, puede que sea una buena idea.
- Los espacios de nombres son una gran idea, ¡tengamos más de esos!

Ver también:

Si quieres darle un toque a tu escritorio, puedes descargar este fondo de pantalla del Zen de Python que queda muy chulo.

1.3.6 Consejos para programar

Un listado de consejos muy interesantes cuando nos enfrentamos a la programación, basados en la experiencia de @codewithvoid:

1. Escribir código es el último paso del proceso.
2. Para resolver problemas: pizarra mejor que teclado.
3. Escribir código sin planificar = estrés.
4. Pareces más inteligente siendo claro, no siendo listo.
5. La constancia a largo plazo es mejor que la intensidad a corto plazo.
6. La solución primero. La optimización después.
7. Gran parte de la programación es resolución de problemas.
8. Piensa en múltiples soluciones antes de decidirte por una.
9. Se aprende construyendo proyectos, no tomando cursos.
10. Siempre elije simplicidad. Las soluciones simples son más fáciles de escribir.
11. Los errores son inevitables al escribir código. Sólo te informan sobre lo que no debes hacer.
12. Fallar es barato en programación. Aprende mediante la práctica.
13. Gran parte de la programación es investigación.
14. La programación en pareja te enseñará mucho más que escribir código tu solo.
15. Da un paseo cuando estés bloqueado con un error.
16. Convierte en un hábito el hecho de pedir ayuda. Pierdes cero credibilidad pidiendo ayuda.
17. El tiempo gastado en entender el problema está bien invertido.

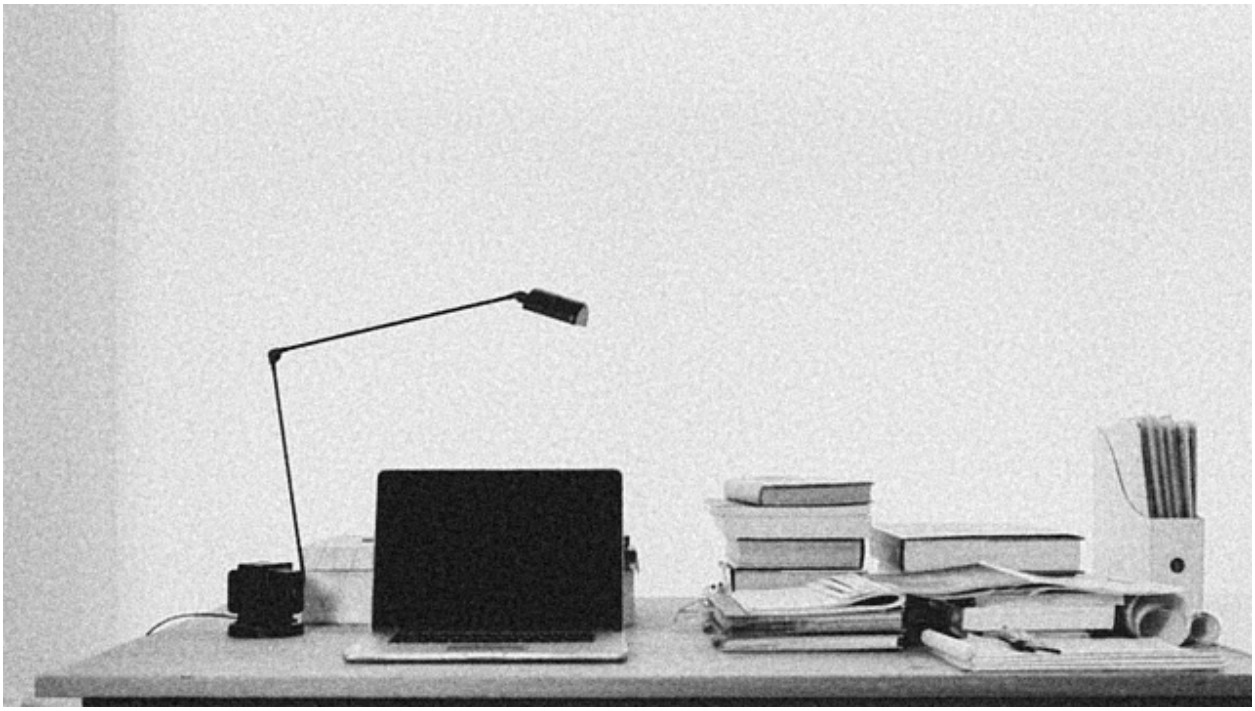
18. Cuando estés bloqueado con un problema: sé curioso, no te frustres.
19. Piensa en posibles escenarios y situaciones extremas antes de resolver el problema.
20. No te estreses con la sintaxis de lenguaje de programación. Entiende conceptos.
21. Aprende a ser un buen corrector de errores. Esto se amortiza.
22. Conoce pronto los atajos de teclado de tu editor favorito.
23. Tu código será tan claro como lo tengas en tu cabeza.
24. Gastarás el doble de tiempo en corregir errores que en escribir código.
25. Saber buscar bien en Google es una habilidad valiosa.
26. Lee código de otras personas para inspirarte.
27. Únete a comunidades de desarrollo para aprender con otros/as programadores/as.

CAPÍTULO 2

Entornos de desarrollo

Para poder utilizar Python debemos preparar nuestra máquina con las herramientas necesarias. Este capítulo trata sobre la instalación y configuración de los elementos adecuados para el desarrollo con el lenguaje de programación Python.

2.1 Thonny



Thonny es un programa muy interesante para empezar a aprender Python, ya que engloba tres de las herramientas fundamentales para trabajar con el lenguaje: **intérprete**, **editor** y **depurador**.[1]

Cuando vamos a trabajar con Python debemos tener instalado, como mínimo, un *intérprete* del lenguaje (para otros lenguajes sería un *compilador*). El **intérprete** nos permitirá *ejecutar* nuestro código para obtener los resultados deseados. La idea del intéprete es lanzar instrucciones «sueltas» para probar determinados aspectos.

Pero normalmente queremos ir un poco más allá y poder escribir programas algo más largos, por lo que también necesitaremos un **editor**. Un editor es un programa que nos permite crear ficheros de código (en nuestro caso con extensión `*.py`), que luego son ejecutados por el intérprete.

Hay otra herramienta interesante dentro del entorno de desarrollo que sería el **depurador**. Lo podemos encontrar habitualmente en la bibliografía por su nombre inglés *debugger*. Es el módulo que nos permite ejecutar paso a paso nuestro código y visualizar qué está ocurriendo en cada momento. Se suele usar normalmente para encontrar fallos (*bugs*) en nuestros programas y poder solucionarlos (*debug/fix*).

Cuando nos encontramos con un programa que proporciona estas funciones (e incluso otras adicionales) para el trabajo de programación, nos referimos a él como un *Entorno Integrado de Desarrollo*, conocido popularmente por sus siglas en inglés **IDE** (por Integrated Development

[1] Foto original de portada por freddie marriage en Unsplash.

Environment). Thonny es un IDE gratuito, sencillo y apto para principiantes.

2.1.1 Instalación

Para instalar Thonny debemos acceder a su web y descargar la aplicación para nuestro sistema operativo. La ventaja es que está disponible tanto para **Windows**, **Mac** y **Linux**. Una vez descargado el fichero lo ejecutamos y seguimos su instalación paso por paso.

Una vez terminada la instalación ya podemos lanzar la aplicación que se verá parecida a la siguiente imagen:

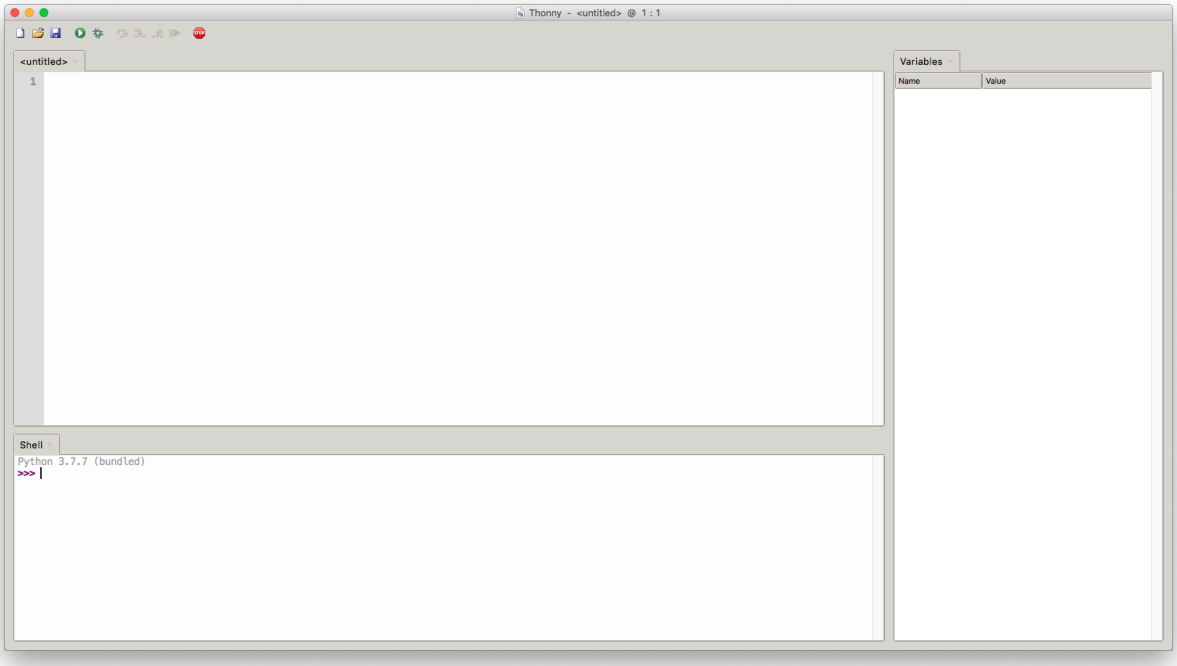


Figura 1: Aspecto de Thonny al arrancarlo

Nota: Es posible que el aspecto del programa varíe ligeramente según el sistema operativo, configuración de escritorio, versión utilizada o idioma (*en mi caso está en inglés*), pero a efectos de funcionamiento no hay diferencia.

Podemos observar que la pantalla está dividida en 3 paneles:

- *Panel principal* que contiene el **editor** e incluye la etiqueta `<untitled>` donde escribiremos nuestro *código fuente* Python.
- *Panel inferior* con la etiqueta *Shell* que contiene el **intérprete** de Python. En el momento de la escritura del presente documento, Thonny incluye la versión de Python 3.7.7.

- *Panel derecho* que contiene el **depurador**. Más concretamente se trata de la ventana de variables donde podemos *inspeccionar* el valor de las mismas.

2.1.2 Probando el intérprete

El intérprete de Python (por lo general) se identifica claramente porque posee un **prompt**[2] con tres angulos hacia la derecha `>>>`. En Thonny lo podemos encontrar en el panel inferior, pero se debe tener en cuenta que el intérprete de Python es una herramienta autocontenida y que la podemos ejecutar desde el símbolo del sistema o la terminal:

Lista 1: Invocando el intérprete de Python 3.7 desde una terminal en MacOS

```
$ python3.7
Python 3.7.4 (v3.7.4:e09359112e, Jul  8 2019, 14:54:52)
[Clang 6.0 (clang-600.0.57)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Para hacer una prueba inicial del intérprete vamos a retomar el primer programa que se suele hacer. Es el llamado *«Hello, World»*. Para ello escribimos lo siguiente en el intérprete y pulsamos la tecla `ENTER`:

```
>>> print('Hello, World')
Hello, World
```

Lo que hemos hecho es indicarle a Python que ejecute como **entrada** la instrucción `print('Hello, World')`. La **salida** es el texto `Hello, World` que lo vemos en la siguiente línea (*ya sin el prompt* `>>>`).

2.1.3 Probando el editor

Ahora vamos a realizar la misma operación, pero en vez de ejecutar la instrucción directamente en el intérprete, vamos a crear un fichero y guardarlo con la sentencia que nos interesa. Para ello escribimos `print('Hello, World')` en el panel de edición (*superior*) y luego guardamos el archivo con el nombre `helloworld.py`[3]:

Importante: Los ficheros que contienen programas hechos en Python siempre deben tener la extensión `.py`

---

[2] Término inglés que se refiere al símbolo que precede la línea de comandos.

[3] La carpeta donde se guarden los archivos de código no es crítico para su ejecución, pero sí es importante mantener un orden y una organización para tener localizados nuestros ficheros y proyectos.

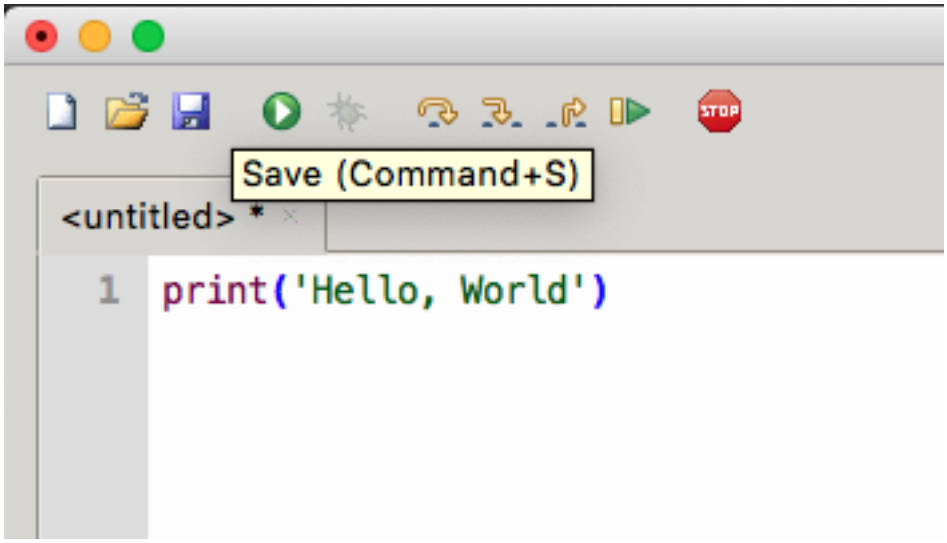


Figura 2: Guardando nuestro primer programa en Python

Ahora ya podemos *ejecutar* nuestro fichero `helloworld.py`. Para ello pulsamos el botón verde con triángulo blanco (en la barra de herramientas) o bien damos a la tecla `F5`. Veremos que en el panel de *Shell* nos aparece la salida esperada. Lo que está pasando «entre bambalinas» es que el intérprete de Python está recibiendo como entrada el fichero que hemos creado; lo ejecuta y devuelve la salida para que Thonny nos lo muestre en el panel correspondiente.

2.1.4 Probando el depurador

Nos falta por probar el depurador o «debugger». Aunque su funcionamiento va mucho más allá, de momento nos vamos a quedar en la posibilidad de inspeccionar las variables de nuestro programa. Desafortunadamente `helloworld.py` es muy simple y ni siquiera contiene variables, pero podemos hacer una pequeña modificación al programa para poder incorporarlas:

```
1 msg = 'Hello, World'
2 print(msg)
```

Aunque ya lo veremos en profundidad, lo que hemos hecho es añadir una variable `msg` en la *línea 1* para luego utilizarla al mostrar por pantalla su contenido. Si ahora volvemos a ejecutar nuestro programa veremos que en el panel de variables nos aparece la siguiente información:

Name	Value
msg	'Hello, World'

También existe la posibilidad, a través del depurador, de ir ejecutando nuestro programa **paso a paso**. Para ello basta con pulsar en el botón que tiene un *insecto*. Ahí comienza la

sesión de depuración y podemos avanzar instrucción por instrucción usando la tecla `F7`:

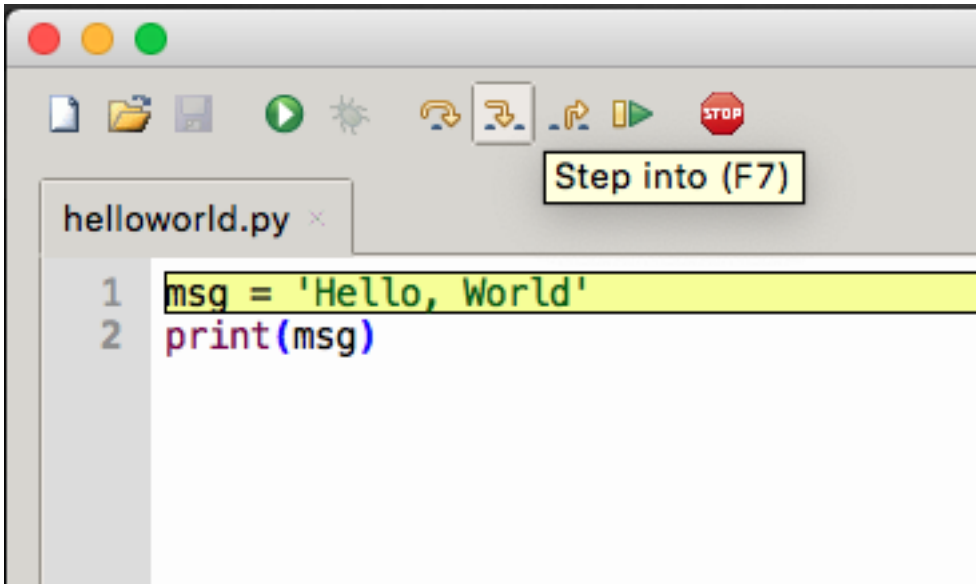


Figura 3: Depurando nuestro primer programa en Python

2.2 Contexto real



Hemos visto que *Thonny* es una herramienta especialmente diseñada para el aprendizaje de Python, integrando diferentes módulos que facilitan su gestión. Si bien lo podemos utilizar

para un desarrollo más *«serio»*, se suele recurrir a un flujo de trabajo algo diferente en **contextos más reales**.[1]

2.2.1 Python

La forma más habitual de instalar Python (junto con sus librerías) es descargarlo e instalarlo desde su página oficial:

- Versiones de Python para Windows
- Versiones de Python para Mac
- Versiones de Python para Linux

Truco: Tutorial para instalar Python en Windows.

Anaconda

Otra de las alternativas para disponer de Python en nuestro sistema y que además es muy utilizada, es **Anaconda**. Se trata de un *conjunto de herramientas*, orientadas en principio a la *ciencia de datos*, pero que podemos utilizarlas para desarrollo general en Python (junto con otras librerías adicionales).

Existen versiones de pago, pero la distribución *Individual Edition* es «open-source» y gratuita. Se puede descargar desde su página web. Anaconda trae por defecto una gran cantidad de paquetes Python en su distribución.

Ver también:

Miniconda es un instalador mínimo que trae por defecto Python y un pequeño número de paquetes útiles.

2.2.2 Gestión de paquetes

La instalación limpia[2] de Python ya ofrece de por sí muchos paquetes y módulos que vienen por defecto. Es lo que se llama la librería estándar. Pero una de las características más destacables de Python es su inmenso «ecosistema» de paquetes disponibles en el Python Package Index (PyPI).

Para gestionar los paquetes que tenemos en nuestro sistema se utiliza la herramienta pip, una utilidad que también se incluye en la instalación de Python. Con ella podremos instalar,

[1] Foto original de portada por SpaceX en Unsplash.

[2] También llamada «vanilla installation» ya que es la que viene por defecto y no se hace ningúna personalización.

desinstalar y actualizar paquetes, según nuestras necesidades. A continuación se muestran las instrucciones que usaríamos para cada una de estas operaciones:

Lista 2: Instalación, desinstalación y actualización del paquete `pandas` utilizando `pip`

```
$ pip install pandas
$ pip uninstall pandas
$ pip install pandas --upgrade
```

Consejo: Para el caso de *Anaconda* usaríamos `conda install pandas` (aunque ya viene preinstalado).

2.2.3 Entornos virtuales

Nivel intermedio

Cuando trabajamos en distintos proyectos, no todos ellos requieren los mismos paquetes ni siquiera la misma versión de Python. La gestión de estas situaciones no es sencilla si únicamente instalamos paquetes y manejamos configuraciones a nivel global (*a nivel de máquina*). Es por ello que surge el concepto de **entornos virtuales**. Como su propio nombre indica se trata de crear distintos entornos en función de las necesidades de cada proyecto, y esto nos permite establecer qué versión de Python usaremos y qué paquetes instalaremos.

La manera más sencilla de crear un entorno virtual es la siguiente:

```
1 $ cd myproject
2 $ python -m venv --prompt myproject .venv
3 $ source .venv/bin/activate
```

- *Línea 1*: Entrar en la carpeta de nuestro proyecto.
- *Línea 2*: Crear una carpeta `.venv` con los ficheros que constituyen el entorno virtual.
- *Línea 3*: Activar el entorno virtual. A partir de aquí todo lo que se instale quedará dentro del entorno virtual.

virtualenv

El paquete de Python que nos proporciona la funcionalidad de crear y gestionar entornos virtuales se denomina virtualenv. Su instalación es sencilla a través del gestor de paquetes `pip`:

```
$ pip install virtualenv
```

Si bien con `virtualenv` tenemos las funcionalidades necesarias para trabajar con entornos virtuales, destacaría una herramienta llamada virtualenvwrapper que funciona *por encima* de `virtualenv` y que facilita las operaciones sobre entornos virtuales. Su instalación es equivalente a cualquier otro paquete Python:

```
$ pip install virtualenvwrapper
```

Veamos a continuación algunos de los comandos que nos ofrece:

```
$ ~/project1 > mkvirtualenv env1
Using base prefix '/Library/Frameworks/Python.framework/Versions/3.7'
New python executable in /Users/sdelquin/.virtualenvs/env1/bin/python3.7
Also creating executable in /Users/sdelquin/.virtualenvs/env1/bin/python
Installing setuptools, pip, wheel...
done.
virtualenvwrapper.user_scripts creating /Users/sdelquin/.virtualenvs/env1/bin/
↪predeactivate
virtualenvwrapper.user_scripts creating /Users/sdelquin/.virtualenvs/env1/bin/
↪postdeactivate
virtualenvwrapper.user_scripts creating /Users/sdelquin/.virtualenvs/env1/bin/
↪preactivate
virtualenvwrapper.user_scripts creating /Users/sdelquin/.virtualenvs/env1/bin/
↪postactivate
virtualenvwrapper.user_scripts creating /Users/sdelquin/.virtualenvs/env1/bin/get_
↪env_details
$ (env1) ~/project1 > pip install requests
Collecting requests
Using cached requests-2.24.0-py2.py3-none-any.whl (61 kB)
Collecting idna<3,>=2.5
Using cached idna-2.10-py2.py3-none-any.whl (58 kB)
Collecting certifi>=2017.4.17
Using cached certifi-2020.6.20-py2.py3-none-any.whl (156 kB)
Collecting urllib3!=1.25.0,!=1.25.1,<1.26,>=1.21.1
Using cached urllib3-1.25.10-py2.py3-none-any.whl (127 kB)
Collecting chardet<4,>=3.0.2
Using cached chardet-3.0.4-py2.py3-none-any.whl (133 kB)
Installing collected packages: idna, certifi, urllib3, chardet, requests
Successfully installed certifi-2020.6.20 chardet-3.0.4 idna-2.10 requests-2.24.0␣
↪urllib3-1.25.10
```

(continué en la próxima página)

(proviene de la página anterior)

```
$ (env1) ~/project1 > deactivate
$ ~/project1 > workon env1
$ (env1) ~/project1 > lssitepackages
__pycache__                 distutils-precedence.pth    pkg_resources              ␣
↪urllib3-1.25.10.dist-info
_distutils_hack             easy_install.py             requests                   ␣
↪wheel
certifi                     idna                        requests-2.24.0.dist-info  ␣
↪wheel-0.34.2.dist-info
certifi-2020.6.20.dist-info idna-2.10.dist-info         setuptools
chardet                     pip                         setuptools-49.3.2.dist-info
chardet-3.0.4.dist-info     pip-20.2.2.dist-info        urllib3
$ (env1) ~/project1 >
```

- `$ mkvirtualenv env1`: crea un entorno virtual llamado `env1`
- `$ pip install requests`: instala el paquete `requests` dentro del entorno virtual `env1`
- `$ workon env1`: activa el entorno virtual `env1`
- `$ lssitepackages`: lista los paquetes instalados en el entorno virtual activo

pyenv

pyenv permite cambiar fácilmente entre múltiples versiones de Python en un mismo sistema. Su instalación engloba varios pasos y está bien explicada en la página del proyecto.

La mayor diferencia con respecto a *virtualenv* es que no instala las distintas versiones de Python a nivel global del sistema. En vez de eso, se suele crear una carpeta `.pyenv` en el HOME del usuario, donde todo está aislado sin generar intrusión en el sistema operativo.

Podemos hacer cosas como:

- Listar las versiones de Python instaladas:

```
$ pyenv versions
  3.7.4
  * 3.5.0 (set by /Users/yuu/.pyenv/version)
  miniconda3-3.16.0
  pypy-2.6.0
```

- Descubrir la versión global «activa» de Python:

```
$ python --version
Python 3.5.0
```

- Cambiar la versión global «activa» de Python:

```
$ pyenv global 3.7.4

$ python --version
Python 3.7.4
```

- Instalar una nueva versión de Python:

```
$ pyenv install 3.9.1
...
```

- Activar una versión de Python local por carpetas:

```
$ cd /cool-project
$ pyenv local 3.9.1
$ python --version
Python 3.9.1
```

También existe un módulo denominado pyenv-virtualenv para manejar entornos virtuales utilizando las ventajas que proporciona *pyenv*.

2.2.4 Editores

Existen multitud de editores en el mercado que nos pueden servir perfectamente para escribir código Python. Algunos de ellos incorporan funcionalidades extra y otros simplemente nos permiten editar ficheros. Cabe destacar aquí el concepto de **Entorno de Desarrollo Integrado**, más conocido por sus siglas en inglés **IDE**[3]. Se trata de una aplicación informática que proporciona servicios integrales para el desarrollo de software.

Podríamos decir que *Thonny* es un IDE de aprendizaje, pero existen muchos otros. Veamos un listado de editores de código que se suelen utilizar para desarrollo en Python:

- **Editores generales o IDEs con soporte para Python:**
  - Eclipse + PyDev
  - Sublime Text
  - Atom
  - GNU Emacs
  - Vi-Vim
  - Visual Studio (+ Python Tools)
  - Visual Studio Code (+ Python Tools)
- **Editores o IDEs específicos para Python:**

[3] Integrated Development Environment.

- PyCharm
- Spyder
- Thonny

Cada editor tiene sus características (ventajas e inconvenientes). Supongo que la preferencia por alguno de ellos estará en base a la experiencia y a las necesidades que surjan. La parte buena es que hay diversidad de opciones para elegir.

2.2.5 Jupyter Notebook

Jupyter Notebook es una aplicación «open-source» que permite crear y compartir documentos que contienen código, ecuaciones, visualizaciones y texto narrativo. Podemos utilizarlo para propósito general aunque suele estar más enfocado a *ciencia de datos*: limpieza y transformación de datos, simulación numérica, modelado estadístico, visualización o «machine-learning»[4].

Podemos verlo como un intérprete de Python (contiene un «kernel»[5] que permite ejecutar código) con la capacidad de incluir documentación en formato Markdown, lo que potencia sus funcionalidades y lo hace adecuado para preparar cualquier tipo de material vinculado con lenguajes de programación.

Aunque su uso está más extendido en el mundo Python, existen muchos otros «kernels» sobre los que trabajar en Jupyter Notebook.

Ver también:

Sección sobre *Jupyter*.

Truco: Visual Studio Code también dispone de integración con Jupyter Notebooks.

2.2.6 repl.it

repl.it es un **servicio web que ofrece un entorno de desarrollo integrado** para programar en más de 50 lenguajes (Python incluido).

Es gratuito y de uso colaborativo. Se requiere una cuenta en el sistema para utilizarlo. El hecho de no requerir instalación ni configuración previa lo hace atractivo en determinadas circunstancias.

En su versión gratuita ofrece:

[4] Término inglés utilizado para hacer referencia a algoritmos de aprendizaje automático.

[5] Proceso específico para un lenguaje de programación que ejecuta instrucciones y actúa como interfaz de entrada/salida.

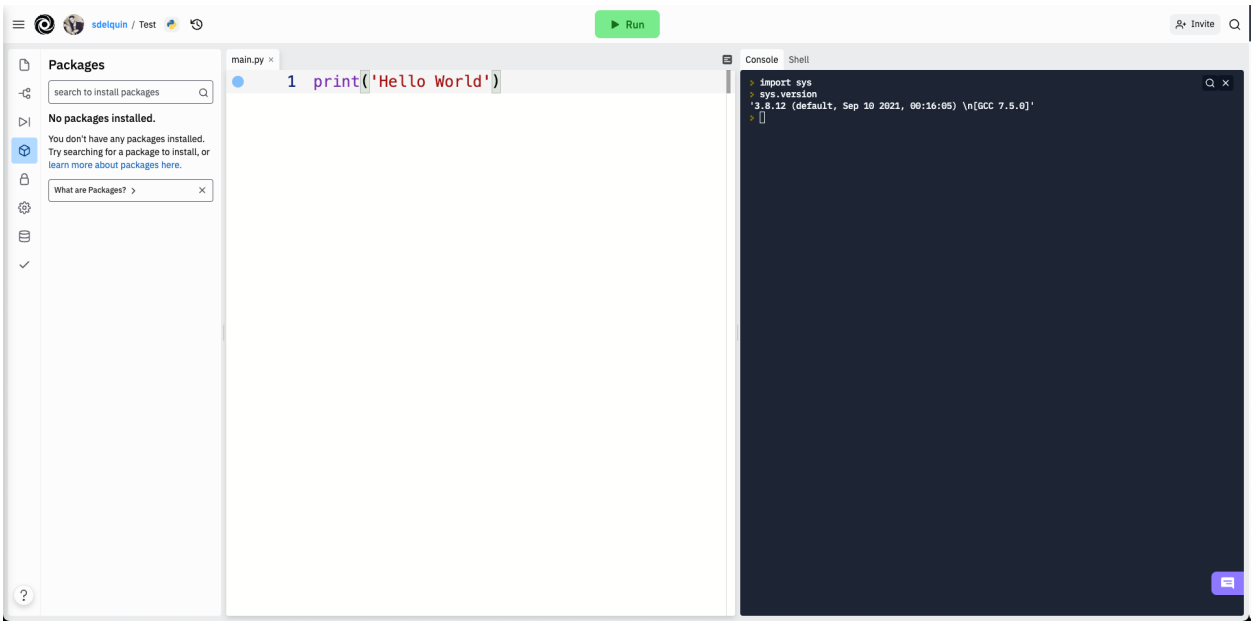


Figura 4: repl.it

- Almacenamiento de 500MB.
- Python 3.8.2 (febrero de 2022).
- 117 paquetes preinstalados (febrero de 2022).
- Navegador (y subida) de ficheros integrado.
- Gestor de paquetes integrado.
- Integración con GitHub.
- Gestión de secretos (datos sensibles).
- Base de datos clave-valor ya integrada.
- Acceso (limitado) al sistema operativo y sistema de ficheros.

2.2.7 WSL

Si estamos trabajando en un sistema **Windows 10** es posible que nos encontremos más cómodos usando una terminal tipo «Linux», entre otras cosas para poder usar con facilidad las herramientas vistas en esta sección y preparar el entorno de desarrollo Python. Durante mucho tiempo esto fue difícil de conseguir hasta que *Microsoft* sacó WSL.

WSL[6] nos proporciona una *consola con entorno Linux* que podemos utilizar en nuestro *Windows 10* sin necesidad de instalar una máquina virtual o crear una partición para un

[6] Windows Subsystem for Linux.

Linux nativo. Es importante también saber que existen dos versiones de WSL hoy en día: WSL y WSL2. La segunda es bastante reciente (publicada a mediados de 2019), tiene mejor rendimiento y se adhiere más al comportamiento de un Linux nativo.

Para la instalación de WSL[7] hay que seguir los siguientes pasos:

1. Lanzamos Powershell con permisos de administrador.
2. Activamos la característica de WSL:

```
$ Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-
↪Subsystem-Linux
```

3. Descargamos la imagen de Ubuntu 20.04 que usaremos:

```
$ Invoke-WebRequest -Uri https://aka.ms/wslubuntu2004 -OutFile Ubuntu.appx -
↪UseBasicParsing
```

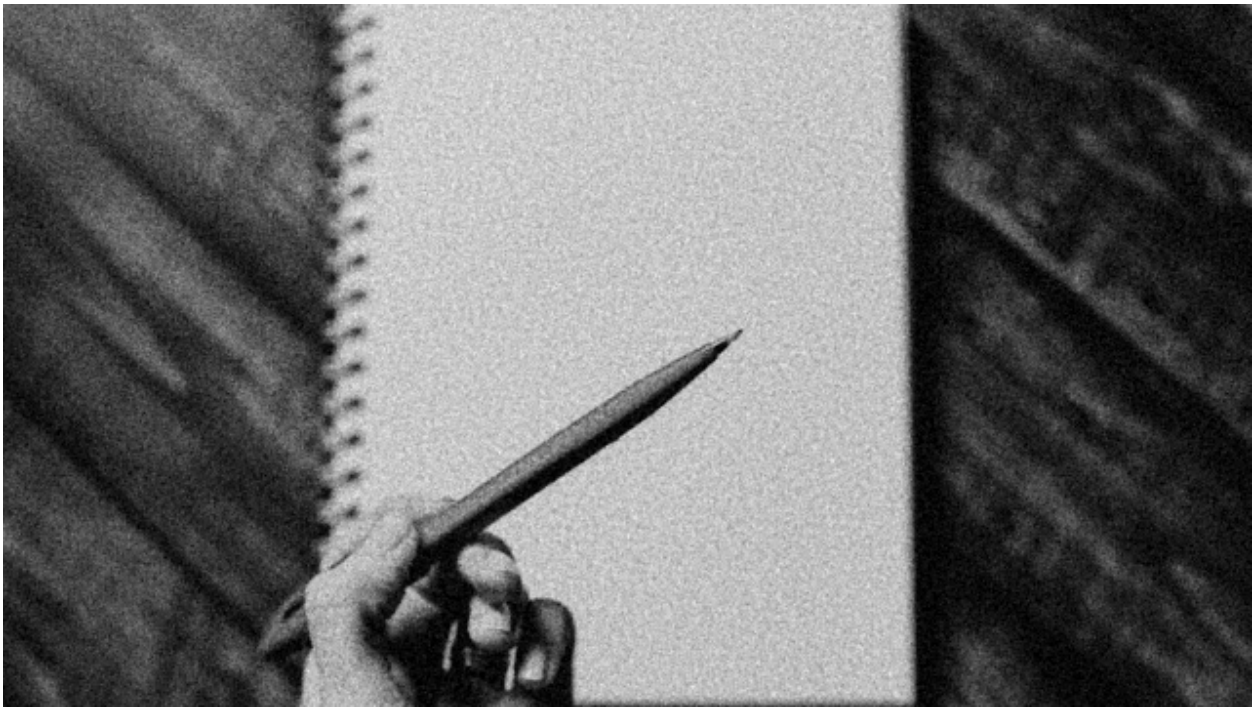
4. Finalmente, la instalamos:

```
$ Add-AppxPackage .\Ubuntu.appx
```

En este punto, WSL debería estar instalado correctamente, y debería también aparecer en el *menú Inicio*.

[7] Tutorial de instalación de WSL.

2.3 VSCode



Visual Studio Code (VSCode) es un entorno de desarrollo integrado[1] gratuito y de código abierto que ha ganado mucha relevancia en los últimos años. Permite trabajar fácilmente con multitud de lenguajes de programación y dispone de una gran cantidad de extensiones.[2]

2.3.1 Instalación

VSCode tiene disponibles paquetes autoinstalables para todos los sistemas operativos.

2.3.2 Extensiones recomendadas

Para escribir un «mejor» código Python en VSCode sería deseable tener instaladas las siguientes extensiones:

- Python
- Ruff
- Mypy Type Checker
- isort

[1] También conocido por IDE siglas en inglés de Integrated Development Environment.
[2] Foto original de portada por Kelly Sikkema en Unsplash.

2.3.3 Atajos de teclado

Conocer los atajos de teclado de tu editor favorito es fundamental para mejorar el flujo de trabajo y ser más productivo. Veamos los principales atajos de teclado de Visual Studio Code[3].

Ajustes generales

Acción	Atajo
Abrir paleta de comandos	`Ctrl + Shift + P`
Abrir archivo	`Ctrl + P`
Nueva ventana	`Ctrl + Shift + N`
Cerrar ventana	`Ctrl + Shift + W`
Ajustes del perfil	`Ctrl + ,`

Usabilidad

Acción	Atajo
Crear un nuevo archivo	`Ctrl + N`
Abrir archivo	`Ctrl + O`
Guardar archivo	`Ctrl + S`
Cerrar	`Ctrl + F4`
Abrir Terminal	`Ctrl + '`
Panel de problemas	`Ctrl + Shift + M`

Edición básica

Acción	Atajo
Cortar linea	`Ctrl + X`
Copiar linea	`Ctrl + C`
Borrar linea	`Ctrl + Shift + K`
Insertar linea abajo	`Enter`
Insertar linea arriba	`Ctrl + Shift + Enter`
Buscar en archivo abierto	`Ctrl + F`
Reemplazar	`Ctrl + H`
Linea de comentario	`Ctrl + /`
Bloque de comentario	`Shift + Alt + A`

continué en la próxima página

[3] Fuente: Gastón Danielsen en Dev.To.

Tabla 3 – proviene de la página anterior

Acción	Atajo
Salto de linea	`Alt + Z`
Seleccionar lineas	`Alt + Click Mouse`
Tabular linea	`Tab`
Destabular linea	`Shift + Tab`
Renombrar símbolo	`F2`

Pantalla

Acción	Atajo
Acercar Zoom	`Ctrl + +`
Alejar Zoom	`Ctrl + -`
Barra lateral	`Ctrl + B`
Abrir debug	`Ctrl + Shift + D`
Panel de salida	`Ctrl + Shift + U`
Control de source	`Ctrl + Shift + G`
Acceder a extensiones	`Ctrl + Shift + X`
Abrir terminal integrado	`Ctrl + Shift + Ñ`

Truco: En macOS 🍏 sustituir `Ctrl` por `Command`.

2.3.4 Depurando código

La **depuración de programas** es el proceso de **identificar y corregir errores de programación**. Es conocido también por el término inglés **debugging**, cuyo significado es eliminación de bugs (bichos), manera en que se conoce informalmente a los errores de programación.

Existen varias herramientas de depuración (o *debuggers*). Algunas de ellas en modo texto (terminal) y otras con entorno gráfico (ventanas).

- La herramienta más extendida para **depurar en modo texto** es el módulo pdb (The Python Debugger). Viene incluido en la instalación base de Python y es realmente potente.
- Aunque existen varias herramientas para **depurar en entorno gráfico**, nos vamos a centrar en **Visual Studio Code**.

Lo primero será abrir el fichero (carpeta) donde vamos a trabajar:

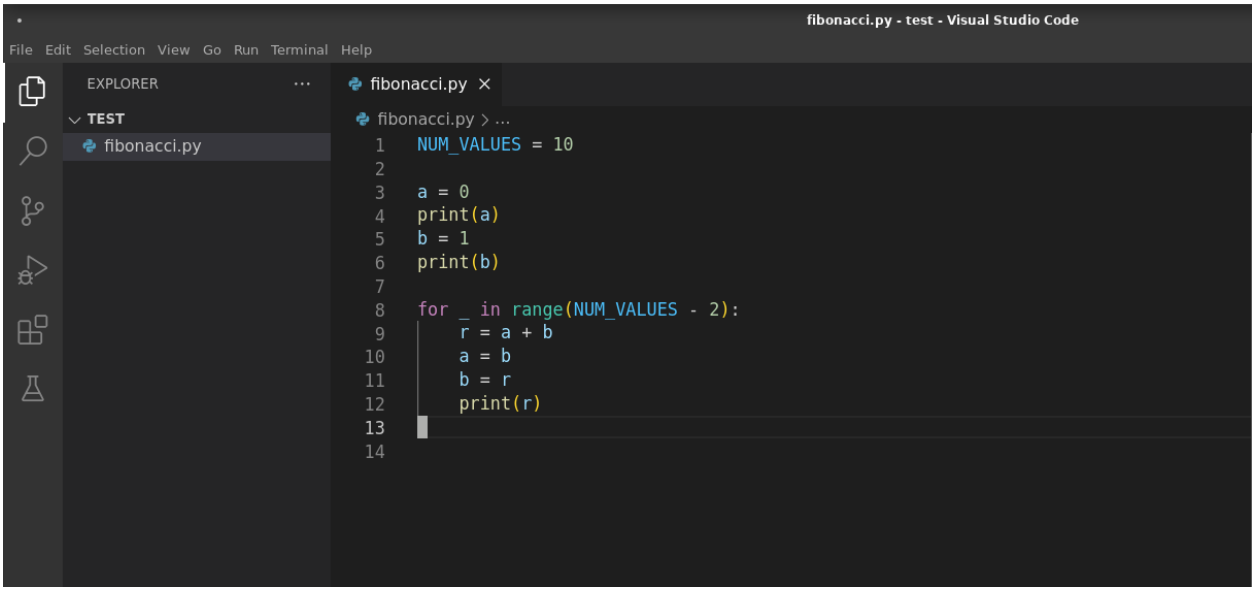


Figura 5: Apertura del fichero a depurar

Punto de ruptura

A continuación pondremos un **punto de ruptura** (también llamado **breakpoint**). Esto implica que la ejecución se pare en ese punto y viene indicado por un punto rojo 🔴. Para ponerlo nos tenemos que acercar a la columna que hay a la izquierda del número de línea y hacer clic.

En este ejemplo ponemos un punto de ruptura en la línea 10:

También es posible añadir **puntos de ruptura condicionales** pulsando con el botón derecho y luego **Add Conditional Breakpoint...**:

Lanzar la depuración

Ahora ya podemos **lanzar la depuración** pulsando la tecla `F5`. Nos aparecerá el siguiente mensaje en el que dejaremos la opción por defecto **Archivo de Python** y pulsamos la tecla :

Ahora ya se inicia el «modo depuración» y veremos una pantalla similar a la siguiente:

Zonas de la interfaz en modo depuración:

1. Código con barra en amarillo que indica la próxima línea que se va a ejecutar.
2. Visualización automática de valores de variables.
3. Visualización personalizada de valores de variables (o expresiones).
4. Salida de la terminal.

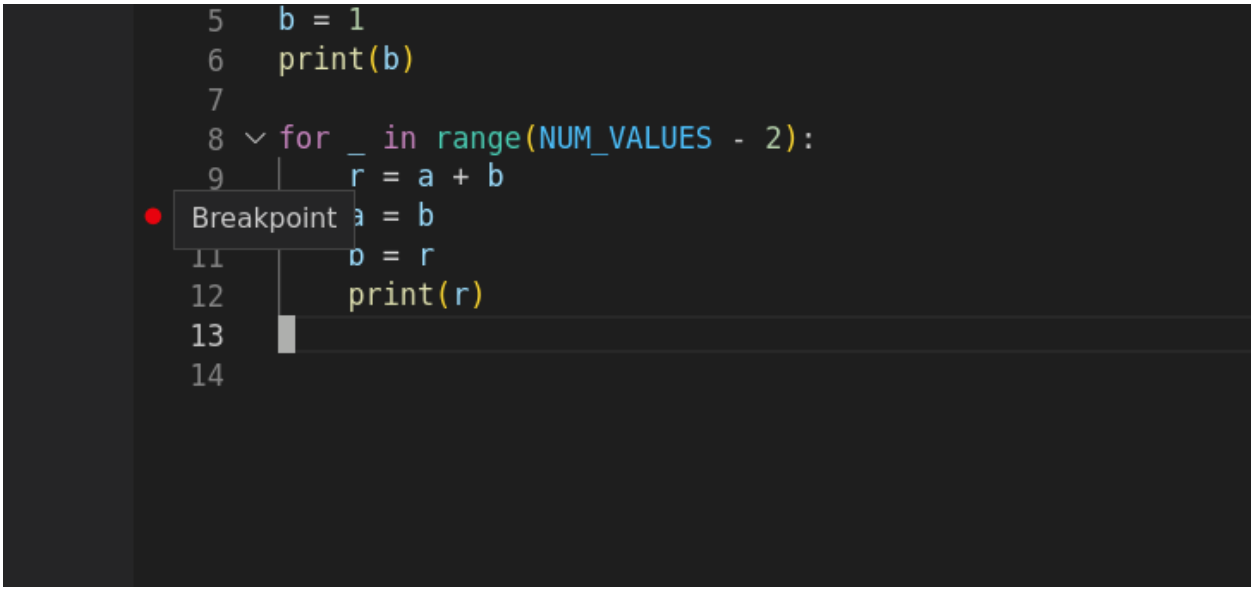


Figura 6: Punto de ruptura

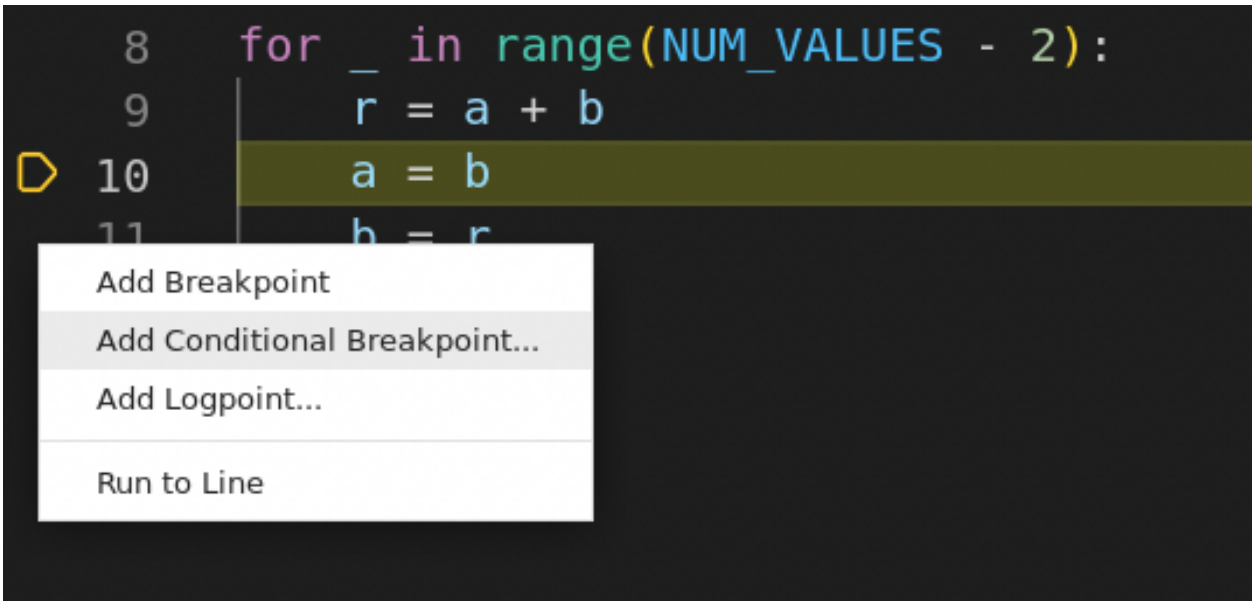


Figura 7: Punto de ruptura condicional

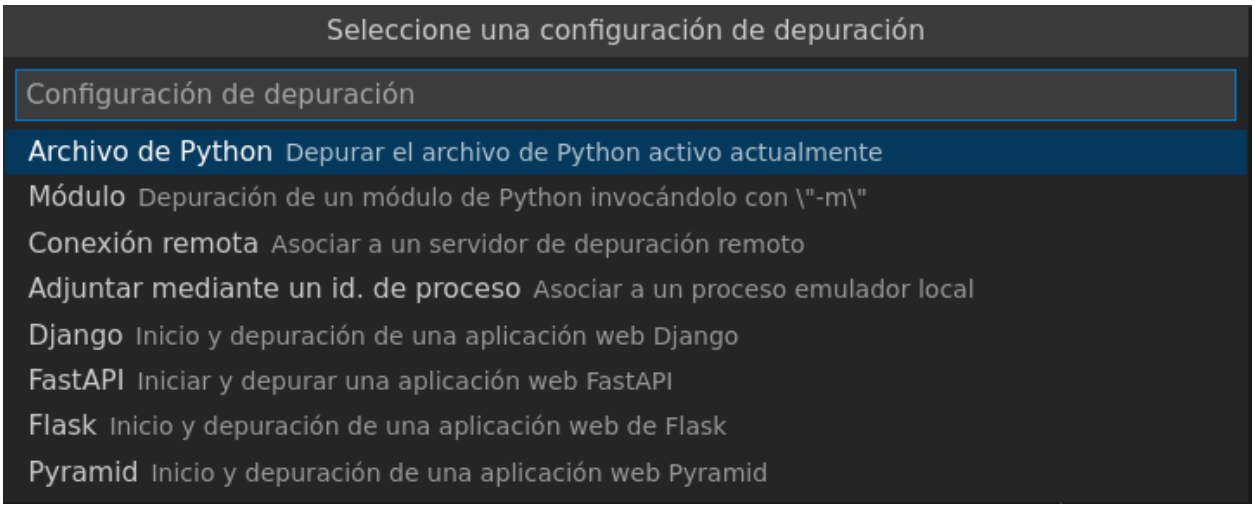


Figura 8: Configuración de la depuración

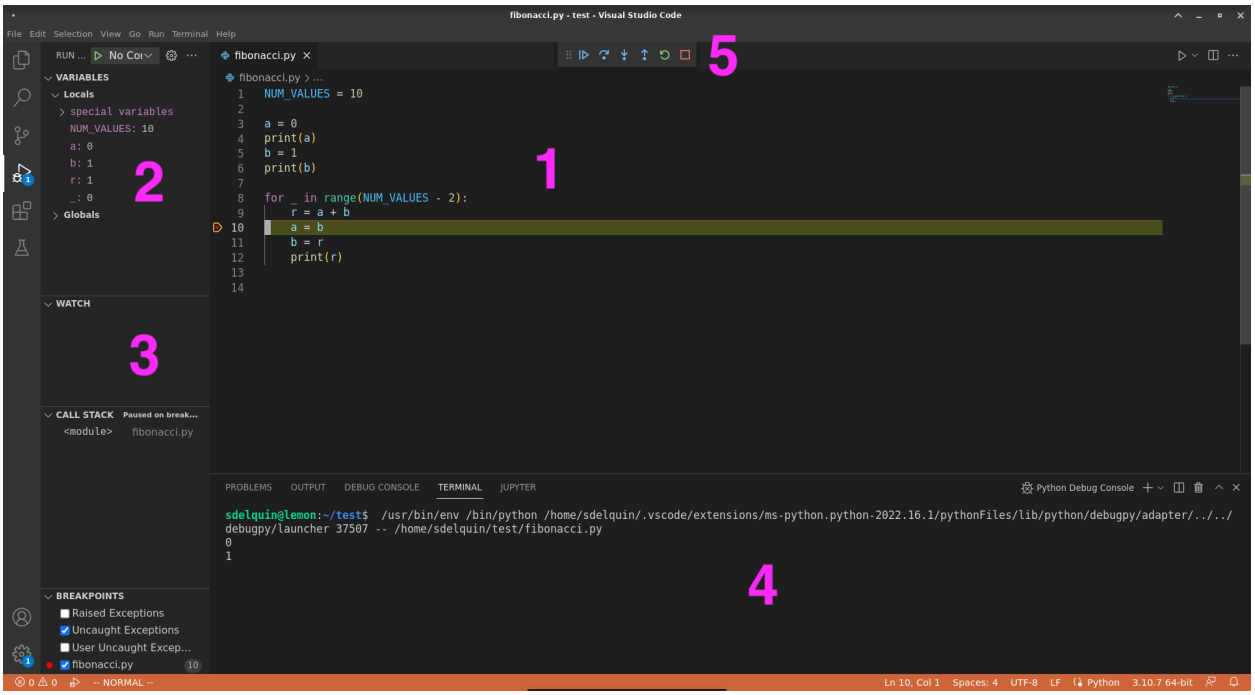


Figura 9: Interfaz en modo depuración

5. Barra de herramientas para depuración.

Controles para la depuración

Veamos con mayor detalle la **barra de herramientas para depuración**:

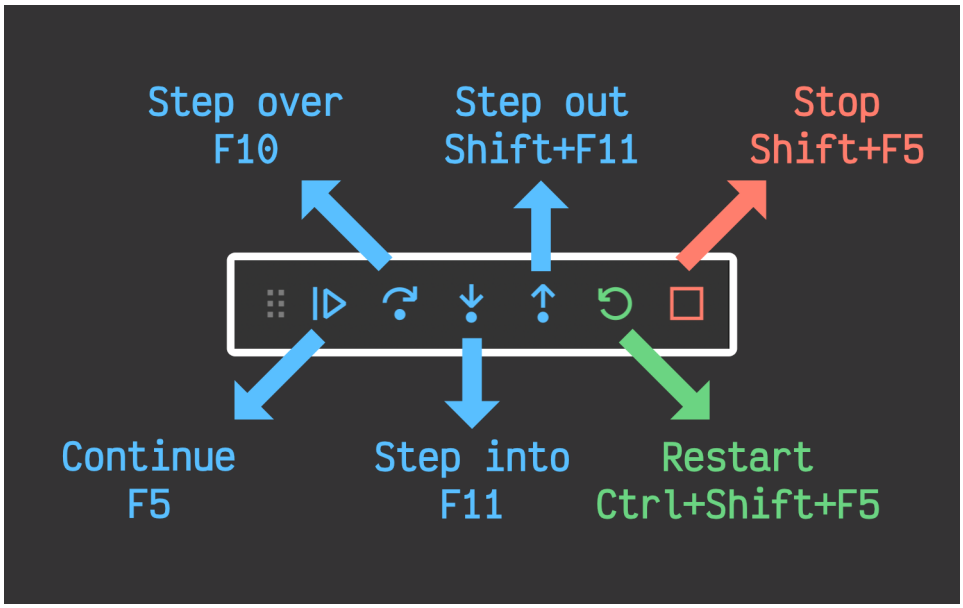


Figura 10: Barra de herarmientas para depuración

Acción	Atajo	Significado
Continue	`F5`	Continuar la ejecución del programa hasta el próximo punto de ruptura o hasta su finalización
Step over	`F10`	Ejecutar la siguiente instrucción del programa
Step into	`F11`	Ejecutar la siguiente instrucción del programa entrando en un contexto inferior
Step out	+ `F11`	Ejecutar la siguiente instrucción del programa saliendo a un contexto superior
Restart	+ + `F5`	Reiniciar la depuración del programa
Stop	+ `F5`	Detener la depuración del programa

Seguimiento de variables

Como hemos indicado previamente, la zona de **Variables** ya nos informa **automáticamente de los valores de las variables** que tengamos en el contexto actual de ejecución:

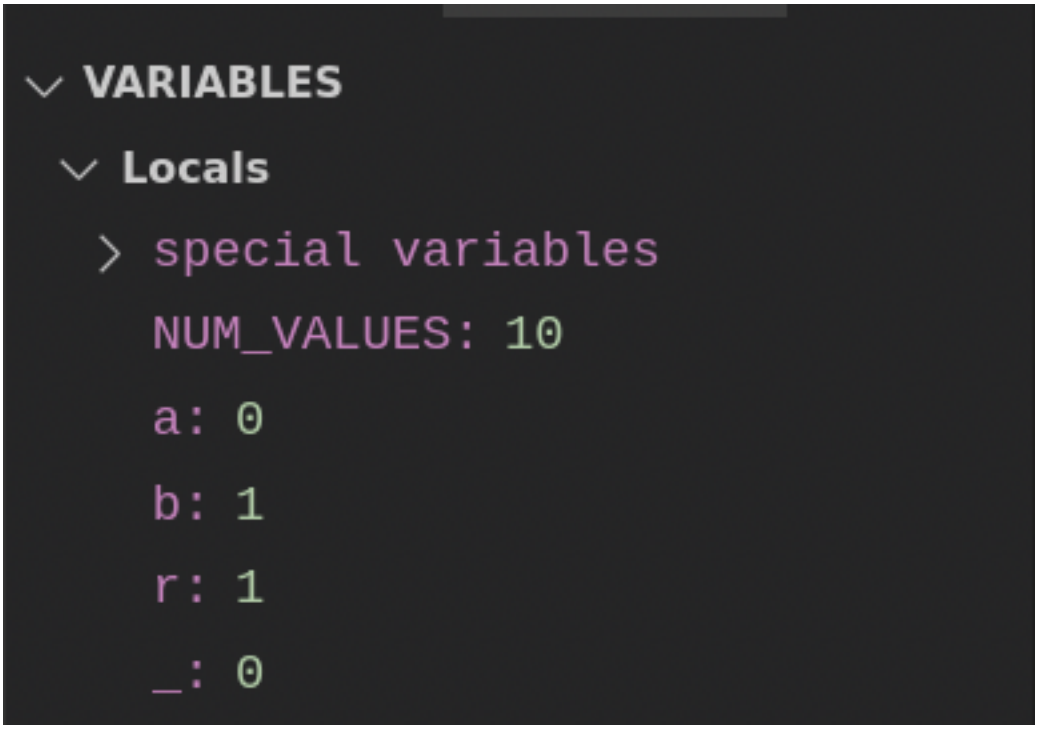


Figura 11: Panel para visualizar variables

Pero también es posible **añadir manualmente** el seguimiento de otras variables o expresiones personalizadas desde la zona **Watch**:

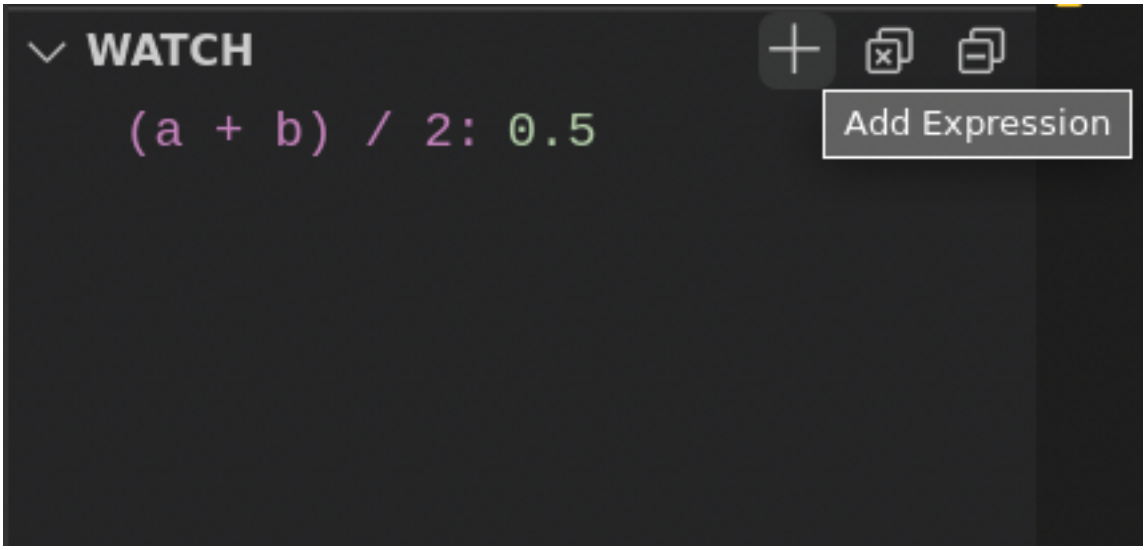


Figura 12: Panel para seguimiento de expresiones

CAPÍTULO 3

Tipos de datos

Igual que en el mundo real cada objeto pertenece a una categoría, en programación manejamos objetos que tienen asociado un tipo determinado. En este capítulo se verán los tipos de datos básicos con los que podemos trabajar en Python.

3.1 Datos



Los programas están formados por **código** y **datos**. Pero a nivel interno de la memoria del ordenador no son más que una secuencia de bits. La interpretación de estos bits depende del lenguaje de programación, que almacena en la memoria no sólo el puro dato sino distintos metadatos.[1]

Cada «trozo» de memoria contiene realmente un objeto, de ahí que se diga que en Python **todo son objetos**. Y cada objeto tiene, al menos, los siguientes campos:

- Un **tipo** del dato almacenado.
- Un **identificador** único para distinguirlo de otros objetos.
- Un **valor** consistente con su tipo.

3.1.1 Tipos de datos

A continuación se muestran los distintos tipos de datos que podemos encontrar en Python, sin incluir aquellos que proveen paquetes externos:

[1] Foto original de portada por Alexander Sinn en Unsplash.

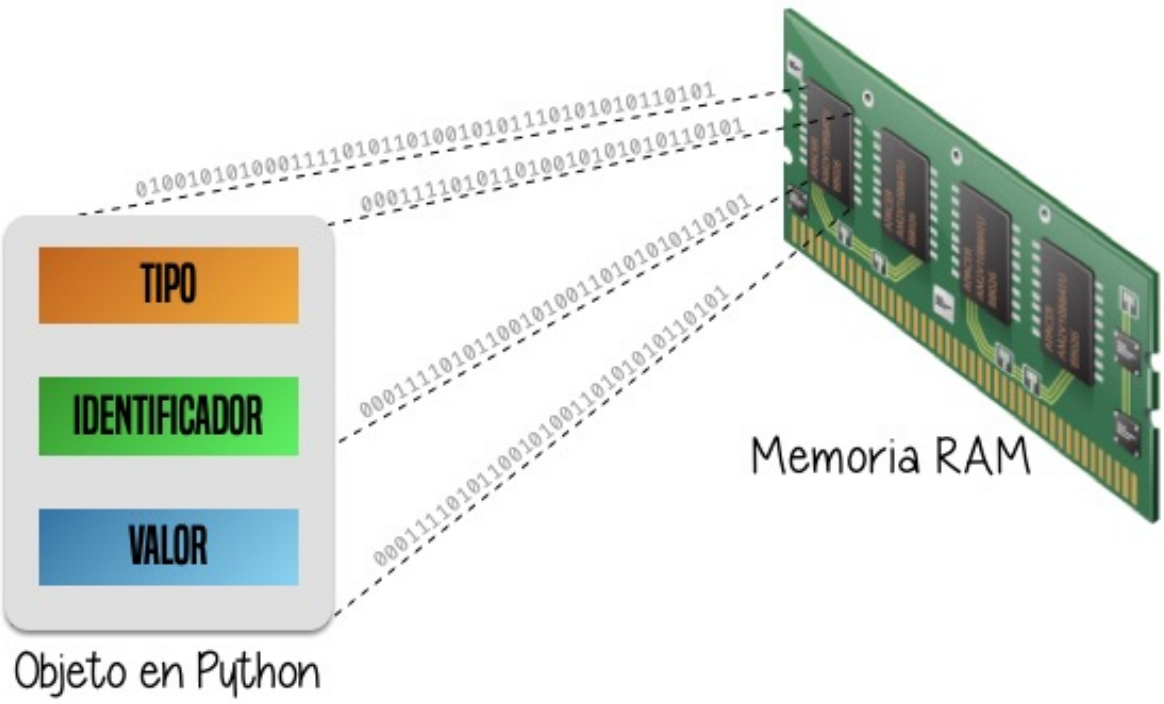


Figura 1: Esquema (*metadatos*) de un objeto en Python

Tabla 1: Tipos de datos en Python

Nombre	Tipo	Ejemplos
Booleano	bool	`True`, `False`
Entero	int	`21`, `34500`, `34_500`
Flotante	float	`3.14`, `1.5e3`
Complejo	complex	`2j`, `3 + 5j`
Cadena	str	`'tfn'`, `'''tenerife - islas canarias'''`
Tupla	tuple	`(1, 3, 5)`
Lista	list	`['Chrome', 'Firefox']`
Conjunto	set	`set([2, 4, 6])`
Diccionario	dict	`{'Chrome': 'v79' , 'Firefox': 'v71'}`

3.1.2 Variables

Las **variables** son fundamentales ya que permiten definir **nombres** para los **valores** que tenemos en memoria y que vamos a usar en nuestro programa.



Figura 2: Uso de un *nombre* de variable

Reglas para nombrar variables

En Python existen una serie de reglas para los nombres de variables:

1. **Sólo pueden contener los siguientes caracteres**[2]**:**
    - Letras minúsculas.
    - Letras mayúsculas.
    - Dígitos.
    - Guiones bajos (_).
2. Deben **empezar con una letra o un guión bajo**, nunca con un dígito.
3. No pueden ser una **palabra reservada** del lenguaje («keywords»).

[2] Para ser exactos, sí se pueden utilizar otros caracteres, e incluso *emojis* en los nombres de variables, aunque no suele ser una práctica extendida, ya que podría dificultar la legibilidad.

Podemos obtener un listado de las palabras reservadas del lenguaje de la siguiente forma:

```
>>> help('keywords')

Here is a list of the Python keywords.  Enter any keyword to get more help.

False               class               from                or
None                continue            global              pass
True                def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
break               for                 not
```

Nota: Por lo general se prefiere dar nombres **en inglés** a las variables que utilicemos, ya que así hacemos nuestro código más «internacional» y con la posibilidad de que otras personas puedan leerlo, entenderlo y – llegado el caso – modificarlo. Es sólo una recomendación, nada impide que se haga en castellano.

Importante: Los nombres de variables son «case-sensitive»[3]. Por ejemplo, `stuff` y `Stuff` son nombres diferentes.

Ejemplos de nombres de variables

Veamos a continuación una tabla con nombres de variables:

Tabla 2: Ejemplos de nombres de variables

Válido	Inválido	Razón
`a`	`3`	Empieza por un dígito
`a3`	`3a`	Empieza por un dígito
`a_b_c___95`	`another-name`	Contiene un carácter no permitido
`_abc`	`with`	Es una palabra reservada del lenguaje
`_3a`	`3_a`	Empieza por un dígito

[3] Sensible a cambios en mayúsculas y minúsculas.

Convenciones para nombres

Mientras se sigan las *reglas* que hemos visto para nombrar variables no hay problema en la forma en la que se escriban, pero sí existe una convención para la **nomenclatura de las variables**. Se utiliza el llamado `snake_case` en el que utilizamos **caracteres en minúsculas** (incluyendo dígitos si procede) junto con **guiones bajos** – cuando sean necesarios para su legibilidad –.[4] Por ejemplo, para nombrar una variable que almacene el número de canciones en nuestro ordenador, podríamos usar `num_songs`.

Esta convención, y muchas otras, están definidas en un documento denominado PEP 8. Se trata de una **guía de estilo** para escribir código en Python. Los PEPs[5] son las propuestas que se hacen para la mejora del lenguaje.

Aunque hay múltiples herramientas disponibles para la comprobación del estilo de código, una bastante accesible es http://pep8online.com/ ya que no necesita instalación, simplemente pegar nuestro código y verificar.

Constantes

Un caso especial y que vale la pena destacar son las **constantes**. Podríamos decir que es un tipo de variable pero que su valor no cambia a lo largo de nuestro programa. Por ejemplo la velocidad de la luz. Sabemos que su valor es constante de 300.000 km/s. En el caso de las constantes utilizamos **mayúsculas** (incluyendo guiones bajos si es necesario) para nombrarlas. Para la velocidad de la luz nuestra constante se podría llamar: `LIGHT_SPEED`.

Elegir buenos nombres

Se suele decir que una persona programadora (con cierta experiencia), a lo que dedica más tiempo, es a buscar un buen nombre para sus variables. Quizás pueda resultar algo excesivo pero da una idea de lo importante que es esta tarea. Es fundamental que los nombres de variables sean **autoexplicativos**, pero siempre llegando a un compromiso entre ser concisos y claros.

Supongamos que queremos buscar un nombre de variable para almacenar el número de elementos que se deben manejar en un pedido:

1. `n`
2. `num_elements`
3. `number_of_elements`
4. `number_of_elements_to_be_handled`

[4] Más información sobre convenciones de nombres en PEP 8.
[5] Del término inglés «Python Enhancement Proposals».

No existe una regla mágica que nos diga cuál es el nombre perfecto, pero podemos aplicar el *sentido común* y, a través de la experiencia, ir detectando aquellos nombres que sean más adecuados. En el ejemplo anterior, quizás podríamos descartar de principio la opción *1* y la *4* (por ser demasiado cortas o demasiado largas); nos quedaríamos con las otras dos. Si nos fijamos bien, casi no hay mucha información adicional de la opción *3* con respecto a la *2*. Así que podríamos concluir que la opción *2* es válida para nuestras necesidades. En cualquier caso esto dependerá siempre del contexto del problema que estemos tratando.

Como regla general:

- Usar **nombres** para *variables* (ejemplo `article`).
- Usar **verbos** para *funciones* (ejemplo `get_article()`).
- Usar **adjetivos** para *booleanos* (ejemplo `available`).

3.1.3 Asignación

En Python se usa el símbolo `=` para **asignar** un valor a una variable:



Figura 3: Asignación de *valor* a *nombre* de variable

Nota: Hay que diferenciar la asignación en Python con la igualación en matemáticas. El símbolo `=` lo hemos aprendido desde siempre como una *equivalencia* entre dos *expresiones algebraicas*, sin embargo en Python nos indica una *sentencia de asignación*, del valor (en la derecha) al nombre (en la izquierda).

Algunos ejemplos de asignaciones a *variables*:

```
>>> total_population = 157_503
>>> avg_temperature = 16.8
>>> city_name = 'San Cristóbal de La Laguna'
```

Algunos ejemplos de asignaciones a *constantes*:

```
>>> SOUND_SPEED = 343.2
>>> WATER_DENSITY = 997
>>> EARTH_NAME = 'La Tierra'
```

Python nos ofrece la posibilidad de hacer una **asignación múltiple** de la siguiente manera:

```
>>> tres = three = drei = 3
```

En este caso las tres variables utilizadas en el «lado izquierdo» tomarán el valor `3`.

Recordemos que los nombres de variables deben seguir unas *reglas establecidas*, de lo contrario obtendremos un **error sintáctico** del intérprete de Python:

```
>>> 7floor = 40  # el nombre empieza por un dígito
  File "<stdin>", line 1
    7floor = 40
         ^
SyntaxError: invalid syntax

>>> for = 'Bucle'  # el nombre usa la palabra reservada "for"
  File "<stdin>", line 1
    for = 'Bucle'
        ^
SyntaxError: invalid syntax

>>> screen-size = 14  # el nombre usa un carácter no válido
  File "<stdin>", line 1
SyntaxError: can't assign to operator
```

Asignando una variable a otra variable

Las asignaciones que hemos hecho hasta ahora han sido de un **valor literal** a una variable. Pero nada impide que podamos hacer asignaciones de una variable a otra variable:

```
>>> people = 157503
>>> total_population = people
>>> total_population
157503
```

Eso sí, la variable que utilicemos como valor de asignación **debe existir previamente**, ya que si no es así, obtendremos un error informando de que no está definida:

```
>>> total_population = lot_of_people
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'lot_of_people' is not defined
```

De hecho, en el *lado derecho* de la asignación pueden aparecer *expresiones* más complejas que se verán en su momento.

Conocer el valor de una variable

Hemos visto previamente cómo asignar un valor a una variable, pero aún no sabemos cómo «comprobar» el valor que tiene dicha variable. Para ello podemos utilizar dos estrategias:

1. Si estamos en un **intérprete** («shell» o consola) de Python, basta con que usemos el nombre de la variable:

```
>>> final_stock = 38934
>>> final_stock
38934
```

2. Si estamos escribiendo un programa desde el **editor**, debmos hacer uso de `print()`:

```
final_stock = 38934
print(final_stock)
```

Nota: `print()` sirve también cuando estamos en una sesión interactiva de Python («shell»)

Conocer el tipo de una variable

Para poder descubrir el tipo de un literal o una variable, Python nos ofrece la función `type()`. Veamos algunos ejemplos de su uso:

```
>>> type(9)
int

>>> type(1.2)
float

>>> height = 3718
>>> type(height)
int

>>> SOUND_SPEED = 343.2
>>> type(SOUND_SPEED)
float
```

Advertencia: Aunque está permitido, **NUNCA** llames `type` a una variable porque destruirías la función que nos permite conocer el tipo de un objeto.

Ejercicio

Utilizando la consola interactiva de Python `>>>`, realiza las siguientes tareas:

1. Asigna un valor entero `2001` a la variable `space_odyssey` y muestra su valor.
2. Descubre el tipo del literal `'Good night & Good luck'`.
3. Identifica el tipo del literal `True`.
4. Asigna la expresión `10 * 3.0` a la variable `result` y muestra su tipo.

3.1.4 Mutabilidad

Nivel avanzado

Las variables son nombres, no lugares. Detrás de esta frase se esconde la reflexión de que cuando asignamos un valor a una variable, lo que realmente está ocurriendo es que se hace **apuntar** el nombre de la variable a una zona de memoria en el que se representa el objeto (con su valor):

```
>>> a = 5
```

Si ahora «copiamos» el valor de `a` en otra variable `b` se podría esperar que hubiera otro espacio en memoria para dicho valor, pero como ya hemos dicho, son referencias a memoria:

```
>>> b = a
```

La función `id()` nos permite conocer la dirección de memoria[6] de un objeto en Python. A través de ella podemos comprobar que los dos objetos que hemos creado «apuntan» a la misma zona de memoria:

```
>>> id(a)
4445989712

>>> id(b)
4445989712
```

La prueba de que la zona de memoria no la ocupa el «nombre» de la variable, es que podemos ver cómo se asigna una dirección de memoria únicamente al «valor» literal:

[6] Esto es un detalle de implementación de CPython.

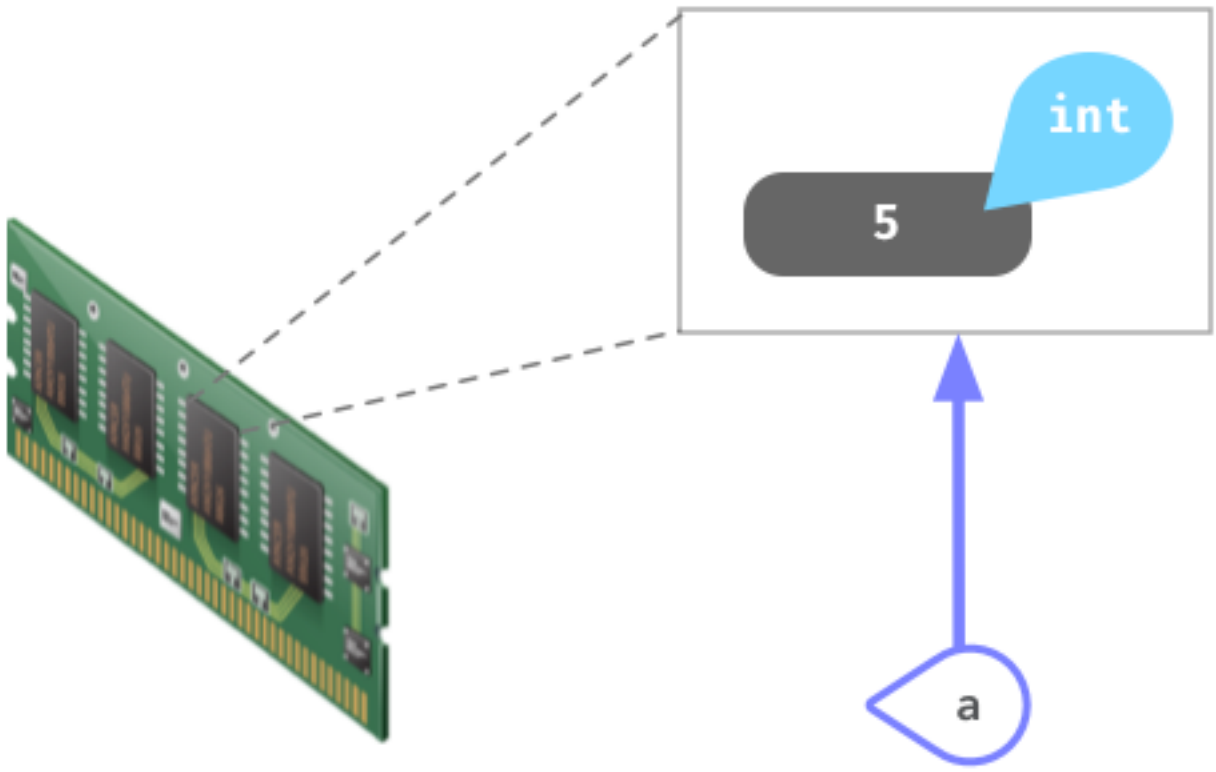


Figura 4: Representación de la asignación de valor a variable

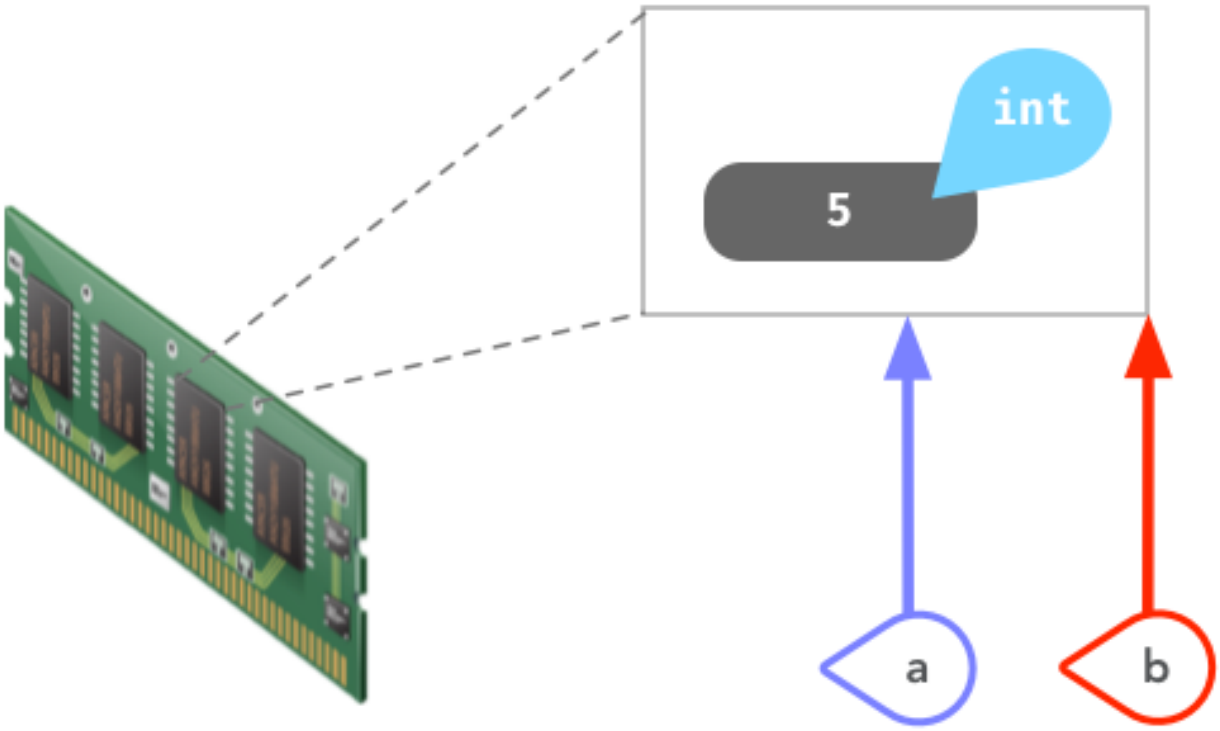


Figura 5: Representación de la asignación de una variable a otra variable

```
>>> id(10)
4333546384

>>> id(20)
4333546704
```

Cada vez que asignamos un nuevo valor a una variable, ésta apunta a una nueva zona de memoria:

```
>>> a = 5
>>> id(a)
4310690224

>>> a = 7
>>> id(a)
4310690288
```

Cuando la zona de memoria que ocupa el objeto se puede modificar hablamos de tipos de datos **mutables**. En otro caso hablamos de tipos de datos **inmutables**.

Por ejemplo, las **listas** son un tipo de dato mutable ya que podemos modificar su contenido (aunque la asignación de un nuevo valor sigue generando un nuevo espacio de memoria).

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/lvCyXeL

Tipos de objetos en Python según su naturaleza de cambio:

Inmutable	Mutable
`bool`	`list`
`int`	`set`
`float`	`dict`
`str`	
`tuple`	

Importante: El hecho de que un tipo de datos sea inmutable significa que no podemos modificar su valor «in-situ», pero siempre podremos asignarle un nuevo valor (hacerlo apuntar a otra zona de memoria).

3.1.5 Funciones «built-in»

Nivel intermedio

Hemos ido usando una serie de *funciones* sin ser especialmente conscientes de ello. Esto se debe a que son funciones «built-in» o incorporadas por defecto en el propio lenguaje Python.

Tabla 3: Funciones «built-in»

`abs()`	`delattr()`	`hash()`	`memoryview()`	`set()`
`all()`	`dict()`	`help()`	`min()`	`setattr()`
`any()`	`any()`	`hex()`	`next()`	`slice()`
`ascii()`	`divmod()`	`id()`	`object()`	`sorted()`
`bin()`	`enumerate()`	`input()`	`oct()`	`staticmethod()`
`bool()`	`eval()`	`int()`	`open()`	`str()`
`breakpoint()`	`exec()`	`isinstance()`	`ord()`	`sum()`
`bytearray()`	`filter()`	`issubclass()`	`pow()`	`super()`
`bytes()`	`float()`	`iter()`	`print()`	`tuple()`
`callable()`	`format()`	`len()`	`property()`	`type()`
`chr()`	`frozenset()`	`list()`	`range()`	`vars()`
`classmethod()`	`getattr()`	`locals()`	`repr()`	`zip()`
`compile()`	`globals()`	`map()`	`reversed()`	`__import__()`
`complex()`	`hasattr()`	`max()`	`round()`	

Los detalles de estas funciones se puede consultar en la documentación oficial de Python.

3.1.6 Pidiendo ayuda

En Python podemos pedir ayuda con la función `help()`.

Supongamos que queremos obtener información sobre `id`. Desde el intérprete de Python ejecutamos lo siguiente:

```
>>> help(id)
Help on built-in function id in module builtins:

id(obj, /)
    Return the identity of an object.

    This is guaranteed to be unique among simultaneously existing objects.
    (CPython uses the object's memory address.)
```

Existe una *forma alternativa* de obtener ayuda: añadiendo el signo de interrogación `?` al término de búsqueda:

```
>>> id?
Signature: id(obj, /)
Docstring:
Return the identity of an object.

This is guaranteed to be unique among simultaneously existing objects.
(CPython uses the object's memory address.)
Type:      builtin_function_or_method
```

AMPLIAR CONOCIMIENTOS

- Basic Data Types in Python
- Variables in Python
- Immutability in Python

3.2 Números



En esta sección veremos los tipos de datos númericos que ofrece Python centrándonos en **booleanos**, **enteros** y **flotantes**.[1]

3.2.1 Booleanos

George Boole es considerado como uno de los fundadores del campo de las ciencias de la computación y fue el creador del Álgebra de Boole que da lugar, entre otras estructuras algebraicas, a la Lógica binaria. En esta lógica las variables sólo pueden tomar dos valores discretos: **verdadero** o **falso**.

El tipo de datos `bool` proviene de lo explicado anteriormente y admite dos posibles valores:

- `True` que se corresponde con *verdadero* (y también con **1** en su representación numérica).
- `False` que se corresponde con *falso* (y también con **0** en su representación numérica).

Veamos un ejemplo de su uso:

```
>>> is_opened = True
>>> is_opened
True
```

(continué en la próxima página)

[1] Foto original de portada por Brett Jordan en Unsplash.

(proviene de la página anterior)

```
>>> has_sugar = False
>>> has_sugar
False
```

La primera variable `is_opened` está representando el hecho de que algo esté abierto, y al tomar el valor `True` podemos concluir que sí. La segunda variable `has_sugar` nos indica si una bebida tiene azúcar; dado que toma el valor `False` inferimos que no lleva azúcar.

Atención: Tal y como se explicó en *este apartado*, los nombres de variables son «case-sensitive». De igual modo el tipo booleano toma valores `True` y `False` con **la primera letra en mayúsculas**. De no ser así obtendríamos un error sintáctico.

```
>>> is_opened = true
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'true' is not defined
>>> has_sugar = false
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'false' is not defined
```

3.2.2 Enteros

Los números enteros no tienen decimales pero sí pueden contener signo y estar expresados en alguna base distinta de la usual (base 10).

Literales enteros

Veamos algunos ejemplos de números enteros:

```
>>> 8
8
>>> 0
0
>>> 08
  File "<stdin>", line 1
    08
     ^
SyntaxError: invalid token
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> 99
99
>>> +99
99
>>> -99
-99
>>> 3000000
3000000
>>> 3_000_000
3000000
```

Dos detalles a tener en cuenta:

- No podemos comenzar un número entero por `0`.
- Python permite dividir los números enteros con *guiones bajos* `_` para clarificar su lectura/escritura. A efectos prácticos es como si esos guiones bajos no existieran.

Operaciones con enteros

A continuación se muestra una tabla con las distintas operaciones sobre enteros que podemos realizar en Python:

Tabla 4: Operaciones con enteros en Python

Operador	Descripción	Ejemplo	Resultado
`+`	Suma	`3 + 9`	`12`
`-`	Resta	`6 - 2`	`4`
`*`	Multiplicación	`5 * 5`	`25`
`/`	División flotante	`9 / 2`	`4.5`
`//`	División entera	`9 // 2`	`4`
`%`	Módulo	`9 % 4`	`1`
`**`	Exponenciación	`2 ** 4`	`16`

Veamos algunas pruebas de estos operadores:

```
>>> 2 + 8 + 4
14
>>> 4 ** 4
256
>>> 7 / 3
2.3333333333333335
>>> 7 // 3
2
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> 6 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

Es de buen estilo de programación **dejar un espacio** entre cada operador. Además hay que tener en cuenta que podemos obtener errores dependiendo de la operación (más bien de los *operandos*) que estemos utilizando, como es el caso de la *división por cero*.

Igualmente es importante tener en cuenta la **prioridad** de los distintos operadores:

Prioridad	Operador
1 (mayor)	()
2	**
3	-a +a
4	* / // %
5 (menor)	+ -

Ejemplos de prioridad de operadores:

```
>>> 2 ** 2 + 4 / 2
6.0

>>> 2 ** (2 + 4) / 2
32.0

>>> 2 ** (2 + 4 / 2)
16.0
```

Asignación aumentada

Python nos ofrece la posibilidad de escribir una asignación aumentada mezclando la *asignación* y un *operador*.

Supongamos que disponemos de 100 vehículos en stock y que durante el pasado mes se han vendido 20 de ellos. Veamos cómo sería el código con asignación tradicional vs. asignación aumentada:

Lista 1: Asignación tradicional

```
>>> total_cars = 100
>>> sold_cars = 20
>>> total_cars = total_cars - sold_cars
```

(continué en la próxima página)

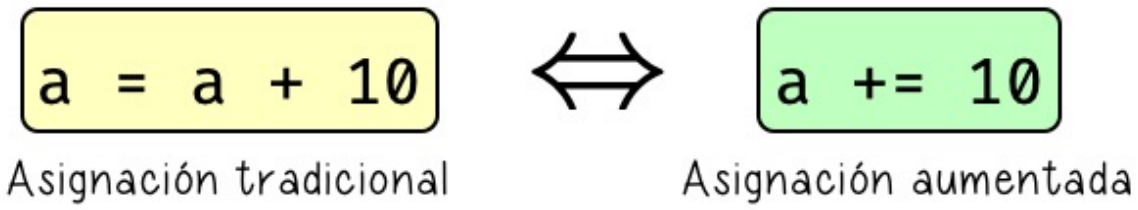


Figura 6: Asignación aumentada en Python

(proviene de la página anterior)

```
>>> total_cars
80
```

Lista 2: Asignación aumentada

```
>>> total_cars = 100
>>> sold_cars = 20
>>> total_cars -= sold_cars
>>> total_cars
80
```

Estas dos formas son equivalentes a nivel de resultados y funcionalidad, pero obviamente tienen diferencias de escritura y legibilidad. De este mismo modo, podemos aplicar un formato compacto al resto de operaciones:

```
>>> random_number = 15

>>> random_number += 5
>>> random_number
20

>>> random_number *= 3
>>> random_number
60

>>> random_number //= 4
>>> random_number
15

>>> random_number **= 1
>>> random_number
15
```

Módulo

La operación **módulo** (también llamado **resto**), cuyo símbolo en Python es `%`, se define como el resto de dividir dos números. Veamos un ejemplo para enteder bien su funcionamiento:

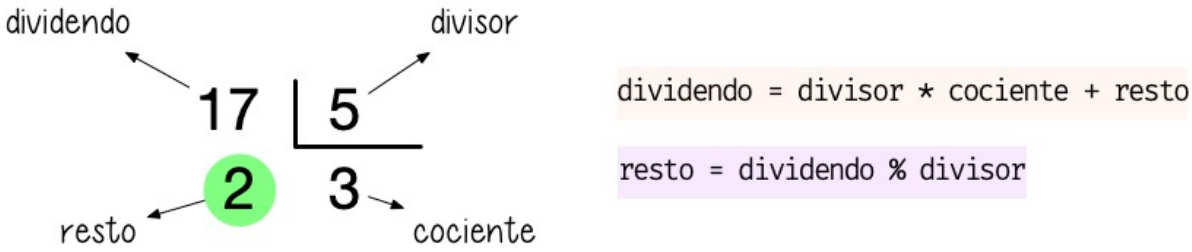


Figura 7: Operador «módulo» en Python

```
>>> dividendo = 17
>>> divisor = 5

>>> cociente = dividendo // divisor  # división entera
>>> resto = dividendo % divisor

>>> cociente
3
>>> resto
2
```

Exponenciación

Para elevar un número a otro en Python utilizamos el operador de exponenciación `**`:

```
>>> 4 ** 3
64
>>> 4 * 4 * 4
64
```

Se debe tener en cuenta que también podemos elevar un número entero a un **número decimal**. En este caso es como si estuviéramos haciendo una *raíz*[2]. Por ejemplo:

$$4^{\frac{1}{2}} = 4^{0.5} = \sqrt{4} = 2$$

Hecho en Python:

[2] No siempre es una raíz cuadrada porque se invierten numerador y denominador.

```
>>> 4 ** 0.5
2.0
```

Ejercicio

pycheck: **quadratic**

Valor absoluto

Python ofrece la función `abs()` para obtener el valor absoluto de un número:

```
>>> abs(-1)
1

>>> abs(1)
1

>>> abs(-3.14)
3.14

>>> abs(3.14)
3.14
```

Límite de un entero

Nivel avanzado

¿Cómo de grande puede ser un `int` en Python? La respuesta es **de cualquier tamaño**. Por poner un ejemplo, supongamos que queremos representar un centillón. Este valor viene a ser un «1» seguido por ¡600 ceros! ¿Será capaz Python de almacenarlo?

```
>>> centillion = 10 ** 600

>>> centillion
1000000000
```

```
>>> 10 ** 10_000
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: Exceeds the limit (4300) for integer string conversion; use sys.set_int_
↪max_str_digits() to increase the limit
```

Obtenemos un error… pero subsanable, ya que hay forma de ampliar este **límite inicial de 4300 dígitos** usando la función `sys.set_int_max_str_digits()`

3.2.3 Flotantes

Los números en **punto flotante**[3] tienen **parte decimal**. Veamos algunos ejemplos de flotantes en Python.

Lista 3: Distintas formas de escribir el flotante *4.0*

```
>>> 4.0
4.0
>>> 4.
4.0
>>> 04.0
4.0
>>> 04.
4.0
>>> 4.000_000
4.0
>>> 4e0  # 4.0 * (10 ** 0)
4.0
```

Conversión de tipos

El hecho de que existan distintos tipos de datos en Python (y en el resto de lenguajes de programación) es una ventaja a la hora de representar la información del mundo real de la mejor manera posible. Pero también se hace necesario buscar mecanismos para convertir unos tipos de datos en otros.

[3] Punto o coma flotante es una notación científica usada por computadores.

Conversión implícita

Cuando mezclamos enteros, booleanos y flotantes, Python realiza automáticamente una conversión implícita (o **promoción**) de los valores al tipo de «mayor rango». Veamos algunos ejemplos de esto:

```
>>> True + 25
26
>>> 7 * False
0
>>> True + False
1
>>> 21.8 + True
22.8
>>> 10 + 11.3
21.3
```

Podemos resumir la conversión implícita en la siguiente tabla:

Tipo 1	Tipo 2	Resultado
`bool`	`int`	`int`
`bool`	`float`	`float`
`int`	`float`	`float`

Se puede ver claramente que la conversión numérica de los valores booleanos es:

- `True` 👉 `1`
- `False` 👉 `0`

Conversión explícita

Aunque más adelante veremos el concepto de **función**, desde ahora podemos decir que existen una serie de funciones para realizar conversiones explícitas de un tipo a otro:

`bool()` Convierte el tipo a *booleano*.

`int()` Convierte el tipo a *entero*.

`float()` Convierte el tipo a *flotante*.

Veamos algunos ejemplos de estas funciones:

```
>>> bool(1)
True
>>> bool(0)
```

(continué en la próxima página)

(proviene de la página anterior)

```
False
>>> int(True)
1
>>> int(False)
0
>>> float(1)
1.0
>>> float(0)
0.0
>>> float(True)
1.0
>>> float(False)
0.0
```

En el caso de que usemos la función `int()` sobre un valor flotante nos retornará su **parte baja**:

$$int(x) = \lfloor x \rfloor$$

Por ejemplo:

```
>>> int(3.1)
3
>>> int(3.5)
3
>>> int(3.9)
3
```

Para **obtener el tipo** de una variable *ya hemos visto* la función `type()`:

```
>>> is_raining = False
>>> type(is_raining)
bool

>>> sound_level = 35
>>> type(sound_level)
int

>>> temperature = 36.6
>>> type(temperature)
float
```

Pero también existe la posibilidad seguimos **comprobar el tipo** que tiene una variable mediante la función `isinstance()`:

```
>>> isinstance(is_raining, bool)
True
>>> isinstance(sound_level, int)
True
>>> isinstance(temperature, float)
True
```

Ejercicio

pycheck: **sin_approx**

Errores de aproximación

Nivel intermedio

Supongamos el siguiente cálculo:

```
>>> (19 / 155) * (155 / 19)
    0.9999999999999999
```

Debería dar 1.0, pero no es así puesto que la representación interna de los valores en **coma flotante** sigue el estándar IEEE 754 y estamos trabajando con aritmética finita.

Aunque existen distintas formas de solventar esta limitación, de momento veremos una de las más sencillas utilizando la función «built-in» round() que nos permite redondear un número flotante a un número determinado de decimales:

```
>>> pi = 3.14159265359

>>> round(pi)
3
>>> round(pi, 1)
3.1
>>> round(pi, 2)
3.14
>>> round(pi, 3)
3.142
>>> round(pi, 4)
3.1416
>>> round(pi, 5)
3.14159
```

Para el caso del error de aproximación que nos ocupa:

```
>>> result = (19 / 155) * (155 / 19)

>>> round(result, 1)
1.0
```

Prudencia: `round()` aproxima al valor más cercano, mientras que `int()` obtiene siepre el entero «por abajo».

Límite de un flotante

A diferencia de los *enteros*, los números flotantes sí que tienen un límite en Python. Para descubrirlo podemos ejecutar el siguiente código:

```
>>> import sys

>>> sys.float_info.min
2.2250738585072014e-308

>>> sys.float_info.max
1.7976931348623157e+308
```

3.2.4 Bases

Nivel intermedio

Los valores numéricos con los que estamos acostumbrados a trabajar están en **base 10** (o decimal). Esto indica que disponemos de 10 «símbolos» para representar las cantidades. En este caso del `0` al `9`.

Pero también es posible representar números en **otras bases**. Python nos ofrece una serie de **prefijos** y **funciones** para este cometido.

Base binaria

Cuenta con **2** símbolos para representar los valores: `0` y `1`.

Prefijo: `0b`

```
>>> 0b1001
9
>>> 0b1100
12
```

Función: `bin()`

```
>>> bin(9)
'0b1001'
>>> bin(12)
'0b1100'
```

Prudencia: Esta función devuelve una *cadena de texto*.

Base octal

Cuenta con **8** símbolos para representar los valores: `0`, `1`, `2`, `3`, `4`, `5`, `6` y `7`.

Prefijo: `0o`

```
>>> 0o6243
3235
>>> 0o1257
687
```

Función: `oct()`

```
>>> oct(3235)
'0o6243'
>>> oct(687)
'0o1257'
```

Prudencia: Esta función devuelve una *cadena de texto*.

Base hexadecimal

Cuenta con **16** símbolos para representar los valores: `0`, `1`, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, `A`, `B`, `C`, `D`, `E` y `F`.

Prefijo: `0x`

```
>>> 0x7F2A
32554
>>> 0x48FF
18687
```

Función: `hex()`

```
>>> hex(32554)
'0x7f2a'
>>> hex(18687)
'0x48ff'
```

Prudencia: Esta función devuelve una *cadena de texto*.

Nota: Las letras para la representación hexadecimal no atienden a mayúsculas y minúsculas.

EJERCICIOS DE REPASO

1. pycheck: **circle_area**
2. pycheck: **sphere_volume**
3. pycheck: **triangle_area**
4. pycheck: **interest_rate**
5. pycheck: **euclid_distance**
6. pycheck: **century_year**
7. pycheck: **red_square**
8. pycheck: **igic**
9. pycheck: **super_fast**
10. pycheck: **move_twice**
11. pycheck: **pillars**
12. pycheck: **clock_time**
13. pycheck: **xor**
14. pycheck: **ring_area**

EJERCICIOS EXTERNOS

1. Cat years, dog years
2. Aspect ratio cropping
3. USD => CNY
4. Third angle of a triangle
5. Keep hydrated!
6. Price of mangoes
7. Total pressure calculation
8. NBA full 48 minutes average
9. Age range compatibility equation
10. Formatting decimal places

AMPLIAR CONOCIMIENTOS

- The Python Square Root Function
- How to Round Numbers in Python
- Operators and Expressions in Python

3.3 Cadenas de texto



Las cadenas de texto son **secuencias** de **caracteres**. También se les conoce como «strings» y nos permiten almacenar información textual de forma muy cómoda.[1]

Es importante destacar que Python 3 almacena los caracteres codificados en el estándar Unicode, lo que es una gran ventaja con respecto a versiones antiguas del lenguaje. Además permite representar una cantidad ingente de símbolos incluyendo los famosos emojis 😎.

3.3.1 Creando «strings»

Para escribir una cadena de texto en Python basta con rodear los caracteres con comillas simples[6]:

```
>>> 'Mi primera cadena en Python'
'Mi primera cadena en Python'
```

Para incluir *comillas dobles* dentro de la cadena de texto no hay mayor inconveniente:

```
>>> 'Los llamados "strings" son secuencias de caracteres'
'Los llamados "strings" son secuencias de caracteres'
```

[1] Foto original de portada por Roman Kraft en Unsplash.

[6] También es posible utilizar comillas dobles. Yo me he decantado por las comillas simples ya que quedan más limpias y suele ser el formato que devuelve el propio intérprete de Python.

Para incluir *comillas simples* dentro de la cadena de texto cambiamos las comillas exteriores a comillas dobles:

```
>>> "Los llamados 'strings' son secuencias de caracteres"
"Los llamados 'strings' son secuencias de caracteres"
```

Truco: Efectivamente, como se puede ver, las cadenas de texto en Python se pueden escribir con comillas simples o con comillas dobles. Es indiferente. **En mi caso personal prefiero usar comillas simples**.

Elijas lo que elijas, ¡haz siempre lo mismo!

Comillas triples

Hay una forma alternativa de crear cadenas de texto y es utilizar *comillas triples*. Su uso está pensado principalmente para **cadenas multilínea**:

```
>>> poem = """To be, or not to be, that is the question:
... Whether 'tis nobler in the mind to suffer
... The slings and arrows of outrageous fortune,
... Or to take arms against a sea of troubles"""
```

En este caso sí que se debería **utilizar comillas dobles** siguiendo las indicaciones de la guía de estilo de Python:

> In Python, single-quoted strings and double-quoted strings are the same. This PEP does not make a recommendation for this. Pick a rule and stick to it. When a string contains single or double quote characters, however, use the other one to avoid backslashes in the string. It improves readability.
>
> For triple-quoted strings, always use double quote characters to be consistent with the docstring convention in PEP 257.

Importante: Los tres puntos ... que aparecen a la izquierda de las líneas no están incluidos en la cadena de texto. Es el símbolo que ofrece el intérprete de Python cuando saltamos de línea.

Cadena vacía

La cadena vacía es aquella que no contiene ningún carácter. Aunque a priori no lo pueda parecer, es un recurso importante en cualquier código. Su representación en Python es la siguiente:

```
>>> ''
''
```

3.3.2 Conversión

Podemos crear «strings» a partir de otros tipos de datos usando la función `str()`:

```
>>> str(True)
'True'
>>> str(10)
'10'
>>> str(21.7)
'21.7'
```

Para el caso contrario de convertir un «string» a un valor numérico, tenemos a disposición las funciones ya vistas:

```
>>> int('10')
10
>>> float('21.7')
21.7
```

Pero hay que tener en cuenta un detalle. La función `int()` también admite la **base** en la que se encuentra el número. Eso significa que podemos pasar un número, por ejemplo, en **hexadecimal** (como «string») y lo podríamos convertir a su valor entero:

```
>>> int('FF', 16)
255
```

Nota: La base por defecto que utiliza `int()` para convertir cadenas de texto es la **base decimal**.

3.3.3 Secuencias de escape

Python permite **escapar** el significado de algunos caracteres para conseguir otros resultados. Si escribimos una barra invertida \ antes del carácter en cuestión, le otorgamos un significado especial.

Quizás la *secuencia de escape* más conocida es `\n` que representa un *salto de línea*, pero existen muchas otras:

```
# Salto de línea
>>> msg = 'Primera línea\nSegunda línea\nTercera línea'
>>> print(msg)
Primera línea
Segunda línea
Tercera línea

# Tabulador
>>> msg = 'Valor = \t40'
>>> print(msg)
Valor =     40

# Comilla simple
>>> msg = 'Necesitamos \'escapar\' la comilla simple'
>>> print(msg)
Necesitamos 'escapar' la comilla simple

# Barra invertida
>>> msg = 'Capítulo \\ Sección \\ Encabezado'
>>> print(msg)
Capítulo \ Sección \ Encabezado
```

Nota: Al utilizar la función `print()` es cuando vemos realmente el resultado de utilizar los caracteres escapados.

Expresiones literales

Nivel intermedio

Hay situaciones en las que nos interesa que los caracteres especiales pierdan ese significado y poder usarlos de otra manera. Existe un modificar de cadena que proporciona Python para tratar el texto *en bruto*. Es el llamado «raw data» y se aplica anteponiendo una `r` a la cadena de texto.

Veamos algunos ejemplos:

```
>>> text = 'abc\ndef'
>>> print(text)
abc
def

>>> text = r'abc\ndef'
>>> print(text)
abc\ndef

>>> text = 'a\tb\tc'
>>> print(text)
a    b    c

>>> text = r'a\tb\tc'
>>> print(text)
a\tb\tc
```

Consejo: El modificador `r''` es muy utilizado para la escritura de **expresiones regulares**.

3.3.4 Más sobre `print()`

Hemos estado utilizando la función `print()` de forma sencilla, pero admite algunos parámetros interesantes:

```
1  >>> msg1 = '¿Sabes por qué estoy acá?'
2  >>> msg2 = 'Porque me apasiona'
3
4  >>> print(msg1, msg2)
5  ¿Sabes por qué estoy acá? Porque me apasiona
6
7  >>> print(msg1, msg2, sep='|')
8  ¿Sabes por qué estoy acá?|Porque me apasiona
9
10 >>> print(msg2, end='!!')
11 Porque me apasiona!!
```

Línea 4: Podemos imprimir todas las variables que queramos separándolas por comas.

Línea 7: El *separador por defecto* entre las variables es un *espacio*, podemos cambiar el carácter que se utiliza como separador entre cadenas.

Línea 10: El *carácter de final de texto* es un *salto de línea*, podemos cambiar el carácter que se utiliza como final de texto.

3.3.5 Leer datos desde teclado

Los programas se hacen para tener interacción con el usuario. Una de las formas de interacción es solicitar la entrada de datos por teclado. Como muchos otros lenguajes de programación, Python también nos ofrece la posibilidad de leer la información introducida por teclado. Para ello se utiliza la función `input()`:

```
>>> name = input('Introduzca su nombre: ')
Introduzca su nombre: Sergio
>>> name
'Sergio'
>>> type(name)
str

>>> age = input('Introduzca su edad: ')
Introduzca su edad: 41
>>> age
'41'
>>> type(age)
str
```

Nota: La función `input()` siempre nos devuelve un objeto de tipo cadena de texto o `str`. Tenerlo muy en cuenta a la hora de trabajar con números, ya que debemos realizar una *conversión explícita*.

Advertencia: Aunque está permitido, **NUNCA** llames `input` a una variable porque destruirías la función que nos permite leer datos desde teclado. Y tampoco uses nombres derivados como `_input` o `input_` ya que no son nombres representativos que *identifiquen el propósito de la variable*.

Ejercicio

Escriba un programa en Python que *lea por teclado* dos números enteros y muestre por pantalla el resultado de realizar las operaciones básicas entre ellos.

Ejemplo

- Valores de entrada 7 y 4.
- Salida esperada:

```
7+4=11
7-4=3
```

(continué en la próxima página)

(proviene de la página anterior)

```
7*4=28
7/4=1.75
```

Consejo:

- Aproveche todo el potencial que ofrece *print()* para conseguir la salida esperada
- No utilice «f-strings».
- Guarde el programa en un fichero `calc.py` y ejecútelo desde la terminal con: `python calc.py`

3.3.6 Operaciones con «strings»

Combinar cadenas

Podemos combinar dos o más cadenas de texto utilizando el operador `+`:

```
>>> proverb1 = 'Cuando el río suena'
>>> proverb2 = 'agua lleva'

>>> proverb1 + proverb2
'Cuando el río suenaagua lleva'

>>> proverb1 + ', ' + proverb2  # incluimos una coma
'Cuando el río suena, agua lleva'
```

Repetir cadenas

Podemos repetir dos o más cadenas de texto utilizando el operador `*`:

```
>>> reaction = 'Wow'

>>> reaction * 4
'WowWowWowWow'
```

Obtener un carácter

Los «strings» están **indexados** y cada carácter tiene su propia posición. Para obtener un único carácter dentro de una cadena de texto es necesario especificar su **índice** dentro de corchetes `[...]`.

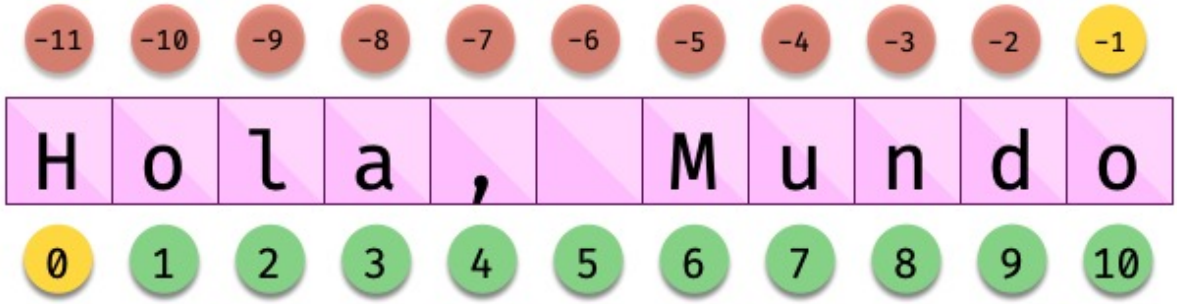


Figura 8: Indexado de una cadena de texto

Veamos algunos ejemplos de acceso a caracteres:

```
>>> sentence = 'Hola, Mundo'

>>> sentence[0]
'H'
>>> sentence[-1]
'o'
>>> sentence[4]
','
>>> sentence[-5]
'M'
```

Truco: Nótese que existen tanto **índices positivos** como **índices negativos** para acceder a cada carácter de la cadena de texto. A priori puede parecer redundante, pero es muy útil en determinados casos.

En caso de que intentemos acceder a un índice que no existe, obtendremos un error por *fuera de rango*:

```
>>> sentence[50]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

Advertencia: Téngase en cuenta que el indexado de una cadena de texto siempre empieza en **0** y termina en **una unidad menos de la longitud** de la cadena.

Las cadenas de texto son tipos de datos *inmutables*. Es por ello que no podemos modificar un carácter directamente:

```
>>> song = 'Hey Jude'

>>> song[4] = 'D'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

Truco: Existen formas de modificar una cadena de texto que veremos más adelante, aunque realmente no estemos transformando el original sino creando un nuevo objeto con las modificaciones.

Advertencia: No hay que confundir las *constantes* con los tipos de datos inmutables. Es por ello que las variables que almacenan cadenas de texto, a pesar de ser inmutables, no se escriben en mayúsculas.

Trocear una cadena

Es posible extraer «trozos» («rebanadas») de una cadena de texto[2]. Tenemos varias aproximaciones para ello:

`[:]` Extrae la secuencia entera desde el comienzo hasta el final. Es una especia de **copia** de toda la cadena de texto.

`[start:]` Extrae desde `start` hasta el final de la cadena.

`[:end]` Extrae desde el comienzo de la cadena hasta `end` *menos 1*.

`[start:end]` Extrae desde `start` hasta `end` *menos 1*.

`[start:end:step]` Extrae desde `start` hasta `end` *menos 1* haciendo saltos de tamaño `step`.

Veamos la aplicación de cada uno de estos accesos a través de un ejemplo:

[2] El término usado en inglés es *slice*.

```
>>> proverb = 'Agua pasada no mueve molino'

>>> proverb[:]
'Agua pasada no mueve molino'

>>> proverb[12:]
'no mueve molino'

>>> proverb[:11]
'Agua pasada'

>>> proverb[5:11]
'pasada'

>>> proverb[5:11:2]
'psd'
```

Importante: El troceado siempre llega a una unidad menos del índice final que hayamos especificado. Sin embargo el comienzo sí coincide con el que hemos puesto.

Longitud de una cadena

Para obtener la longitud de una cadena podemos hacer uso de `len()`, una función común a prácticamente todos los tipos y estructuras de datos en Python:

```
>>> proverb = 'Lo cortés no quita lo valiente'
>>> len(proverb)
30

>>> empty = ''
>>> len(empty)
0
```

Pertenencia de un elemento

Si queremos comprobar que una determinada subcadena se encuentra en una cadena de texto utilizamos el operador `in` para ello. Se trata de una expresión que tiene como resultado un valor «booleano» verdadero o falso:

```
>>> proverb = 'Más vale malo conocido que bueno por conocer'
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> 'malo' in proverb
True

>>> 'bueno' in proverb
True

>>> 'regular' in proverb
False
```

Habría que prestar atención al caso en el que intentamos descubrir si una subcadena **no está** en la cadena de texto:

```
>>> dna_sequence = 'ATGAAATTGAAATGGGA'

>>> not('C' in dna_sequence)  # Primera aproximación
True

>>> 'C' not in dna_sequence  # Forma pitónica
True
```

Limpiar cadenas

Cuando leemos datos del usuario o de cualquier fuente externa de información, es bastante probable que se incluyan en esas cadenas de texto, *caracteres de relleno*[3] al comienzo y al final. Python nos ofrece la posibilidad de eliminar estos caracteres u otros que no nos interesen.

La función `strip()` se utiliza para eliminar caracteres del principio y del final de un «string». También existen variantes de esta función para aplicarla únicamente al comienzo o únicamente al final de la cadena de texto.

Supongamos que debemos procesar un fichero con números de serie de un determinado artículo. Cada línea contiene el valor que nos interesa pero se han «colado» ciertos caracteres de relleno que debemos limpiar:

```
>>> serial_number = '\n\t   \n 48374983274832    \n\n\t   \t   \n'

>>> serial_number.strip()
'48374983274832'
```

Nota: Si no se especifican los caracteres a eliminar, `strip()` usa por defecto cualquier

[3] Se suele utilizar el término inglés «padding» para referirse a estos caracteres.

combinación de *espacios en blanco*, *saltos de línea* `\n` y *tabuladores* `\t`.

A continuación vamos a hacer «limpieza» por la izquierda (*comienzo*) y por la derecha (*final*) utilizando la función `lstrip()` y `rstrip()` respectivamente:

Lista 4: «Left strip»

```
>>> serial_number.lstrip()
'48374983274832    \n\n\t   \t   \n'
```

Lista 5: «Right strip»

```
>>> serial_number.rstrip()
'\n\t   \n 48374983274832'
```

Como habíamos comentado, también existe la posibilidad de especificar los caracteres que queremos borrar:

```
>>> serial_number.strip('\n')
'\t   \n 48374983274832    \n\n\t   \t   '
```

Importante: La función `strip()` no modifica la cadena que estamos usando (*algo obvio porque los «strings» son inmutables*) sino que devuelve una nueva cadena de texto con las modificaciones pertinentes.

Realizar búsquedas

Aunque hemos visto que la forma pitónica de saber si una subcadena se encuentra dentro de otra es *a través del operador in*, Python nos ofrece distintas alternativas para realizar búsquedas en cadenas de texto.

Vamos a partir de una variable que contiene un trozo de la canción Mediterráneo de *Joan Manuel Serrat* para ejemplificar las distintas opciones que tenemos:

```
>>> lyrics = """Quizás porque mi niñez
... Sigue jugando en tu playa
... Y escondido tras las cañas
... Duerme mi primer amor
... Llevo tu luz y tu olor
... Por dondequiera que vaya"""
```

Comprobar si una cadena de texto **empieza o termina por alguna subcadena**:

```
>>> lyrics.startswith('Quizás')
True

>>> lyrics.endswith('Final')
False
```

Encontrar la **primera ocurrencia** de alguna subcadena:

```
>>> lyrics.find('amor')
93

>>> lyrics.index('amor')  # Same behaviour?
93
```

Tanto `find()` como `index()` devuelven el **índice** de la primera ocurrencia de la subcadena que estemos buscando, pero se diferencian en su comportamiento cuando la subcadena buscada no existe:

```
>>> lyrics.find('universo')
-1

>>> lyrics.index('universo')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: substring not found
```

Contabilizar el **número de veces que aparece** una subcadena:

```
>>> lyrics.count('mi')
2

>>> lyrics.count('tu')
3

>>> lyrics.count('él')
0
```

Ejercicio

pycheck: **lost_word**

Reemplazar elementos

Podemos usar la función `replace()` indicando la *subcadena a reemplazar*, la *subcadena de reemplazo* y *cuántas instancias* se deben reemplazar. Si no se especifica este último argumento, la sustitución se hará en todas las instancias encontradas:

```
>>> proverb = 'Quien mal anda mal acaba'

>>> proverb.replace('mal', 'bien')
'Quien bien anda bien acaba'

>>> proverb.replace('mal', 'bien', 1)  # sólo 1 reemplazo
'Quien bien anda mal acaba'
```

Mayúsculas y minúsculas

Python nos permite realizar variaciones en los caracteres de una cadena de texto para pasarlos a mayúsculas y/o minúsculas. Veamos las distintas opciones disponibles:

```
>>> proverb = 'quien a buen árbol se arrima Buena Sombra le cobija'

>>> proverb
'quien a buen árbol se arrima Buena Sombra le cobija'

>>> proverb.capitalize()
'Quien a buen árbol se arrima buena sombra le cobija'

>>> proverb.title()
'Quien A Buen Árbol Se Arrima Buena Sombra Le Cobija'

>>> proverb.upper()
'QUIEN A BUEN ÁRBOL SE ARRIMA BUENA SOMBRA LE COBIJA'

>>> proverb.lower()
'quien a buen árbol se arrima buena sombra le cobija'

>>> proverb.swapcase()
'QUIEN A BUEN ÁRBOL SE ARRIMA bUENA sOMBRA LE COBIJA'
```

Identificando caracteres

Hay veces que recibimos información textual de distintas fuentes de las que necesitamos identificar qué tipo de caracteres contienen. Para ello Python nos ofrece un grupo de funciones.

Veamos **algunas** de estas funciones:

Lista 6: Detectar si todos los caracteres son letras o números

```
>>> 'R2D2'.isalnum()
True
>>> 'C3-PO'.isalnum()
False
```

Lista 7: Detectar si todos los caracteres son números

```
>>> '314'.isnumeric()
True
>>> '3.14'.isnumeric()
False
```

Lista 8: Detectar si todos los caracteres son letras

```
>>> 'abc'.isalpha()
True
>>> 'a-b-c'.isalpha()
False
```

Lista 9: Detectar mayúsculas/minúsculas

```
>>> 'BIG'.isupper()
True
>>> 'small'.islower()
True
>>> 'First Heading'.istitle()
True
```

3.3.7 Interpolación de cadenas

En este apartado veremos cómo **interpolar** valores dentro de cadenas de texto utilizando diferentes formatos. Interpolar (en este contexto) significa sustituir una variable por su valor dentro de una cadena de texto.

Veamos los estilos que proporciona Python para este cometido:

Nombre	Símbolo	Soportado
Estilo antiguo	`%`	>= Python2
Estilo «nuevo»	`.format`	>= Python2.6
«f-strings»	`f''`	>= Python3.6

Aunque aún podemos encontrar código con el estilo antiguo y el estilo nuevo en el formateo de cadenas, vamos a centrarnos en el análisis de los **«f-strings»** que se están utilizando bastante en la actualidad.

«f-strings»

Los **f-strings** aparecieron en Python 3.6 y se suelen usar en código de nueva creación. Es la forma más potente – y en muchas ocasiones más eficiente – de formar cadenas de texto incluyendo valores de otras variables.

La **interpolación** en cadenas de texto es un concepto que existe en la gran mayoría de lenguajes de programación y hace referencia al hecho de sustituir los nombres de variables por sus valores cuando se construye un «string».

Para indicar en Python que una cadena es un «f-string» basta con precederla de una `f` e incluir las variables o expresiones a interpolar entre llaves `{...}`.

Supongamos que disponemos de los datos de una persona y queremos formar una frase de bienvenida con ellos:

```
>>> name = 'Elon Musk'
>>> age = 49
>>> fortune = 43_300

>>> f'Me llamo {name}, tengo {age} años y una fortuna de {fortune} millones'
'Me llamo Elon Musk, tengo 49 años y una fortuna de 43300 millones'
```

Advertencia: Si olvidamos poner la **f** delante del «string» no conseguiremos sustitución de variables.

Podría surgir la duda de cómo incluir llaves dentro de la cadena de texto, teniendo en cuenta que las llaves son símbolos especiales para la interpolación de variables. La respuesta es duplicar las llaves:

```
>>> x = 10

>>> f'The variable is {{ x = {x} }}'
'The variable is { x = 10 }'
```

Formateando cadenas

Nivel intermedio

Los «f-strings» proporcionan una gran variedad de **opciones de formateado**: ancho del texto, número de decimales, tamaño de la cifra, alineación, etc. Muchas de estas facilidades se pueden consultar en el artículo Best of Python3.6 f-strings[4]

Dando formato a valores enteros:

```
>>> mount_height = 3718

>>> f'{mount_height:10d}'
'      3718'

>>> f'{mount_height:010d}'
'0000003718'
```

Truco: Utilizamos el modificador `d` que viene de *entero decimal*.

Dando formato a valores flotantes:

[4] Escrito por Nirant Kasliwal en Medium.

```
>>> PI = 3.14159265

>>> f'{PI:f}'  # 6 decimales por defecto
'3.141593'

>>> f'{PI:.3f}'
'3.142'

>>> f'{PI:12f}'
'    3.141593'

>>> f'{PI:7.2f}'
'   3.14'

>>> f'{PI:07.2f}'
'0003.14'

>>> f'{PI:.010f}'
'3.1415926500'

>>> f'{PI:e}'
'3.141593e+00'
```

Truco: Utilizamos el modificador `f` que viene de *flotante*.

Dando formato a cadenas de texto

```
>>> text1 = 'how'
>>> text2 = 'are'
>>> text3 = 'you'

>>> f'{text1:<7s}|{text2:^11s}|{text3:>7s}'
'how    |    are    |    you'

>>> f'{text1:-<7s}|{text2:·^11s}|{text3:->7s}'
'how----|····are····|----you'
```

Truco: Utilizamos el modificador `s` que viene de *string*.

Convirtiendo valores enteros a otras bases:

```
>>> value = 65_321
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> f'{value:b}'
'1111111100101001'

>>> f'{value:o}'
'177451'

>>> f'{value:x}'
'ff29'
```

Por supuesto en el caso de otras bases también es posible aplicar los mismos **modificadores de ancho y de relleno** vistos para números enteros decimales. Por ejemplo:

```
>>> f'{value:07x}'
'000ff29'
```

Ver también:

Nótese la diferencia de obtener el cambio de base con este método frente a las *funciones de cambio de base* ya vistas previamente que añaden el prefijo de cada base `0b`, `0o` y `0x`.

Modo «debug»

A partir de Python 3.8, los «f-strings» permiten imprimir el nombre de la variable y su valor, como un atajo para depurar nuestro código. Para ello sólo tenemos que incluir un símbolo `=` después del nombre de la variable:

```
>>> serie = 'The Simpsons'
>>> imdb_rating = 8.7
>>> num_seasons = 30

>>> f'{serie=}'
"serie='The Simpsons'"

>>> f'{imdb_rating=}'
'imdb_rating=8.7'

>>> f'{serie[4:]=}'  # incluso podemos añadir expresiones!
"serie[4:]='Simpsons'"

>>> f'{imdb_rating / num_seasons=}'
'imdb_rating / num_seasons=0.29'
```

Modo «representación»

Si imprimimos el valor de una variable utilizando un «f-string», obviamente veremos ese valor tal y como esperaríamos:

```
>>> name = 'Steven Spielberg'

>>> print(f'{name}')
Steven Spielberg
```

Pero si quisiéramos ver la **representación** del objeto, tal y como se almacena internamente, podríamos utilizar el modificador `!r` en el «f-string»:

```
>>> name = 'Steven Spielberg'

>>> print(f'{name!r}')
'Steven Spielberg'
```

En este caso se han añadido las comillas denotando que es una cadena de texto. Este modificador se puede aplicar a cualquier otro tipo de dato.

Ejercicio

Dada la variable:

```
e = 2.71828
```

, obtenga los siguientes resultados utilizando «f-strings»:

```
'2.718'
'2.718280'
'    2.72'  # 4 espacios en blanco
'2.718280e+00'
'00002.7183'
'           2.71828'  # 12 espacios en blanco
```

Aproveche para hacer el ejercicio directamente en el intérprete de Python: >>>

3.3.8 Caracteres Unicode

Los programas de ordenador deben manejar una **amplia variedad de caracteres**. Simplemente por el hecho de la internacionalización hay que mostrar mensajes en distintos idiomas (inglés, francés, japonés, español, etc.). También es posible incluir «emojis» u otros símbolos.

Python utiliza el estándar **Unicode** para representar caracteres. Eso significa que tenemos acceso a una amplia carta de caracteres que nos ofrece este estándar de codificación.

Unicode asigna a cada carácter dos atributos:

1. Un **código numérico** único (habitualmente en hexadecimal).
2. Un **nombre** representativo.

Supongamos un ejemplo sobre el típico «emoji» de un **cohete** definido en este cuadro:



Figura 9: Representación Unicode del carácter ROCKET

La función `chr()` permite representar un carácter **a partir de su código**:

```
>>> rocket_code = 0x1F680
>>> rocket = chr(rocket_code)
>>> rocket
'🚀'
```

La función `ord()` permite obtener el código (decimal) de un carácter **a partir de su representación**:

```
>>> rocket_code = hex(ord(rocket))
>>> rocket_code
'0x1f680'
```

El modificador `\N` permite representar un carácter **a partir de su nombre**:

```
>>> '\N{ROCKET}'
'🚀'
```

Ejercicio

pycheck: **find_unicode**

ASCII

En los principios de la computación los caracteres se representaban utilizando el código ASCII. En un primer momento solo incluía letras mayúsculas y números, pero en 1967 se agregaron las letras minúsculas y algunos caracteres de control, formando así lo que se conoce como US-ASCII, es decir los **caracteres del 0 al 127**.

Podemos obtener alguno de sus caracteres imprimibles mediante Python:

```
>>> chr(48)
'0'

>>> chr(57)
'9'

>>> chr(65)
'A'

>>> chr(90)
'Z'
```

Comparar cadenas

Cuando comparamos dos cadenas de texto lo hacemos en términos **lexicográficos**. Es decir, se van comparando los caracteres de ambas cadenas uno a uno y se va mirando cuál está «antes».

Por ejemplo:

```
>>> 'arca' < 'arpa' # 'ar' es igual para ambas
True

>>> ord('c')
99
>>> ord('p')
112
```

Nota: Internamente se utiliza la función `ord()` para comparar qué carácter está «antes».

Otros ejemplos:

```
>>> 'a' < 'antes'
True

>>> 'antes' < 'después'
True

>>> 'después' < 'ahora'
False

>>> 'ahora' < 'a'
False
```

Tener en cuenta que en Python la letras mayúsculas van antes que las minúsculas:

```
>>> 'A' < 'a'
True

>>> ord('A')
65
>>> ord('a')
97
```

3.3.9 Casos de uso

Nivel avanzado

Hemos estado usando muchas funciones de objetos tipo «string» (y de otros tipos previamente). Pero quizás no sabemos aún como podemos descubrir todo lo que podemos hacer con ellos y los **casos de uso** que nos ofrece.

Python proporciona una *función «built-in»* llamada `dir()` para inspeccionar un determinado tipo de objeto:

```
>>> text = 'This is it!'

>>> dir(text)
['__add__',
 '__class__',
 '__contains__',
 '__delattr__',
```

(continué en la próxima página)

(proviene de la página anterior)

```
'__dir__',
'__doc__',
'__eq__',
'__format__',
'__ge__',
'__getattribute__',
'__getitem__',
'__getnewargs__',
'__gt__',
'__hash__',
'__init__',
'__init_subclass__',
'__iter__',
'__le__',
'__len__',
'__lt__',
'__mod__',
'__mul__',
'__ne__',
'__new__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rmod__',
'__rmul__',
'__setattr__',
'__sizeof__',
'__str__',
'__subclasshook__',
'capitalize',
'casefold',
'center',
'count',
'encode',
'endswith',
'expandtabs',
'find',
'format',
'format_map',
'index',
'isalnum',
'isalpha',
'isascii',
'isdecimal',
'isdigit',
```

(continué en la próxima página)

(proviene de la página anterior)

```
 'isidentifier',
 'islower',
 'isnumeric',
 'isprintable',
 'isspace',
 'istitle',
 'isupper',
 'join',
 'ljust',
 'lower',
 'lstrip',
 'maketrans',
 'partition',
 'replace',
 'rfind',
 'rindex',
 'rjust',
 'rpartition',
 'rsplit',
 'rstrip',
 'split',
 'splitlines',
 'startswith',
 'strip',
 'swapcase',
 'title',
 'translate',
 'upper',
 'zfill']
```

Esto es aplicable tanto a variables como a literales e incluso a tipos de datos (clases) explícitos:

```
>>> dir(10)
['__abs__',
 '__add__',
 '__and__',
 '__bool__',
 ...
 'imag',
 'numerator',
 'real',
 'to_bytes']

>>> dir(float)
```

(continué en la próxima página)

(proviene de la página anterior)

```
['__abs__',
 '__add__',
 '__bool__',
 '__class__',
 ...
 'hex',
 'imag',
 'is_integer',
 'real']
```

EJERCICIOS DE REPASO

1. pycheck: **format_hexcolor**
2. pycheck: **switch_name**
3. pycheck: **samba_split**
4. pycheck: **nif_digit**
5. pycheck: **n_repeat**
6. pycheck: **str_metric**
7. pycheck: **h2md**
8. pycheck: **count_sheeps**
9. pycheck: **strip1**
10. pycheck: **swap_name**
11. pycheck: **find_integral**
12. pycheck: **multiply_jack**
13. pycheck: **first_last_digit**

AMPLIAR CONOCIMIENTOS

- A Guide to the Newer Python String Format Techniques
- Strings and Character Data in Python
- How to Convert a Python String to int
- Your Guide to the Python print<> Function
- Basic Input, Output, and String Formatting in Python

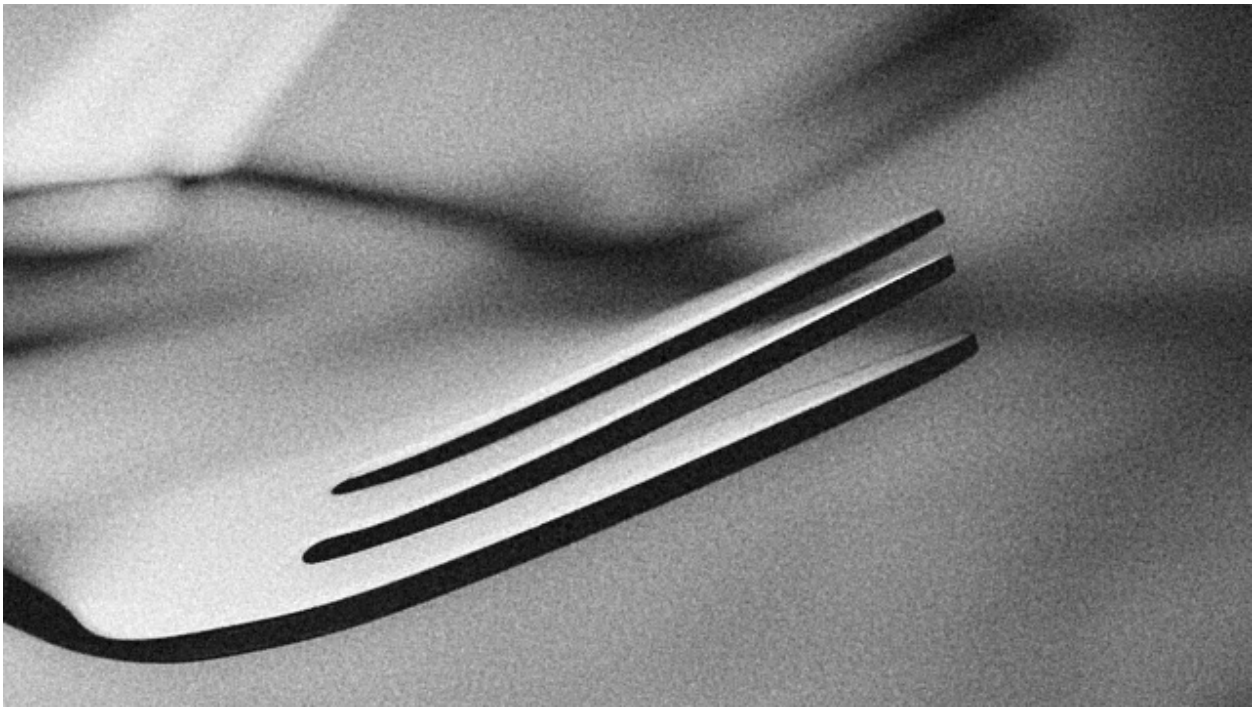
- Unicode & Character Encodings in Python: A Painless Guide
- Python String Formatting Tips & Best Practices
- Python 3's f-Strings: An Improved String Formatting Syntax
- Splitting, Concatenating, and Joining Strings in Python
- Conditional Statements in Python
- Python String Formatting Best Practices

CAPÍTULO 4

Control de flujo

Todo programa informático está formado por *instrucciones* que se ejecutan en forma secuencial de «arriba» a «abajo», de igual manera que leeríamos un libro. Este orden constituye el llamado **flujo** del programa. Es posible modificar este flujo secuencial para que tome *bifurcaciones* o *repita* ciertas instrucciones. Las sentencias que nos permiten hacer estas modificaciones se engloban en el *control de flujo*.

4.1 Condicionales



En esta sección veremos las sentencias `if` y `match-case` junto a las distintas variantes que pueden asumir, pero antes de eso introduciremos algunas cuestiones generales de *escritura de código*.[1]

4.1.1 Definición de bloques

A diferencia de otros lenguajes que utilizan llaves para definir los bloques de código, cuando Guido Van Rossum *creó el lenguaje* quiso evitar estos caracteres por considerarlos innecesarios. Es por ello que en Python los bloques de código se definen a través de **espacios en blanco, preferiblemente 4**.[2] En términos técnicos se habla del **tamaño de indentación**.

Consejo: Esto puede resultar extraño e incómodo a personas que vienen de otros lenguajes de programación pero desaparece rápido y se siente natural a medida que se escribe código.

---

[1] Foto original de portada por ali nafezarefi en Unsplash.
[2] Reglas de indentación definidas en PEP 8

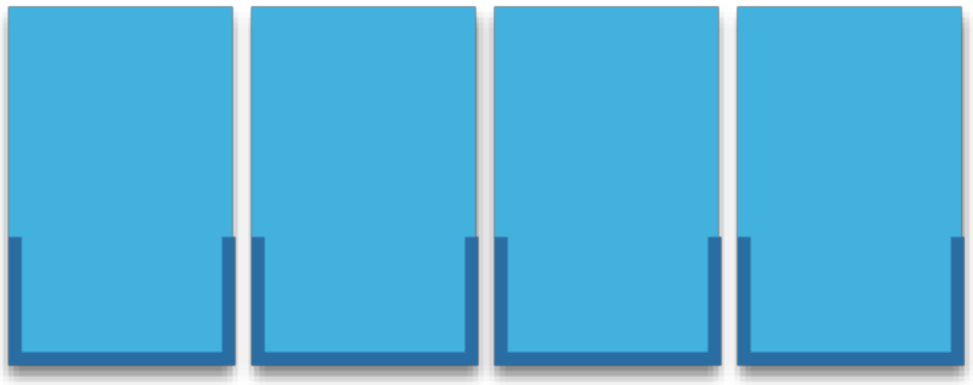


Figura 1: Python recomienda 4 espacios en blanco para indentar

4.1.2 Comentarios

Los comentarios son anotaciones que podemos incluir en nuestro programa y que nos permiten aclarar ciertos aspectos del código. Estas indicaciones son ignoradas por el intérprete de Python.

Los comentarios se incluyen usando el símbolo almohadilla # y comprenden hasta el final de la línea.

Lista 1: Comentario en bloque

```
# Universe age expressed in days
universe_age = 13800 * (10 ** 6) * 365
```

Los comentarios también pueden aparecer en la misma línea de código, aunque la guía de estilo de Python no aconseja usarlos en demasía:

Lista 2: Comentario en línea

```
stock = 0   # Release additional articles
```

Reglas para escribir buenos comentarios:[6]

1. Los comentarios no deberían duplicar el código.
2. Los buenos comentarios no arreglan un código poco claro.
3. Si no puedes escribir un comentario claro, puede haber un problema en el código.
4. Los comentarios deberían evitar la confusión, no crearla.
5. Usa comentarios para explicar código no idiomático.
6. Proporciona enlaces a la fuente original del código copiado.

[6] Referencia: Best practices for writing code comments

7. Incluye enlaces a referencias externas que sean de ayuda.
8. Añade comentarios cuando arregles errores.
9. Usa comentarios para destacar implementaciones incompletas.

4.1.3 Ancho del código

Los programas suelen ser más legibles cuando las líneas no son excesivamente largas. La longitud máxima de línea recomendada por la guía de estilo de Python es de **80 caracteres**.

Sin embargo, esto genera una cierta polémica hoy en día, ya que los tamaños de pantalla han aumentado y las resoluciones son mucho mayores que hace años. Así las líneas de más de 80 caracteres se siguen visualizando correctamente. Hay personas que son más estrictas en este límite y otras más flexibles.

En caso de que queramos **romper una línea de código** demasiado larga, tenemos dos opciones:

1. Usar la *barra invertida* `\`:

```
>>> factorial = 4 * 3 * 2 * 1

>>> factorial = 4 * \
...             3 * \
...             2 * \
...             1
```

2. Usar los *paréntesis* `(...)`:

```
>>> factorial = 4 * 3 * 2 * 1

>>> factorial = (4 *
...              3 *
...              2 *
...              1)
```

4.1.4 La sentencia `if`

La sentencia condicional en Python (al igual que en muchos otros lenguajes de programación) es `if`. En su escritura debemos añadir una **expresión de comparación** terminando con dos puntos al final de la línea. Veamos un ejemplo:

```
>>> temperature = 40
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> if temperature > 35:
...     print('Aviso por alta temperatura')
...
Aviso por alta temperatura
```

Nota: Nótese que en Python no es necesario incluir paréntesis `(` y `)` al escribir condiciones. Hay veces que es recomendable por claridad o por establecer prioridades.

En el caso anterior se puede ver claramente que la condición se cumple y por tanto se ejecuta la instrucción que tenemos dentro del cuerpo de la condición. Pero podría no ser así. Para controlar ese caso existe la sentencia `else`. Veamos el mismo ejemplo anterior pero añadiendo esta variante:

```
>>> temperature = 20

>>> if temperature > 35:
...     print('Aviso por alta temperatura')
... else:
...     print('Parámetros normales')
...
Parámetros normales
```

Podríamos tener incluso condiciones dentro de condiciones, lo que se viene a llamar técnicamente **condiciones anidadas**[3]. Veamos un ejemplo ampliando el caso anterior:

```
>>> temperature = 28

>>> if temperature < 20:
...     if temperature < 10:
...         print('Nivel azul')
...     else:
...         print('Nivel verde')
... else:
...     if temperature < 30:
...         print('Nivel naranja')
...     else:
...         print('Nivel rojo')
...
Nivel naranja
```

Python nos ofrece una mejora en la escritura de condiciones anidadas cuando aparecen consecutivamente un `else` y un `if`. Podemos sustituirlos por la sentencia `elif`:

[3] El anidamiento (o «nesting») hace referencia a incorporar sentencias unas dentro de otras mediante la inclusión de diversos niveles de profundidad (indentación).



Figura 2: Construcción de la sentencia `elif`

Apliquemos esta mejora al código del ejemplo anterior:

```
>>> temperature = 28

>>> if temperature < 20:
...     if temperature < 10:
...         print('Nivel azul')
...     else:
...         print('Nivel verde')
... elif temperature < 30:
...     print('Nivel naranja')
... else:
...     print('Nivel rojo')
...
Nivel naranja
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/wd58B4t

4.1.5 Asignaciones condicionales

Nivel intermedio

Supongamos que queremos asignar un nivel de riesgo de incendio en función de la temperatura. En su **versión clásica** escribiríamos:

```
>>> temperature = 35

>>> if temperature < 30:
...     fire_risk = 'LOW'
... else:
...     fire_risk = 'HIGH'
...
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> fire_risk
'HIGH'
```

Sin embargo, esto lo podríamos abreviar con una **asignación condicional de una única línea**:

```
>>> fire_risk = 'LOW' if temperature < 30 else 'HIGH'

>>> fire_risk
'HIGH'
```

4.1.6 Operadores de comparación

Cuando escribimos condiciones debemos incluir alguna expresión de comparación. Para usar estas expresiones es fundamental conocer los operadores que nos ofrece Python:

Operador	Símbolo
Igualdad	==
Desigualdad	!=
Menor que	<
Menor o igual que	<=
Mayor que	>
Mayor o igual que	>=

A continuación vamos a ver una serie de ejemplos con expresiones de comparación. Téngase en cuenta que estas expresiones habría que incluirlas dentro de la sentencia condicional en el caso de que quisiéramos tomar una acción concreta:

```
# Asignación de valor inicial
>>> value = 8

>>> value == 8
True

>>> value != 8
False

>>> value < 12
True

>>> value <= 7
False
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> value > 4
True

>>> value >= 9
False
```

Python ofrece la posibilidad de ver si un valor está entre dos límites de manera directa. Así, por ejemplo, para descubrir si `x` está entre *4* y *12* haríamos:

```
>>> 4 <= x <= 12
True
```

4.1.7 Operadores lógicos

Podemos escribir condiciones más complejas usando los operadores lógicos:

- `and`
- `or`
- `not`

```
# Asignación de valor inicial
>>> x = 8

>>> x > 4 or x > 12  # True or False
True

>>> x < 4 or x > 12  # False or False
False

>>> x > 4 and x > 12  # True and False
False

>>> x > 4 and x < 12  # True and True
True

>>> not(x != 8)  # not False
True
```

Véanse las **tablas de la verdad** para cada operador lógico:

Nota:

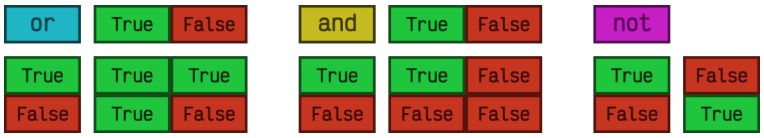


Figura 3: Resultados al aplicar operadores lógicos

1. Una expresión de comparación siempre devuelve un valor *booleano*, es decir `True` o `False`.
2. El uso de paréntesis, en función del caso, puede aclarar la expresión de comparación.

Ejercicio

pycheck: **leap_year**

Cortocircuito lógico

Es interesante comprender que **las expresiones lógicas no se evalúan por completo si se dan una serie de circunstancias**. Aquí es donde entra el concepto de **cortocircuito** que no es más que una forma de denominar a este escenario.

Supongamos un ejemplo en el que utilizamos un **teléfono móvil** que mide la batería por la variable `power` de 0 a 100% y la cobertura 4G por la variable `signal_4g` de 0 a 100%.

Para poder **enviar un mensaje por Telegram** necesitamos tener al menos un 25% de batería y al menos un 10% de cobertura:

```
>>> power = 10
>>> signal_4g = 60

>>> power > 25 and signal_4g > 10
False
```

Dado que estamos en un `and` y la primera condición `power > 25` no se cumple, se produce un **cortocircuito** y no se sigue evaluando el resto de la expresión porque ya se sabe que va a dar `False`.

Otro ejemplo. Para poder **hacer una llamada VoIP** necesitamos tener al menos un 40% de batería o al menos un 30% de cobertura:

```
>>> power = 50
>>> signal_4g = 20
```

(continué en la próxima página)

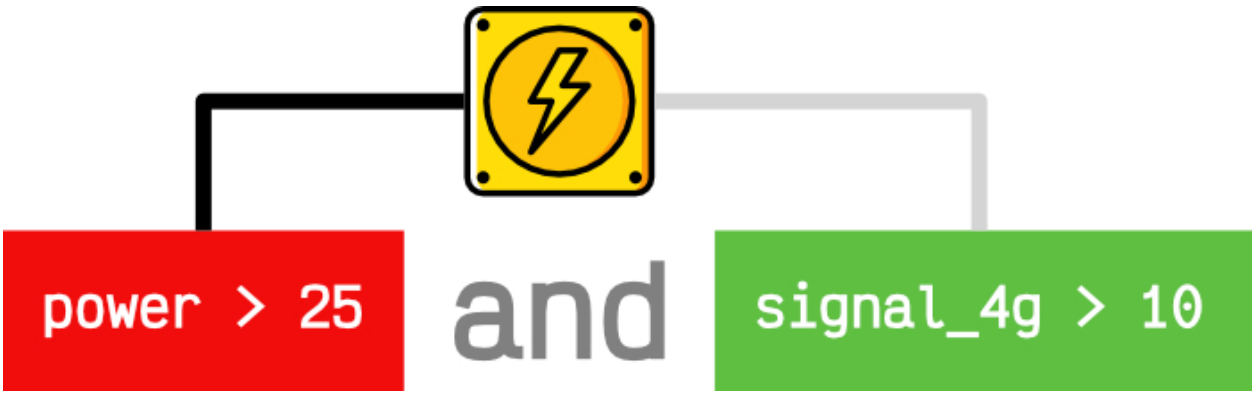


Figura 4: Cortocircuito para expresión lógica «and»

(proviene de la página anterior)

```
>>> power > 40 or signal_4g > 30
True
```

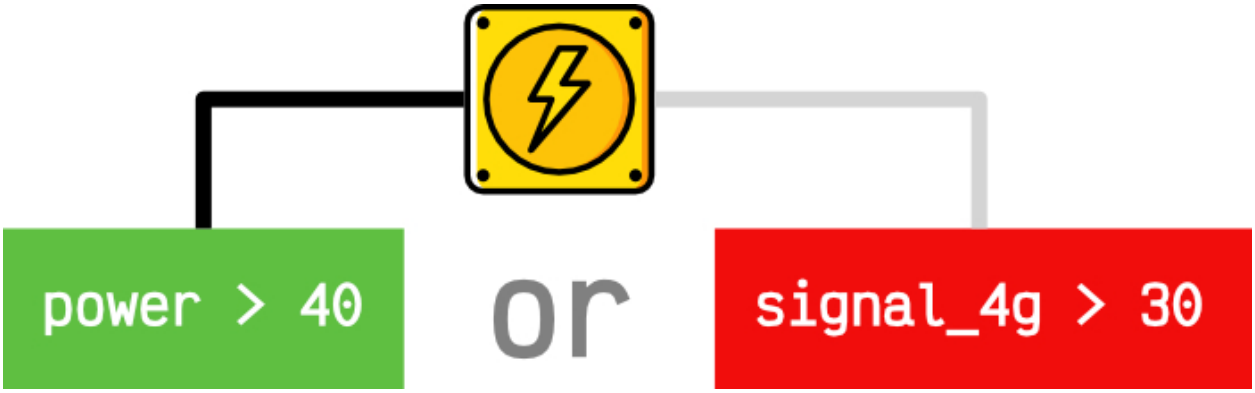


Figura 5: Cortocircuito para expresión lógica «or»

Dado que estamos en un `or` y la primera condición `power > 40` se cumple, se produce un **cortocircuito** y no se sigue evaluando el resto de la expresión porque ya se sabe que va a dar `True`.

Nota: Si no se produjera un cortocircuito en la evaluación de la expresión, se seguiría comprobando todas las condiciones posteriores hasta llegar al final de la misma.

«Booleanos» en condiciones

Cuando queremos preguntar por la **veracidad** de una determinada variable «booleana» en una condición, la primera aproximación que parece razonable es la siguiente:

```
>>> is_cold = True

>>> if is_cold == True:
...     print('Coge chaqueta')
... else:
...     print('Usa camiseta')
...
Coge chaqueta
```

Pero podemos *simplificar* esta condición tal que así:

```
>>> if is_cold:
...     print('Coge chaqueta')
... else:
...     print('Usa camiseta')
...
Coge chaqueta
```

Hemos visto una comparación para un valor «booleano» verdadero (`True`). En el caso de que la comparación fuera para un valor falso lo haríamos así:

```
>>> is_cold = False

>>> if not is_cold:  # Equivalente a if is_cold == False
...     print('Usa camiseta')
... else:
...     print('Coge chaqueta')
...
Usa camiseta
```

De hecho, si lo pensamos, estamos reproduciendo bastante bien el *lenguaje natural*:

- Si hace frío, coge chaqueta.
- Si no hace frío, usa camiseta.

Ejercicio

pycheck: **marvel_akinator**

Valor nulo

Nivel intermedio

`None` es un valor especial de Python que almacena el **valor nulo**[4]. Veamos cómo se comporta al incorporarlo en condiciones de veracidad:

```
>>> value = None

>>> if value:
...     print('Value has some useful value')
... else:
...     # value podría contener None, False (u otro)
...     print('Value seems to be void')
...
Value seems to be void
```

Para distinguir `None` de los valores propiamente booleanos, se recomienda el uso del operador `is`. Veamos un ejemplo en el que tratamos de averiguar si un valor **es nulo**:

```
>>> value = None

>>> if value is None:
...     print('Value is clearly None')
... else:
...     # value podría contener True, False (u otro)
...     print('Value has some useful value')
...
Value is clearly None
```

De igual forma, podemos usar esta construcción para el caso contrario. La forma «pitónica» de preguntar si algo **no es nulo** es la siguiente:

```
>>> value = 99

>>> if value is not None:
...     print(f'{value=}')
...
value=99
```

Nivel avanzado

Cabe preguntarse por qué utilizamos `is` en vez del operador `==` al comprobar si un valor es nulo, ya que ambas aproximaciones nos dan el mismo resultado[7]:

[4] Lo que en otros lenguajes se conoce como `nil`, `null`, `nothing`.
[7] Uso de `is` en comparación de valores nulos explicada aquí por Jared Grubb.

```
>>> value = None

>>> value is None
True

>>> value == None
True
```

La respuesta es que el operador `is` comprueba únicamente si los identificadores (posiciones en memoria) de dos objetos son iguales, mientras que la comparación `==` puede englobar *muchas otras acciones*. De este hecho se deriva que su ejecución sea mucho más rápida y que se eviten «falsos positivos».

Cuando ejecutamos un programa Python existe una serie de objetos precargados en memoria. Uno de ellos es `None`:

```
>>> id(None)
4314501456
```

Cualquier variable que igualemos al valor nulo, únicamente será una referencia al mismo objeto `None` en memoria:

```
>>> value = None

>>> id(value)
4314501456
```

Por lo tanto, ver si un objeto es `None` es simplemente comprobar que su identificador coincida con el de `None`, que es exactamente el cometido de la función `is()`:

```
>>> id(value) == id(None)
True

>>> value is None
True
```

Truco: Python carga inicialmente en memoria objetos como `True` o `False`, pero también los números enteros que van desde el -5 hasta el 256. Se entiende que tiene que ver con optimizaciones a nivel de rendimiento.

4.1.8 Veracidad

Nivel intermedio

Cuando trabajamos con expresiones que incorporan valores booleanos, se produce una *conversión implícita* que transforma los tipos de datos involucrados a valores `True` o `False`.

Lo primero que debemos entender de cara comprobar la **veracidad** son los valores que **evalúan a falso** o **evalúan a verdadero**.

Veamos las únicas «cosas» que son evaluadas a `False` en Python:

```
>>> bool(False)
False

>>> bool(None)
False

>>> bool(0)
False

>>> bool(0.0)
False

>>> bool('')  # cadena vacía
False

>>> bool([])  # lista vacía
False

>>> bool(())  # tupla vacía
False

>>> bool({})  # diccionario vacío
False

>>> bool(set())  # conjunto vacío
False
```

Importante: El resto de objetos son evaluados a `True` en Python.

Veamos algunos ejemplos que son evaluados a `True` en Python:

```
>>> bool('False')
True
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> bool(' ')
True

>>> bool(1e-10)
True

>>> bool([0])
True

>>> bool('🐓')
True
```

Asignación lógica

Es posible utilizar *operadores lógicos* en **sentencias de asignación** sacando partido de las tablas de la verdad que funcionan para estos casos.

Veamos un ejemplo de **asignación lógica** utilizando el operador `or`:

```
>>> b = 0
>>> c = 5

>>> a = b or c

>>> a
5
```

En la línea resaltada podemos ver que se está aplicando una **expresión lógica**, por lo tanto se aplica una conversión implícita de los valores enteros a valores «booleanos». En este sentido el valor `0` se **evalúa a falso** y el valor `5` se evalúa a verdadero. Como estamos en un `or` el resultado será verdadero, que en este caso es el valor 5 asignado finalmente a la variable `a`.

Veamos **el mismo ejemplo de antes** pero utilizando el operador `and`:

```
>>> b = 0
>>> c = 5

>>> a = b and c

>>> a
0
```

En este caso, como estamos en un `and` el resultado será falso, por lo que el valor 0 es asignado finalmente a la variable `a`.

4.1.9 Sentencia `match-case`

Una de las novedades más esperadas (y quizás controvertidas) de Python 3.10 fue el llamado Structural Pattern Matching que introdujo en el lenguaje una nueva sentencia condicional. Ésta se podría asemejar a la sentencia «switch» que ya existe en otros lenguajes de programación.

Comparando valores

En su versión más simple, el «pattern matching» permite comparar un valor de entrada con una serie de literales. Algo así como un conjunto de sentencias «if» encadenadas. Veamos esta aproximación mediante un ejemplo:

```
>>> color = '#FF0000'

>>> match color:
...     case '#FF0000':
...         print('🔴')
...     case '#00FF00':
...         print('🟢')
...     case '#0000FF':
...         print('🔵')
...
🔴
```

¿Qué ocurre si el valor que comparamos no existe entre las opciones disponibles? Pues en principio, nada, ya que este caso no está cubierto. Si lo queremos controlar, hay que añadir una nueva regla utilizando el subguión _ como patrón:

```
>>> color = '#AF549B'

>>> match color:
...     case '#FF0000':
...         print('🔴')
...     case '#00FF00':
...         print('🟢')
...     case '#0000FF':
...         print('🔵')
...     case _:
...         print('Unknown color!')
...
Unknown color!
```

Ejercicio

pycheck: **simple_op**

Patrones avanzados

Nivel avanzado

La sentencia `match-case` va mucho más allá de una simple comparación de valores. Con ella podremos deconstruir estructuras de datos, capturar elementos o mapear valores.

Para ejemplificar varias de sus funcionalidades, vamos a partir de una *tupla* que representará un punto en el plano (2 coordenadas) o en el espacio (3 coordenadas). Lo primero que vamos a hacer es detectar en qué dimensión se encuentra el punto:

```
>>> point = (2, 5)

>>> match point:
...     case (x, y):
...         print(f'({x},{y}) is in plane')
...     case (x, y, z):
...         print(f'({x},{y},{z}) is in space')
...
(2,5) is in plane

>>> point = (3, 1, 7)

>>> match point:
...     case (x, y):
...         print(f'({x},{y}) is in plane')
...     case (x, y, z):
...         print(f'({x},{y},{z}) is in space')
...
(3,1,7) is in space
```

En cualquier caso, esta aproximación permitiría un punto formado por «strings»:

```
>>> point = ('2', '5')

>>> match point:
...     case (x, y):
...         print(f'({x},{y}) is in plane')
...     case (x, y, z):
...         print(f'({x},{y},{z}) is in space')
...
(2,5) is in plane
```

Por lo tanto, en un siguiente paso, podemos restringir nuestros patrones a valores enteros:

```
>>> point = ('2', '5')

>>> match point:
...     case (int(), int()):
...         print(f'{point} is in plane')
...     case (int(), int(), int()):
...         print(f'{point} is in space')
...     case _:
...         print('Unknown!')
...
Unknown!

>>> point = (3, 9, 1)

>>> match point:
...     case (int(), int()):
...         print(f'{point} is in plane')
...     case (int(), int(), int()):
...         print(f'{point} is in space')
...     case _:
...         print('Unknown!')
...
(3, 9, 1) is in space
```

Imaginemos ahora que nos piden calcular la distancia del punto al origen. Debemos tener en cuenta que, a priori, desconocemos si el punto está en el plano o en el espacio:

```
>>> point = (8, 3, 5)

>>> match point:
...     case (int(x), int(y)):
...         dist_to_origin = (x ** 2 + y ** 2) ** (1 / 2)
...     case (int(x), int(y), int(z)):
...         dist_to_origin = (x ** 2 + y ** 2 + z ** 2) ** (1 / 2)
...     case _:
...         print('Unknown!')
...

>>> dist_to_origin
9.899494936611665
```

Con este enfoque, nos aseguramos que los puntos de entrada deben tener todas sus coordenadas como valores enteros:

```
>>> point = ('8', 3, 5)  # Nótese el 8 como "string"
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> match point:
...     case (int(x), int(y)):
...         dist_to_origin = (x ** 2 + y ** 2) ** (1 / 2)
...     case (int(x), int(y), int(z)):
...         dist_to_origin = (x ** 2 + y ** 2 + z ** 2) ** (1 / 2)
...     case _:
...         print('Unknown!')
...
Unknown!
```

Cambiando de ejemplo, veamos un fragmento de código en el que tenemos que **comprobar la estructura de un bloque de autenticación** definido mediante un *diccionario*. Los métodos válidos de autenticación son únicamente dos: bien usando nombre de usuario y contraseña, o bien usando correo electrónico y «token» de acceso. Además, los valores deben venir en formato cadena de texto:

```
>>> # Lista de diccionarios
>>> auths = [
...     {'username': 'sdelquin', 'password': '1234'},
...     {'email': 'sdelquin@gmail.com', 'token': '4321'},
...     {'email': 'test@test.com', 'password': 'ABCD'},
...     {'username': 'sdelquin', 'password': 1234}
... ]

>>> for auth in auths:
...     print(auth)
...     match auth:
...         case {'username': str(username), 'password': str(password)}:
...             print('Authenticating with username and password')
...             print(f'{username}: {password}')
...         case {'email': str(email), 'token': str(token)}:
...             print('Authenticating with email and token')
...             print(f'{email}: {token}')
...         case _:
...             print('Authenticating method not valid!')
...     print('---')
...
{'username': 'sdelquin', 'password': '1234'}
Authenticating with username and password
sdelquin: 1234
---
{'email': 'sdelquin@gmail.com', 'token': '4321'}
Authenticating with email and token
sdelquin@gmail.com: 4321
---
```

(continué en la próxima página)

(proviene de la página anterior)

```
{'email': 'test@test.com', 'password': 'ABCD'}
Authenticating method not valid!
---
{'username': 'sdelquin', 'password': 1234}
Authenticating method not valid!
---
```

Cambiando de ejemplo, a continuación veremos un código que nos indica si, dada la edad de una persona, puede beber alcohol:

```
>>> age = 21

>>> match age:
...     case 0 | None:
...         print('Not a person')
...     case n if n < 17:
...         print('Nope')
...     case n if n < 22:
...         print('Not in the US')
...     case _:
...         print('Yes')
...
Not in the US
```

- En la **línea 4** podemos observar el uso del operador **OR**.
- En las **líneas 6 y 8** podemos observar el uso de condiciones dando lugar a **cláusulas guarda**.

4.1.10 Operador morsa

Nivel avanzado

A partir de Python 3.8 se incorpora el operador morsa[5] que permite unificar **sentencias de asignación dentro de expresiones**. Su nombre proviene de la forma que adquiere `:=`

Supongamos un ejemplo en el que computamos el perímetro de una circunferencia, indicando al usuario que debe incrementarlo siempre y cuando no llegue a un mínimo establecido.

Versión tradicional

```
>>> radius = 4.25
... perimeter = 2 * 3.14 * radius
... if perimeter < 100:
```

(continué en la próxima página)

[5] Se denomina así porque el operador `:=` tiene similitud con los colmillos de una morsa.

(proviene de la página anterior)

```
...     print('Increase radius to reach minimum perimeter')
...     print('Actual perimeter: ', perimeter)
...
Increase radius to reach minimum perimeter
Actual perimeter:  26.69
```

Versión con operador morsa

```
>>> radius = 4.25
... if (perimeter := 2 * 3.14 * radius) < 100:
...     print('Increase radius to reach minimum perimeter')
...     print('Actual perimeter: ', perimeter)
...
Increase radius to reach minimum perimeter
Actual perimeter:  26.69
```

Consejo: Como hemos comprobado, el operador morsa permite realizar asignaciones dentro de expresiones, lo que, en muchas ocasiones, permite obtener un código más compacto. Sería conveniente encontrar un equilibrio entre la expresividad y la legibilidad.

EJERCICIOS DE REPASO

1. pycheck: **rps**
2. pycheck: **min3values**
3. pycheck: **blood_donation**
4. pycheck: **facemoji**
5. pycheck: **shortcuts**

EJERCICIOS EXTERNOS

1. Return the day
2. Return negative
3. What's the real floor?
4. Area or Perimeter
5. Check same case

6. Simple multiplication
7. Quarter of the year
8. Grade book
9. Transportation on vacation
10. Safen User Input Part I - htmlspecialchars
11. Remove an exclamation mark from the end of string
12. Pythagorean triple
13. How much water do I need?
14. Set Alarm
15. Compare within margin
16. Will you make it?
17. Plural
18. Student's final grade
19. Drink about
20. Switch it up!
21. Floating point comparison
22. No zeros for heros
23. Tip calculator
24. Grader
25. Evil or Odious
26. Validate code with simple regex
27. Fuel calculator

AMPLIAR CONOCIMIENTOS

- How to Use the Python or Operator
- Conditional Statements in Python (if/elif/else)

4.2 Bucles



Cuando queremos hacer algo más de una vez, necesitamos recurrir a un **bucle**. En esta sección veremos las distintas sentencias en Python que nos permiten repetir un bloque de código.[1]

4.2.1 La sentencia `while`

El primer mecanismo que existe en Python para repetir instrucciones es usar la sentencia `while`. La semántica tras esta sentencia es: «Mientras se cumpla la condición haz algo».

Veamos un sencillo bucle que repite un saludo mientras así se desee:

```
>>> want_greet = 'S'  # importante dar un valor inicial

>>> while want_greet == 'S':
...     print('Hola qué tal!')
...     want_greet = input('¿Quiere otro saludo? [S/N] ')
... print('Que tenga un buen día')
Hola qué tal!
¿Quiere otro saludo? [S/N] S
Hola qué tal!
¿Quiere otro saludo? [S/N] S
```

(continué en la próxima página)

[1] Foto original de portada por Gary Lopater en Unsplash.

(proviene de la página anterior)

```
Hola qué tal!
¿Quiere otro saludo? [S/N] N
Que tenga un buen día
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/Zwwr8fub

La condición del bucle se comprueba en cada nueva repetición. En este caso chequeamos que la variable `want_greet` sea igual a `'S'`. Dentro del cuerpo del bucle estamos mostrando un mensaje y pidiendo la opción al usuario.

Romper un bucle while

Python ofrece la posibilidad de *romper* o finalizar un bucle *antes de que se cumpla la condición de parada*.

Supongamos que en el ejemplo anterior, establecemos un máximo de 4 saludos:

```
>>> MAX_GREETS = 4

>>> num_greets = 0
>>> want_greet = 'S'

>>> while want_greet == 'S':
...     print('Hola qué tal!')
...     num_greets += 1
...     if num_greets == MAX_GREETS:
...         print('Máximo número de saludos alcanzado')
...         break
...     want_greet = input('¿Quiere otro saludo? [S/N] ')
... print('Que tenga un buen día')
Hola qué tal!
¿Quiere otro saludo? [S/N] S
Hola qué tal!
¿Quiere otro saludo? [S/N] S
Hola qué tal!
¿Quiere otro saludo? [S/N] S
Hola qué tal!
Máximo número de saludos alcanzado
Que tenga un buen día
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/0wwtyaDF

Como hemos visto en este ejemplo, `break` nos permite finalizar el bucle una vez que hemos llegado al máximo número de saludos. Pero si no hubiéramos llegado a dicho límite, el bucle habría seguido hasta que el usuario indicara que no quiere más saludos.

Otra forma de resolver este ejercicio sería incorporar una condición al bucle:

```
while want_greet == 'S' and num_questions < MAX_GREETS:
    ...
```

Comprobar la rotura

Nivel intermedio

Python nos ofrece la posibilidad de **detectar si el bucle ha acabado de forma ordinaria**, esto es, ha finalizado por no cumplirse la condición establecida. Para ello podemos hacer uso de la sentencia `else` como parte del propio bucle. Si el bucle while finaliza normalmente (sin llamada a `break`) el flujo de control pasa a la sentencia opcional `else`.

Veamos su comportamiento siguiendo con el ejemplo que venimos trabajando:

```
>>> MAX_GREETS = 4

>>> num_greets = 0
>>> want_greet = 'S'

>>> while want_greet == 'S':
...     print('Hola qué tal!')
...     num_greets += 1
...     if num_greets == MAX_GREETS:
...         print('Máximo número de saludos alcanzado')
...         break
...     want_greet = input('¿Quiere otro saludo? [S/N] ')
... else:
...     print('Usted no quiere más saludos')
... print('Que tenga un buen día')
Hola qué tal!
¿Quiere otro saludo? [S/N] S
Hola qué tal!
¿Quiere otro saludo? [S/N] N
Usted no quiere más saludos
Que tenga un buen día
```

Importante: Si hubiéramos agotado el número de saludos NO se habría ejecutado la cláusula `else` del bucle ya que habríamos roto el flujo con un `break`.

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/jwwtpivu

Continuar un bucle

Nivel intermedio

Hay situaciones en las que, en vez de romper un bucle, nos interesa **saltar adelante hacia la siguiente repetición**. Para ello Python nos ofrece la sentencia `continue` que hace precisamente eso, descartar el resto del código del bucle y saltar a la siguiente iteración.

Continuamos con el ejemplo anterior y vamos a contar el número de respuestas válidas:

```
>>> want_greet = 'S'
>>> valid_options = 0

>>> while want_greet == 'S':
...     print('Hola qué tal!')
...     want_greet = input('¿Quiere otro saludo? [S/N] ')
...     if want_greet not in 'SN':
...         print('No le he entendido pero le saludo')
...         want_greet = 'S'
...         continue
...     valid_options += 1
... print(f'{valid_options} respuestas válidas')
... print('Que tenga un buen día')
Hola qué tal!
¿Quiere otro saludo? [S/N] S
Hola qué tal!
¿Quiere otro saludo? [S/N] A
No le he entendido pero le saludo
Hola qué tal!
¿Quiere otro saludo? [S/N] B
No le he entendido pero le saludo
Hola qué tal!
¿Quiere otro saludo? [S/N] N
2 respuestas válidas
Que tenga un buen día
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/YwwtlAp8

Bucle infinito

Si no establecemos correctamente la **condición de parada** o bien el valor de alguna variable está fuera de control, es posible que lleguemos a una situación de bucle infinito, del que nunca podamos salir. Veamos un ejemplo de esto:

```
>>> num = 1

>>> while num != 10:
...     num += 2
...
# CTRL-C
KeyboardInterrupt
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
```

El problema que surje es que la variable `num` toma los valores `1, 3, 5, 7, 9, 11, ...` por lo que nunca se cumple la condición de parada del bucle. Esto hace que repitamos «eternamente» la instrucción de incremento.

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/AfrZroa

Una posible solución a este error es reescribir la condición de parada en el bucle:

```
>>> num = 1

>>> while num < 10:
...     num += 2
...
```

Truco: Para abortar una situación de *bucle infinito* podemos pulsar en el teclado la combinación `CTRL-C`. Se puede ver reflejado en el intérprete de Python por `KeyboardInterrupt`.

Hay veces que un **supuesto bucle «infinito»** puede ayudarnos a resolver un problema. Imaginemos que queremos escribir un programa que ayude al profesorado a introducir las notas de un examen. Si la nota no está en el intervalo $[0, 10]$ mostramos un mensaje de error, en otro caso seguimos pidiendo valores:

```
>>> while True:
...     mark = float(input('Introduzca nueva nota: '))
...     if not(0 <= mark <= 10):
...         print('Nota fuera de rango')
...         break
```

(continué en la próxima página)

(proviene de la página anterior)

```
...     print(mark)
...
Introduzca nueva nota: 5
5.0
Introduzca nueva nota: 3
3.0
Introduzca nueva nota: 11
Nota fuera de rango
```

El código anterior se podría enfocar haciendo uso del *operador morsa*:

```
>>> while 0 <= (mark := float(input('Introduzca una nueva nota: '))) <= 10:
...     print(mark)
... print('Nota fuera de rango')
Introduzca una nueva nota: 5
5.0
Introduzca una nueva nota: 3
3.0
Introduzca una nueva nota: 11
Nota fuera de rango
```

Ejercicio

Escriba un programa que encuentre todos los múltiplos de 5 menores que un valor dado:

Ejemplo

- Entrada: `36`
- Salida: `5 10 15 20 25 30 35`

4.2.2 La sentencia `for`

Python permite recorrer aquellos tipos de datos que sean **iterables**, es decir, que admitan *iterar*[2] sobre ellos. Algunos ejemplos de tipos y estructuras de datos que permiten ser iteradas (*recorridas*) son: cadenas de texto, listas, diccionarios, ficheros, etc. La sentencia `for` nos permite realizar esta acción.

A continuación se plantea un ejemplo en el que vamos a recorrer (iterar) una cadena de texto:

[2] Realizar cierta acción varias veces. En este caso la acción es tomar cada elemento.

```
>>> word = 'Python'

>>> for letter in word:
...     print(letter)
...
P
y
t
h
o
n
```

La clave aquí está en darse cuenta que el bucle va tomando, en cada iteración, cada uno de los elementos de la variable que especifiquemos. En este caso concreto `letter` va tomando cada una de las letras que existen en `word`, porque una cadena de texto está formada por elementos que son caracteres.

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/Pft6R2e

Importante: La variable que utilizamos en el bucle `for` para ir tomando los valores puede tener **cualquier nombre**. Al fin y al cabo es una variable que definimos según nuestras necesidades. Tener en cuenta que se suele usar un nombre en singular.

Romper un bucle for

Una sentencia break dentro de un `for` rompe el bucle, *igual que veíamos* para los bucles `while`. Veamos un ejemplo con el código anterior. En este caso vamos a recorrer una cadena de texto y pararemos el bucle cuando encontremos una letra *t* minúscula:

```
>>> word = 'Python'

>>> for letter in word:
...     if letter == 't':
...         break
...     print(letter)
...
P
y
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/zfyqkbJ

Truco: Tanto la *comprobación de rotura de un bucle* como la *continuación a la siguiente iteración* se llevan a cabo del mismo modo que hemos visto con los bucles de tipo `while`.

Ejercicio

pycheck: **count_vowels**

Secuencias de números

Es muy habitual hacer uso de secuencias de números en bucles. Python no tiene una instrucción específica para ello. Lo que sí aporta es una función `range()` que devuelve un *flujo de números* en el rango especificado. Una de las grandes ventajas es que la «lista» generada no se construye explícitamente, sino que cada valor se genera bajo demanda. Esta técnica mejora el consumo de recursos, especialmente en términos de memoria.

La técnica para la generación de secuencias de números es muy similar a la utilizada en los *«slices»* de cadenas de texto. En este caso disponemos de la función `range(start, stop, step)`:

- **start**: Es *opcional* y tiene valor por defecto **0**.
- **stop**: es *obligatorio* (siempre se llega a 1 menos que este valor).
- **step**: es *opcional* y tiene valor por defecto **1**.

`range()` devuelve un *objeto iterable*, así que iremos obteniendo los valores paso a paso con una sentencia `for ... in`[3]. Veamos diferentes ejemplos de uso:

Rango: $[0, 1, 2]$

```
>>> for i in range(0, 3):
...     print(i)
...
0
1
2

>>> for i in range(3):  # No hace falta indicar el inicio si es 0
...     print(i)
...
0
1
2
```

[3] O convertir el objeto a una secuencia como una lista.

Rango: $[1, 3, 5]$

```
>>> for i in range(1, 6, 2):
...     print(i)
...
1
3
5
```

Rango: $[2, 1, 0]$

```
>>> for i in range(2, -1, -1):
...     print(i)
...
2
1
0
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/vfywE45

Truco: Se suelen utilizar nombres de variables `i`, `j`, `k` para lo que se denominan **contadores**. Este tipo de variables toman valores numéricos enteros como en los ejemplos anteriores. No conviene generalizar el uso de estas variables a situaciones en las que, claramente, tenemos la posibilidad de asignar un nombre semánticamente más significativo. Esto viene de tiempos antiguos en FORTRAN donde `i` era la primera letra que tenía valor entero por defecto.

Ejercicio

pycheck: **prime**

Usando el guión bajo

Nivel avanzado

Hay situaciones en las que **no necesitamos usar la variable** que toma valores en el rango, sino que únicamente queremos repetir una acción un número determinado de veces.

Para estos casos se suele recomendar usar el **guión bajo _** como **nombre de variable**, que da a entender que no estamos usando esta variable de forma explícita:

```
>>> for _ in range(10):
...     print('Repeat me 10 times!')
...
Repeat me 10 times!
Repeat me 10 times!
Repeat me 10 times!
Repeat me 10 times!
Repeat me 10 times!
Repeat me 10 times!
Repeat me 10 times!
Repeat me 10 times!
Repeat me 10 times!
Repeat me 10 times!
```

Ejercicio

pycheck: **pow**

4.2.3 Bucles anidados

Como ya vimos en las *sentencias condicionales*, el *anidamiento* es una técnica por la que incluimos distintos niveles de encapsulamiento de sentencias, unas dentro de otras, con mayor nivel de profundidad. En el caso de los bucles también es posible hacer anidamiento.

Veamos un ejemplo de 2 bucles anidados en el que generamos todas las tablas de multiplicar:

```
>>> for num_table in range(1, 10):
...     for mul_factor in range(1, 10):
...         result = num_table * mul_factor
...         print(f'{num_table} * {mul_factor} = {result}')
...
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
2 x 1 = 2
2 x 2 = 4
```

(continué en la próxima página)

[4] Foto de Matrioskas por *Marina Yufereva*'_' en Escáner Cultural.



Figura 6: Muñecas rusas Matrioskas para ejemplificar el anidamiento[4]

(proviene de la página anterior)

```
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
7 x 1 = 7
7 x 2 = 14
```

(continué en la próxima página)

(proviene de la página anterior)

```
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
```

Lo que está ocurriendo en este código es que, para cada valor que toma la variable `i`, la otra variable `j` toma todos sus valores. Como resultado tenemos una combinación completa de los valores en el rango especificado.

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/pwwtctK6

Nota:

- Podemos añadir todos los niveles de anidamiento que queramos. Eso sí, hay que tener en cuenta que cada nuevo nivel de anidamiento supone un importante aumento de la complejidad ciclomática de nuestro código, lo que se traduce en mayores tiempos de ejecución.
- Los bucles anidados también se pueden aplicar en la sentencia `while`.

Ejercicio

Dado su tamaño, muestre por pantalla un mosaico donde la diagonal principal esté representada por `X`, la parte inferior por `D` y la parte superior por `U`.

Ejemplo

- Entrada: `5`
- Salida:

```
X U U U U
D X U U U
D D X U U
D D D X U
D D D D X
```

EJERCICIOS DE REPASO

1. **Escriba un programa que encuentre la mínima secuencia de múltiplos de 3 (distintos) c**

    - Entrada: `45`
    - Salida: `0, 3, 6, 9, 12, 15`

2. Escriba un programa que pida nombre y apellidos de una persona (usando un solo `input`) y repita la pregunta mientras el nombre no esté en formato título (`solución`).

```
¿Su nombre? ana torres blanco
Error. Debe escribirlo correctamente
¿Su nombre? Ana torres blanco
Error. Debe escribirlo correctamente
¿Su nombre? Ana Torres blanco
Error. Debe escribirlo correctamente
¿Su nombre? Ana Torres Blanco
```

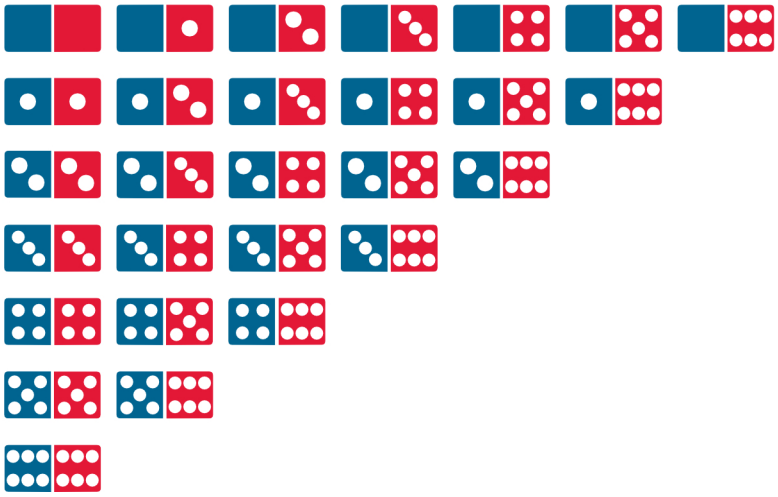
3. Escriba un programa en Python que realice las siguientes 9 multiplicaciones. ¿Nota algo raro en el resultado? (`solución`)

$$1 \cdot 1$$
$$11 \cdot 11$$
$$111 \cdot 111$$
$$\vdots$$
$$111111111 \cdot 111111111$$

4. **Escriba un programa en Python que acepte dos valores enteros (x e y) que representar**

- Entrada: `objetivo_x=7; objetivo_y=8;`
- Salida: `(0, 0) (1, 2) (3, 3) (4, 5) (6, 6) (7, 8)`

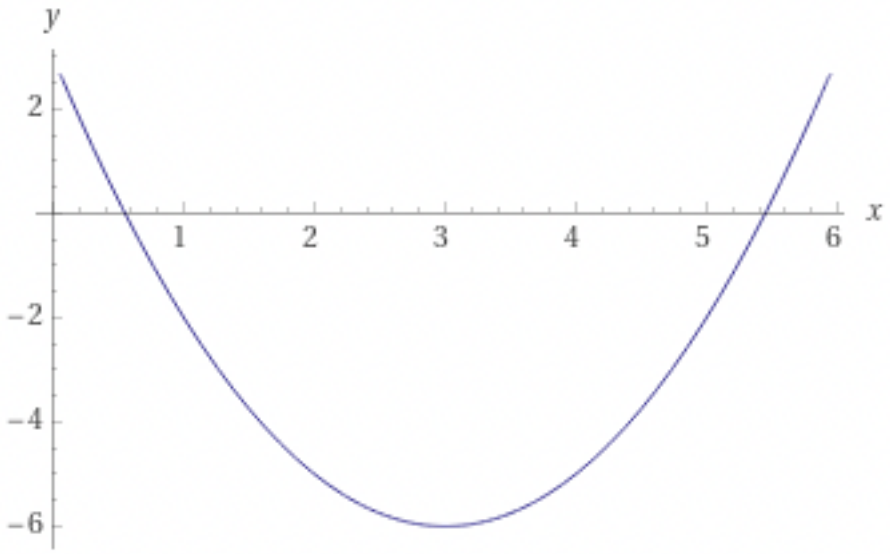
5. Escriba un programa que muestre por pantalla todas las fichas del dominó. La ficha «en blanco» se puede representar con un 0 (`solución`).



```
0|0 0|1 0|2 0|3 0|4 0|5 0|6
1|1 1|2 1|3 1|4 1|5 1|6
2|2 2|3 2|4 2|5 2|6
3|3 3|4 3|5 3|6
4|4 4|5 4|6
5|5 5|6
6|6
```

6. Escriba un programa que calcule el valor de x para el que la función $f(x) = x^2 - 6x + 3$ obtiene su menor resultado. Centre la búsqueda en el rango $[-9, 9]$ sólo con valores enteros (`solución`).

El resultado es: $x = 3$ y $f(x) = -6$



7. Escriba un programa que muestre (por filas) la Tabla ASCII, empezando con el código 33 y terminando con el 127 (`solución`):

```
033 !   034 ”   035 #   036 $   037 %
038 &   039 ’   040 (   041 )   042 *
043 +   044 ,   045 -   046 .   047 /
048 0   049 1   050 2   051 3   052 4
053 5   054 6   055 7   056 8   057 9
058 :   059 ;   060 <   061 =   062 >
063 ?   064 @   065 A   066 B   067 C
068 D   069 E   070 F   071 G   072 H
073 I   074 J   075 K   076 L   077 M
078 N   079 O   080 P   081 Q   082 R
083 S   084 T   085 U   086 V   087 W
088 X   089 Y   090 Z   091 [   092 \
093 ]   094 ^   095 _   096 `   097 a
098 b   099 c   100 d   101 e   102 f
103 g   104 h   105 i   106 j   107 k
108 l   109 m   110 n   111 o   112 p
113 q   114 r   115 s   116 t   117 u
118 v   119 w   120 x   121 y   122 z
123 {   124 |   125 }   126 ~   127
```

8. Escriba un programa que permita al usuario adivinar un número. Indicar si el número buscado es menor o mayor que el que se está preguntando y mostrar igualmente el número de intentos hasta encontrar el número objetivo (`solución`):

```
Introduzca número: 50
Mayor
Introduzca número: 100
```

(continué en la próxima página)

(proviene de la página anterior)

```
Menor
Introduzca número: 90
Menor
Introduzca número: 87
✅ ¡Enhorabuena! Has encontrado el número en 4 intentos
```

9. pycheck: **gcd**
10. pycheck: **hamming**
11. pycheck: **sprod_cart**
12. pycheck: **cumsq_prod**
13. pycheck: **isalphabetic**
14. pycheck: **tennis_game**
15. pycheck: **tennis_set**
16. pycheck: **kpower**
17. pycheck: **fibonacci**

EJERCICIOS EXTERNOS

1. Summation
2. Find nearest square number
3. Bin to decimal
4. altERnaTIng cAsE
5. Fake binary
6. Correct the mistakes of the character recognition software
7. String cleaning
8. Sum of multiples
9. ASCII Total
10. Collatz Conjecture (3n+1)

AMPLIAR CONOCIMIENTOS

- The Python range() Function
- How to Write Pythonic Loops
- For Loops in Python (Definite Iteration)
- Python «while» Loops (Indefinite Iteration)

CAPÍTULO 5

Estructuras de datos

Si bien ya hemos visto una sección sobre *Tipos de datos*, podríamos hablar de tipos de datos más complejos en Python que se constituyen en **estructuras de datos**. Si pensamos en estos elementos como *átomos*, las estructuras de datos que vamos a ver sería *moléculas*. Es decir, combinamos los tipos básicos de formas más complejas. De hecho, esta distinción se hace en el Tutorial oficial de Python. Trataremos distintas estructuras de datos como listas, tuplas, diccionarios y conjuntos.

5.1 Listas



Las listas permiten **almacenar objetos** mediante un **orden definido** y con posibilidad de duplicados. Las listas son estructuras de datos **mutables**, lo que significa que podemos añadir, eliminar o modificar sus elementos.[1]

5.1.1 Creando listas

Una lista está compuesta por *cero* o *más elementos*. En Python debemos escribir estos elementos separados por *comas* y dentro de *corchetes*. Veamos algunos ejemplos de listas:

```
>>> empty_list = []

>>> languages = ['Python', 'Ruby', 'Javascript']

>>> fibonacci = [0, 1, 1, 2, 3, 5, 8, 13]

>>> data = ['Tenerife', {'cielo': 'limpio', 'temp': 24}, 3718, (28.2933947, -16.
↪5226597)]
```

Nota: Una lista puede contener tipos de **datos heterogéneos**, lo que la hace una estructura

[1] Foto original de portada por Mike Arney en Unsplash.

de datos muy versátil.

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/Ofiiare

Advertencia: Aunque está permitido, **NUNCA** llames `list` a una variable porque destruirías la función que nos permite crear listas. Y tampoco uses nombres derivados como `_list` o `list_` ya que no son nombres representativos que *identifiquen el propósito de la variable*.

Ejercicio

Entre en el intérprete interactivo de Python (`>>>`) y cree una lista con las 5 ciudades que más le gusten.

5.1.2 Conversión

Para convertir otros tipos de datos en una lista podemos usar la función `list()`:

```
>>> # conversión desde una cadena de texto
>>> list('Python')
['P', 'y', 't', 'h', 'o', 'n']
```

Si nos fijamos en lo que ha pasado, al convertir la cadena de texto `Python` se ha creado una lista con *6* elementos, donde cada uno de ellos representa un carácter de la cadena. Podemos *extender* este comportamiento a cualquier otro tipo de datos que permita ser iterado (*iterables*).

Otro ejemplo interesante de conversión puede ser la de los *rangos*. En este caso queremos obtener una **lista explícita** con los valores que constituyen el rango $[0, 9]$:

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Lista vacía

Existe una manera particular de usar `list()` y es no pasarle ningún argumento. En este caso estaremos queriendo convertir el «vacío» en una lista, con lo que obtendremos una *lista vacía*:

```
>>> list()
[]
```

Truco: Para crear una lista vacía, se suele recomendar el uso de `[]` frente a `list()`, no sólo por ser más *pitónico* sino por tener (en promedio) un mejor rendimiento en tiempos de ejecución.

5.1.3 Operaciones con listas

Obtener un elemento

Igual que en el caso de las *cadenas de texto*, podemos obtener un elemento de una lista a través del **índice** (lugar) que ocupa. Veamos un ejemplo:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']

>>> shopping[0]
'Agua'

>>> shopping[1]
'Huevos'

>>> shopping[2]
'Aceite'

>>> shopping[-1]  # acceso con índice negativo
'Aceite'
```

El *índice* que usemos para acceder a los elementos de una lista tiene que estar comprendido entre los límites de la misma. Si usamos un índice antes del comienzo o después del final obtendremos un error (*excepción*):

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']

>>> shopping[3]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
```

(continué en la próxima página)

(proviene de la página anterior)

```
IndexError: list index out of range

>>> shopping[-5]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

Trocear una lista

El troceado de listas funciona de manera totalmente análoga al *troceado de cadenas*. Veamos algunos ejemplos:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping[0:3]
['Agua', 'Huevos', 'Aceite']

>>> shopping[:3]
['Agua', 'Huevos', 'Aceite']

>>> shopping[2:4]
['Aceite', 'Sal']

>>> shopping[-1:-4:-1]
['Limón', 'Sal', 'Aceite']

>>> # Equivale a invertir la lista
>>> shopping[::-1]
['Limón', 'Sal', 'Aceite', 'Huevos', 'Agua']
```

En el troceado de listas, a diferencia de lo que ocurre al obtener elementos, no debemos preocuparnos por acceder a *índices inválidos* (fuera de rango) ya que Python los restringirá a los límites de la lista:

```
>>> shopping
['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping[10:]
[]

>>> shopping[-100:2]
['Agua', 'Huevos']
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> shopping[2:100]
['Aceite', 'Sal', 'Limón']
```

Importante: Ninguna de las operaciones anteriores modifican la lista original, simplemente devuelven una lista nueva.

Invertir una lista

Python nos ofrece, al menos, tres mecanismos para invertir los elementos de una lista:

Conservando la lista original: *Opción 1*: Mediante *troceado* de listas con *step* negativo:

```
>>> shopping
['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping[::-1]
['Limón', 'Sal', 'Aceite', 'Huevos', 'Agua']
```

Opción 2: Mediante la función `reversed()`:

```
>>> shopping
['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> list(reversed(shopping))
['Limón', 'Sal', 'Aceite', 'Huevos', 'Agua']
```

Modificando la lista original: Utilizando la función `reverse()` (nótese que es sin *«d»* al final):

```
>>> shopping
['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping.reverse()

>>> shopping
['Limón', 'Sal', 'Aceite', 'Huevos', 'Agua']
```

Añadir al final de la lista

Una de las operaciones más utilizadas en listas es añadir elementos al final de las mismas. Para ello Python nos ofrece la función `append()`. Se trata de un método «destructivo» que modifica la lista original:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']

>>> shopping.append('Atún')

>>> shopping
['Agua', 'Huevos', 'Aceite', 'Atún']
```

Creando desde vacío

Una forma muy habitual de trabajar con listas es empezar con una vacía e ir añadiendo elementos poco a poco. Se podría hablar de un **patrón creación**.

Supongamos un ejemplo en el que queremos construir una lista con los números pares del $[0, 20)$:

```
>>> even_numbers = []

>>> for i in range(20):
...     if i % 2 == 0:
...         even_numbers.append(i)
...

>>> even_numbers
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/2fiS9Ax

Añadir en cualquier posición de una lista

Ya hemos visto cómo añadir elementos al final de una lista. Sin embargo, Python ofrece una función `insert()` que vendría a ser una generalización de la anterior, para incorporar elementos en cualquier posición. Simplemente debemos especificar el índice de inserción y el elemento en cuestión. También se trata de una función *destructiva*[2]:

[2] Cuando hablamos de que una función/método es «destructiva/o» significa que modifica la lista (objeto) original, no que la destruye.

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']

>>> shopping.insert(1, 'Jamón')

>>> shopping
['Agua', 'Jamón', 'Huevos', 'Aceite']

>>> shopping.insert(3, 'Queso')

>>> shopping
['Agua', 'Jamón', 'Huevos', 'Queso', 'Aceite']
```

Nota: El índice que especificamos en la función `insert()` lo podemos intepretar como la posición *delante* (a la izquierda) de la cual vamos a colocar el nuevo valor en la lista.

Al igual que ocurría con el *troceado de listas*, en este tipo de inserciones no obtendremos un error si especificamos índices fuera de los límites de la lista. Estos se ajustarán al principio o al final en función del valor que indiquemos:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']

>>> shopping.insert(100, 'Mermelada')

>>> shopping
['Agua', 'Huevos', 'Aceite', 'Mermelada']

>>> shopping.insert(-100, 'Arroz')

>>> shopping
['Arroz', 'Agua', 'Huevos', 'Aceite', 'Mermelada']
```

Consejo: Aunque es posible utilizar `insert()` para añadir **elementos al final de una lista**, siempre se recomienda usar `append()` por su mayor legibilidad:

```
>>> values = [1, 2, 3]
>>> values.append(4)
>>> values
[1, 2, 3, 4]

>>> values = [1, 2, 3]
>>> values.insert(len(values), 4)  # don't do it!
>>> values
[1, 2, 3, 4]
```

Repetir elementos

Al igual que con las *cadenas de texto*, el operador `*` nos permite repetir los elementos de una lista:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']

>>> shopping * 3
['Agua',
 'Huevos',
 'Aceite',
 'Agua',
 'Huevos',
 'Aceite',
 'Agua',
 'Huevos',
 'Aceite']
```

Combinar listas

Python nos ofrece dos aproximaciones para combinar listas:

Conservando la lista original: Mediante el operador `+` o `+=`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']
>>> fruitshop = ['Naranja', 'Manzana', 'Piña']

>>> shopping + fruitshop
['Agua', 'Huevos', 'Aceite', 'Naranja', 'Manzana', 'Piña']
```

Modificando la lista original: Mediante la función `extend()`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']
>>> fruitshop = ['Naranja', 'Manzana', 'Piña']

>>> shopping.extend(fruitshop)

>>> shopping
['Agua', 'Huevos', 'Aceite', 'Naranja', 'Manzana', 'Piña']
```

Hay que tener en cuenta que `extend()` funciona adecuadamente si pasamos una **lista como argumento**. En otro caso, quizás los resultados no sean los esperados. Veamos un ejemplo:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']

>>> shopping.extend('Limón')
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> shopping
['Agua', 'Huevos', 'Aceite', 'L', 'i', 'm', 'ó', 'n']
```

El motivo es que `extend()` «recorre» (o itera) sobre cada uno de los elementos del objeto en cuestión. En el caso anterior, al ser una cadena de texto, está formada por caracteres. De ahí el resultado que obtenemos.

Se podría pensar en el uso de `append()` para combinar listas. La realidad es que no funciona exactamente como esperamos; la segunda lista se añadiría como una *sublista* de la principal:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']
>>> fruitshop = ['Naranja', 'Manzana', 'Piña']

>>> shopping.append(fruitshop)

>>> shopping
['Agua', 'Huevos', 'Aceite', ['Naranja', 'Manzana', 'Piña']]
```

Modificar una lista

Del mismo modo que se *accede a un elemento* utilizando su índice, también podemos modificarlo:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite']

>>> shopping[0]
'Agua'

>>> shopping[0] = 'Jugo'

>>> shopping
['Jugo', 'Huevos', 'Aceite']
```

En el caso de acceder a un *índice no válido* de la lista, incluso para modificar, obtendremos un error:

```
>>> shopping[100] = 'Chocolate'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list assignment index out of range
```

Modificar con troceado

No sólo es posible modificar un elemento de cada vez, sino que podemos asignar valores a trozos de una lista:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping[1:4]
['Huevos', 'Aceite', 'Sal']

>>> shopping[1:4] = ['Atún', 'Pasta']

>>> shopping
['Agua', 'Atún', 'Pasta', 'Limón']
```

Nota: La lista que asignamos no necesariamente debe tener la misma longitud que el trozo que sustituimos.

Borrar elementos

Python nos ofrece, al menos, cuatro formas para borrar elementos en una lista:

Por su índice: Mediante la sentencia `del`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> del shopping[3]

>>> shopping
['Agua', 'Huevos', 'Aceite', 'Limón']
```

Por su valor: Mediante la función `remove()`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping.remove('Sal')

>>> shopping
['Agua', 'Huevos', 'Aceite', 'Limón']
```

> **Advertencia:** Si existen valores duplicados, la función `remove()` sólo borrará la primera ocurrencia.

Por su índice (con extracción): La sentencia `del` y la función `remove()` efectivamente borran el elemento indicado de la lista, pero no «devuelven»[3] nada. Sin embargo, Python nos ofrece la función `pop()` que además de borrar, nos «recupera» el elemento; algo así como una *extracción*. Lo podemos ver como una combinación de *acceso* + *borrado*:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> product = shopping.pop()  # shopping.pop(-1)
>>> product
'Limón'

>>> shopping
['Agua', 'Huevos', 'Aceite', 'Sal']

>>> product = shopping.pop(2)
>>> product
'Aceite'

>>> shopping
['Agua', 'Huevos', 'Sal']
```

Nota: Si usamos la función `pop()` sin pasarle ningún argumento, por defecto usará el índice *-1*, es decir, el último elemento de la lista. Pero también podemos indicarle el índice del elemento a extraer.

Por su rango: Mediante troceado de listas:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping[1:4] = []

>>> shopping
['Agua', 'Limón']
```

[3] Más adelante veremos el comportamiento de las funciones. Devolver o retornar un valor es el resultado de aplicar una función.

Borrado completo de la lista

Python nos ofrece, al menos, dos formas para borrar una lista por completo:

1. Utilizando la función `clear()`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping.clear()  # Borrado in-situ

>>> shopping
[]
```

2. «Reinicializando» la lista a vacío con `[]`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping = []  # Nueva zona de memoria

>>> shopping
[]
```

Nivel avanzado

La diferencia entre ambos métodos tiene que ver con cuestiones internas de gestión de memoria y de rendimiento:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']
>>> id(shopping)
4416018560
>>> shopping.clear()
>>> id(shopping)  # se mantiene la misma "posición de memoria"
4416018560

>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']
>>> id(shopping)
4458688576
>>> shopping = []
>>> id(shopping)  # se crea una nueva "posición de memoria"
4458851520
```

Ver también:

La memoria que queda «en el limbo» después de asignar un nuevo valor a la lista es detectada por el **recolector de basura** de Python, quien se encarga de liberar aquellos datos que no están referenciados por ninguna variable.

A efectos de **velocidad de ejecución**, `shopping.clear()` «parece» ir más rápido que `shopping = []`.

Encontrar un elemento

Si queremos descubrir el **índice** que corresponde a un determinado valor dentro la lista podemos usar la función `index()` para ello:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping.index('Huevos')
1
```

Tener en cuenta que si el elemento que buscamos no está en la lista, obtendremos un error:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> shopping.index('Pollo')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: 'Pollo' is not in list
```

Nota: Si buscamos un valor que existe más de una vez en una lista, la función `index()` sólo nos devolverá el índice de la primera ocurrencia.

Advertencia: En listas no disponemos de la función `find()` que sí estaba disponible para *cadenas de texto*.

Pertenencia de un elemento

Si queremos comprobar la existencia de un determinado elemento en una lista, podríamos buscar su índice, pero la forma **pitónica** de hacerlo es utilizar el operador `in`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> 'Aceite' in shopping
True

>>> 'Pollo' in shopping
False
```

Nota: El operador `in` siempre devuelve un valor booleano, es decir, verdadero o falso.

Ejercicio

pycheck: **isogram**

Número de ocurrencias

Para contar cuántas veces aparece un determinado valor dentro de una lista podemos usar la función `count()`:

```
>>> sheldon_greeting = ['Penny', 'Penny', 'Penny']

>>> sheldon_greeting.count('Howard')
0

>>> sheldon_greeting.count('Penny')
3
```

Dividir una cadena de texto en lista

Una tarea muy común al trabajar con cadenas de texto es dividirlas por algún tipo de *separador*. En este sentido, Python nos ofrece la función `split()`, que debemos usar anteponiendo el «string» que queramos dividir:

```
>>> proverb = 'No hay mal que por bien no venga'
>>> proverb.split()
['No', 'hay', 'mal', 'que', 'por', 'bien', 'no', 'venga']

>>> tools = 'Martillo,Sierra,Destornillador'
>>> tools.split(',')
['Martillo', 'Sierra', 'Destornillador']
```

Nota: Si no se especifica un separador, `split()` usa por defecto cualquier secuencia de espacios en blanco, tabuladores y saltos de línea.

La función `split()` devuelve una lista donde cada elemento es una parte de la cadena de texto original:

```
>>> game = 'piedra-papel-tijera'

>>> type(game_tools := game.split('-'))
list
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> game_tools
['piedra', 'papel', 'tijera']
```

Ejercicio

pycheck: **num_words**

Particionado de cadenas de texto

Existe una forma algo más «elaborada» de dividir una cadena a través del **particionado**. Para ello podemos valernos de la función `partition()` que proporciona Python.

Esta función toma un argumento como separador, y divide la cadena de texto en 3 partes: lo que queda a la izquierda del separador, el separador en sí mismo y lo que queda a la derecha del separador:

```
>>> text = '3 + 4'

>>> text.partition('+')
('3 ', '+', ' 4')
```

Ver también:

En este caso el resultado de la función `partition()` es una *tupla*.

Unir una lista en cadena de texto

Dada una lista, podemos convetirla a una cadena de texto, uniendo todos sus elementos mediante algún **separador**. Para ello hacemos uso de la función `join()` con la siguiente estructura:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> ','.join(shopping)
'Agua,Huevos,Aceite,Sal,Limón'

>>> ' '.join(shopping)
'Agua Huevos Aceite Sal Limón'

>>> '|'.join(shopping)
'Agua|Huevos|Aceite|Sal|Limón'
```

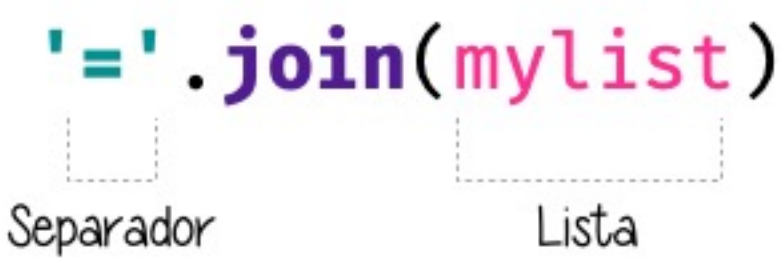


Figura 1: Estructura de llamada a la función `join()`

Hay que tener en cuenta que `join()` sólo funciona si *todos sus elementos son cadenas de texto*:

```
>>> ', '.join([1, 2, 3, 4, 5])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: sequence item 0: expected str instance, int found
```

Truco: Esta función `join()` es realmente la **opuesta** a la función `split()`.

Ejercicio

pycheck: **fixdate**

Ordenar una lista

Python proporciona, al menos, dos formas de ordenar los elementos de una lista:

Conservando lista original: Mediante la función `sorted()` que devuelve una nueva lista ordenada:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> sorted(shopping)
['Aceite', 'Agua', 'Huevos', 'Limón', 'Sal']
```

Modificando la lista original: Mediante la función `sort()`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> shopping.sort()

>>> shopping
['Aceite', 'Agua', 'Huevos', 'Limón', 'Sal']
```

Ambos métodos admiten un *parámetro* «booleano» `reverse` para indicar si queremos que la ordenación se haga en **sentido inverso**:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> sorted(shopping, reverse=True)
['Sal', 'Limón', 'Huevos', 'Agua', 'Aceite']
```

Longitud de una lista

Podemos conocer el número de elementos que tiene una lista con la función `len()`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> len(shopping)
5
```

Iterar sobre una lista

Al igual que *hemos visto con las cadenas de texto*, también podemos *iterar* sobre los elementos de una lista utilizando la sentencia `for`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> for product in shopping:
...     print(product)
...
Agua
Huevos
Aceite
Sal
Limón
```

Nota: También es posible usar la sentencia `break` en este tipo de bucles para abortar su ejecución en algún momento que nos interese.

Ejercicio

pycheck: **chars_list**

Iterar usando enumeración

Hay veces que no sólo nos interesa «visitar» cada uno de los elementos de una lista, sino que también queremos **saber su índice** dentro de la misma. Para ello Python nos ofrece la función `enumerate()`:

```
>>> shopping = ['Agua', 'Huevos', 'Aceite', 'Sal', 'Limón']

>>> for i, product in enumerate(shopping):
...     print(i, product)
...
0 Agua
1 Huevos
2 Aceite
3 Sal
4 Limón
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/TfiuIZ0

Truco: Es posible utilizar el parámetro `start` con `enumerate()` para indicar el índice en el que queremos comenzar. Por defecto es 0.

Iterar sobre múltiples listas

Python ofrece la posibilidad de iterar sobre **múltiples listas en paralelo** utilizando la función `zip()`. Se basa en ir «juntando» ambas listas elemento a elemento:

Veamos un ejemplo en el que añadimos ciertos detalles a nuestra lista de la compra:

```
>>> shopping = ['Agua', 'Aceite', 'Arroz']
>>> details = ['mineral natural', 'de oliva virgen', 'basmati']

>>> for product, detail in zip(shopping, details):
...     print(product, detail)
...
```

(continué en la próxima página)

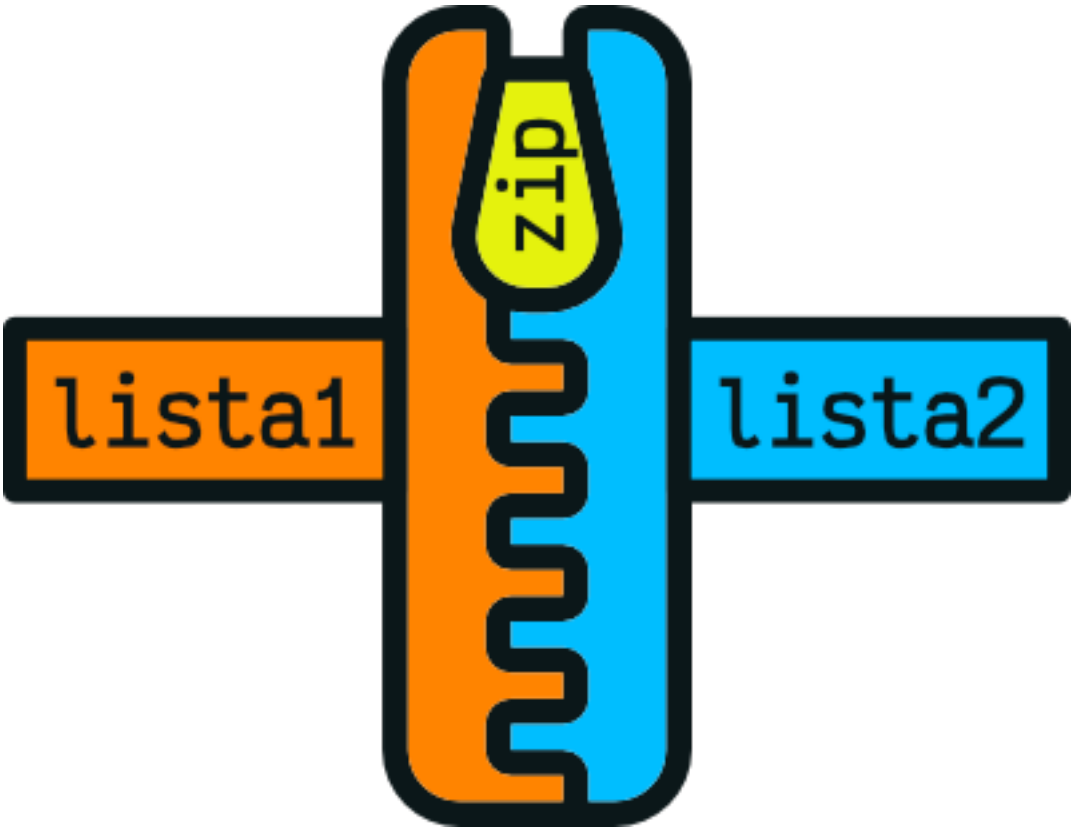


Figura 2: Funcionamiento de `zip()`

(proviene de la página anterior)

```
Agua mineral natural
Aceite de oliva virgen
Arroz basmati
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/lfioilG

Nota: En el caso de que las listas no tengan la misma longitud, la función `zip()` realiza la combinación hasta que se agota la lista más corta.

Dado que `zip()` produce un *iterador*, si queremos obtener una **lista explícita** con la combinación en paralelo de las listas, debemos construir dicha lista de la siguiente manera:

```
>>> shopping = ['Agua', 'Aceite', 'Arroz']
>>> details = ['mineral natural', 'de oliva virgen', 'basmati']

>>> list(zip(shopping, details))
[('Agua', 'mineral natural'),
 ('Aceite', 'de oliva virgen'),
 ('Arroz', 'basmati')]
```

Ejercicio

pycheck: **dot_product**

Comparar listas

Supongamos este ejemplo:

```
>>> [1, 2, 3] < [1, 2, 4]
True
```

Python llega a la conclusión de que la lista `[1, 2, 3]` es menor que `[1, 2, 4]` porque va comparando elemento a elemento:

- El `1` es igual en ambas listas.
- El `2` es igual en ambas litas.
- El `3` es menor que el `4`, por lo que la primera lista es menor que la segunda.

Entender la forma en la que se comparan dos listas es importante para poder aplicar otras funciones y obtener los resultados deseados.

Ver también:

Esta comparación funciona de forma totalmente análoga a la *comparación de cadenas de texto*.

5.1.4 Cuidado con las copias

Nivel intermedio

Las listas son estructuras de datos *mutables* y esta característica nos obliga a tener cuidado cuando realizamos copias de listas, ya que la modificación de una de ellas puede afectar a la otra.

Veamos un ejemplo sencillo:

```
>>> original_list = [4, 3, 7, 1]

>>> copy_list = original_list

>>> original_list[0] = 15

>>> original_list
[15, 3, 7, 1]

>>> copy_list
[15, 3, 7, 1]
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/pfi5PC5

Nota: A través de *Python Tutor* se puede ver claramente el motivo de por qué ocurre esto. Dado que las variables «apuntan» a la misma zona de memoria, al modificar una de ellas, el cambio también se ve reflejado en la otra.

Una **posible solución** a este problema es hacer una «copia dura». Para ello Python proporciona la función `copy()`:

```
>>> original_list = [4, 3, 7, 1]

>>> copy_list = original_list.copy()

>>> original_list[0] = 15

>>> original_list
```

(continué en la próxima página)

(proviene de la página anterior)

```
[15, 3, 7, 1]

>>> copy_list
[4, 3, 7, 1]
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/Dfi6oLk

Existe **otra aproximación** a este problema, y es utilizar un *troceado* completo de la lista, lo que nos devuelve una «copia desvinculada» de manera implícita:

```
>>> original_list = [4, 3, 7, 1]

>>> copy_list = original_list[:]

>>> id(original_list) != id(copy_list)
True
```

Truco: En el caso de que estemos trabajando con listas que contienen elementos mutables, debemos hacer uso de la función `deepcopy()` dentro del módulo `copy` de la librería estándar.

5.1.5 Veracidad múltiple

Si bien podemos usar *sentencias condicionales* para comprobar la veracidad de determinadas expresiones, Python nos ofrece dos funciones «built-in» con las que podemos evaluar si se cumplen **todas** las condiciones `all()` o si se cumple **alguna** condición `any()`. Estas funciones trabajan sobre iterables, y el caso más evidente es una **lista**.

Supongamos un ejemplo en el que queremos comprobar si una determinada palabra cumple las siguientes condiciones:

- Su longitud total es mayor que 4.
- Empieza por «p».
- Contiene, al menos, una «y».

Veamos la **versión clásica**:

```
>>> word = 'python'

>>> if len(word) > 4 and word.startswith('p') and word.count('y') >= 1:
...     print('Cool word!')
```

(continué en la próxima página)

(proviene de la página anterior)

```
... else:
...     print('No thanks')
...
Cool word!
```

Veamos la **versión con veracidad múltiple** usando `all()`, donde se comprueba que se cumplan **todas** las expresiones:

```
>>> word = 'python'

>>> enough_length = len(word) > 4            # True
>>> right_beginning = word.startswith('p')   # True
>>> min_ys = word.count('y') >= 1            # True

>>> is_cool_word = all([enough_length, right_beginning, min_ys])

>>> if is_cool_word:
...     print('Cool word!')
... else:
...     print('No thanks')
...
Cool word!
```

Veamos la **versión con veracidad múltiple** usando `any()`, donde se comprueba que se cumpla **alguna** expresión:

```
>>> word = 'yeah'

>>> enough_length = len(word) > 4            # False
>>> right_beginning = word.startswith('p')   # False
>>> min_ys = word.count('y') >= 1            # True

>>> is_fine_word = any([enough_length, right_beginning, min_ys])

>>> if is_fine_word:
...     print('Fine word!')
... else:
...     print('No thanks')
...
Fine word!
```

Consejo: Este enfoque puede ser interesante cuando se manejan muchas condiciones o bien cuando queremos separar las condiciones y agruparlas en una única lista.

A tener en cuenta la *peculiaridad* de estas funciones cuando trabajan con la **lista vacía**:

```
>>> all([])
True

>>> any([])
False
```

5.1.6 Listas por comprensión

Nivel intermedio

Las **listas por comprensión** establecen una técnica para crear listas de forma más **compacta** basándose en el concepto matemático de conjuntos definidos por comprensión.

Podríamos decir que su sintaxis sigue un modelo **VLC (Value-Loop-Condition)** tal y como se muestra en la siguiente figura:

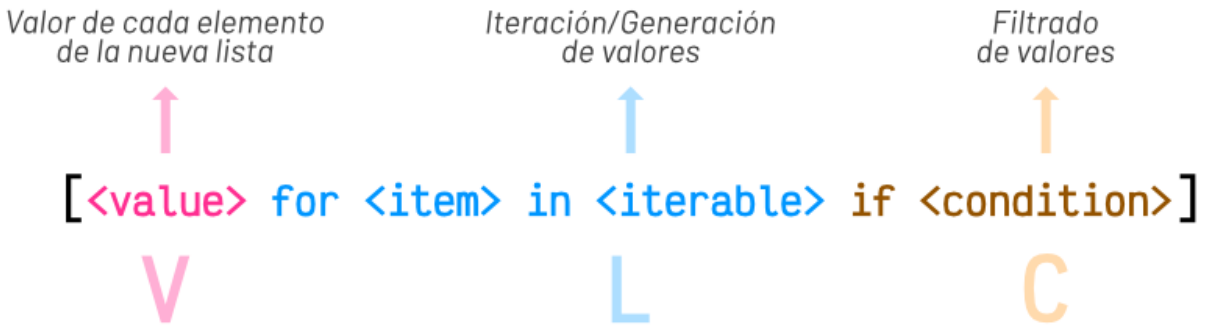


Figura 3: Estructura de una lista por comprensión

En primer lugar veamos un ejemplo en el que convertimos una cadena de texto con valores numéricos en una lista con los mismos valores pero convertidos a enteros. En su **versión clásica** haríamos algo tal que así:

```
>>> values = '32,45,11,87,20,48'

>>> int_values = []

>>> for value in values.split(','):
...     int_value = int(value)
...     int_values.append(int_value)
...

>>> int_values
[32, 45, 11, 87, 20, 48]
```

Ahora veamos el código utilizando una **lista por comprensión**:

```
>>> values = '32,45,11,87,20,48'

>>> int_values = [int(value) for value in values.split(',')]

>>> int_values
[32, 45, 11, 87, 20, 48]
```

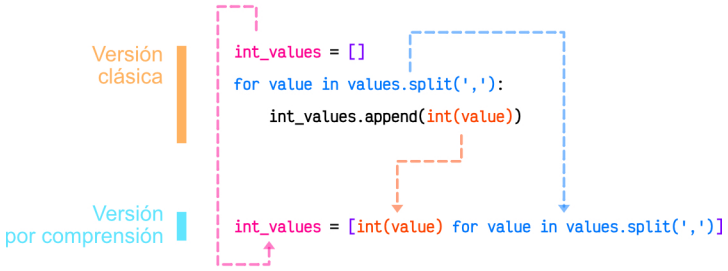


Figura 4: Transformación de estructura clásica en lista por comprensión

Condiciones en comprensiones

También existe la posibilidad de incluir condiciones en las **listas por comprensión**.

Continuando con el ejemplo anterior, supongamos que sólo queremos crear la lista con aquellos valores que empiecen por el dígito 4:

```
>>> values = '32,45,11,87,20,48'

>>> int_values = [int(v) for v in values.split(',') if v.startswith('4')]

>>> int_values
[45, 48]
```

Anidamiento en comprensiones

Nivel avanzado

En la iteración que usamos dentro de la lista por comprensión es posible usar *bucles anidados*.

Veamos un ejemplo en el que generamos todas las combinaciones de una serie de valores:

```
>>> values = '32,45,11,87,20,48'
>>> svalues = values.split(',')
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> combinations = [f'{v1}x{v2}' for v1 in svalues for v2 in svalues]

>>> combinations
['32x32',
 '32x45',
 '32x11',
 '32x87',
 '32x20',
 '32x48',
 '45x32',
 '45x45',
 ...
 '48x45',
 '48x11',
 '48x87',
 '48x20',
 '48x48']
```

Consejo: Las listas por comprensión son una herramienta muy potente y nos ayuda en muchas ocasiones, pero hay que tener cuidado de no generar **expresiones excesivamente complejas**. En estos casos es mejor una *aproximación clásica*.

Ejercicio

pycheck: **fcomp**

5.1.7 sys.argv

Cuando queramos ejecutar un programa Python desde **línea de comandos**, tendremos la posibilidad de acceder a los argumentos de dicho programa. Para ello se utiliza una lista «especial» que la encontramos dentro del módulo `sys` y que se denomina `argv`:

Veamos una aplicación de lo anterior en un programa que convierte un número decimal a una determinada base, ambos argumentos pasados por línea de comandos:

`dec2base.py`

```
1 import sys
2
3 number = int(sys.argv[1])
4 tobase = int(sys.argv[2])
```

(continué en la próxima página)

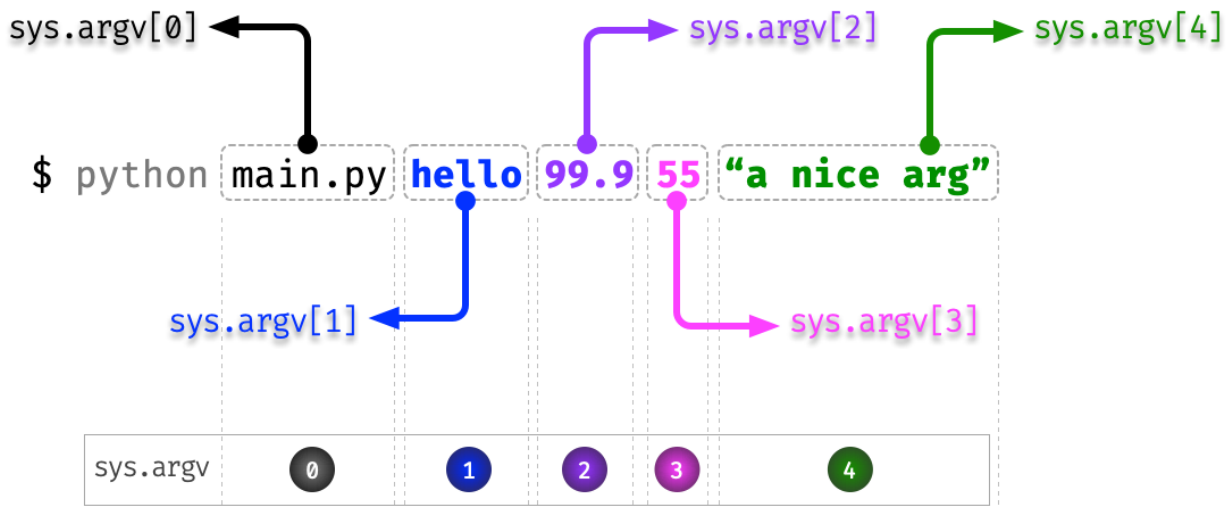


Figura 5: Acceso a parámetros en línea de comandos

(proviene de la página anterior)

```
5
6   match tobase:
7       case 2:
8           result = f'{number:b}'
9       case 8:
10          result = f'{number:o}'
11      case 16:
12          result = f'{number:x}'
13      case _:
14          result = None
15
16  if result is None:
17      print(f'Base {tobase} not implemented!')
18  else:
19      print(result)
```

Si lo ejecutamos obtenemos lo siguiente:

```
$ python dec2base.py 65535 2
1111111111111111
```

5.1.8 Funciones matemáticas

Python nos ofrece, entre otras[4], estas tres funciones matemáticas básicas que se pueden aplicar sobre listas.

Suma de todos los valores: Mediante la función `sum()`:

```
>>> data = [5, 3, 2, 8, 9, 1]
>>> sum(data)
28
```

Mínimo de todos los valores: Mediante la función `min()`:

```
>>> data = [5, 3, 2, 8, 9, 1]
>>> min(data)
1
```

Máximo de todos los valores: Mediante la función `max()`:

```
>>> data = [5, 3, 2, 8, 9, 1]
>>> max(data)
9
```

Ejercicio

Lea *desde línea de comandos* una serie de números y obtenga la media de dichos valores (redondeando a 2 cifras decimales).

La llamada se haría de la siguiente manera:

```
$ python avg.py 32 56 21 99 12 21
```

Plantilla de código para el programa:

```
import sys

# En values tendremos una lista con los valores (como strings)
values = sys.argv[1:]

# Su código debajo de aquí
```

Ejemplo

- Entrada: `32 56 21 99 12 17`
- Salida: `40.17`

[4] Existen multitud de paquetes científicos en Python para trabajar con listas o vectores numéricos. Una de las más famosas es la librería Numpy.

5.1.9 Listas de listas

Nivel intermedio

Como ya hemos visto en varias ocasiones, las listas son estructuras de datos que pueden contener elementos heterogéneos. Estos elementos pueden ser a su vez listas.

A continuación planteamos un ejemplo del contexto deportivo. Un equipo de fútbol suele tener una disposición en el campo organizada en líneas de jugadores/as. En aquella alineación con la que España ganó la copa del mundo en 2023 había una disposición *4-3-3* con las siguientes jugadoras:



Figura 6: Lista de listas (como equipo de fútbol)

Veamos una posible representación de este equipo de fútbol usando **una lista compuesta de listas**. Primero definimos cada una de las líneas:

```
>>> goalkeeper = 'Cata'
>>> defenders = ['Olga', 'Laia', 'Irene', 'Ona']
>>> midfielders = ['Jenni', 'Teresa', 'Aitana']
>>> forwards = ['Mariona', 'Salma', 'Alba']
```

Y ahora las juntamos en una única lista:

```
>>> team = [goalkeeper, defenders, midfielders, forwards]

>>> team
['Cata',
 ['Olga', 'Laia', 'Irene', 'Ona'],
 ['Jenni', 'Teresa', 'Aitana'],
 ['Mariona', 'Salma', 'Alba']]
```

Podemos comprobar el **acceso a distintos elementos**:

```
>>> team[0]  # portera
'Cata'

>>> team[1][0]  # lateral izquierdo
'Olga'

>>> team[2]  # centrocampistas
['Jenni', 'Teresa', 'Aitana']

>>> team[3][1]  # delantera centro
'Salma'
```

También podemos **recorrer toda la alineación**:

```
>>> for playline in team:
...     if isinstance(playline, list):
...         for player in playline:
...             print(player, end=' ')
...         print()
...     else:
...         print(playline)
...
Cata
Olga Laia Irene Ona
Jenni Teresa Aitana
Mariona Salma Alba
```

Ejercicio

pycheck: **mulmatrix2**

Ejercicio

---

pycheck: **mulmatrix**

EJERCICIOS DE REPASO

1. pycheck: **max_value**
2. pycheck: **max_value_with_min**
3. pycheck: **remove_dups**
4. pycheck: **flatten_list**
5. pycheck: **remove_consecutive_dups**
6. pycheck: **all_same**
7. pycheck: **sum_diagonal**
8. pycheck: **powers2**
9. pycheck: **dec2bin**
10. pycheck: **sum_mixed**
11. pycheck: **n_mult**
12. pycheck: **remove_second**
13. pycheck: **nth_power**
14. pycheck: **name_initials**
15. pycheck: **non_consecutive**
16. pycheck: **mult_reduce**
17. pycheck: **digit_rev_list**
18. pycheck: **time_plus_minutes**
19. pycheck: **sum_positive**
20. pycheck: **add_inverse**
21. pycheck: **descending_numbers**
22. pycheck: **merge_sorted**
23. pycheck: **min_value**
24. pycheck: **min_value_with_max**
25. pycheck: **trimmed_sum**

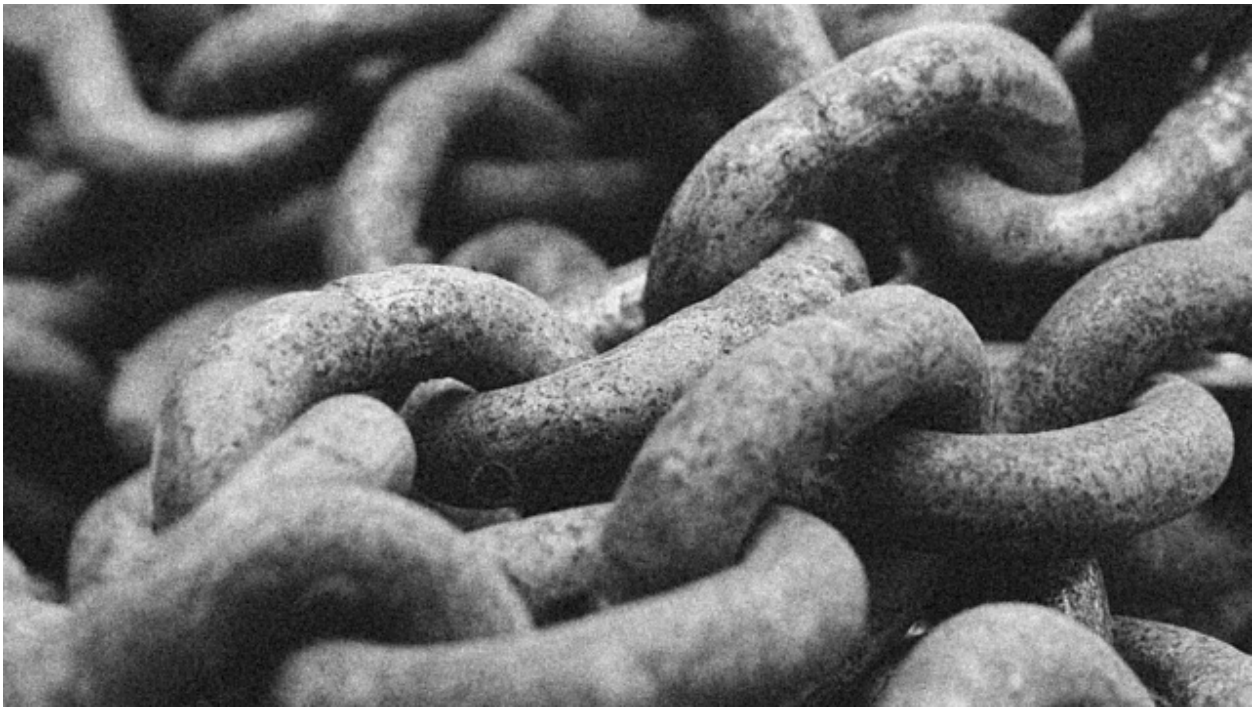
26. pycheck: **wolves**
27. pycheck: **maxmin_values**
28. pycheck: **cascading_subsets**
29. pycheck: **diff_cuboid**
30. pycheck: **fl_strip**
31. pycheck: **logical_chain**
32. pycheck: **smallest_unused_id**
33. pycheck: **find_odds**
34. pycheck: **reagent_formula**
35. pycheck: **whats_next**
36. pycheck: **npartition**
37. pycheck: **add_length**
38. pycheck: **reversing_words**
39. pycheck: **barycenter**
40. pycheck: **kpower**
41. pycheck: **sort_numbers**
42. pycheck: **flatten_list_deep**
43. pycheck: **first_duplicated**
44. pycheck: **fill_values**
45. pycheck: **frange**
46. pycheck: **qual_number**

AMPLIAR CONOCIMIENTOS

- Linked Lists in Python: An Introduction
- Python Command Line Arguments
- Sorting Data With Python
- When to Use a List Comprehension in Python
- Using the Python zip() Function for Parallel Iteration
- Lists and Tuples in Python
- How to Use sorted() and sort() in Python

- Using List Comprehensions Effectively

5.2 Tuplas



El concepto de **tupla** es muy similar al de *lista*. Aunque hay algunas diferencias menores, lo fundamental es que, mientras una *lista* es mutable y se puede modificar, una *tupla* no admite cambios y por lo tanto, es **inmutable**.[1]

5.2.1 Creando tuplas

Podemos pensar en crear tuplas tal y como *lo hacíamos con listas*, pero usando **paréntesis** en lugar de *corchetes*:

```
>>> empty_tuple = ()

>>> tenerife_geoloc = (28.46824, -16.25462)

>>> three_wise_men = ('Melchor', 'Gaspar', 'Baltasar')
```

Truco: Al igual que con las listas, las tuplas admiten diferentes tipos de datos: `('a', 1,`

[1] Foto original de portada por engin akyurt en Unsplash.

```
True)
```

Tuplas de un elemento

Hay que prestar especial atención cuando vamos a crear una **tupla de un único elemento**. La intención primera sería hacerlo de la siguiente manera:

```
>>> one_item_tuple = ('Papá Noel')

>>> one_item_tuple
'Papá Noel'

>>> type(one_item_tuple)
str
```

Realmente, hemos creado una variable de tipo `str` (cadena de texto). Para crear una tupla de un elemento debemos añadir una **coma** al final:

```
>>> one_item_tuple = ('Papá Noel',)

>>> one_item_tuple
('Papá Noel',)

>>> type(one_item_tuple)
tuple
```

Tuplas sin paréntesis

Según el caso, hay veces que nos podemos encontrar con tuplas que no llevan paréntesis. Quizás no está tan extendido, pero a efectos prácticos tiene el mismo resultado. Veamos algunos ejemplos de ello:

```
>>> one_item_tuple = 'Papá Noel',

>>> three_wise_men = 'Melchor', 'Gaspar', 'Baltasar'

>>> tenerife_geoloc = 28.46824, -16.25462
```

Advertencia: Aunque está permitido, **NUNCA** llames `tuple` a una variable porque destruirías la función que nos permite crear tuplas. Y tampoco uses nombres derivados como `_tuple` o `tuple_` ya que no son nombres representativos que *identifiquen el propósito de la variable*.

5.2.2 Modificar una tupla

Como ya hemos comentado previamente, las tuplas con estructuras de datos **inmutables**. Una vez que las creamos con un valor, no podemos modificarlas. Veamos qué ocurre si lo intentamos:

```
>>> three_wise_men = 'Melchor', 'Gaspar', 'Baltasar'

>>> three_wise_men[0] = 'Tom Hanks'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

5.2.3 Conversión

Para convertir otros tipos de datos en una tupla podemos usar la función `tuple()`:

```
>>> shopping = ['Agua', 'Aceite', 'Arroz']

>>> tuple(shopping)
('Agua', 'Aceite', 'Arroz')
```

Esta conversión es válida para aquellos tipos de datos que sean *iterables*: cadenas de caracteres, listas, diccionarios, conjuntos, etc. Un ejemplo que no funciona es intentar convertir un número en una tupla:

```
>>> tuple(5)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'int' object is not iterable
```

El uso de la función `tuple()` sin argumentos equivale a crear una tupla vacía:

```
>>> tuple()
()
```

Truco: Para crear una tupla vacía, se suele recomendar el uso de `()` frente a `tuple()`, no sólo por ser más *pitónico* sino por tener (en promedio) un mejor rendimiento en tiempos de ejecución.

5.2.4 Operaciones con tuplas

Con las tuplas podemos realizar *todas las operaciones que vimos con listas* **salvo las que conlleven una modificación** «in-situ» de la misma:

- `reverse()`
- `append()`
- `extend()`
- `remove()`
- `clear()`
- `sort()`

Truco: Sí es posible aplicar `sorted()` o `reversed()` sobre una tupla ya que no estamos modificando su valor sino creando un nuevo objeto.

Ver también:

La comparación de tuplas funciona exactamente igual que la *comparación de listas*.

5.2.5 Desempaquetado de tuplas

El **desempaquetado** es una característica de las tuplas que nos permite *asignar una tupla a variables independientes*:

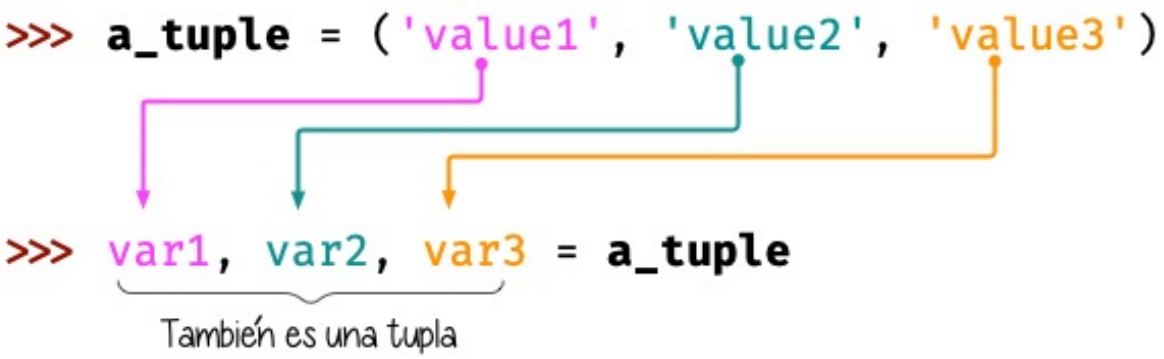


Figura 7: Desempaquetado de tuplas

Veamos un ejemplo con código:

```
>>> three_wise_men = ('Melchor', 'Gaspar', 'Baltasar')

>>> king1, king2, king3 = three_wise_men

>>> king1
'Melchor'
>>> king2
'Gaspar'
>>> king3
'Baltasar'
```

Python proporciona la función «built-in» `divmod()` que devuelve el cociente y el resto de una división usando una única llamada. Lo interesante (para el caso que nos ocupa) es que se suele utilizar el desempaquetado de tuplas para obtener los valores:

```
>>> quotient, remainder = divmod(7, 3)

>>> quotient
2
>>> remainder
1
```

Intercambio de valores

A través del desempaquetado de tuplas podemos llevar a cabo *el intercambio de los valores de dos variables* de manera directa:

```
>>> value1 = 40
>>> value2 = 20

>>> value1, value2 = value2, value1

>>> value1
20
>>> value2
40
```

Nota: A priori puede parecer que esto es algo «natural», pero en la gran mayoría de lenguajes de programación no es posible hacer este intercambio de forma «directa» ya que necesitamos recurrir a una tercera variable «auxiliar» como almacén temporal en el paso intermedio de traspaso de valores.

Desempaquetado extendido

No tenemos que ceñirnos a realizar desempaquetado uno a uno. También podemos extenderlo e indicar ciertos «grupos» de elementos mediante el operador `*`.

Veamos un ejemplo:

```
>>> ranking = ('G', 'A', 'R', 'Y', 'W')

>>> head, *body, tail = ranking

>>> head
'G'

>>> body
['A', 'R', 'Y']

>>> tail
'W'
```

Podemos aplicar combinaciones del enfoque anterior. Por ejemplo usando sólo dos elementos:

```
>>> ranking = ('G', 'A', 'R', 'Y', 'W')

>>> head, *body = ranking
>>> head
'G'
>>> body
['A', 'R', 'Y', 'W']

>>> *body, tail = ranking
>>> body
['G', 'A', 'R', 'Y']
>>> tail
'W'
```

Lo que se tiene que cumplir es que **el número de elementos de destino debe ser menor o igual que el número de elementos de origen**:

```
>>> ranking
('G', 'A', 'R', 'Y', 'W')

>>> len(ranking)
5

>>> r1, r2, r3, r4, r5, r6 = ranking
Traceback (most recent call last):
```

(continué en la próxima página)

(proviene de la página anterior)

```
  File "<stdin>", line 1, in <module>
ValueError: not enough values to unpack (expected 6, got 5)
```

Desempaquetado genérico

El desempaquetado de tuplas es extensible a cualquier tipo de datos que sea **iterable**. Veamos algunos ejemplos de ello.

Sobre *cadenas de texto*:

```
>>> oxygen = 'O2'
>>> first, last = oxygen
>>> first, last
('O', '2')

>>> text = 'Hello, World!'
>>> head, *body, tail = text
>>> head, body, tail
('H', ['e', 'l', 'l', 'o', ',', ' ', 'W', 'o', 'r', 'l', 'd'], '!')
```

Sobre *listas*:

```
>>> writer1, writer2, writer3 = ['Virginia Woolf', 'Jane Austen', 'Mary Shelley']
>>> writer1, writer2, writer3
('Virginia Woolf', 'Jane Austen', 'Mary Shelley')

>>> text = 'Hello, World!'
>>> word1, word2 = text.split()
>>> word1, word2
('Hello,', 'World!')
```

5.2.6 ¿Tuplas por comprensión?

Los tipos de datos mutables (*listas, diccionarios y conjuntos*) sí permiten comprensiones pero no así los tipos de datos inmutables como *cadenas de texto* y *tuplas*.

Si intentamos crear una **tupla por comprensión** utilizando paréntesis alrededor de la expresión, vemos que no obtenemos ningún error al ejecutarlo:

```
>>> myrange = (number for number in range(1, 6))
```

Sin embargo no hemos conseguido una tupla por comprensión sino un generador:

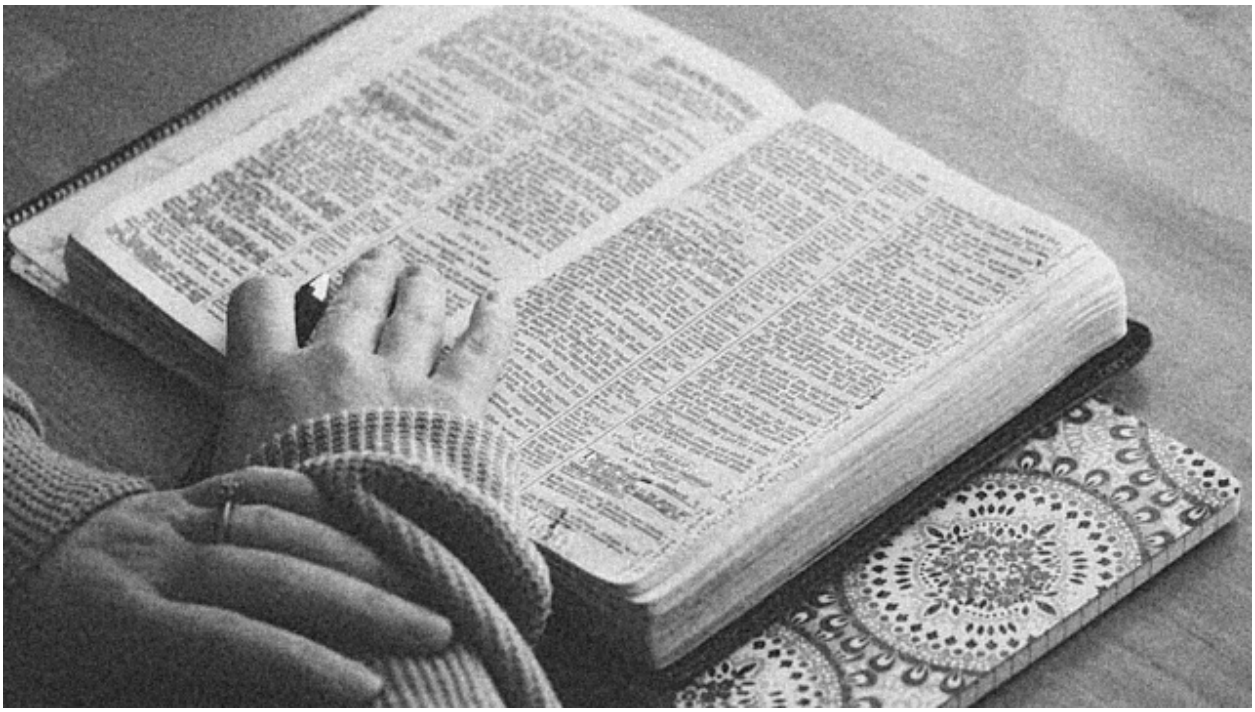
```
>>> myrange
<generator object <genexpr> at 0x10b3732e0>
```

5.2.7 Tuplas vs Listas

Aunque puedan parecer estructuras de datos muy similares, sabemos que las tuplas carecen de ciertas operaciones, especialmente las que tienen que ver con la modificación de sus valores, ya que no son inmutables. Si las listas son más flexibles y potentes, ¿por qué íbamos a necesitar tuplas? Veamos 4 potenciales ventajas del uso de tuplas frente a las listas:

1. Las tuplas ocupan **menos espacio** en memoria.
2. En las tuplas existe **protección** frente a cambios indeseados.
3. Las tuplas se pueden usar como **claves de diccionarios** (son *«hashables»*).
4. Las namedtuples son una alternativa sencilla a los objetos.

5.3 Diccionarios



Podemos trasladar el concepto de *diccionario* de la vida real al de *diccionario* en Python. Al fin y al cabo un diccionario es un objeto que contiene palabras, y cada palabra tiene asociado un significado. Haciendo el paralelismo, diríamos que en Python un diccionario es

también un objeto indexado por **claves** (las palabras) que tienen asociados unos **valores** (los significados).[1]

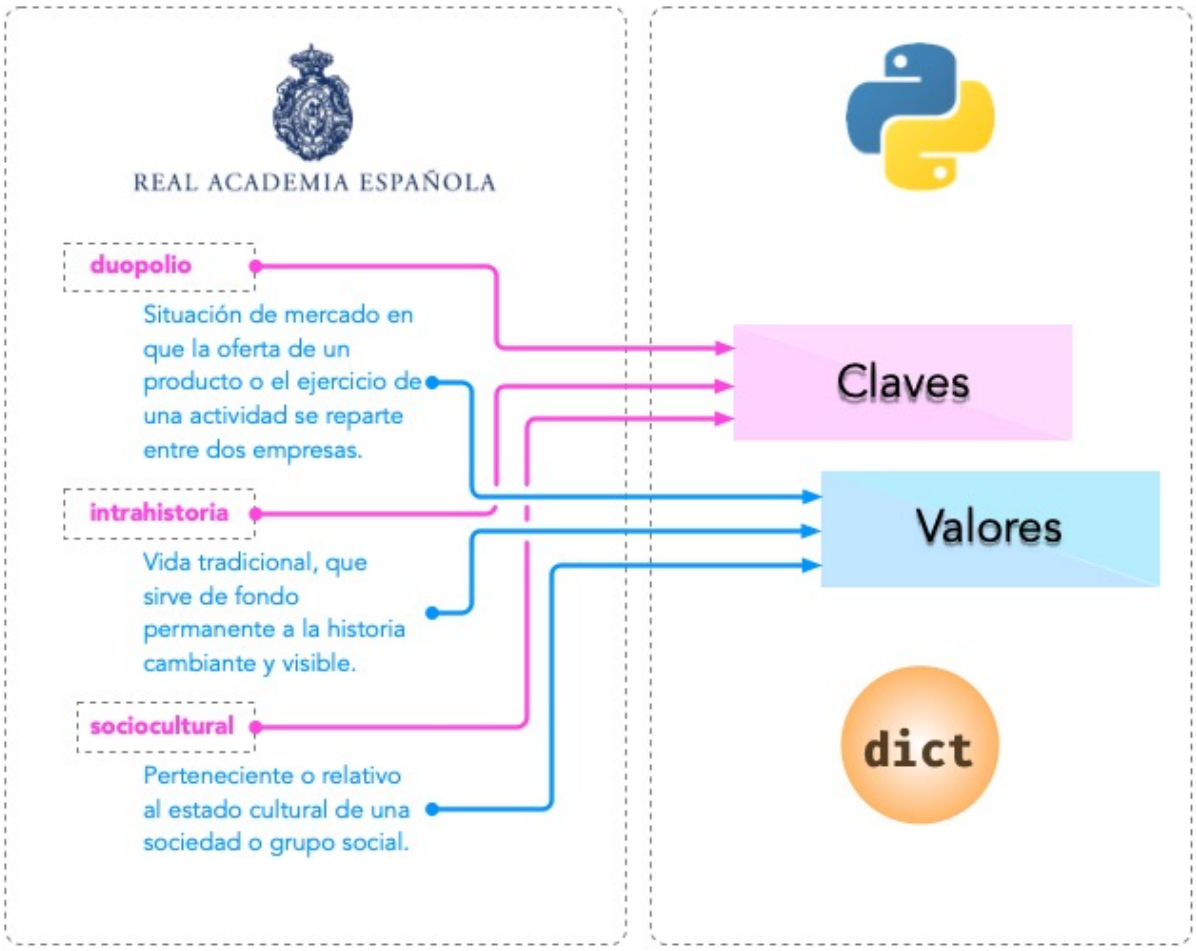


Figura 8: Analogía de un diccionario en Python

Los diccionarios en Python tienen las siguientes *características*:

- Mantienen el **orden** en el que se insertan las claves.[2]
- Son **mutables**, con lo que admiten añadir, borrar y modificar sus elementos.
- Las **claves** deben ser **únicas**. A menudo se utilizan las *cadenas de texto* como claves, pero en realidad podría ser cualquier tipo de datos inmutable: enteros, flotantes, tuplas (entre otros).

[1] Foto original de portada por Aaron Burden en Unsplash.

[2] Aunque históricamente Python no establecía que las claves de los diccionarios tuvieran que mantener su orden de inserción, a partir de Python 3.7 este comportamiento cambió y se garantizó el orden de inserción de las claves como parte oficial de la especificación del lenguaje.

- Tienen un **acceso muy rápido** a sus elementos, debido a la forma en la que están implementados internamente.[3]

Nota: En otros lenguajes de programación, a los diccionarios se les conoce como *arrays asociativos*, *«hashes»* o *«hashmaps»*.

5.3.1 Creando diccionarios

Para crear un diccionario usamos llaves `{}` rodeando asignaciones `clave: valor` que están separadas por comas. Veamos algunos ejemplos de diccionarios:

```
>>> empty_dict = {}

>>> rae = {
...     'bifronte': 'De dos frentes o dos caras',
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa'
... }

>>> population_can = {
...     2015: 2_135_209,
...     2016: 2_154_924,
...     2017: 2_177_048,
...     2018: 2_206_901,
...     2019: 2_220_270
... }
```

En el código anterior podemos observar la creación de un diccionario vacío, otro donde sus claves y sus valores son cadenas de texto y otro donde las claves y los valores son valores enteros.

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/Sfav2Yw

Advertencia: Aunque está permitido, **NUNCA** llames `dict` a una variable porque destruirías la función que nos permite crear diccionarios. Y tampoco uses nombres derivados como `_dict` o `dict_` ya que no son nombres representativos que *identifiquen el propósito de la variable*.

Ejercicio

[3] Véase este análisis de complejidad y rendimiento de distintas estructuras de datos en CPython.

Entre en el intérprete interactivo de Python (>>>) y cree un diccionario con los nombres (como claves) de 5 personas de su familia y sus edades (como valores).

5.3.2 Conversión

Para convertir otros tipos de datos en un diccionario podemos usar la función `dict()`:

```
>>> # Diccionario a partir de una lista de cadenas de texto
>>> dict(['a1', 'b2'])
{'a': '1', 'b': '2'}

>>> # Diccionario a partir de una tupla de cadenas de texto
>>> dict(('a1', 'b2'))
{'a': '1', 'b': '2'}

>>> # Diccionario a partir de una lista de listas
>>> dict([['a', 1], ['b', 2]])
{'a': 1, 'b': 2}
```

Nota: Si nos fijamos bien, cualquier iterable que tenga una estructura interna de 2 elementos es susceptible de convertirse en un diccionario a través de la función `dict()`.

Diccionario vacío

Existe una manera particular de usar `dict()` y es no pasarle ningún argumento. En este caso estaremos queriendo convertir el «vacío» en un diccionario, con lo que obtendremos un *diccionario vacío*:

```
>>> dict()
{}
```

Truco: Para crear un diccionario vacío, se suele recomendar el uso de `{}` frente a `dict()`, no sólo por ser más *pitónico* sino por tener (en promedio) un mejor rendimiento en tiempos de ejecución.

Creación con `dict()`

También es posible utilizar la función `dict()` para crear dicionarios y no tener que utilizar llaves y comillas:

Supongamos que queremos transformar la siguiente tabla en un diccionario:

Atributo	Valor
`name`	Guido
`surname`	Van Rossum
`job`	Python creator

Utilizando la construcción mediante `dict` podemos pasar clave y valor como **argumentos** de la función:

```
>>> person = dict(
...     name='Guido',
...     surname='Van Rossum',
...     job='Python creator'
... )

>>> person
{'name': 'Guido', 'surname': 'Van Rossum', 'job': 'Python creator'}
```

El inconveniente que tiene esta aproximación es que las **claves deben ser identificadores válidos** en Python. Por ejemplo, no se permiten espacios:

```
>>> person = dict(
...     name='Guido van Rossum',
...     date of birth='31/01/1956'
  File "<stdin>", line 3
    date of birth='31/01/1956'
          ^
SyntaxError: invalid syntax
```

Nivel intermedio

Es posible crear un diccionario especificando sus claves y un único valor de «relleno»:

```
>>> dict.fromkeys('aeiou', 0)
{'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}
```

Nota: Es válido pasar cualquier «iterable» como referencia a las claves.

5.3.3 Operaciones con diccionarios

Obtener un elemento

Para obtener un elemento de un diccionario basta con escribir la **clave** entre corchetes. Veamos un ejemplo:

```
>>> rae = {
...     'bifronte': 'De dos frentes o dos caras',
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa'
... }

>>> rae['anarcoide']
'Que tiende al desorden'
```

Si intentamos acceder a una clave que no existe, obtendremos un error:

```
>>> rae['acceso']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'acceso'
```

Usando `get()`

Existe una función muy útil para «superar» los posibles errores de acceso por claves inexistentes. Se trata de `get()` y su comportamiento es el siguiente:

1. Si la clave que buscamos existe, nos devuelve su valor.
2. Si la clave que buscamos no existe, nos devuelve `None`[4] salvo que le indiquemos otro valor por defecto, pero en ninguno de los dos casos obtendremos un error.

```
1  >>> rae
2  {'bifronte': 'De dos frentes o dos caras',
3   'anarcoide': 'Que tiende al desorden',
4   'montuvio': 'Campesino de la costa'}
5
6  >>> rae.get('bifronte')
7  'De dos frentes o dos caras'
8
9  >>> rae.get('programación')
10
```

(continué en la próxima página)

[4] `None` es la palabra reservada en Python para la «nada». Más información en esta web.

(proviene de la página anterior)

```
11 >>> rae.get('programación', 'No disponible')
12 'No disponible'
```

Línea 6: Equivalente a `rae['bifronte']`.

Línea 9: La clave buscada no existe y obtenemos `None`.[5]

Línea 11: La clave buscada no existe y nos devuelve el valor que hemos aportado por defecto.

Añadir o modificar un elemento

Para añadir un elemento a un diccionario sólo es necesario hacer referencia a la *clave* y asignarle un *valor*:

- Si la clave **ya existía** en el diccionario, **se reemplaza** el valor existente por el nuevo.
- Si la clave **es nueva**, **se añade** al diccionario con su valor. *No vamos a obtener un error a diferencia de las listas.*

Partimos del siguiente diccionario para ejemplificar estas acciones:

```
>>> rae = {
...     'bifronte': 'De dos frentes o dos caras',
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa'
... }
```

Vamos a **añadir** la palabra *enjuiciar* a nuestro diccionario de la Real Academia de La Lengua:

```
>>> rae['enjuiciar'] = 'Someter una cuestión a examen, discusión y juicio'

>>> rae
{'bifronte': 'De dos frentes o dos caras',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa',
 'enjuiciar': 'Someter una cuestión a examen, discusión y juicio'}
```

Supongamos ahora que queremos **modificar** el significado de la palabra *enjuiciar* por otra acepción:

[5] Realmente no estamos viendo nada en la consola de Python porque la representación en cadena de texto es vacía.

```
>>> rae['enjuiciar'] = 'Instruir, juzgar o sentenciar una causa'

>>> rae
{'bifronte': 'De dos frentes o dos caras',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa',
 'enjuiciar': 'Instruir, juzgar o sentenciar una causa'}
```

Creando desde vacío

Una forma muy habitual de trabajar con diccionarios es utilizar el **patrón creación** partiendo de uno vacío e ir añadiendo elementos poco a poco.

Supongamos un ejemplo en el que queremos construir un diccionario donde las claves son las letras vocales y los valores son sus posiciones:

```
>>> VOWELS = 'aeiou'

>>> enum_vowels = {}

>>> for i, vowel in enumerate(VOWELS, start=1):
...     enum_vowels[vowel] = i
...

>>> enum_vowels
{'a': 1, 'e': 2, 'i': 3, 'o': 4, 'u': 5}
```

Nota: Hemos utilizando la función `enumerate()` que ya vimos para las listas en el apartado: *Iterar usando enumeración*.

Ejercicio

pycheck: **cities**

Pertenencia de una clave

La forma **pitónica** de comprobar la existencia de una clave dentro de un diccionario, es utilizar el operador `in`:

```
>>> 'bifronte' in rae
True

>>> 'almohada' in rae
False

>>> 'montuvio' not in rae
False
```

Nota: El operador `in` siempre devuelve un valor booleano, es decir, verdadero o falso.

Ejercicio

pycheck: **count_letters**

Longitud de un diccionario

Podemos conocer el número de elementos («clave-valor») que tiene un diccionario con la función `len()`:

```
>>> rae
{'bifronte': 'De dos frentes o dos caras',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa',
 'enjuiciar': 'Instruir, juzgar o sentenciar una causa'}

>>> len(rae)
4
```

Obtener todos los elementos

Python ofrece mecanismos para obtener todos los elementos de un diccionario. Partimos del siguiente diccionario:

```
>>> rae
{'bifronte': 'De dos frentes o dos caras',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa',
 'enjuiciar': 'Instruir, juzgar o sentenciar una causa'}
```

Obtener todas las claves de un diccionario: Mediante la función `keys()`:

```
>>> rae.keys()
dict_keys(['bifronte', 'anarcoide', 'montuvio', 'enjuiciar'])
```

Obtener todos los valores de un diccionario: Mediante la función `values()`:

```
>>> rae.values()
dict_values([
    'De dos frentes o dos caras',
    'Que tiende al desorden',
    'Campesino de la costa',
    'Instruir, juzgar o sentenciar una causa'
])
```

Obtener todos los pares «clave-valor» de un diccionario: Mediante la función `items()`:

```
>>> rae.items()
dict_items([
    ('bifronte', 'De dos frentes o dos caras'),
    ('anarcoide', 'Que tiende al desorden'),
    ('montuvio', 'Campesino de la costa'),
    ('enjuiciar', 'Instruir, juzgar o sentenciar una causa')
])
```

Nota: Para este último caso cabe destacar que los «items» se devuelven como una lista de *tuplas*, donde cada tupla tiene dos elementos: el primero representa la clave y el segundo representa el valor.

Iterar sobre un diccionario

En base a *los elementos que podemos obtener*, Python nos proporciona tres maneras de iterar sobre un diccionario.

Iterar sobre claves:

```
>>> for word in rae.keys():
...     print(word)
...
bifronte
anarcoide
montuvio
enjuiciar
```

Iterar sobre valores:

```
>>> for meaning in rae.values():
...     print(meaning)
...
De dos frentes o dos caras
Que tiende al desorden
Campesino de la costa
Instruir, juzgar o sentenciar una causa
```

Iterar sobre «clave-valor»:

```
>>> for word, meaning in rae.items():
...     print(f'{word}: {meaning}')
...
bifronte: De dos frentes o dos caras
anarcoide: Que tiende al desorden
montuvio: Campesino de la costa
enjuiciar: Instruir, juzgar o sentenciar una causa
```

Nota: En este último caso, recuerde el uso de los *«f-strings»* para formatear cadenas de texto.

Ejercicio

pycheck: **avg_population**

Borrar elementos

Python nos ofrece, al menos, tres formas para borrar elementos en un diccionario:

Por su clave: Mediante la sentencia `del`:

```
>>> rae = {
...     'bifronte': 'De dos frentes o dos caras',
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa'
... }

>>> del rae['bifronte']

>>> rae
{'anarcoide': 'Que tiende al desorden', 'montuvio': 'Campesino de la costa'}
```

Por su clave (con extracción): Mediante la función `pop()` podemos extraer un elemento del diccionario por su clave. Vendría a ser una combinación de `get()` + `del`:

```
>>> rae = {
...     'bifronte': 'De dos frentes o dos caras',
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa'
... }

>>> rae.pop('anarcoide')
'Que tiende al desorden'

>>> rae
{'bifronte': 'De dos frentes o dos caras', 'montuvio': 'Campesino de la costa'}

>>> rae.pop('bucle')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'bucle'
```

> **Advertencia:** Si la clave que pretendemos extraer con `pop()` no existe, obtendremos un error.

Borrado completo del diccionario:

1. Utilizando la función `clear()`:

```
>>> rae = {
...     'bifronte': 'De dos frentes o dos caras',
```

(continué en la próxima página)

(proviene de la página anterior)

```
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa'
... }

>>> rae.clear()

>>> rae
{}
```

En este caso borramos el contenido de la variable.

2. «Reinicializando» el diccionario a vacío con `{}`:

```
>>> rae = {
...     'bifronte': 'De dos frentes o dos caras',
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa'
... }

>>> rae = {}

>>> rae
{}
```

En este caso creamos una nueva variable «vacía».

Ejercicio

pycheck: **merge_dicts**

Combinar diccionarios

Dados dos (o más) diccionarios, es posible «mezclarlos» para obtener una combinación de los mismos. Esta combinación se basa en dos premisas:

1. Si la clave no existe, se añade con su valor.
2. Si la clave ya existe, se añade con el valor del «último» diccionario en la mezcla.[6]

Python ofrece dos mecanismos para realizar esta combinación. Vamos a partir de los siguientes diccionarios para ejemplificar su uso:

[6] En este caso «último» hace referencia al diccionario que se encuentra más a la derecha en la expresión.

```
>>> rae1 = {
...     'bifronte': 'De dos frentes o dos caras',
...     'enjuiciar': 'Someter una cuestión a examen, discusión y juicio'
... }

>>> rae2 = {
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa',
...     'enjuiciar': 'Instruir, juzgar o sentenciar una causa'
... }
```

Sin modificar los diccionarios originales: Mediante el operador **:

```
>>> {**rae1, **rae2}
{'bifronte': 'De dos frentes o dos caras',
 'enjuiciar': 'Instruir, juzgar o sentenciar una causa',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa'}
```

A partir de **Python 3.9** podemos utilizar el operador | para combinar dos diccionarios:

```
>>> rae1 | rae2
{'bifronte': 'De dos frentes o dos caras',
 'enjuiciar': 'Instruir, juzgar o sentenciar una causa',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa'}
```

Modificando los diccionarios originales: Mediante la función `update()`:

```
>>> rae1.update(rae2)

>>> rae1
{'bifronte': 'De dos frentes o dos caras',
 'enjuiciar': 'Instruir, juzgar o sentenciar una causa',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa'}
```

Nota: Tener en cuenta que el orden en el que especificamos los diccionarios a la hora de su combinación (mezcla) es relevante en el resultado final. En este caso *el orden de los factores sí altera el producto.*

5.3.4 Cuidado con las copias

Nivel intermedio

Al igual que ocurría con *las listas*, si hacemos un cambio en un diccionario, se verá reflejado en todas las variables que hagan referencia al mismo. Esto se deriva de su propiedad de ser *mutable*. Veamos un ejemplo concreto:

```
>>> original_rae = {
...     'bifronte': 'De dos frentes o dos caras',
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa'
... }

>>> copy_rae = original_rae

>>> original_rae['bifronte'] = 'bla bla bla'

>>> original_rae
{'bifronte': 'bla bla bla',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa'}

>>> copy_rae
{'bifronte': 'bla bla bla',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa'}
```

Una **posible solución** a este problema es hacer una «copia dura». Para ello Python proporciona la función `copy()`:

```
>>> original_rae = {
...     'bifronte': 'De dos frentes o dos caras',
...     'anarcoide': 'Que tiende al desorden',
...     'montuvio': 'Campesino de la costa'
... }

>>> copy_rae = original_rae.copy()

>>> original_rae['bifronte'] = 'bla bla bla'

>>> original_rae
{'bifronte': 'bla bla bla',
'anarcoide': 'Que tiende al desorden',
'montuvio': 'Campesino de la costa'}

>>> copy_rae
```

(continué en la próxima página)

(proviene de la página anterior)

```
{'bifronte': 'De dos frentes o dos caras',
 'anarcoide': 'Que tiende al desorden',
 'montuvio': 'Campesino de la costa'}
```

Truco: En el caso de que estemos trabajando con diccionarios que contienen elementos mutables, debemos hacer uso de la función `deepcopy()` dentro del módulo `copy` de la librería estándar.

5.3.5 Diccionarios por comprensión

Nivel intermedio

De forma análoga a cómo se escriben las *listas por comprensión*, podemos aplicar este método a los diccionarios usando llaves `{ }`.

Veamos un ejemplo en el que creamos un **diccionario por comprensión** donde las claves son palabras y los valores son sus longitudes:

```
>>> words = ('sun', 'space', 'rocket', 'earth')

>>> words_length = {word: len(word) for word in words}

>>> words_length
{'sun': 3, 'space': 5, 'rocket': 6, 'earth': 5}
```

También podemos aplicar **condiciones** a estas comprensiones. Continuando con el ejemplo anterior, podemos incorporar la restricción de sólo incluir palabras que no empiecen por vocal:

```
>>> words = ('sun', 'space', 'rocket', 'earth')

>>> words_length = {w: len(w) for w in words if w[0] not in 'aeiou'}

>>> words_length
{'sun': 3, 'space': 5, 'rocket': 6}
```

Nota: Se puede consultar el PEP-274 para ver más ejemplos sobre diccionarios por comprensión.

Ejercicio

pycheck: **split_marks**

5.3.6 Objetos «hashables»

Nivel avanzado

La única restricción que deben cumplir las **claves** de un diccionario es ser **«hashables»**[7]. Un objeto es «hashable» si se le puede asignar un valor «hash» que no cambia en ejecución durante toda su vida.

Para encontrar el «hash» de un objeto, Python usa la función `hash()`, que devuelve un número entero y es utilizado para indexar la *tabla «hash»* que se mantiene internamente:

```
>>> hash(999)
999

>>> hash(3.14)
322818021289917443

>>> hash('hello')
-8103770210014465245

>>> hash(('a', 'b', 'c'))
-2157188727417140402
```

Para que un objeto sea «hashable», debe ser **inmutable**:

```
>>> hash(['a', 'b', 'c'])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

Nota: De lo anterior se deduce que las claves de los diccionarios, al tener que ser «hasheables», sólo pueden ser objetos inmutables.

La función «built-in» `hash()` realmente hace una llamada al método mágico `__hash__()` del objeto en cuestión:

```
>>> hash('spiderman')
-8105710090476541603
```

(continué en la próxima página)

[7] Se recomienda esta ponencia de Víctor Terrón sobre objetos «hashables».

(proviene de la página anterior)

```
>>> 'spiderman'.__hash__()
-8105710090476541603
```

EJERCICIOS DE REPASO

1. pycheck: **group_words**
2. pycheck: **same_dict_values**
3. pycheck: **build_super_dict**
4. pycheck: **clear_dict_values**
5. pycheck: **fix_keys**
6. pycheck: **order_stock**
7. pycheck: **inventory_moves**
8. pycheck: **sort_dict**
9. pycheck: **money_back**
10. pycheck: **money_back_max**
11. pycheck: **first_ntimes**
12. pycheck: **fix_id**

AMPLIAR CONOCIMIENTOS

- Using the Python defaultdict Type for Handling Missing Keys
- Python Dictionary Iteration: Advanced Tips & Tricks
- Python KeyError Exceptions and How to Handle Them
- Dictionaries in Python
- How to Iterate Through a Dictionary in Python
- Shallow vs Deep Copying of Python Objects

5.4 Conjuntos



Un **conjunto** en Python representa una serie de **valores únicos** y **sin orden establecido**. Mantiene muchas similitudes con el concepto matemático de conjunto[1]

5.4.1 Creando conjuntos

Para crear un conjunto basta con separar sus valores por *comas* y rodearlos de llaves `{}`:

```
>>> lottery = {21, 10, 46, 29, 31, 94}

>>> lottery
{10, 21, 29, 31, 46, 94}
```

La excepción la tenemos a la hora de crear un **conjunto vacío**, ya que, siguiendo la lógica de apartados anteriores, deberíamos hacerlo a través de llaves:

```
>>> wrong_empty_set = {}

>>> type(wrong_empty_set)
dict
```

[1] Foto original de portada por Duy Pham en Unsplash.

Advertencia: Si hacemos esto, lo que obtenemos es un *diccionario vacío*.

La única opción que tenemos es utilizar la función `set()`:

```
>>> empty_set = set()

>>> empty_set
set()

>>> type(empty_set)
set
```

Advertencia: Aunque está permitido, **NUNCA** llames `set` a una variable porque destruirías la función que nos permite crear conjuntos. Y tampoco uses nombres derivados como `_set` o `set_` ya que no son nombres representativos que *identifiquen el propósito de la variable*.

5.4.2 Conversión

Para convertir otros tipos de datos en un conjunto podemos usar la función `set()` sobre cualquier iterable:

```
>>> set('aplatanada')
{'a', 'd', 'l', 'n', 'p', 't'}

>>> set([1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5])
{1, 2, 3, 4, 5}

>>> set(('ADENINA', 'TIMINA', 'TIMINA', 'GUANINA', 'ADENINA', 'CITOSINA'))
{'ADENINA', 'CITOSINA', 'GUANINA', 'TIMINA'}

>>> set({'manzana': 'rojo', 'plátano': 'amarillo', 'kiwi': 'verde'})
{'kiwi', 'manzana', 'plátano'}
```

Importante: Como se ha visto en los ejemplos anteriores, `set()` se suele utilizar en muchas ocasiones como una forma de **extraer los valores únicos** de otros tipos de datos. En el caso de los diccionarios se extraen las claves, que, por definición, son únicas.

Nota: El hecho de que en los ejemplos anteriores los elementos de los conjuntos estén

ordenados es únicamente un «detalle de implementación» en el que no se puede confiar.

5.4.3 Operaciones con conjuntos

Obtener un elemento

En un conjunto no existe un orden establecido para sus elementos, por lo tanto **no podemos acceder a un elemento en concreto**.

De este hecho se deriva igualmente que **no podemos modificar un elemento existente**, ya que ni siquiera tenemos acceso al mismo. Python sí nos permite añadir o borrar elementos de un conjunto.

Añadir un elemento

Para añadir un elemento a un conjunto debemos utilizar la función `add()`. Como ya hemos indicado, al no importar el orden dentro del conjunto, la inserción no establece a priori la posición donde se realizará.

A modo de ejemplo, vamos a partir de un conjunto que representa a los cuatro integrantes originales de *The Beatles*. Luego añadiremos a un nuevo componente:

```
>>> # John Lennon, Paul McCartney, George Harrison y Ringo Starr
>>> beatles = set(['Lennon', 'McCartney', 'Harrison', 'Starr'])

>>> beatles.add('Best')  # Pete Best

>>> beatles
{'Best', 'Harrison', 'Lennon', 'McCartney', 'Starr'}
```

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/E8Q3p1m

Truco: Una pequeña **regla mnemotécnica** para diferenciar `add()` de `append()` es que la función `append()` significa añadir al final, y como los conjuntos no mantienen un orden, esta función se aplica únicamente a listas. Por descarte, la función `add()` se aplica sobre conjuntos.

Este pequeño fragmento de código nos demuestra claramente que, aunque lo intentemos por fuerza bruta, nunca vamos a poder insertar elementos repetidos en un conjunto:

```
>>> items = set()

>>> for _ in range(1_000_000):
...     items.add(1)
...

>>> items
{1}
```

Ejercicio

pycheck: **tupleset**

Objetos hashables

Los elementos de un conjunto deben ser *«hashables»*.

Por ejemplo, **una lista** no podría ser un objeto válido para un conjunto (ya que no es «hashable»). Supongamos que estamos contruyendo un conjunto con los elementos químicos de la tabla periódica:

```
>>> periodic_table = set()
>>> metals = ['Fe', 'Mg', 'Au', 'Au', 'Zn']

>>> periodic_table.add(metals)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

Sin embargo, podríamos conseguir lo que buscamos si, en vez de listas, usáramos **tuplas** para almacenar los elementos químicos (ya que sí son «hashables»):

```
>>> periodic_table = set()

>>> metals = ('Fe', 'Mg', 'Au', 'Au', 'Zn')
>>> periodic_table.add(metals)

>>> non_metals = ('C', 'H', 'O', 'F', 'Cl')
>>> periodic_table.add(non_metals)

>>> periodic_table
{('Fe', 'Mg', 'Au', 'Au', 'Zn'), ('C', 'H', 'O', 'F', 'Cl')}
```

Borrar elementos

Para borrar un elemento de un conjunto podemos utilizar la función `remove()`. Siguiendo con el ejemplo anterior, vamos a borrar al último «beatle» añadido:

```
>>> beatles
{'Best', 'Harrison', 'Lennon', 'McCartney', 'Starr'}

>>> beatles.remove('Best')

>>> beatles
{'Harrison', 'Lennon', 'McCartney', 'Starr'}
```

Si tratamos de **borrar un elemento que no existe** en un conjunto obtendremos un `KeyError` (al estilo de los *diccionarios*):

```
>>> beatles.remove('Sinatra')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Sinatra'
```

Longitud de un conjunto

Podemos conocer el número de elementos (*cardinalidad*) que tiene un conjunto con la función `len()`:

```
>>> beatles
{'Harrison', 'Lennon', 'McCartney', 'Starr'}

>>> len(beatles)
4
```

Ejercicio

pycheck: **diverse_word**

Iterar sobre un conjunto

Tal y como hemos visto para otros tipos de datos *iterables*, la forma de recorrer los elementos de un conjunto es utilizar la sentencia `for`:

```
>>> for beatle in beatles:
...     print(beatle)
...
Harrison
McCartney
Starr
Lennon
```

Consejo: Como en el ejemplo anterior, es muy común utilizar una *variable en singular* para recorrer un iterable (en plural). No es una regla fija ni sirve para todos los casos, pero sí suele ser una *buena práctica*.

Pertenencia de un elemento

Al igual que con otros tipos de datos, Python nos ofrece el operador `in` para determinar si un elemento pertenece a un conjunto:

```
>>> beatles
{'Harrison', 'Lennon', 'McCartney', 'Starr'}

>>> 'Lennon' in beatles
True

>>> 'Fari' in beatles
False
```

Obviamente también disponemos de la «negación» del operador:

```
>>> 'Fari' not in beatles
True
```

Ejercicio

pycheck: **half_out**

Ordenando un conjunto

Ya hemos comentado que los conjuntos **no mantienen un orden**. ¿Pero qué ocurre si intentamos ordenarlo?

```
>>> marks = {8, 4, 6, 2, 9, 5}

>>> sorted(marks)
[2, 4, 5, 6, 8, 9]
```

Obtenemos **una lista con los elementos ordenados**.

Hay que tener en cuenta que, lógicamente, no podremos hacer uso de la función `sort()` sobre un conjunto:

```
>>> marks.sort()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'set' object has no attribute 'sort'
```

5.4.4 Teoría de conjuntos

Vamos a partir de dos conjuntos $A = \{1, 2\}$ y $B = \{2, 3\}$ para ejemplificar las distintas operaciones que se pueden hacer entre ellos basadas en los Diagramas de Venn y la Teoría de Conjuntos:

```
>>> A = {1, 2}
>>> B = {2, 3}
```

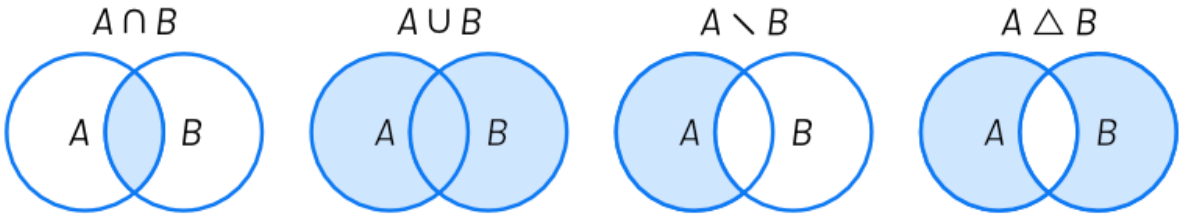


Figura 9: Diagramas de Venn

Intersección

$A \cap B$ – Elementos que están a la vez en A y en B:

```
>>> A & B
{2}

>>> A.intersection(B)
{2}
```

Unión



$A \cup B$ – Elementos que están tanto en A como en B:

```
>>> A | B
{1, 2, 3}

>>> A.union(B)
{1, 2, 3}
```

Diferencia



$A \setminus B$ – Elementos que están en A y no están en B:

```
>>> A - B
{1}

>>> A.difference(B)
{1}
```

Diferencia simétrica

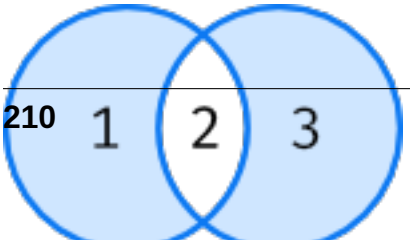


$A \triangle B$ – Elementos que están en A o en B pero no en ambos conjuntos:

```
>>> A ^ B
{1, 3}

>>> A.symmetric_difference(B)
{1, 3}
```

Podemos comprobar que la definición de la diferencia simétrica se cumple también en Python:



```
>>> A ^ B == (A | B) - (A & B)
True
```

Inclusión

- Un conjunto B es un **subconjunto** de otro conjunto A si todos los elementos de B están incluidos en A.
- Un conjunto A es un **superconjunto** de otro conjunto B si todos los elementos de B están incluidos en A.

Veamos un ejemplo con los siguientes conjuntos:

```
>>> A = {2, 4, 6, 8, 10}
>>> B = {4, 6, 8}
```

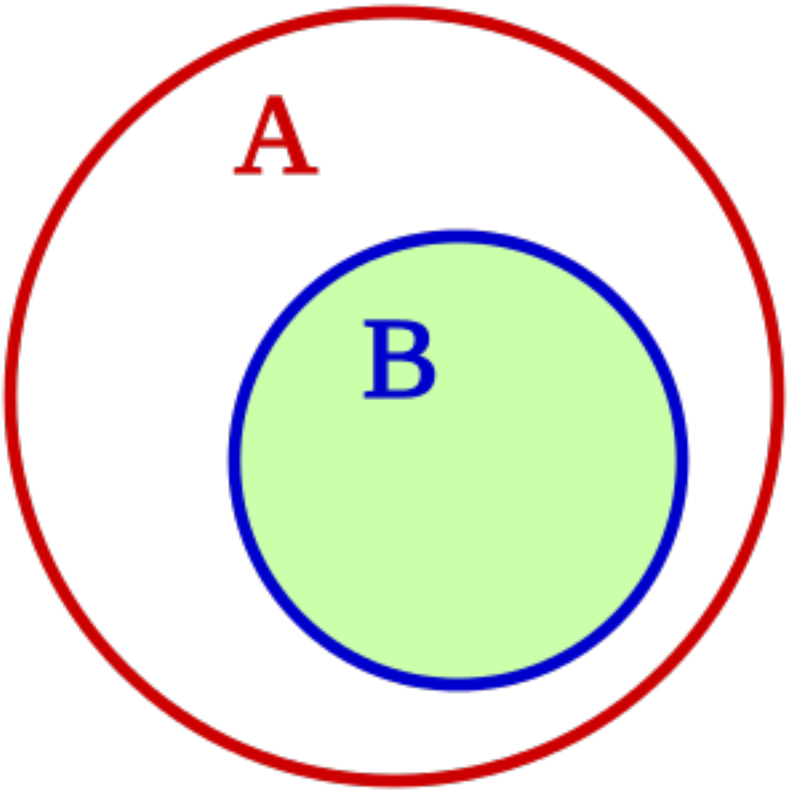


Figura 10: Subconjuntos y Superconjuntos

En Python podemos realizar comprobaciones de inclusión (subconjuntos y superconjuntos) utilizando operadores clásicos de comparación:

$B \subset A$

```
>>> B < A  # subconjunto
True
```

$B \subseteq A$

```
>>> B <= A
True
```

$A \supset B$

```
>>> A > B  # superconjunto
True
```

$A \supseteq B$

```
>>> B >= A
True
```

El hecho de que algunos elementos sí pertenezcan a otro conjunto no hace que sea un subconjunto. En el siguiente ejemplo tanto 3 como 5 del conjunto B están en el conjunto A, pero al no estar el elemento 1 no se trata de un subconjunto:

```
>>> A = {3, 5, 7, 9}
>>> B = {1, 3, 5}
```

```
>>> B < A
False
```

5.4.5 Conjuntos por comprensión

Los conjuntos, al igual que las *listas* y los *diccionarios*, también se pueden crear por comprensión.

Veamos un ejemplo en el que construimos un conjunto por comprensión con aquellos números enteros múltiplos de 3 en el rango $[0, 20)$:

```
>>> m3 = {number for number in range(0, 20) if number % 3 == 0}

>>> m3
{0, 3, 6, 9, 12, 15, 18}
```

Ejercicio

pycheck: **common_consonants**

5.4.6 Conjuntos inmutables

Python ofrece la posibilidad de crear **conjuntos inmutables** haciendo uso de la función `frozenset()` que recibe cualquier iterable como argumento.

Supongamos que recibimos una serie de calificaciones de exámenes y queremos crear un conjunto inmutable con los posibles niveles (categorías) de calificaciones:

```
>>> marks = [1, 3, 2, 3, 1, 4, 2, 4, 5, 2, 5, 5, 3, 1, 4]

>>> marks_levels = frozenset(marks)

>>> marks_levels
frozenset({1, 2, 3, 4, 5})
```

Veamos qué ocurre si intentamos modificar este conjunto:

```
>>> marks_levels.add(50)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'frozenset' object has no attribute 'add'
```

Un ejemplo en el que podemos aplicar un «frozenset» sería el conjunto de **posibles piezas del ajedrez**. Sabemos que estas piezas siempre son las mismas:

```
>>> CHESS_PIECES = frozenset(('King', 'Queen', 'Bishop', 'Knight', 'Rook', 'Pawn'))

>>> CHESS_PIECES
frozenset({'Bishop', 'King', 'Knight', 'Pawn', 'Queen', 'Rook'})
```

Nota: Los `frozenset` son a los `sets` lo que las tuplas a las listas: una forma de «congelar» los valores para que no se puedan modificar.

EJERCICIOS DE REPASO

1. pycheck: **is_binary**

AMPLIAR CONOCIMIENTOS

- Sets in Python

5.5 Ficheros



Aunque los ficheros encajarían más en un apartado de «*entrada/salida*» ya que representan un **medio de almacenamiento persistente**, también podrían ser vistos como *estructuras de datos*, puesto que nos permiten guardar la información y asignarles un cierto formato.[1]

Un **fichero** es un *conjunto de bytes* almacenados en algún *dispositivo*. El sistema de ficheros es la estructura lógica que alberga los ficheros y está jerarquizado a través de *directorios* (o carpetas). **Cada fichero se identifica unívocamente a través de una ruta que nos permite acceder a él**.

[1] Foto original de portada por Maksym Kaharlytskyi en Unsplash.

5.5.1 Lectura de un fichero

Python ofrece la función `open()` para «abrir» un fichero. Esta apertura se puede realizar en 3 modos distintos:

- **Lectura** del contenido de un fichero existente.
- **Escritura** del contenido en un fichero nuevo.
- **Añadido** al contenido de un fichero existente.

Veamos un ejemplo para leer el contenido de un fichero en el que se encuentran las temperaturas mínimas y máximas de cada día de la última semana. El fichero está en la subcarpeta (*ruta relativa*) `files/temps.dat` y tiene el siguiente contenido:

```
23 29
23 31
26 34
23 33
22 29
22 28
22 28
```

Lo primero será abrir el fichero:

```
>>> f = open('files/temps.dat', 'r')
```

La función `open()` recibe como primer argumento la **ruta al fichero** que queremos manejar (como un «string») y como segundo argumento el **modo de apertura** (también como un «string»). Nos **devuelve el manejador del fichero**, que en este caso lo estamos asignando a una variable llamada `f` pero le podríamos haber puesto cualquier otro nombre.

Nota: Es importante dominar los conceptos de **ruta relativa** y **ruta absoluta** para el trabajo con ficheros. Véase este artículo de DeNovatoANovato.

El **manejador del fichero** se implementa mediante un flujo de entrada/salida para las operaciones de lectura/escritura. Este objeto almacena, entre otras cosas, la *ruta al fichero*, el *modo de apertura* y la *codificación*:

```
>>> f
<_io.TextIOWrapper name='files/temps.dat' mode='r' encoding='UTF-8'>
```

Truco: Existen muchas codificaciones de caracteres para ficheros, pero la más utilizada es UTF-8 ya que es capaz de representar cualquier caracter Unicode al utilizar una longitud variable de 1 a 4 bytes.

Hay que tener en cuenta que la ruta al fichero que abrimos (*en modo lectura*) **debe existir**, ya que de lo contrario obtendremos un error:

```
>>> f = open('foo.txt', 'r')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'foo.txt'
```

Una vez abierto el fichero ya podemos proceder a leer su contenido. Para ello Python nos ofrece la posibilidad de leer todo el fichero de una vez o bien leerlo línea a línea.

Lectura completa de un fichero

Siguiendo con nuestro ejemplo de temperaturas, veamos cómo leer todo el contenido del fichero de una sola vez. Para esta operación, Python nos provee, al menos, de dos funciones:

`read()` Devuelve todo el contenido del fichero como una cadena de texto (`str`):

```
>>> # Podemos obviar 'r' ya que es el modo por defecto!
>>> f = open('files/temps.dat')

>>> f.read()
'23 29\n23 31\n26 34\n23 33\n22 29\n22 28\n22 28\n'
```

`readlines()` Devuelve todo el contenido del fichero como una lista (`list`) donde cada línea es un elemento de la lista:

```
>>> f = open('files/temps.dat')

>>> f.readlines()
['23 29\n', '23 31\n', '26 34\n', '23 33\n', '22 29\n', '22 28\n', '22 28\n']
```

Importante: Nótese que, en ambos casos, los saltos de línea `\n` siguen apareciendo en los datos leídos, por lo que habría que «limpiar» estos caracteres. Para ello se recomienda utilizar *las funciones ya vistas de cadenas de texto*.

Lectura línea a línea

Hay situaciones en las que interesa leer el contenido del fichero línea a línea. Imaginemos un fichero de tamaño considerable (varios GB). Si intentamos leer completamente este fichero de sola una vez podríamos ocupar demasiada RAM y reducir el rendimiento de nuestra máquina.

Es por ello que Python nos ofrece varias aproximaciones a la lectura de ficheros línea a línea. La más usada es iterar sobre el propio *manejador* del fichero, ya que los ficheros son estructuras de datos **iterables**:

```
>>> f = open('files/temps.dat')

>>> for line in f:    # that easy!
...     print(line)
...
23 29

23 31

26 34

23 33

22 29

22 28

22 28
```

Truco: Igual que pasaba anteriormente, la lectura línea por línea también incluye el **salto de línea** `\n` lo que provoca un «doble espacio» entre cada una de las salidas. Bastaría con aplicar `line.strip()` para eliminarlo.

Lectura de una línea

Hay ocasiones en las que nos interesa leer únicamente una sola línea. Es cierto que esto se puede conseguir mediante la aproximación anterior. Sería algo como:

```
>>> f = open('files/temps.dat')

>>> for line in f:
...     print(line)
```

(continué en la próxima página)

(proviene de la página anterior)

```
...     break
...
23 29
```

Pero Python también ofrece la función `readline()` que nos devuelve la siguiente línea del fichero:

```
>>> f = open('files/temps.dat')

>>> f.readline()
'23 29\n'
```

Es importante señalar que cuando utilizamos la función `readline()` **el «puntero de lectura» se desplaza a la siguiente línea del fichero**, con lo que podemos seguir cargando la información según nos interese:

```
>>> f = open('files/temps.dat')

>>> # Lectura de las 3 primeras líneas
>>> for _ in range(3):
...     print(f.readline().strip())
...
23 29
23 31
26 34

>>> # Lectura de las restantes líneas (4)
>>> for line in f:
...     print(line.strip())
...
23 33
22 29
22 28
22 28
```

La función `readline()` devuelve la **cadena vacía** cuando ha llegado (puntero de lectura) al final del fichero. Con esta premisa podemos implementar una forma **poco ortodoxa** de recorrer un fichero usando la función `readline()`:

```
>>> f = open('files/temps.dat')

>>> while line := f.readline():  # No hagas esto!
...     print(line.strip())
...
23 29
```

(continué en la próxima página)

(proviene de la página anterior)

```
23 31
26 34
23 33
22 29
22 28
22 28
```

Los ficheros se agotan

Hay que tener en cuenta que, una vez que leemos un fichero, no lo podemos volver a leer «directamente». O dicho de otra manera, el iterable que lleva implícito «se agota».

Veamos este escenario con el ejemplo anterior:

```
>>> f = open('files/temps.dat')

>>> for line in f:
...     print(line.strip(), end=' ')
...
23 29 23 31 26 34 23 33 22 29 22 28 22 28

>>> for line in f:
...     print(line.strip(), end=' ')
... # No hay salida!!
```

Esto mismo ocurre si utilizamos funciones como `read()` o `readlines()`.

Advertencia: Por este motivo y también por cuestiones de legibilidad del código, deberíamos abrir un fichero una única vez y realizar todas las operaciones de lectura necesarias, siempre que las circunstancias lo permitan.

Hay una posibilidad de **volver a leer desde el principio** y es utilizando la función `seek()`. Esta función permite situar el «puntero de lectura» en cualquier *byte* del fichero. Veamos cómo usarlo:

```
>>> f = open('files/temps.dat')

>>> for line in f:
...     print(line.strip(), end=' ')
...
23 29 23 31 26 34 23 33 22 29 22 28 22 28
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> f.seek(0)  # desplazamiento al principio
0

>>> for line in f:
...     print(line.strip(), end=' ')
...
23 29 23 31 26 34 23 33 22 29 22 28 22 28
```

Enumerando líneas

En ocasiones no sólo necesitamos recorrer cada línea del fichero sino también ir llevando un «índice» que nos indique el número de línea que estamos procesando.

Dado que los manejadores de ficheros también son objetos *iterables* podemos hacer uso de *enumerate()*. Veamos un ejemplo:

```
>>> f = open('files/temps.dat')

>>> for line_no, line in enumerate(f, start=1):
...     print(f'L{line_no}: {line.strip()}')
...
L1: 23 29
L2: 23 31
L3: 26 34
L4: 23 33
L5: 22 29
L6: 22 28
L7: 22 28
```

5.5.2 Escritura en un fichero

Para escribir texto en un fichero hay que abrir dicho fichero en **modo escritura**. Para ello utilizamos el *argumento adicional* en la función `open()` que indica esta operación:

```
>>> f = open('files/canary-iata.dat', 'w')
```

Nota: Si bien el fichero en sí mismo se crea al abrirlo en modo escritura, la **ruta** hasta ese fichero no. Eso quiere decir que debemos asegurarnos que **las carpetas hasta llegar a dicho fichero existen**. En otro caso obtenemos un error de tipo `FileNotFoundError`.

Ahora ya podemos hacer uso de la función `write()` para enviar contenido al fichero abierto.

Supongamos que queremos volcar el contenido de una lista/tupla en dicho fichero. En este caso partimos de los *códigos IATA* de aeropuertos de las Islas Canarias[2].

```
1 >>> canary_iata = ('TFN', 'TFS', 'LPA', 'GMZ', 'VDE', 'SPC', 'ACE', 'FUE')
2
3 >>> for code in canary_iata:
4 ...     f.write(code + '\n')
5 ...
6
7 >>> f.close()
```

Nótese:

Línea 4 Escritura de cada código en el fichero. La función `write()` no incluye el salto de línea por defecto, así que lo añadimos de *manera explícita*.

Línea 7 Cierre del fichero con la función `close()`. Especialmente en el caso de la escritura de ficheros, se recomienda encarecidamente cerrar los ficheros para evitar pérdida de datos.

Advertencia: Siempre que se abre un fichero en **modo escritura** utilizando el argumento `'w'`, el fichero se inicializa, borrando cualquier contenido que pudiera tener.

Otra forma de **escribir la tupla «de una sola vez»** podría ser utilizando la función `join()` con el *salto de línea* como separador:

```
>>> canary_iata = ('TFN', 'TFS', 'LPA', 'GMZ', 'VDE', 'SPC', 'ACE', 'FUE')

>>> f = open('files/canary-iata.dat', 'w')

>>> f.write('\n'.join(canary_iata))

>>> f.close()
```

En el caso de que ya tengamos una **lista (iterable) cuyos elementos tengan el formato de salida que necesitamos** (incluyendo salto de línea si así fuera necesario) podemos utilizar la función `writelines()` que nos ofrece Python.

Siguiendo con el ejemplo anterior, imaginemos un escenario en el que la tupla ya contiene los saltos de línea:

```
>>> canary_iata = ('TFN\n', 'TFS\n', 'LPA\n', 'GMZ\n', 'VDE\n', 'SPC\n', 'ACE\n',
↪'FUE\n')
```

(continué en la próxima página)

[2] Fuente: Smart Drone

(proviene de la página anterior)

```
>>> f = open('files/canary-iata.dat', 'w')

>>> f.writelines(canary_iata)

>>> f.close()
```

Truco: Esta aproximación puede ser interesante cuando leemos de un fichero y escribimos en otro ya que las líneas «vienen» con el salto de línea ya incorporado.

5.5.3 Añadido a un fichero

La única diferencia entre añadir información a un fichero y *escribir información en un fichero* es el modo de apertura del fichero. En este caso utilizamos `'a'` por «append»:

```
>>> f = open('more-data.txt', 'a')
```

En este caso el fichero `more-data.txt` se abrirá en *modo añadir* con lo que las llamadas a la función `write()` hará que aparezcan nueva información al final del contenido ya existente en dicho fichero.

5.5.4 Usandos contextos

Python ofrece *gestores de contexto* como una solución para establecer reglas de entrada y salida a un determinado bloque de código.

En el caso que nos ocupa, usaremos la sentencia `with` y el contexto creado se ocupará de cerrar adecuadamente el fichero que hemos abierto, liberando así sus recursos:

```
>>> with open('files/temps.dat') as f:
...     for line in f:
...         min_temp, max_temp = line.strip().split()
...         print(min_temp, max_temp)
...
23 29
23 31
26 34
23 33
22 29
22 28
22 28
```

Línea 1 Apertura del fichero en *modo lectura* utilizando el gestor de contexto definido por la palabra reservada `with`.

Línea 2 Lectura del fichero línea a línea utilizando la iteración sobre el *manejador del fichero*.

Línea 3 Limpieza de saltos de línea con `strip()` encadenando la función `split()` para separar las dos temperaturas por el carácter *espacio*. Ver *limpiar una cadena* y *dividir una cadena*.

Línea 4 Imprimir por pantalla la temperatura mínima y la máxima.

Nota: Es una buena práctica usar `with` cuando se manejan ficheros. La ventaja es que el fichero se cierra adecuadamente en cualquier circunstancia, incluso si se produce cualquier **tipo de error**.

Hay que prestar atención a la hora de escribir valores numéricos en un fichero, ya que el método `write()` por defecto espera ver un «string» como argumento:

```
>>> lottery = [43, 21, 99, 18, 37, 99]

>>> with open('files/lottery.dat', 'w') as f:
...     for number in lottery:
...         f.write(number)
...
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
TypeError: write() argument must be str, not int
```

Importante: Para evitar este tipo de **errores**, se debe convertir a `str` aquellos valores que queramos usar con la función `write()` para escribir información en un fichero de texto. Los *f-strings* son tu aliado.

EJERCICIOS DE REPASO

1. pycheck: **wc**
2. pycheck: **read_csv**
3. pycheck: **txt2md**
4. pycheck: **avg_temps**
5. pycheck: **find_words**
6. pycheck: **sum_matrix**

7. pycheck: **longest_word**
8. pycheck: **word_freq**
9. pycheck: **get_line**
10. pycheck: **replace_chars**
11. pycheck: **histogram**
12. pycheck: **submarine**
13. pycheck: **common_words**

AMPLIAR CONOCIMIENTOS

- Reading and Writing Files in Python
- Python Context Managers and the «with» Statement

CAPÍTULO 6

Modularidad

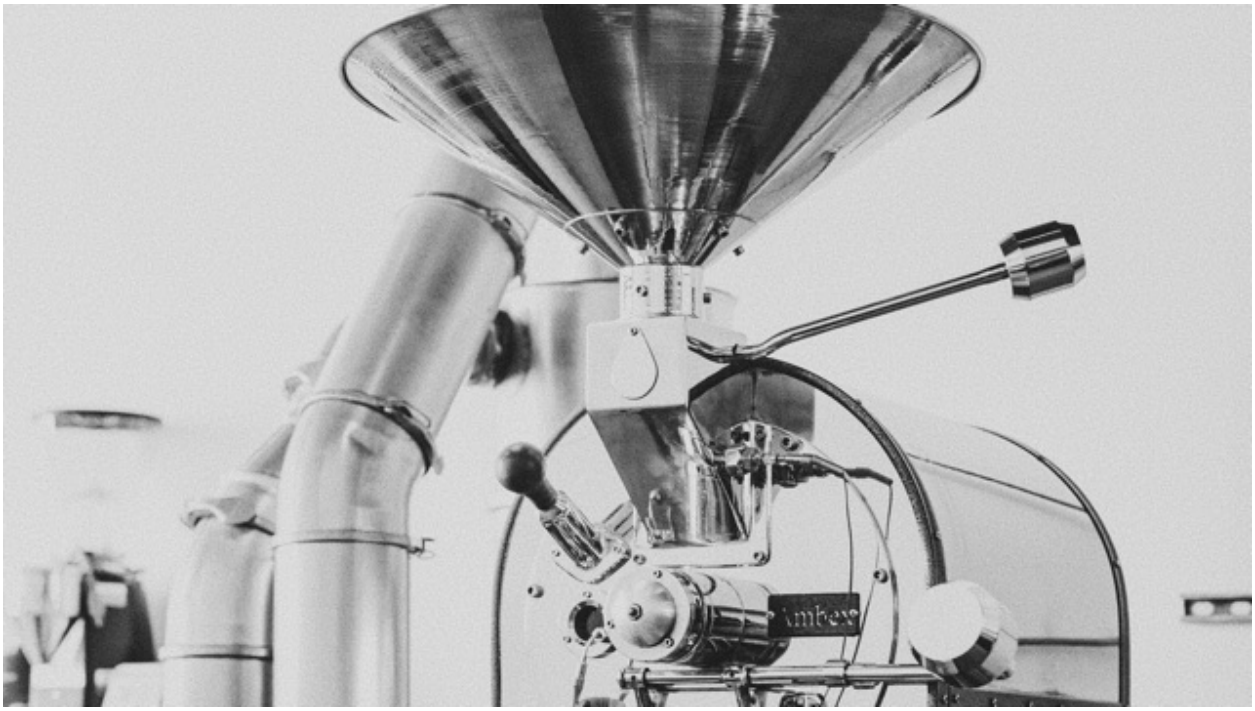
La **modularidad** es la característica de un sistema que permite que sea estudiado, visto o entendido como la unión de varias partes que interactúan entre sí y que trabajan solidariamente para alcanzar un objetivo común, realizando cada una de ellas una tarea necesaria para la consecución de dicho objetivo.

Cada una de esas partes en que se encuentre dividido el sistema recibe el nombre de **módulo**. Idealmente un módulo debe poder cumplir las condiciones de caja negra, es decir, ser independiente del resto de los módulos y comunicarse con ellos (con todos o sólo con una parte) a través de entradas y salidas bien definidas.[1]

En este capítulo veremos las facilidades que nos proporciona Python para trabajar en la línea de modularidad del código.

[1] Definición de modularidad en Wikipedia

6.1 Funciones



El concepto de **función** es básico en prácticamente cualquier lenguaje de programación. Se trata de una estructura que nos permite agrupar código. Persigue dos objetivos claros:

1. **No repetir** fragmentos de código en un programa.
2. **Reutilizar** el código en distintos escenarios.

Una función viene *definida* por su *nombre*, sus *parámetros* y su *valor de retorno*. Esta parametrización de las funciones las convierten en una poderosa herramienta ajustable a las circunstancias que tengamos. Al *invocarla* estaremos solicitando su ejecución y obtendremos unos resultados.[1]

6.1.1 Definir una función

Para definir una función utilizamos la palabra reservada `def` seguida del **nombre**[6] de la función. A continuación aparecerán 0 o más **parámetros** separados por comas (entre paréntesis), finalizando la línea con **dos puntos** : En la siguiente línea empezaría el **cuerpo** de la función que puede contener 1 o más **sentencias**, incluyendo (o no) una **sentencia de retorno** con el resultado mediante `return`.

[1] Foto original por Nathan Dumlao en Unsplash.

[6] Las *reglas aplicadas a nombres de variables* también se aplican a nombres de funciones.

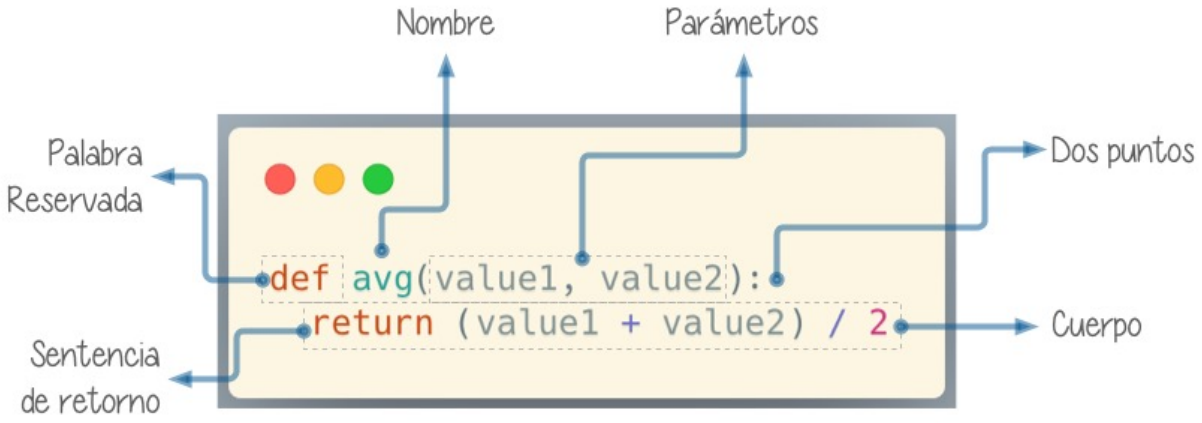


Figura 1: Definición de una función en Python

Advertencia: Prestar especial atención a los dos puntos : porque suelen olvidarse en la *definición de la función*.

Hagamos una primera función sencilla que no recibe parámetros:

```
def say_hello():
    print('Hello!')
```

Nota: Nótese la *indentación* (sangrado) del *cuerpo* de la función.

Los **nombres de las funciones** siguen *las mismas reglas que las variables* y, como norma general, se suelen utilizar **verbos en infinitivo** para su definición: `load_data`, `store_values`, `reset_cart`, `filter_results`, `block_request`, …

Invocar una función

Para invocar (o «llamar») a una función sólo tendremos que escribir su nombre seguido de paréntesis. En el caso de la función sencilla (vista anteriormente) se haría así:

```
>>> def say_hello():
...     print('Hello!')
...

>>> say_hello()
Hello!
```

Nota: Como era de esperar, al invocar a esta función obtenemos un mensaje por pantalla, fruto de la ejecución del cuerpo de la función.

Cuando queremos **invocar a una función dentro de un fichero** `*.py` lo haremos del mismo modo que hemos visto en el intérprete interactivo:

```
def say_hello():
    print('Hello!')

# Llamada a la función (primer nivel de indentación)
say_hello()
```

Importante: La función debe estar definida **antes** del punto en el que sea llamada.

Retornar un valor

Las funciones pueden retornar (o «devolver») un valor. Veamos un ejemplo muy sencillo:

```
>>> def one():
...     return 1
...

>>> one()
1
```

Importante: No confundir `return` con `print()`. El valor de retorno de una función nos permite usarlo fuera de su contexto. El hecho de añadir `print()` al cuerpo de una función es algo «coyuntural» y no modifica el resultado de la lógica interna.

Nota: En la sentencia `return` podemos incluir variables y expresiones, no únicamente literales.

Pero no sólo podemos invocar a la función directamente, también la podemos asignar a variables y utilizarla:

```
>>> value = one()

>>> print(value)
1
```

También la podemos integrar en otras expresiones, por ejemplo en condicionales:

```
>>> if one() == 1:
...     print('It works!')
... else:
...     print('Something is broken')
...
It works!
```

Si una función no incluye un `return` de forma explícita, devolverá *None* de forma implícita:

```
>>> def empty():
...     x = 0
...     # return None

>>> print(empty())
None
```

Existe la posibilidad de usar la sentencia `return` «a secas» (que también devuelve `None`) y hace que «salgamos» inmediatamente de la función:

```
>>> def quick():
...     return
...

>>> print(quick())
None
```

Advertencia: En general, esto **no se considera una buena práctica** salvo que sepamos lo que estamos haciendo. Si la función debe devolver `None` es preferible ser **explícito** y utilizar `return None`. Aunque es posible que en ciertos escenarios nos interese dicha aproximación.

Retornando múltiples valores

Una función puede retornar más de un valor. El «secreto» es hacerlo **mediante una tupla**:

```
>>> def multiple():
...     return 0, 1  # es una tupla!
...
```

Veamos qué ocurre si invocamos a esta función:

```
>>> result = multiple()

>>> result
(0, 1)

>>> type(result)
tuple
```

Por lo tanto, podremos aplicar el *desempaquetado de tuplas* sobre el valor retornado por la función:

```
>>> a, b = multiple()

>>> a
0

>>> b
1
```

6.1.2 Parámetros y argumentos

Si una función no dispusiera de valores de entrada, su comportamiento quedaría muy limitado. Es por ello que los **parámetros** nos permiten variar los datos que consume una función para obtener distintos resultados. Vamos a empezar a crear funciones que reciben **parámetros**.

En este caso escribiremos una función que recibe un valor numérico y devuelve su raíz cuadrada:

```
>>> def sqrt(value):
...     return value ** (1/2)
...

>>> sqrt(4)
2.0
```

Nota: En este caso, el valor `4` es un **argumento** de la función.

Cuando llamamos a una función con *argumentos*, los valores de estos argumentos se copian en los correspondientes *parámetros* dentro de la función:

Truco: La sentencia `pass` permite «no hacer nada». Es una especie de «*placeholder*».

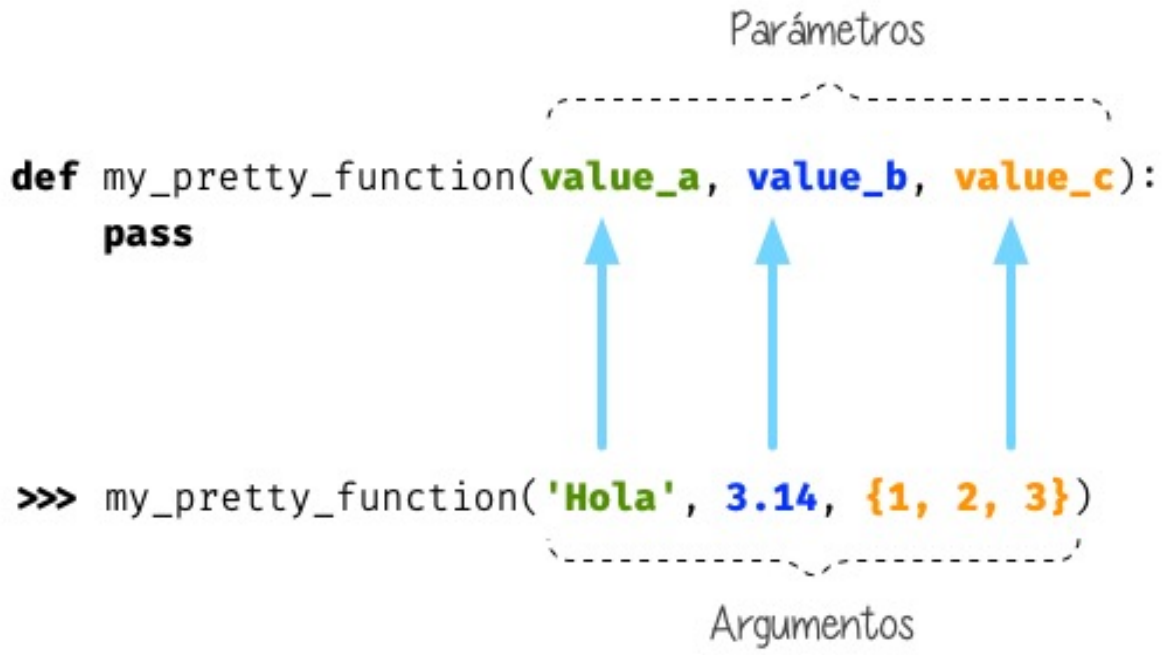


Figura 2: Parámetros y argumentos de una función

Veamos otra función con dos parámetros y algo más de lógica de negocio:[2]

```
>>> def _min(a, b):
...     if a < b:
...         return a
...     else:
...         return b
...

>>> _min(7, 9)
7
```

Nótese que la sentencia `return` puede escribirse en **múltiples ocasiones** y puede encontrarse en **cualquier lugar** de la función, no necesariamente al final del cuerpo. Esta técnica puede ser beneficiosa en distintos escenarios.

Uno de esos escenarios se relaciona con el concepto de **cláusula guarda**: una pieza de código que normalmente está al comienzo de la función y comprueba una serie de condiciones para continuar con la ejecución o cortarla[10].

[2] Término para identificar el «algoritmo» o secuencia de instrucciones derivadas del procesamiento que corresponda.

[10] Para más información sobre las cláusulas guarda, véase este artículo de Miguel G. Flores

Teniendo en cuenta que la sentencia `return` finaliza la ejecución de una función, es viable **eliminar la sentencia** `else` del ejemplo visto anteriormente:

```
>>> def _min(a, b):
...     if a < b:
...         return a
...     return b

>>> _min(7, 9)
7
```

Ejercicio

pycheck: **squared_sum**

Argumentos posicionales

Los **argumentos posicionales** son aquellos argumentos que se copian en sus correspondientes parámetros **por orden de escritura**.

Vamos a mostrar un ejemplo definiendo una función que construye una «cpu» a partir de 3 parámetros:

```
>>> def build_cpu(vendor, num_cores, freq):
...     return dict(
...         vendor=vendor,
...         num_cores=num_cores,
...         freq=freq
...     )
...
```

Una posible llamada a la función con argumentos posicionales sería la siguiente:

```
>>> build_cpu('AMD', 8, 2.7)
{'vendor': 'AMD', 'num_cores': 8, 'freq': 2.7}
```

Lo que ha sucedido es un **mapeo** directo entre argumentos y parámetros en el mismo orden que estaban definidos:

Parámetro	Argumento
vendor	AMD
num_cores	8
freq	2.7

Pero es evidente que una clara desventaja del uso de argumentos posicionales es que se necesita **recordar el orden** de los argumentos. Un error en la posición de los argumentos puede generar resultados indeseados:

```
>>> build_cpu(8, 2.7, 'AMD')
{'vendor': 8, 'num_cores': 2.7, 'freq': 'AMD'}
```

Argumentos nominales

En esta aproximación los argumentos no son copiados en un orden específico sino que **se asignan por nombre a cada parámetro**. Ello nos permite evitar el problema de conocer cuál es el orden de los parámetros en la definición de la función. Para utilizarlo, basta con realizar una asignación de cada argumento en la propia llamada a la función.

Veamos la misma llamada que hemos hecho en el ejemplo de construcción de la «cpu» pero ahora utilizando paso de argumentos nominales:

```
>>> build_cpu(vendor='AMD', num_cores=8, freq=2.7)
{'vendor': 'AMD', 'num_cores': 8, 'freq': 2.7}
```

Se puede ver claramente que el orden de los argumentos no influye en el resultado final:

```
>>> build_cpu(num_cores=8, freq=2.7, vendor='AMD')
{'vendor': 'AMD', 'num_cores': 8, 'freq': 2.7}
```

Argumentos posicionales y nominales

Python permite **mezclar argumentos posicionales y nominales** en la llamada a una función:

```
>>> build_cpu('INTEL', num_cores=4, freq=3.1)
{'vendor': 'INTEL', 'num_cores': 4, 'freq': 3.1}
```

Pero hay que tener en cuenta que, en este escenario, **los argumentos posicionales siempre deben ir antes** que los argumentos nominales. Esto tiene mucho sentido ya que, de no hacerlo así, Python no tendría forma de discernir a qué parámetro corresponde cada argumento:

```
>>> build_cpu(num_cores=4, 'INTEL', freq=3.1)
  File "<stdin>", line 1
SyntaxError: positional argument follows keyword argument
```

Argumentos mutables e inmutables

Nivel intermedio

Cuando realizamos modificaciones a los argumentos de una función es importante tener en cuenta si son **mutables** (listas, diccionarios, conjuntos, …) o **inmutables** (tuplas, enteros, flotantes, cadenas de texto, …) ya que podríamos obtener efectos colaterales no deseados.

Supongamos que nos piden escribir una función que reciba una lista y que devuelva sus valores elevados al cuadrado. Pero lo hacemos «malamente»:

```
>>> values = [2, 3, 4]

>>> def square_it(values):
...     # NO HAGAS ESTO
...     for i in range(len(values)):
...         values[i] **= 2
...     return values

>>> square_it(values)
[4, 9, 16]

>>> values  # ???
[4, 9, 16]
```

Advertencia: Esto **no es una buena práctica**. O bien documentar que el argumento puede modificarse o bien retornar un nuevo valor. Por regla general, no se recomienda que las funciones modifiquen argumentos de entrada, salvo que sea específicamente lo que estamos buscando.

Parámetros por defecto

Es posible especificar **valores por defecto** en los parámetros de una función. En el caso de que no se proporcione un valor al argumento en la llamada a la función, el parámetro correspondiente tomará el valor definido por defecto.

Siguiendo con el ejemplo de la «cpu», podemos asignar *2.0GHz* como frecuencia por defecto. La definición de la función cambiaría ligeramente:

```
>>> def build_cpu(vendor, num_cores, freq=2.0):
...     return dict(
...         vendor=vendor,
...         num_cores=num_cores,
...         freq=freq
```

(continué en la próxima página)

(proviene de la página anterior)

```
...     )
...
```

Llamada a la función sin especificar frecuencia de «cpu»:

```
>>> build_cpu('INTEL', 2)
{'vendor': 'INTEL', 'num_cores': 2, 'freq': 2.0}
```

Llamada a la función indicando una frecuencia concreta de «cpu»:

```
>>> build_cpu('INTEL', 2, 3.4)
{'vendor': 'INTEL', 'num_cores': 2, 'freq': 3.4}
```

Nivel intermedio

Es importante tener presente que los valores por defecto en los parámetros se calculan cuando se **define** la función, no cuando se **ejecuta**. Veamos un ejemplo siguiendo con el caso anterior:

```
>>> DEFAULT_FREQ = 2.0

>>> def build_cpu(vendor, num_cores, freq=DEFAULT_FREQ):
...     return dict(
...         vendor=vendor,
...         num_cores=num_cores,
...         freq=freq
...     )
...

>>> build_cpu('AMD', 4)
{'vendor': 'AMD', 'num_cores': 4, 'freq': 2.0}

>>> DEFAULT_FREQ = 3.5

>>> build_cpu('AMD', 4)
{'vendor': 'AMD', 'num_cores': 4, 'freq': 2.0}
```

Ejercicio

pycheck: **factorial**

Modificando parámetros mutables

Nivel avanzado

Hay que tener cuidado a la hora de manejar los parámetros que pasamos a una función ya que *podemos obtener resultados indeseados*, especialmente cuando trabajamos con *tipos de datos mutables*.

Supongamos una función que añade elementos a una lista que pasamos como argumento. La idea es que si no pasamos la lista, ésta siempre empiece siendo vacía. Hagamos una serie de pruebas pasando alguna lista como segundo argumento:

```
>>> def buggy(arg, result=[]):
...     result.append(arg)
...     print(result)
...

>>> buggy('a', [])
['a']

>>> buggy('b', [])
['b']

>>> buggy('a', ['x', 'y', 'z'])
['x', 'y', 'z', 'a']

>>> buggy('b', ['x', 'y', 'z'])
['x', 'y', 'z', 'b']
```

Aparentemente todo está funcionando de manera correcta, pero veamos qué ocurre en las siguientes llamadas:

```
>>> def buggy(arg, result=[]):
...     result.append(arg)
...     print(result)
...

>>> buggy('a')
['a']

>>> buggy('b')  # Se esperaría ['b']
['a', 'b']
```

Obviamente algo no ha funcionado correctamente. Se esperaría que `result` tuviera una lista vacía en cada ejecución. Sin embargo esto no sucede por estas dos razones:

1. El valor por defecto se establece cuando se define la función.

2. La variable `result` apunta a una zona de memoria en la que se modifican sus valores.

Ejecución **paso a paso** a través de *Python Tutor*:

https://cutt.ly/sBNpVT2

A riesgo de perder el *parámetro por defecto*, una posible solución sería la siguiente:

```
>>> def works(arg):
...     result = []
...     result.append(arg)
...     return result
...

>>> works('a')
['a']

>>> works('b')
['b']
```

La forma de arreglar el código anterior utilizando un parámetro con valor por defecto sería utilizar un **tipo de dato inmutable** y tener en cuenta cuál es la primera llamada:

```
>>> def nonbuggy(arg, result=None):
...     if result is None:
...         result = []
...     result.append(arg)
...     print(result)
...

>>> nonbuggy('a')
['a']

>>> nonbuggy('b')
['b']

>>> nonbuggy('a', ['x', 'y', 'z'])
['x', 'y', 'z', 'a']

>>> nonbuggy('b', ['x', 'y', 'z'])
['x', 'y', 'z', 'b']
```

Empaquetar/Desempaquetar argumentos

Nivel intermedio

Python nos ofrece la posibilidad de empaquetar y desempaquetar argumentos cuando estamos invocando a una función, tanto para **argumentos posicionales** como para **argumentos nominales**.

Y de esto se deriva el hecho de que podamos utilizar un **número variable de argumentos** en una función, algo que puede ser muy interesante según el caso de uso que tengamos.

Empaquetar/Desempaquetar argumentos posicionales

Si utilizamos el operador `*` delante del nombre de un parámetro posicional, estaremos indicando que los argumentos pasados a la función se empaqueten en una **tupla**.

Veamos un ejemplo en el que vamos a **implementar una función para sumar un número variable de valores**. La función que tenemos disponible en Python no cubre este caso:

```
>>> sum(4, 3, 2, 1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: sum() takes at most 2 arguments (4 given)
```

Para superar esta «limitación» vamos a hacer uso del `*` para empaquetar los argumentos posicionales:

```
>>> def _sum(*values):
...     print(f'{values=}')
...     result = 0
...     for value in values:  # values es una tupla
...         result += value
...     return result
...

>>> _sum(4, 3, 2, 1)
values=(4, 3, 2, 1)
10
```

Existe la posibilidad de usar el asterisco `*` en la llamada a la función para **desempaquetar** los argumentos posicionales:

```
>>> values = (4, 3, 2, 1)

>>> _sum(values)
Traceback (most recent call last):
```

(continué en la próxima página)

(proviene de la página anterior)

```
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 4, in _sum
TypeError: unsupported operand type(s) for +=: 'int' and 'tuple'

>>> # Desempaquetado: _sum(4, 3, 2, 1)
>>> _sum(*values)
values=(4, 3, 2, 1)
10
```

Empaquetar/Desempaquetar argumentos nominales

Si utilizamos el operador ** delante del nombre de un parámetro nominal, estaremos indicando que los argumentos pasados a la función se empaqueten en un **diccionario**.

Supongamos un ejemplo en el que queremos **encontrar la persona con mayor calificación de un examen**. Haremos uso del ** para empaquetar los argumentos nominales:

```
>>> def best_student(**marks):
...     print(f'{marks=}')
...     max_mark = -1
...     for student, mark in marks.items():  # marks es un diccionario
...         if mark > max_mark:
...             max_mark = mark
...             best_student = student
...     return best_student
...

>>> best_student(ana=8, antonio=6, inma=9, javier=7)
marks={'ana': 8, 'antonio': 6, 'inma': 9, 'javier': 7}
'inma'
```

Al igual que veíamos previamente, existe la posibilidad de usar doble asterisco ** en la llamada a la función para **desempaquetar** los argumentos nominales:

```
>>> marks = dict(ana=8, antonio=6, inma=9, javier=7)

>>> best_student(marks)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: best_student() takes 0 positional arguments but 1 was given

>>> # Desempaquetado: best_student(ana=8, antonio=6, inma=9, javier=7)
>>> best_student(**marks)
```

(continué en la próxima página)

(proviene de la página anterior)

```
marks={'ana': 8, 'antonio': 6, 'inma': 9, 'javier': 7}
'inma'
```

Convenciones

En muchas ocasiones se utiliza `args` como nombre de parámetro para argumentos posicionales y `kwargs` como nombre de parámetro para argumentos nominales. Esto son únicamente **convenciones**, no hay obligación de utilizar estos nombres. Así, podemos encontrar funciones definidas de la siguiente manera:

```
>>> def func(*args, **kwargs):
...     # TODO
...     pass
...
```

Forzando modo de paso de argumentos

Si bien Python nos da flexibilidad para pasar argumentos a nuestras funciones en modo nominal o posicional, existen opciones para forzar que dicho paso sea obligatorio para una determinada modalidad.

Argumentos sólo nominales

Nivel avanzado

A partir de Python 3.0 se ofrece la posibilidad de obligar a que determinados parámetros de la función sean pasados sólo por nombre.

Para ello, en la definición de los parámetros de la función, tendremos que incluir un parámetro especial `*` que delimitará el tipo de parámetros. Así, todos los parámetros a la derecha del separador estarán **obligados** a ser nominales:

Ejemplo:

```
>>> def sum_power(a, b, *, power=False):
...     if power:
...         a **= 2
...         b **= 2
...     return a + b
...

>>> sum_power(3, 4)
```

(continué en la próxima página)

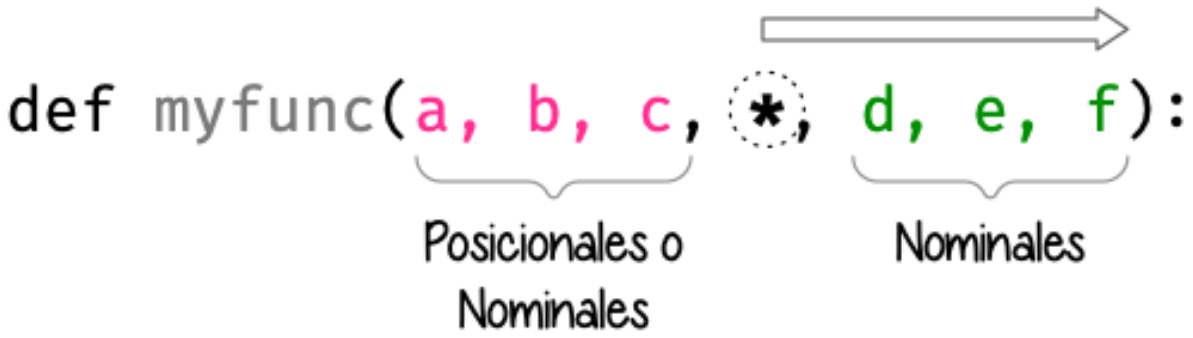


Figura 3: Separador para especificar parámetros sólo nominales

(proviene de la página anterior)

```
7

>>> sum_power(a=3, b=4)
7

>>> sum_power(3, 4, power=True)
25

>>> sum_power(3, 4, True)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: sum_power() takes 2 positional arguments but 3 were given
```

Argumentos sólo posicionales

Nivel avanzado

A partir de Python 3.8 se ofrece la posibilidad de obligar a que determinados parámetros de la función sean pasados sólo por posición.

Para ello, en la definición de los parámetros de la función, tendremos que incluir un parámetro especial `/` que delimitará el tipo de parámetros. Así, todos los parámetros a la izquierda del delimitador estarán **obligados** a ser posicionales:

Ejemplo:

```
>>> def sum_power(a, b, /, power=False):
...     if power:
...         a **= 2
...         b **= 2
```

(continué en la próxima página)



Figura 4: Separador para especificar parámetros sólo posicionales

(proviene de la página anterior)

```
...     return a + b
...

>>> sum_power(3, 4)
7

>>> sum_power(3, 4, True)
25

>>> sum_power(3, 4, power=True)
25

>>> sum_power(a=3, b=4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: sum_power() got some positional-only arguments passed as keyword␣
↪arguments: 'a, b'
```

Fijando argumentos posicionales y nominales

Si mezclamos las dos estrategias anteriores podemos forzar a que una función reciba argumentos de un modo concreto.

Continuando con el ejemplo anterior, podríamos hacer lo siguiente:

```
>>> def sum_power(a, b, /, *, power=False):
...     if power:
...         a **= 2
...         b **= 2
```

(continué en la próxima página)

(proviene de la página anterior)

```
...     return a + b
...

>>> sum_power(3, 4, power=True)  # Único modo posible de llamada
25
```

Ejercicio

pycheck: **consecutive_freqs**

Funciones como parámetros

Nivel avanzado

Las funciones se pueden utilizar en cualquier contexto de nuestro programa. Son objetos que pueden ser asignados a variables, usados en expresiones, devueltos como valores de retorno o pasados como argumentos a otras funciones.

Veamos un primer ejemplo en el que pasamos una función como argumento:

```
>>> def success():
...     print('Yeah!')
...

>>> type(success)
function

>>> def doit(f):
...     f()
...

>>> doit(success)
Yeah!
```

Veamos un segundo ejemplo en el que pasamos, no sólo una función como argumento, sino los valores con los que debe operar:

```
>>> def repeat_please(text, times=1):
...     return text * times
...

>>> type(repeat_please)
function
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> def doit(f, arg1, arg2):
...     return f(arg1, arg2)
...

>>> doit(repeat_please, 'Functions as params', 2)
'Functions as paramsFunctions as params'
```

6.1.3 Documentación

Ya hemos visto que en Python podemos incluir *comentarios* para explicar mejor determinadas zonas de nuestro código.

Del mismo modo podemos (y en muchos casos **debemos**) adjuntar **documentación** a la definición de una función incluyendo una cadena de texto (**docstring**) al comienzo de su cuerpo:

```
>>> def sqrt(value):
...     'Returns the square root of the value'
...     return value ** (1/2)
...
```

La forma más ortodoxa de escribir un `docstring` es utilizando *triples comillas*:

```
>>> def closest_int(value):
...     """Returns the closest integer to the given value.
...     The operation is:
...         1. Compute distance to floor.
...         2. If distance less than a half, return floor.
...            Otherwise, return ceil.
...     """
...     floor = int(value)
...     if value - floor < 0.5:
...         return floor
...     else:
...         return floor + 1
...
```

Para ver el `docstring` de una función, basta con utilizar `help`:

```
>>> help(closest_int)

Help on function closest_int in module __main__:

```

(continué en la próxima página)

(proviene de la página anterior)

```
closest_int(value)
    Returns the closest integer to the given value.
    The operation is:
        1. Compute distance to floor.
        2. If distance less than a half, return floor.
           Otherwise, return ceil.
```

También es posible extraer información usando el símbolo de interrogación:

```
>>> closest_int?
Signature: closest_int(value)
Docstring:
Returns the closest integer to the given value.
The operation is:
    1. Compute distance to floor.
    2. If distance less than a half, return floor.
    Otherwise, return ceil.
File:      ~/aprendepython/<ipython-input-75-5dc166360da1>
Type:      function
```

Importante: Esto no sólo se aplica a funciones propias, sino a cualquier otra función definida en el lenguaje.

Nota: Si queremos ver el `docstring` de una función en «crudo» (sin formatear), podemos usar `<function>.__doc__`.

Explicación de parámetros

Como ya se ha visto, es posible documentar una función utilizando un `docstring`. Pero la redacción y el formato de esta cadena de texto puede ser muy variada. Existen distintas formas de documentar una función (u otros objetos)[3]:

reStructuredText docstrings Formato de documentación recomendado por Python.

Google docstrings Formato de documentación recomendado por Google.

NumPy-SciPy docstrings Combinación de formatos reStructuredText y Google (usados por el proyecto NumPy).

Epytext docstrings Formato utilizado por Epydoc (una adaptación de Javadoc).

[3] Véase Docstring Formats.

Aunque cada uno tienes sus particularidades, todos comparten una misma estructura:

- Una primera línea de **descripción de la función**.
- A continuación especificamos las características de los **parámetros** (incluyendo sus tipos).
- Por último, indicamos si la función **retorna un valor** y sus características.

Aunque todos los formatos son válidos, nos centraremos en **reStructuredText** por ser el estándar propuesto por Python para la documentación.

Ver también:

Google docstrings y *Numpy docstrings* también son ampliamente utilizados, lo único es que necesitan de un módulo externo denominado Napoleon para que se puedan incluir en la documentación *Sphinx*.

Sphinx

Sphinx es una herramienta para generar documentación usando el lenguaje reStructuredText o RST. Incluye un módulo «built-in» denominado autodoc el cual permite la autogeneración de documentación a partir de los «docstrings» definidos en el código.

Veamos el uso de este formato en la documentación de la siguiente función:

```
>>> def my_power(x, n):
...     """Calculate x raised to the power of n.
...
...     :param x: number representing the base of the operation
...     :type x: int
...     :param n: number representing the exponent of the operation
...     :type n: int
...
...     :return: :math:`x^n`
...     :rtype: int
...     """
...     result = 1
...     for _ in range(n):
...         result *= x
...     return result
...
```

Dentro del «docstring» podemos escribir con sintaxis reStructuredText – véase por ejemplo la expresión matemática en el tag `:return:` – lo que nos proporciona una gran flexibilidad.

Nota: La plataforma Read the Docs aloja la documentación de gran cantidad de proyectos. En muchos de los casos se han usado «docstrings» con el formato Sphinx visto anteriormente.

Un ejemplo de ello es la popular librería de Python requests.

Anotación de tipos

Nivel intermedio

Las anotaciones de tipos o **type-hints**[5] se introdujeron en Python 3.5 y permiten indicar tipos para los parámetros de una función y/o para su valor de retorno (*aunque también funcionan en creación de variables*).

Veamos un ejemplo en el que creamos una función para dividir una cadena de texto por la posición especificada en el parámetro:

```
>>> def ssplit(text: str, split_pos: int) -> tuple:
...     return text[:split_pos], text[split_pos:]
...

>>> ssplit('Always remember us this way', 15)
('Always remember', ' us this way')
```

Como se puede observar, vamos añadiendo los tipos después de cada parámetro utilizando `:` como separador. En el caso del valor de retorno usamos la flecha `->`

Quizás la siguiente ejecución pueda sorprender:

```
>>> ssplit([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5)
([1, 2, 3, 4, 5], [6, 7, 8, 9, 10])
```

Efectivamente como habrás visto, **no hemos obtenido ningún error**, a pesar de que estamos pasando como primer argumento una lista en vez de una cadena de texto. Esto ocurre porque lo que hemos definido es simplemente una anotación de tipo, no una declaración de tipo. Existen herramientas como mypy que sí se encarga de comprobar este escenario.

Valores por defecto

Al igual que ocurre en la definición ordinaria de funciones, cuando usamos anotaciones de tipos también podemos indicar un valor por defecto para los parámetros.

Veamos la forma de hacerlo continuando con el ejemplo anterior:

```
>>> def ssplit(text: str, split_pos: int = -1) -> tuple:
...     if split_pos == -1:
...         split_pos = len(text) // 2
```

(continué en la próxima página)

[5] Conocidos como «type hints» en terminología inglesa.

(proviene de la página anterior)

```
...     return text[:split_pos], text[split_pos:]
...

>>> ssplit('Always remember us this way')
('Always rememb', 'er us this way')
```

Simplemente añadimos el valor por defecto después de indicar el tipo.

Las **anotaciones de tipos** son una herramienta muy potente y que, usada de forma adecuada, permite complementar la documentación de nuestro código y aclarar ciertos aspectos, que a priori, pueden parecer confusos. Su aplicación estará en función de la necesidad detectada por parte del equipo de desarrollo.

Tipos compuestos

Hay escenarios en los que necesitamos más expresividad de cara a la anotación de tipos. ¿Qué ocurre si queremos indicar una *lista de cadenas de texto* o un *conjunto de enteros*?

Veamos algunos ejemplos válidos:

Anotación	Ejemplo
`list[str]`	`['A', 'B', 'C']`
`set[int]`	`{4, 3, 9}`
`dict[str, float]`	`{'x': 3.786, 'y': 2.198, 'z': 4.954}`
`tuple[str, int]`	`('Hello', 10)`
`tuple[float, ...]`	`(1.23, 5.21, 3.62)` o `(4.31, 6.87)` o `(7.11,)`

Múltiples tipos

En el caso de que queramos indicar que un determinado parámetro puede ser de un tipo o de otro hay que especificarlo utilizando el operador[7] `|`.

Veamos algunos ejemplos válidos:

Anotación	Significado
`tuple|dict`	Tupla o diccionario
`list[str|int]`	Lista de cadenas de texto y/o enteros
`set[int|float]`	Conjunto de enteros y/o flotantes

[7] Disponible a partir de Python 3.10.

Ver también:

Guía rápida para de anotación de tipos (mypy)

Ejercicio

pycheck: **mcount**

Número indefinido

Cuando trabajamos con **parámetros que representan un número indefinido de valores**, las anotaciones de tipo sólo hacen referencia al tipo que contiene la tupla, no es necesario indicar que es una tupla.

En el siguiente ejemplo hay una función que calcula el máximo de una serie de valores enteros o flotantes, pero no indicamos que se reciben como tupla:

```
>>> def _max(*args: int | float):
...     ...
...
```

6.1.4 Tipos de funciones

Nivel avanzado

Funciones anónimas «lambda»

Una función lambda tiene las siguientes propiedades:

1. Se escribe en una única sentencia (línea).
2. No tiene nombre (anónima).
3. Su cuerpo conlleva un `return` implícito.
4. Puede recibir cualquier número de parámetros.

Veamos un primer ejemplo de función «lambda» que nos permite **contar el número de palabras en una cadena de texto** dada. La **transformación de su versión clásica en su versión anónima** sería la siguiente:

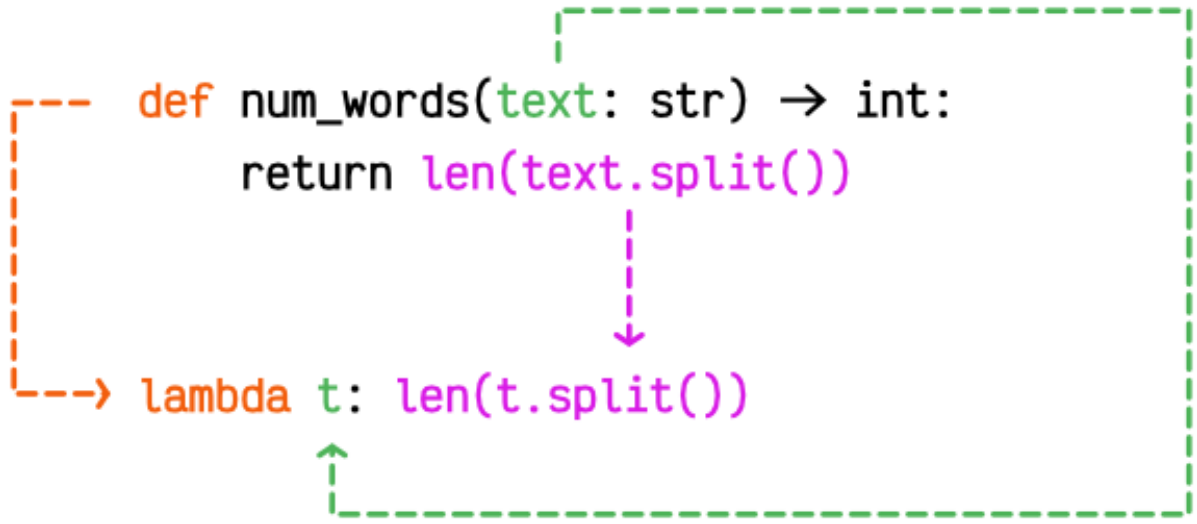


Figura 5: Transformación en función «lambda»

Prudencia: Aunque en muchas ocasiones se suelen «abreviar» los nombres de las variables en una función «lambda» no es obligatorio, y en muchos casos, puede que sea hasta contraproducente.

A continuación probamos el comportamiento de la función anónima «lambda»:

```
>>> num_words = lambda t: len(t.split())

>>> type(num_words)
function

>>> num_words
<function __main__.<lambda>(t)>

>>> num_words('hola socio vamos a ver')
5
```

Veamos otro ejemplo en el que mostramos una tabla con el resultado de aplicar el «and» lógico mediante una función «lambda» que ahora recibe dos parámetros:

```
>>> logic_and = lambda x, y: x & y

>>> for i in range(2):
...     for j in range(2):
...         print(f'{i} & {j} = {logic_and(i, j)}')
...
```

(continué en la próxima página)

(proviene de la página anterior)

```
0 & 0 = 0
0 & 1 = 0
1 & 0 = 0
1 & 1 = 1
```

Lambdas como argumentos

Las funciones «lambda» son bastante utilizadas **como argumentos a otras funciones**. Un ejemplo claro de ello es la función `sorted` que recibe un parámetro opcional `key` donde se define la clave de ordenación.

Veamos cómo usar una función anónima «lambda» para ordenar una tupla de pares *longitud-latitud*:

```
>>> geoloc = (
... (15.623037, 13.258358),
... (55.147488, -2.667338),
... (54.572062, -73.285171),
... (3.152857, 115.327724),
... (-40.454262, 172.318877)
)

>>> # Ordenación por longitud (primer elemento de la tupla)
>>> sorted(geoloc)
[(-40.454262, 172.318877),
 (3.152857, 115.327724),
 (15.623037, 13.258358),
 (54.572062, -73.285171),
 (55.147488, -2.667338)]

>>> # Ordenación por latitud (segundo elemento de la tupla)
>>> sorted(geoloc, key=lambda t: t[1])
[(54.572062, -73.285171),
 (55.147488, -2.667338),
 (15.623037, 13.258358),
 (3.152857, 115.327724),
 (-40.454262, 172.318877)]
```

Ejercicio

pycheck: **sort_ages**

Enfoque funcional

Como se comentó en la *introducción*, Python es un lenguaje de programación multiparadigma. Uno de los paradigmas menos explotados en este lenguaje es la **programación funcional**[4].

Python nos ofrece 3 funciones que encajan verdaderamente bien en este enfoque: `map()`, `filter()` y `reduce()`.

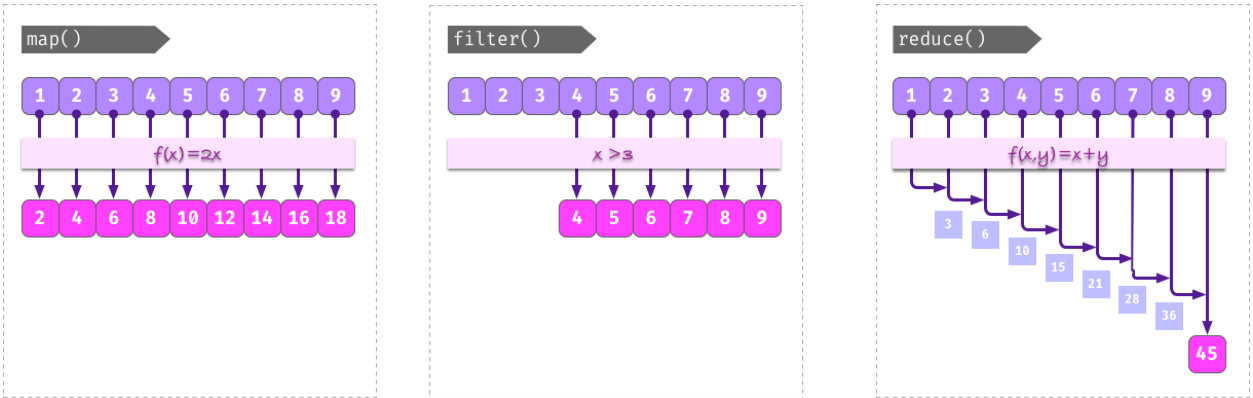


Figura 6: Rutinas muy enfocadas a programación funcional

`map()`

Esta función **aplica otra función** sobre cada elemento de un iterable. Supongamos que queremos aplicar la siguiente función:

$$f(x) = \frac{x^2}{2} \quad \forall x \in [1, 10]$$

```
>>> def f(x):
...     return x**2 / 2
...

>>> data = range(1, 11)

>>> map_gen = map(f, data)

>>> type(map_gen)
map

>>> list(map_gen)
[0.5, 2.0, 4.5, 8.0, 12.5, 18.0, 24.5, 32.0, 40.5, 50.0]
```

[4] Definición de *Programación funcional* en Wikipedia.

Truco: Hay que tener en cuenta que `map()` devuelve un *generador*, no directamente una lista.

Podemos obtener el mismo resultado aplicando una *función anónima «lambda»*:

```
>>> list(map(lambda x: x**2 / 2, data))
[0.5, 2.0, 4.5, 8.0, 12.5, 18.0, 24.5, 32.0, 40.5, 50.0]
```

En Python es posible «simular» un `map()` a través de una *lista por comprensión*:

```
>>> [x**2 / 2 for x in data]
[0.5, 2.0, 4.5, 8.0, 12.5, 18.0, 24.5, 32.0, 40.5, 50.0]
```

`filter()`

Esta función **selecciona** aquellos elementos de un iterable que cumplan una determinada condición. Supongamos que queremos seleccionar sólo aquellos números impares dentro de un rango:

```
>>> def odd_number(x):
...     return x % 2 == 1
...

>>> data = range(1, 21)

>>> filter_gen = filter(odd_number, data)

>>> type(filter_gen)
filter

>>> list(filter_gen)
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

Truco: Hay que tener en cuenta que `filter()` devuelve un *generador*, no directamente una lista.

Podemos obtener el mismo resultado aplicando una *función anónima «lambda»*:

```
>>> list(filter(lambda x: x % 2 == 1, data))
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

En Python es posible «simular» un `filter()` a través de una *lista por comprensión*:

```
>>> [x for x in data if x % 2 == 1]
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

reduce()

Para poder usar esta función debemos usar el módulo `functools`. Nos permite aplicar una función dada sobre todos los elementos de un iterable de manera acumulativa. O dicho en otras palabras, nos permite **reducir** una función sobre un conjunto de valores. Supongamos que queremos realizar el producto de una serie de valores aplicando este enfoque:

```
>>> from functools import reduce

>>> def mult_values(a, b):
...     return a * b
...

>>> data = range(1, 6)

>>> reduce(mult_values, data)  # ((((1 * 2) * 3) * 4) * 5)
120
```

Aplicando una *función anónima «lambda»*...

```
>>> reduce(lambda x, y: x * y, data)
120
```

Consejo: Por cuestiones de legibilidad del código, se suelen preferir las **listas por comprensión** a funciones como `map()` o `filter()`, aunque cada problema tiene sus propias características y sus soluciones más adecuadas. Es un **enfoque «más pitónico»**.

Hazlo pitónico

Trey Hunner explica en una de sus «newsletters» lo que él entiende por **código pitónico**:

«Pitónico es un término extraño que significa diferentes cosas para diferentes personas. Algunas personas piensan que código pitónico va sobre legibilidad. Otras personas piensan que va sobre adoptar características particulares de Python. Mucha gente tiene una definición difusa que no va sobre legibilidad ni sobre características del lenguaje.

Yo normalmente uso el término código pitónico como un sinónimo de código idiomático o la forma en la que la comunidad de Python tiende a hacer las cosas cuando escribe Python.

Eso deja mucho espacio a la interpretación, ya que lo que hace algo idiomático en Python no está particularmente bien definido.

Yo argumento que código pitónico implica adoptar el *desempaquetado de tuplas*, usar *listas por comprensión* cuando sea apropiado, usar *argumentos nominales* cuando tenga sentido, evitar el *uso excesivo de clases*, usar las *estructuras de iteración* adecuadas o evitar *recorrer mediante índices*.

Para mí, código pitónico significa intentar ver el código desde la perspectiva de las herramientas específicas que Python nos proporciona, en oposición a la forma en la que resolveríamos el mismo problema usando las herramientas que nos proporciona JavaScript, Java, C, …»

Generadores

Un **generador**, como su propio nombre indica, se encarga de generar «valores» que podemos tratar de manera individual (y aislada).

Es decir, no construye una secuencia de forma explícita, sino que nos permite ir «consumiendo» un valor de cada vez. Esta propiedad los hace idóneos para situaciones en las que el tamaño de las secuencias podría tener un impacto negativo en el consumo de memoria.

De hecho ya hemos visto algunos generadores y los hemos usado sin ser del todo conscientes. Algo muy parecido a un generador es `range()`[8] que ofrece la posibilidad de crear *secuencias de números*.

Básicamente existen dos implementaciones de generadores:

- Funciones generadoras.
- Expresiones generadoras.

Importante: A diferencia de las funciones ordinarias, los generadores tienen la capacidad de **«recordar» su estado** para recuperarlo en la siguiente iteración y continuar devolviendo nuevos valores.

---

[8] La función `range()` es un tanto especial. Véase este artículo de Trey Hunner.

Funciones generadoras

Las funciones generadoras[9] (o **factorías de generadores**) se escriben como funciones ordinarias con el matiz de incorporar la sentencia `yield` que sustituye, de alguna manera, a `return`. Esta sentencia devuelve el valor indicado y, a la vez, «congela» el estado de la función hasta la siguiente llamada.

Veamos un ejemplo en el que escribimos una **función generadora de números pares**:

```
>>> def evens(lim: int) -> int:
...     for i in range(0, lim + 1, 2):
...         yield i
...

>>> type(evens)
function

>>> evens_gen = evens(20)  # retorna un generador

>>> type(evens_gen)
generator
```

Importante: Las funciones generadoras devuelven **un generador** que debe ser invocado con los parámetros correspondientes para obtener la secuencia de valores que necesitemos.

Una vez creado el generador, ya podemos iterar sobre él:

```
>>> for even in evens_gen:
...     print(even, end=' ')
...
0 2 4 6 8 10 12 14 16 18 20
```

De forma más «directa» (y habitual) podemos iterar sobre la propia llamada a la función generadora:

```
>>> for even in evens(20):
...     print(even, end=' ')
...
0 2 4 6 8 10 12 14 16 18 20
```

Si queremos «explicitar» la lista de valores que contiene un generador, podemos hacerlo convirtiendo a lista:

[9] Para una explicación detallada sobre generadores e iteradores se recomienda la ponencia Yield el amigo que no sabías que tenías de Jacobo de Vera.

```
>>> list(evens(20))
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Un detalle muy importante es que **los generadores «se agotan»**. Es decir, una vez que ya hemos consumido todos sus elementos, no obtendremos nuevos valores:

```
>>> evens_gen = evens(10)

>>> for even in evens_gen:
...     print(even, end=' ')
...
0 2 4 6 8 10

>>> for even in evens_gen:
...     print(even, end=' ')
... # No sale nada... ¡Agotado!
```

Expresiones generadoras

Una **expresión generadora** es sintácticamente muy similar a una *lista por comprensión*, pero utilizamos **paréntesis** en vez de corchetes.

Importante: Una expresión generadora es un generador en sí misma.

Podemos tratar de reproducir el ejemplo visto en *funciones generadoras* en el que creamos números pares hasta el 20:

```
>>> evens_gen = (i for i in range(0, 20, 2))

>>> type(evens_gen)
generator

>>> for i in evens_gen:
...     print(i, end=' ')
...
0 2 4 6 8 10 12 14 16 18
```

Ver también:

Las expresiones generadoras admiten *condiciones* y *anidamiento de bucles*, tal y como se vio con las *listas por comprensión*.

Una expresión generadora se puede explicitar[11], sumar, buscar su máximo o su mínimo, o lo

[11] Cuando hablamos de «explicitar» un generador nos referimos a obtener todos sus valores de forma

que queramos, tal y como lo haríamos con un iterable cualquiera:

```
>>> list(i for i in range(0, 20, 2))
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

>>> sum(i for i in range(0, 20, 2))
90

>>> min(i for i in range(0, 20, 2))
0

>>> max(i for i in range(0, 20, 2))
18
```

Ejercicio

pycheck: **gen_squared**

Funciones interiores

Está permitido definir una función dentro de otra función. Es lo que se conoce como **función interior**.

Veamos un ejemplo en el que extraemos las palabras de un texto que contienen todas las vocales, haciendo uso de una función interior que nos devuelve el número de vocales distintas que tiene cada palabra:

```
>>> def get_words_with_all_vowels(text: str) -> list[str]:
...     VOWELS = 'aeiou'
...     def get_unique_vowels(word: str) -> set[str]:
...         return set(c for c in word if c in VOWELS)
...
...     result = []
...     for word in text.split():
...         if len(get_unique_vowels(word)) == len(VOWELS):
...             result.append(word)
...     return result

>>> get_words_with_all_vowels('La euforia de ver el riachuelo fue inmensa')
['euforia', 'riachuelo']
```

Truco: Estas funciones pueden tener sentido cuando su ámbito de aplicación es muy concreto

directa como una lista (o sucedáneo).

y no se pueden reutilizar fácilmente.

Clausuras

Una **clausura** (del término inglés «*closure*») establece el uso de una *función interior* que se genera dinámicamente y recuerda los valores de los argumentos con los que fue creada.

Veamos en acción una clausura que nos permitirá generar «tablas de multiplicar»:

```
>>> def make_multiplier_of(n: int):
...     def multiplier(x: int) -> int:
...         return x * n
...     return multiplier  # factoría de funciones
...

>>> m3 = make_multiplier_of(3)
>>> type(m3)
function

>>> m3(7)  # 7 * 3
21

>>> m5 = make_multiplier_of(5)
>>> type(m5)
function

>>> m5(8)  # 8 * 5
40

>>> make_multiplier_of(5)(8)  # Llamada directa!
40
```

Importante: En una clausura retornamos una función, no una llamada a una función. Es por esto que se dice que **una clausura** es una **factoría de funciones**.

Decoradores

Hay situaciones en las que necesitamos modificar el comportamiento de funciones existentes pero sin alterar su código. Para estos casos es muy útil usar decoradores.

Un **decorador** es una *función* que recibe como parámetro una función y devuelve otra función. Se podría ver como un caso particular de una *clausura*:

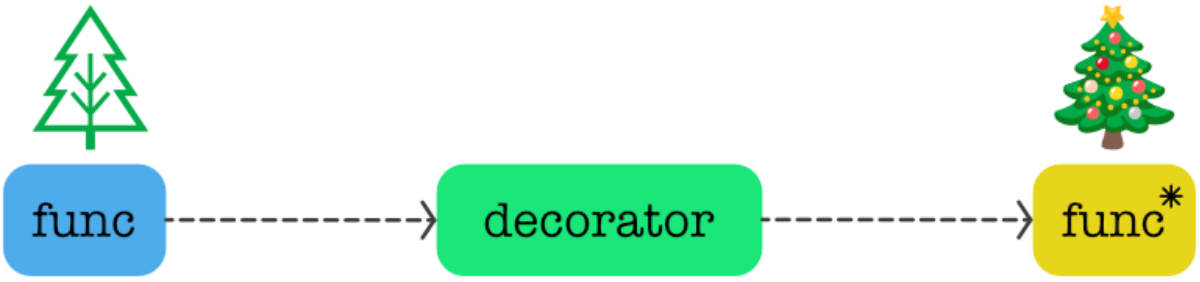


Figura 7: Comportamiento de un decorador

El *esqueleto básico* de un decorador es el siguiente:

```
>>> def my_decorator(func):
...     def wrapper(*args, **kwargs):
...         # some code before calling func
...         return func(*args, **kwargs)
...         # some code after calling func
...     return wrapper
...
```

Elemento	Descripción
`my_decorator`	Nombre del decorador
`wrapper`	Función interior (convención de nombre)
`func`	Función a decorar (convención de nombre)
`*args`	Argumentos posicionales (convención de nombre)
`**kwargs`	Argumentos nominales (convención de nombre)

Veamos un ejemplo de **decorador que convierte el resultado numérico de una función a su representación binaria**:

```
>>> def res2bin(func):
...     def wrapper(*args, **kwargs):
...         result = func(*args, **kwargs)
...         return bin(result)
...     return wrapper
...
```

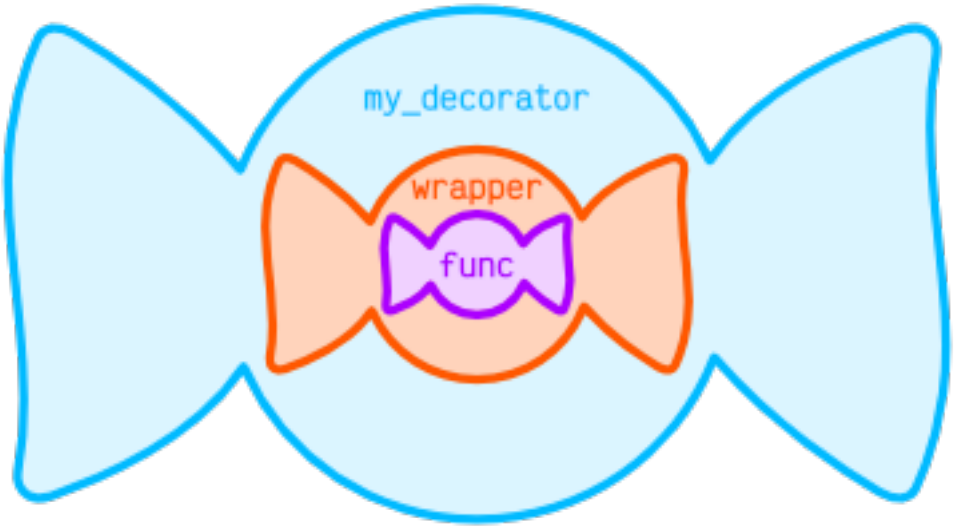


Figura 8: Esqueleto básico de un decorador

Ahora definimos una función ordinaria (que usaremos más adelante) y que computa x^n:

```
>>> def power(x: int, n: int) -> int:
...     return x ** n
...

>>> power(2, 3)
8
>>> power(4, 5)
1024
```

Ahora aplicaremos el decorador definido previamente `res2bin()` sobre la función ordinaria `power()`. Se dice que `res2bin()` es la **función decoradora** y que `power()` es la **función decorada**:

```
>>> decorated_power = res2bin(power)

>>> decorated_power(2, 3)  # 8
'0b1000'
>>> decorated_power(4, 5)  # 1024
'0b10000000000'
```

Usando @ para decorar

Python nos ofrece un «syntactic sugar» para simplificar la aplicación de los decoradores a través del operador `@` justo antes de la definición de la función que queremos decorar:

```
>>> @res2bin
... def power(x: int, n: int) -> int:
...     return x ** n
...

>>> power(2, 3)
'0b1000'
>>> power(4, 5)
'0b10000000000'
```

Ejercicio

pycheck: **abs_decorator**

Manipulando argumentos

Hemos visto un ejemplo de decorador que trabaja sobre el resultado de la función decorada, pero nada impide que trabajemos sobre los argumentos que pasamos a la función decorada.

Supongamos un escenario en el que implementamos **funciones que trabajan con dos operandos** y queremos asegurarnos de que **esos operados son números enteros**. Lo primero será definir el decorador:

```
>>> def assert_int(func):
...     def wrapper(value1: int, value2: int, /) -> int | float | None:
...         if isinstance(value1, int) and isinstance(value2, int):
...             return func(value1, value2)
...         return None
...     return wrapper
...
```

Truco: Dado que sabemos positivamente que las funciones a decorar trabajan con dos operados (dos parámetros) podemos definir la función interior `wrapper(value1, value2)` con dos parámetros, en vez de con un número indeterminado de parámetros.

Ahora creamos una función sencilla que suma dos números y le aplicamos el decorador:

```
>>> @assert_int
... def _sum(a, b):
...     return a + b
...
```

Veamos el comportamiento para diferentes casos de uso:

```
>>> result = _sum(3, 4)
>>> print(result)
7

>>> result = _sum(5, 'a')
>>> print(result)
None

>>> result = _sum('a', 'b')
>>> print(result)
None
```

Múltiples decoradores

Podemos aplicar más de un decorador a cada función. Para ejemplificarlo vamos a crear dos decoradores muy sencillos:

```
>>> def plus5(func):
...     def wrapper(*args, **kwargs):
...         result = func(*args, **kwargs)
...         return result + 5
...     return wrapper
...

>>> def div2(func):
...     def wrapper(*args, **kwargs):
...         result = func(*args, **kwargs)
...         return result // 2
...     return wrapper
...
```

Ahora aplicaremos ambos decoradores sobre una función que realiza el producto de dos números:

```
>>> @plus5
... @div2
... def prod(a, b):
```

(continué en la próxima página)

(proviene de la página anterior)

```
...     return a * b
...

>>> prod(4, 3)
11

>>> ((4 * 3) // 2) + 5
11
```

Orden de ejecución

Cuando tenemos varios decoradores **se aplican desde dentro hacia fuera** ya que la ejecución de un decorador depende de otro decorador.

Una forma sencilla de entender el orden de ejecución de múltiples decoradores es aplicar las funciones decoradoras directamente sobre la función decorada.

Esto:

```
>>> @plus5
... @div2
... def prod(a, b):
...     return a * b
...
```

equivale a:

```
>>> plus5(div2(prod(4, 3)))
```

Decoradores con parámetros

El último «salto mortal» sería definir decoradores con parámetros. El *esqueleto básico* de un decorador con parámetros es el siguiente:

```
>>> def my_decorator_with_params(*args, **kwargs):
...     def decorator(func):
...         def wrapper(*args, **kwargs):
...             return func(*args, **kwargs)
...         return wrapper
...     return decorator
...
```

Atención: Nótese que `my_decorator_with_params()` no es exactamente un decorador sino que es una factoría de decoradores (*clausura*) que devuelve un decorador según los argumentos pasados.

Lo más sencillo es verlo con un ejemplo. Supongamos que queremos forzar a que los parámetros de entrada a la función sean de un tipo concreto (pero parametrizable). Podríamos definir el decorador de la siguiente manera:

```
>>> def assert_type(atype):
...     def decorator(func):
...         def wrapper(*args, **kwargs):
...             all_args_with_atype = all(isinstance(a, atype) for a in args)
...             all_kwargs_with_atype = all(isinstance(a, atype) for a in kwargs.
↪values())
...             if all_args_with_atype and all_kwargs_with_atype:
...                 return func(*args, **kwargs)
...             return None
...         return wrapper
...     return decorator
...
```

Ahora creamos una función sencilla que suma dos números y le aplicamos el decorador:

```
>>> @assert_type(float)
... def _sum(a, b):
...     return a + b
...
```

Veamos el comportamiento para diferentes casos de uso:

```
>>> result = _sum(3, 4)
>>> print(result)
None

>>> result = _sum(3.0, 4.0)
>>> print(result)
7.0

>>> result = _sum(a=3.0, b=4.0)  # Funciona con kwargs!
>>> print(result)
7.0
```

La ventaja que tiene este enfoque es que podemos aplicar «distintos» decoradores modificando sus parámetros. Por ejemplo, supongamos que ahora queremos **asegurar que una función trabaja únicamente con cadenas de texto**:

```
>>> @assert_type(str)
... def split(text):
...     half_size = len(text) // 2
...     return text[:half_size], text[half_size:]
...
```

Veamos su aplicación con distintos tipos de datos:

```
>>> result = split('bienvenida')
>>> print(result)
('bienv', 'enida')

>>> result = split(256)
>>> print(result)
None

>>> result = split([10, 20, 30, 40])
>>> print(result)
None
```

Ejercicio

pycheck: **deco__sort**

Funciones recursivas

La **recursividad** es el mecanismo por el cual una función se llama a sí misma:

```
>>> def call_me():
...     return call_me()
...

>>> call_me()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in call_me
  File "<stdin>", line 2, in call_me
  File "<stdin>", line 2, in call_me
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

Advertencia: Podemos observar que existe un número máximo de llamadas recursivas. Python controla esta situación por nosotros, ya que, de no ser así, podríamos llegar a consumir todos los recursos del sistema.

Veamos ahora un ejemplo más real en el que calcular x^n de manera recursiva. Usaremos la idea de *base* y *exponente* para resolver este reto:

```
>>> def pow(base: int, exponent: int) -> int:
...     if exponent == 0:
...         return 1
...     return base * pow(base, exponent - 1)
...

>>> pow(2, 4)
16

>>> pow(3, 5)
243
```

Condición de parada: En todo código recursivo es necesario establecer una **condición de parada**. En el ejemplo la condición de parada se da cuando el exponente es 0, siendo el resultado 1, ya que todo número elevado a 0 es igual a 1.

Llamada recursiva: Obviamente en todo código recursivo debe haber una **llamada recursiva**. En el ejemplo anterior la recursividad se apoya en la idea de que $2^4 = 2 * 2^3$. Por tanto podemos hacer uso de la misma función recursiva para calcular el resto de valores.

La «pila de llamadas» para el ejemplo de `pow(2, 4)` sería la siguiente:

Ejercicio

pycheck: **factorial_recursive**

Otra aproximación a la recursividad se da en problemas donde tenemos que procesar una secuencia de elementos. Supongamos que nos piden **calcular la suma de las longitudes de una serie de palabras** definidas en una lista:

```
>>> def get_size(words: list[str]) -> int:
...     if len(words) == 0:
...         return 0
...     return len(words[0]) + get_size(words[1:])
...

>>> words = ['this', 'is', 'recursive']
```

(continué en la próxima página)

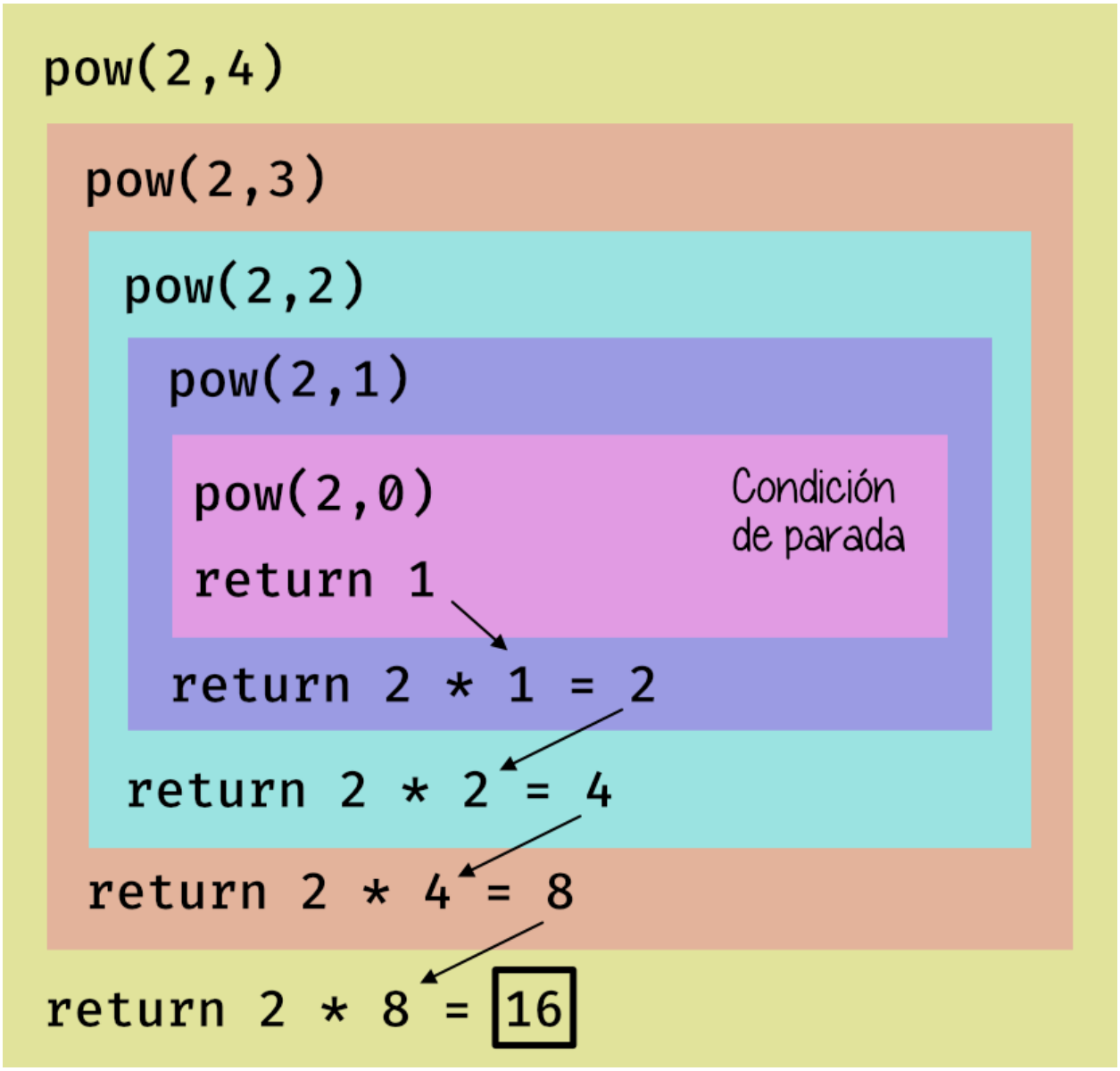


Figura 9: Esquema de llamadas recursivas

(proviene de la página anterior)

```
>>> get_size(words)
15
```

Funcionitis

La «funcionitis» es una «inflamación en la zona funcional» por querer aplicar funciones donde no es necesario. Un ejemplo vale más que mil explicaciones:

```
>>> def in_list(item: int, items: list[int]) -> bool:
...     return item in items
...

>>> in_list(1, [1, 2, 3])
True

>>> 1 in [1, 2, 3]  # That easy!
True
```

Truco: La «funcionitis» es uno de los síntomas de la llamada «sobre-ingeniería» a la que tendemos muchas de las personas que hacemos programación. Hay que intentar evitarla en la medida de lo posible.

6.1.5 Espacios de nombres

Como bien indica el *Zen de Python*:

> *Namespaces are one honking great idea – let's do more of those!*

Que vendría a traducirse como: «Los espacios de nombres son una gran idea – hagamos más de eso». Los **espacios de nombres** permiten definir **ámbitos** o **contextos** en los que agrupar nombres de objetos.

Los espacios de nombres proporcionan un mecanismo de empaquetado, de tal forma que podamos tener incluso nombres iguales que no hacen referencia al mismo objeto (siempre y cuando estén en ámbitos distintos).

Cada *función* define su propio espacio de nombres y es diferente del espacio de nombres global aplicable a todo nuestro programa.

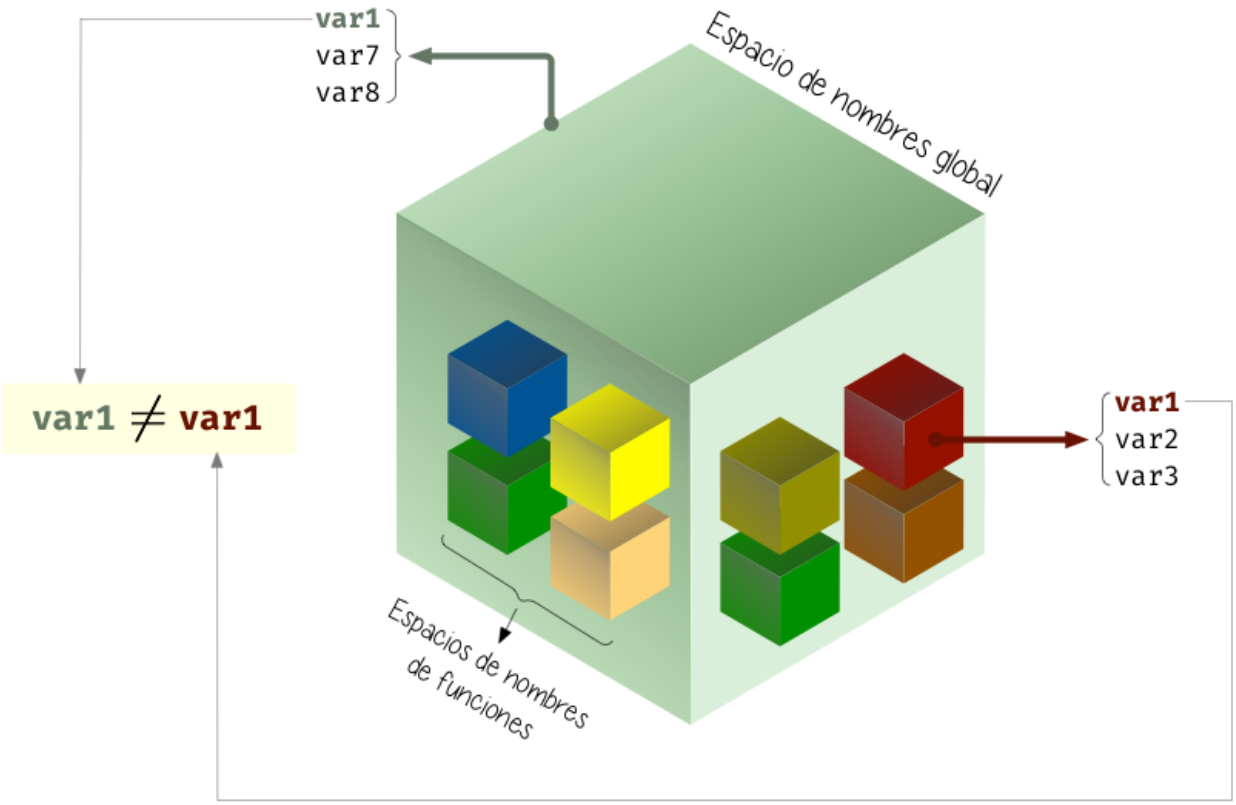


Figura 10: Espacio de nombres global vs espacios de nombres de funciones

Acceso a variables globales

Cuando una variable se define en el *espacio de nombres global* podemos hacer uso de ella con total transparencia dentro del ámbito de las funciones del programa:

```
>>> language = 'castellano'

>>> def catalonia():
...     print(f'{language=}')
...

>>> language
'castellano'

>>> catalonia()
language='castellano'
```

Creando variables locales

En el caso de que asignemos un valor a una variable global dentro de una función, no estaremos modificando ese valor. Por el contrario, estaremos creando una *variable en el espacio de nombres local*:

```
>>> language = 'castellano'

>>> def catalonia():
...     language = 'catalan'
...     print(f'{language=}')
...

>>> language
'castellano'

>>> catalonia()
language='catalan'

>>> language
'castellano'
```

Forzando modificación global

Python nos permite modificar una variable definida en un espacio de nombres global dentro de una función. Para ello debemos usar el modificador `global`:

```
>>> language = 'castellano'

>>> def catalonia():
...     global language
...     language  = 'catalan'
...     print(f'{language=}')
...

>>> language
'castellano'

>>> catalonia()
language='catalan'

>>> language
'catalan'
```

Advertencia: El uso de `global` no se considera una buena práctica ya que puede inducir a confusión y tener efectos colaterales indeseados.

Contenido de los espacios de nombres

Python proporciona dos funciones para acceder al contenido de los espacios de nombres:

`locals()` Devuelve un diccionario con los contenidos del **espacio de nombres local**:

```
>>> language = 'castellano'

>>> def catalonia():
...     language  = 'catalan'
...     print(f'{locals()=}')
...

>>> catalonia()
locals()={'language': 'catalan'}
```

`globals()` Devuelve un diccionario con los contenidos del **espacio de nombres global**:

```
>>> globals()
{'__name__': '__main__',
'__doc__': 'Automatically created module for IPython interactive environment',
'__package__': None,
'__loader__': None,
'__spec__': None,
'__builtin__': <module 'builtins' (built-in)>,
'__builtins__': <module 'builtins' (built-in)>,
'_ih': ['',
"language = 'castellano'",
"def catalonia():\n    language  = 'catalan'\n    print(f'{locals()=}')\n    ",
'language',
'catalonia()',
'globals()'],
'_oh': {3: 'castellano'},
'_dh': ['/Users/sdelquin'],
'In': ['',
"language = 'castellano'",
"def catalonia():\n    language  = 'catalan'\n    print(f'{locals()=}')\n    ",
'language',
'catalonia()',
'globals()'],
'Out': {3: 'castellano'},
'get_ipython': <bound method InteractiveShell.get_ipython of <IPython.terminal.
↪interactiveshell.TerminalInteractiveShell object at 0x10e70c2e0>>,
'exit': <IPython.core.autocall.ExitAutocall at 0x10e761070>,
'quit': <IPython.core.autocall.ExitAutocall at 0x10e761070>,
'_': 'castellano',
'__': '',
'___': '',
'Prompts': IPython.terminal.prompts.Prompts,
'Token': Token,
'MyPrompt': __main__.MyPrompt,
'ip': <IPython.terminal.interactiveshell.TerminalInteractiveShell at␣
↪0x10e70c2e0>,
'_i': 'catalonia()',
'_ii': 'language',
'_iii': "def catalonia():\n    language  = 'catalan'\n    print(f'{locals()=}')\
↪n    ",
'_i1': "language = 'castellano'",
'language': 'castellano',
'_i2': "def catalonia():\n    language  = 'catalan'\n    print(f'{locals()=}')\
↪n    ",
'catalonia': <function __main__.catalonia()>,
'_i3': 'language',
'_3': 'castellano',
```

(continué en la próxima página)

(proviene de la página anterior)

```
'_i4': 'catalonia()',
'_i5': 'globals()'}
```

6.1.6 Consejos para programar

Chris Staudinger comparte estos 7 consejos para mejorar tu código:

1. **Las funciones deberían hacer una única cosa.** *Por ejemplo, un mal diseño sería tener una única función que calcule el total de una cesta de la compra, los impuestos y los gastos de envío. Sin embargo esto se debería hacer con tres funciones separadas. Así conseguimos que el código sea más fácil de matener, reutilizar y depurar.*

2. **Utiliza nombres descriptivos y con significado.** *Los nombres autoexplicativos de variables y funciones mejoran la legibilidad del código. Por ejemplo – deberíamos llamar «total_cost» a una variable que se usa para almacenar el total de un carrito de la compra en vez de «x» ya que claramente explica su propósito.*

3. **No uses variables globales.** *Las variables globales pueden introducir muchos problemas, incluyendo efectos colaterales inesperados y errores de programación difíciles de trazar. Supongamos que tenemos dos funciones que comparten una variable global. Si una función cambia su valor la otra función podría no funcionar como se espera.*

4. **Refactorizar regularmente.** *El código inevitablemente cambia con el tiempo, lo que puede derivar en partes obsoletas, redundantes o desorganizadas. Trata de mantener la calidad del código revisando y refactorizando aquellas zonas que se editan.*

5. **No utilices «números mágicos» o valores «hard-codeados».** *No es lo mismo escribir «99 * 3» que «price * quantity». Esto último es más fácil de entender y usa variables con nombres descriptivos haciéndolo autoexplicativo. Trata de usar constantes o variables en vez de valores «hard-codeados».*

6. **Escribe lo que necesites ahora, no lo que pienses que podrías necesitar en el futuro.** *Los programas simples y centrados en el problema son más flexibles y menos complejos.*

7. **Usa comentarios para explicar el «por qué» y no el «qué».** *El código limpio es autoexplicativo y por lo tanto los comentarios no deberían usarse para explicar lo que hace el código. En cambio, los comentarios debería usarse para proporcionar contexto adicional, como por qué el código está diseñado de una cierta manera.*

EJERCICIOS DE REPASO

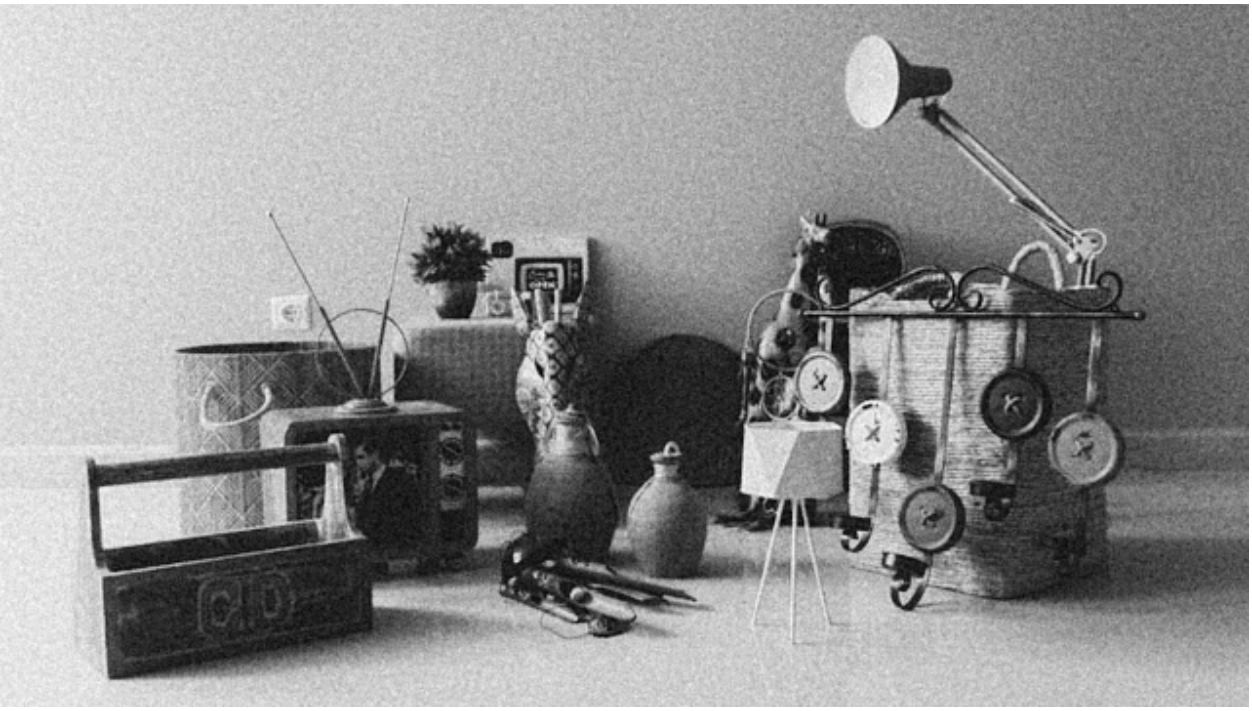
1. pycheck: **num_in_interval**
2. pycheck: **extract_evens**
3. pycheck: **split_case**
4. pycheck: **perfect**
5. pycheck: **palindrome**
6. pycheck: **count_vowels_rec**
7. pycheck: **pangram**
8. pycheck: **cycle_alphabet**
9. pycheck: **bubble_sort**
10. pycheck: **consecutive_seq**
11. pycheck: **magic_square**
12. pycheck: **sum_nested**
13. pycheck: **fibonacci_recursive**
14. pycheck: **hyperfactorial**
15. pycheck: **fibonacci_generator**
16. pycheck: **gcd_recursive**
17. pycheck: **palindrome_recursive**
18. pycheck: **assert_positive**
19. pycheck: **slice_recursive**

AMPLIAR CONOCIMIENTOS

- Comparing Python Objects the Right Way: «is» vs «==»
- Python Scope & the LEGB Rule: Resolving Names in Your Code
- Defining Your Own Python Function
- Null in Python: Understanding Python's NoneType Object
- Python "!=" Is Not "is not": Comparing Objects in Python
- Python args and kwargs: Demystified
- Documenting Python Code: A Complete Guide

- Thinking Recursively in Python
- How to Use Generators and yield in Python
- How to Use Python Lambda Functions
- Python Decorators 101
- Writing Comments in Python
- Introduction to Python Exceptions
- Primer on Python Decorators

6.2 Objetos y Clases



Hasta ahora hemos estado usando objetos de forma totalmente transparente, casi sin ser conscientes de ello. Pero, en realidad, **todo en Python es un objeto**, desde números a funciones. El lenguaje provee ciertos mecanismos para no tener que usar explícitamente técnicas de orientación a objetos.

Llegados a este punto, investigaremos en profundidad la creación y manipulación de clases y objetos, así como todas las técnicas y procedimientos que engloban este paradigma.[1]

[1] Foto original por Rabie Madaci en Unsplash.

6.2.1 Programación orientada a objetos

La programación orientada a objetos (POO) o en sus siglas inglesas **OOP** es una manera de programar (paradigma) que permite llevar al código mecanismos usados con entidades de la vida real.

Sus **beneficios** son los siguientes:

Encapsulamiento Permite **empaquetar** el código dentro de una unidad (objeto) donde se puede determinar el ámbito de actuación.

Abstracción Permite **generalizar** los tipos de objetos a través de las clases y simplificar el programa.

Herencia Permite **reutilizar** código al poder heredar atributos y comportamientos de una clase a otra.

Polimorfismo Permite **crear** múltiples objetos a partir de una misma pieza flexible de código.

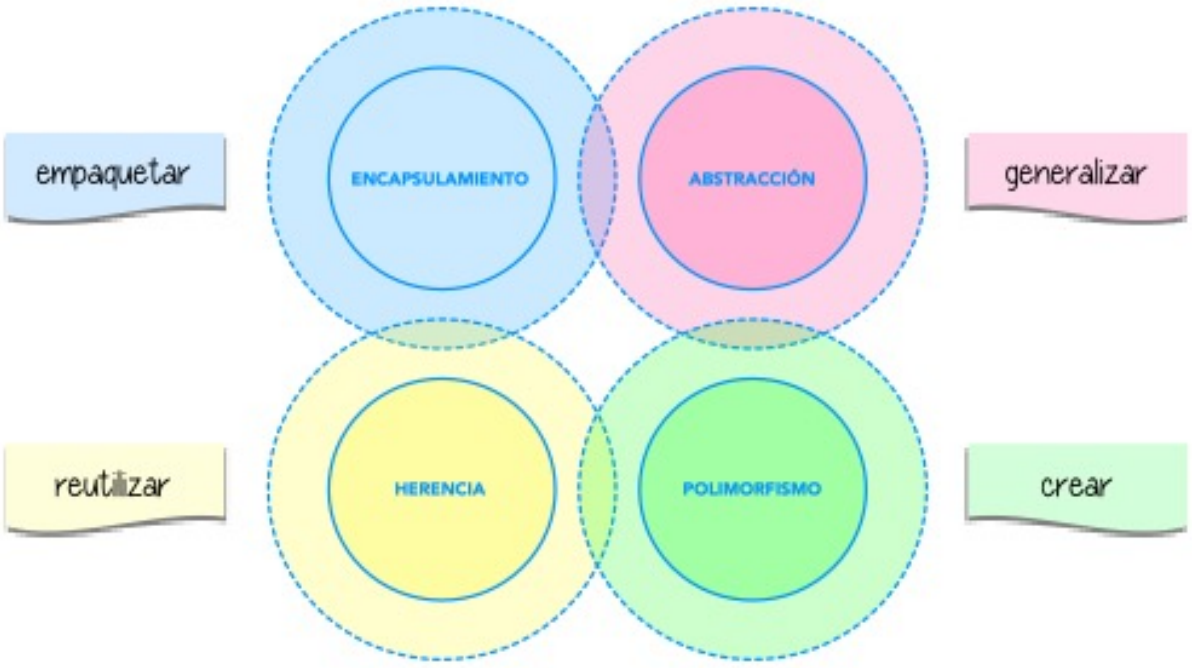


Figura 11: Beneficios de la Programación Orientada a Objetos

¿Qué es un objeto?

Un **objeto** es una **estructura de datos personalizada** que contiene **datos** y **código**:

Elementos	¿Qué son?	¿Nombre en POO?	¿Cómo se identifican?
Datos	Variables	Atributos	Mediante sustantivos
Código	Funciones	Métodos	Mediante verbos

Un objeto representa **una instancia única** de alguna entidad (a través de los valores de sus atributos) e interactúa con otros objetos (o consigo mismo) a través de sus métodos.

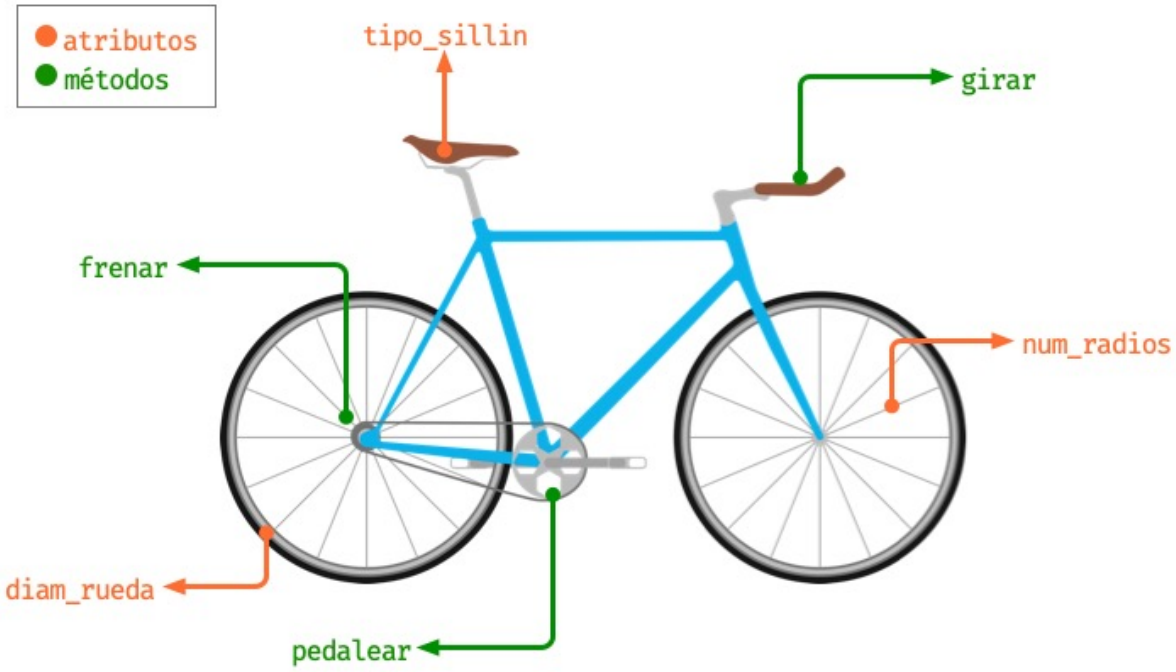


Figura 12: Analogía de atributos y métodos en un objeto «bicicleta»

¿Qué es una clase?

Para crear un objeto primero debemos definir la clase que lo contiene. Podemos pensar en la **clase** como el **molde** con el que se crean nuevos objetos de ese tipo.

En el **proceso de diseño** de una clase hay que tener en cuenta – entre otros – el **principio de responsabilidad única**[7], intentando que los atributos y los métodos que contenga esa clase estén enfocados a un *objetivo único y bien definido*.

[7] Principios SOLID



Figura 13: Ejemplificación de creación de objetos a partir de una clase

6.2.2 Creando objetos

Empecemos por crear nuestra **primera clase**. En este caso vamos a modelar algunos de los droides de la saga StarWars:

Para ello usaremos la palabra reservada `class` seguida del nombre de la clase:

```
>>> class StarWarsDroid:
...     pass  # pass no hace nada
...
```

Consejo: Los nombres de clases se suelen escribir en formato `CamelCase` y en singular[3].

Existen multitud de droides en el universo StarWars. Una vez que hemos definido la clase genérica podemos crear **instancias/objetos** (droides) concretos:

```
>>> c3po = StarWarsDroid()
>>> r2d2 = StarWarsDroid()
>>> bb8 = StarWarsDroid()
```

(continué en la próxima página)

[2] Fuente de la imagen: Astro Mech Droids.
[3] Guía de estilos PEP8 para convenciones de nombres.



Figura 14: Droides de la saga StarWars[2]

(proviene de la página anterior)

```
>>> type(c3po)
__main__.StarWarsDroid
>>> type(r2d2)
__main__.StarWarsDroid
>>> type(bb8)
__main__.StarWarsDroid
```

Añadiendo métodos

Un **método** es una función que forma parte de una clase o de un objeto. En su ámbito tiene acceso a otros métodos y atributos de la clase o del objeto al que pertenece.

La definición de un método (de instancia) es análoga a la de una función ordinaria, pero incorporando un primer parámetro `self` que hace referencia a la instancia actual del objeto.

Una de las acciones más sencillas que se pueden hacer sobre un droide es encenderlo o apagarlo. Vamos a implementar estos dos métodos en nuestra clase:

```
>>> class Droid:
...     def switch_on(self):
...         print("Hi! I'm a droid. Can I help you?")
...
...     def switch_off(self):
...         print("Bye! I'm going to sleep")
...
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> k2so = Droid()

>>> k2so.switch_on()
Hi! I'm a droid. Can I help you?

>>> k2so.switch_off()
Bye! I'm going to sleep
```

Consejo: El nombre `self` es sólo una convención. Este parámetro puede llamarse de otra manera, pero seguir el estándar ayuda a la legibilidad.

Añadiendo atributos

Un **atributo** no es más que una variable, un nombre al que asignamos un valor, con la particularidad de vivir dentro de una clase o de un objeto.

Supongamos que, siguiendo con el ejemplo anterior, queremos guardar en un atributo el estado del droide (encendido/apagado):

```
>>> class Droid:
...     def switch_on(self):
...         self.power_on = True
...         print("Hi! I'm a droid. Can I help you?")
...
...     def switch_off(self):
...         self.power_on = False
...         print("Bye! I'm going to sleep")

>>> k2so = Droid()

>>> k2so.switch_on()
Hi! I'm a droid. Can I help you?
>>> k2so.power_on
True

>>> k2so.switch_off()
Bye! I'm going to sleep
>>> k2so.power_on
False
```

Importante: Siempre que queramos acceder a cualquier método o atributo del objeto habrá

que utilizar la palabra `self`.

Inicialización

Existe un **método especial** que se ejecuta cuando creamos una instancia de un objeto. Este método es `__init__` y nos permite asignar atributos y realizar operaciones con el objeto en el momento de su creación. También es ampliamente conocido como el **constructor**.

Veamos un ejemplo de este método con nuestros droides en el que únicamente guardaremos el nombre del droide como un atributo del objeto:

```
1 >>> class Droid:
2 ...     def __init__(self, name: str):
3 ...         self.name = name
4 ...
5
6 >>> droid = Droid('BB-8')
7
8 >>> droid.name
9 'BB-8'
```

Línea 2 Definición del constructor.

Línea 7 Creación del objeto (y llamada implícita al constructor)

Línea 9 Acceso al atributo `name` creado previamente en el constructor.

Es importante tener en cuenta que si no usamos `self` estaremos creando una variable local en vez de un atributo del objeto:

```
>>> class Droid:
...     def __init__(self, name: str):
...         name = name  # No lo hagas!
...

>>> droid = Droid('BB-8')

>>> droid.name
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'Droid' object has no attribute 'name'
```

Ejercicio

Escriba una clase `MobilePhone` que represente un teléfono móvil.

Atributos:

- `manufacturer` (cadena de texto)
- `screen_size` (flotante)
- `num_cores` (entero)
- `apps` (lista de cadenas de texto)
- `status` (`False`: apagado, `True`: encendido)

Métodos:

- `__init__(self, manufacturer, screen_size, num_cores)`
- `power_on(self)`
- `power_off(self)`
- `install_app(self, app)` *(no instalar la app si ya existe)*
- `uninstall_app(self, app)` *(no borrar la app si no existe)*

¿Serías capaz de extender el método "install_app()" y "uninstall_app()"para instalar/desinstalar varias aplicaciones a la vez?

6.2.3 Atributos

Acceso directo

En el siguiente ejemplo vemos que, aunque el atributo `name` se ha creado en el constructor de la clase, también podemos modificarlo desde «fuera» con un acceso directo:

```
>>> class Droid:
...     def __init__(self, name: str):
...         self.name = name
...

>>> droid = Droid('C-3PO')

>>> droid.name
'C-3PO'

>>> droid.name = 'waka-waka'  # esto sería válido!
```

Python nos permite **añadir atributos dinámicamente** a un objeto incluso después de su creación:

```
>>> droid.manufacturer = 'Cybot Galactica'
>>> droid.height = 1.77
```

Nota: Nótese el acceso a los atributos con `obj.attribute` en vez de lo que veníamos usando en *diccionarios* donde hay que escribir «un poco más» `obj['attribute']`.

Propiedades

Como hemos visto previamente, los atributos definidos en un objeto son accesibles públicamente. Esto puede parecer extraño a personas que vengan de otros lenguajes de programación (véase Java). En Python existe un cierto «sentido de la responsabilidad» a la hora de programar y manejar este tipo de situaciones: Casi todo es posible a priori pero se debe controlar explícitamente.

Una primera solución «pitónica» para la privacidad de los atributos es el uso de **propiedades**. La forma más común de aplicar propiedades es mediante el uso de *decoradores*:

- `@property` para leer el valor de un atributo («getter»).
- `@name.setter` para escribir el valor de un atributo.

Veamos un ejemplo en el que estamos ofuscando el nombre del droide a través de propiedades:

```
>>> class Droid:
...     def __init__(self, name: str):
...         self.hidden_name = name
...
...     @property
...     def name(self) -> str:
...         print('inside the getter')
...         return self.hidden_name
...
...     @name.setter
...     def name(self, name: str) -> None:
...         print('inside the setter')
...         self.hidden_name = name
...

>>> droid = Droid('N1-G3L')

>>> droid.name
inside the getter
'N1-G3L'

>>> droid.name = 'Nigel'
inside the setter

>>> droid.name
```

(continué en la próxima página)

(proviene de la página anterior)

```
inside the getter
'Nigel'
```

En cualquier caso, seguimos pudiendo acceder directamente a `.hidden_name`:

```
>>> droid.hidden_name
'Nigel'
```

Incluso podemos cambiar su valor:

```
>>> droid.hidden_name = 'waka-waka'

>>> droid.name
inside the getter
'waka-waka'
```

Valores calculados

Una propiedad también se puede usar para devolver un **valor calculado** (o computado).

A modo de ejemplo, supongamos que la altura del periscopio de los droides astromecánicos se calcula siempre como un porcentaje de su altura. Veamos cómo implementarlo:

```
>>> class AstromechDroid:
...     def __init__(self, name: str, height: float):
...         self.name = name
...         self.height = height
...
...     @property
...     def periscope_height(self) -> float:
...         return 0.3 * self.height
...

>>> droid = AstromechDroid('R2-D2', 1.05)

>>> droid.periscope_height  # podemos acceder como atributo
0.315

>>> droid.periscope_height = 10  # no podemos modificarlo
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: can't set attribute
```

Las propiedades **no pueden recibir parámetros** ya que no tiene sentido semánticamente:

```
>>> class AstromechDroid:
...     def __init__(self, name: str, height: float):
...         self.name = name
...         self.height = height
...
...     @property
...     def periscope_height(self, from_ground: bool = False) -> float:
...         height_factor = 1.3 if from_ground else 0.3
...         return height_factor * self.height
...

>>> droid = AstromechDroid('R2-D2', 1.05)

>>> droid.periscope_height
0.315

>>> droid.periscope_height(from_ground=True)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'float' object is not callable
```

En este caso tendríamos que implementar un método para resolver el escenario planteado.

Consejo: La ventaja de usar valores calculados sobre simples atributos es que el cambio de valor en un atributo no asegura que actualicemos otro atributo, y además siempre podremos modificar directamente el valor del atributo, con lo que podríamos obtener efectos colaterales indeseados.

Cacheando propiedades

En los ejemplos anteriores hemos creado una propiedad que calcula el alto del periscopio de un droide astromecánico a partir de su altura. El «coste» de este cálculo es bajo, pero imaginemos por un momento que fuera muy alto.

Si cada vez que accedemos a dicha propiedad tenemos que realizar ese cálculo, estaríamos siendo muy ineficientes (en el caso de que la altura del droide no cambiara). Veamos una aproximación a este escenario usando el **cacheado de propiedades**:

```
>>> class AstromechDroid:
...     def __init__(self, name: str, height: float):
...         self.name = name
...         self.height = height  # llamada al setter
...
```

(continué en la próxima página)

(proviene de la página anterior)

```
...     @property
...     def height(self) -> float:
...         return self._height
...
...     @height.setter
...     def height(self, height: float) -> None:
...         self._height = height
...         self._periscope_height = None  # invalidar caché
...
...     @property
...     def periscope_height(self) -> float:
...         if self._periscope_height is None:
...             print('Calculating periscope height...')
...             self._periscope_height = 0.3 * self.height
...         return self._periscope_height
```

Probamos ahora la implementación diseñada, modificando la altura del droide:

```
>>> droid = AstromechDroid('R2-D2', 1.05)

>>> droid.periscope_height
Calculating periscope height...
0.315
>>> droid.periscope_height  # Cacheado!
0.315

>>> droid.height = 1.15

>>> droid.periscope_height
Calculating periscope height...
0.345
>>> droid.periscope_height  # Cacheado!
0.345
```

Ocultando atributos

Python tiene una convención sobre aquellos atributos que queremos hacer **«privados»** (u ocultos): comenzar el nombre con doble subguión `__`

```
>>> class Droid:
...     def __init__(self, name: str):
...         self.__name = name
...
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> droid = Droid('BC-44')

>>> droid.__name  # efectivamente no aparece como atributo
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'Droid' object has no attribute '__name'
```

Lo que realmente ocurre tras el telón se conoce como «*name mangling*» y consiste en modificar el nombre del atributo incorporado la clase como un prefijo. Sabiendo esto podemos acceder al valor del atributo supuestamente privado:

```
>>> droid._Droid__name
'BC-44'
```

Nota: La filosofía de Python permite hacer casi cualquier cosa con los objetos que se manejan, eso sí, el sentido de la responsabilidad se traslada a la persona que desarrolla e incluso a la persona que hace uso del objeto.

Atributos de clase

Podemos asignar atributos a una clase y serán asumidos por todos los objetos instanciados de esa clase.

A modo de ejemplo, en un principio, todos los droides están diseñados para que obedezcan a su dueño. Esto lo conseguiremos a nivel de clase, salvo que ese comportamiento se sobreescriba:

```
>>> class Droid:
...     obeys_owner = True  # obedece a su dueño
...

>>> good_droid = Droid()
>>> good_droid.obeys_owner
True

>>> t1000 = Droid()  # T-1000 (Terminator)
>>> t1000.obeys_owner = False
>>> t1000.obeys_owner
False

>>> Droid.obeys_owner  # el cambio no afecta a nivel de clase
True
```

Truco: Los atributos de clase son accesibles tanto desde la clase como desde las instancias creadas.

Hay que tener en cuenta lo siguiente:

- Si modificamos un atributo de clase desde un objeto, sólo modificamos el valor en el objeto y no en la clase.
- Si modificamos un atributo de clase desde una clase, **modificamos el valor en todos los objetos pasados y futuros**.

Veamos un ejemplo de esto último:

```
>>> class Droid:
...     obeys_owner = True
...

>>> droid1 = Droid()
>>> droid1.obeys_owner
True

>>> droid2 = Droid()
>>> droid2.obeys_owner
True

>>> Droid.obeys_owner = False  # cambia pasado y futuro

>>> droid1.obeys_owner
False
>>> droid2.obeys_owner
False

>>> droid3 = Droid()
>>> droid3.obeys_owner
False
```

La explicación de este fenómeno es sencilla: Todas las instancias (pasadas y futuras) del droide tienen un «atributo» `obeys_owner` que «apunta» a la misma zona de memoria que la del atributo `obeys_owner` de la clase:

```
>>> id(Droid.obeys_owner)
4385213672
>>> id(droid1.obeys_owner)
4385213672
>>> id(droid2.obeys_owner)
4385213672
```

(continué en la próxima página)

(proviene de la página anterior)

```
>>> id(droid3.obeys_owner)
4385213672
```

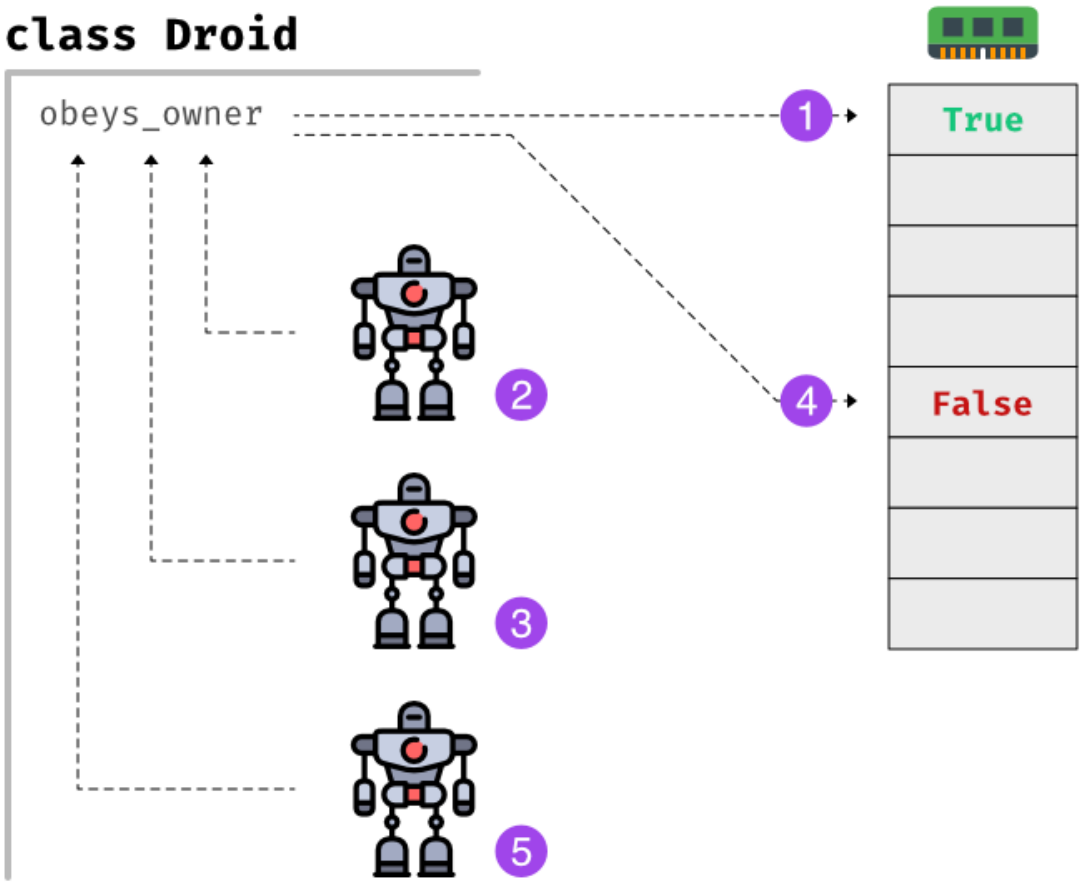


Figura 15: Atributo de clase

Supongamos que tras el cambio «global» de `obeys_owner` lo que buscamos es que **sólo se modifiquen los droides futuros pero no los pasados**.

Para poder abordar este escenario debemos recurrir a **atributos de instancia**.

```
>>> class Droid:
...     obeys_owner = True
...     def __init__(self):
...         self.obeys_owner = Droid.obeys_owner
...
```

Ahora veamos cuál es el comportamiento:

```
>>> droid1 = Droid()
>>> droid1.obeys_owner
True

>>> droid2 = Droid()
>>> droid2.obeys_owner
True

>>> Droid.obeys_owner = False

>>> droid1.obeys_owner
True
>>> droid2.obeys_owner
True

>>> droid3 = Droid()
>>> droid3.obeys_owner
False
```

6.2.4 Métodos

Métodos de instancia

Un **método de instancia** es un método que modifica o accede al estado del objeto al que hace referencia. Recibe `self` como primer parámetro, el cual se convierte en el propio objeto sobre el que estamos trabajando. Python envía este argumento de forma transparente: no hay que pasarlo como argumento.

Veamos un ejemplo en el que, además del constructor, creamos un método de instancia para hacer que un droide se mueva:

```
>>> class Droid:
...     def __init__(self, name: str):  # método de instancia -> constructor
...         self.name = name
...         self.covered_distance = 0
...
...     def move_up(self, steps: int) -> None:  # método de instancia
...         self.covered_distance += steps
...         print(f'Moving {steps} steps')
...

>>> droid = Droid('C1-10P')

>>> droid.move_up(10)
Moving 10 steps
```

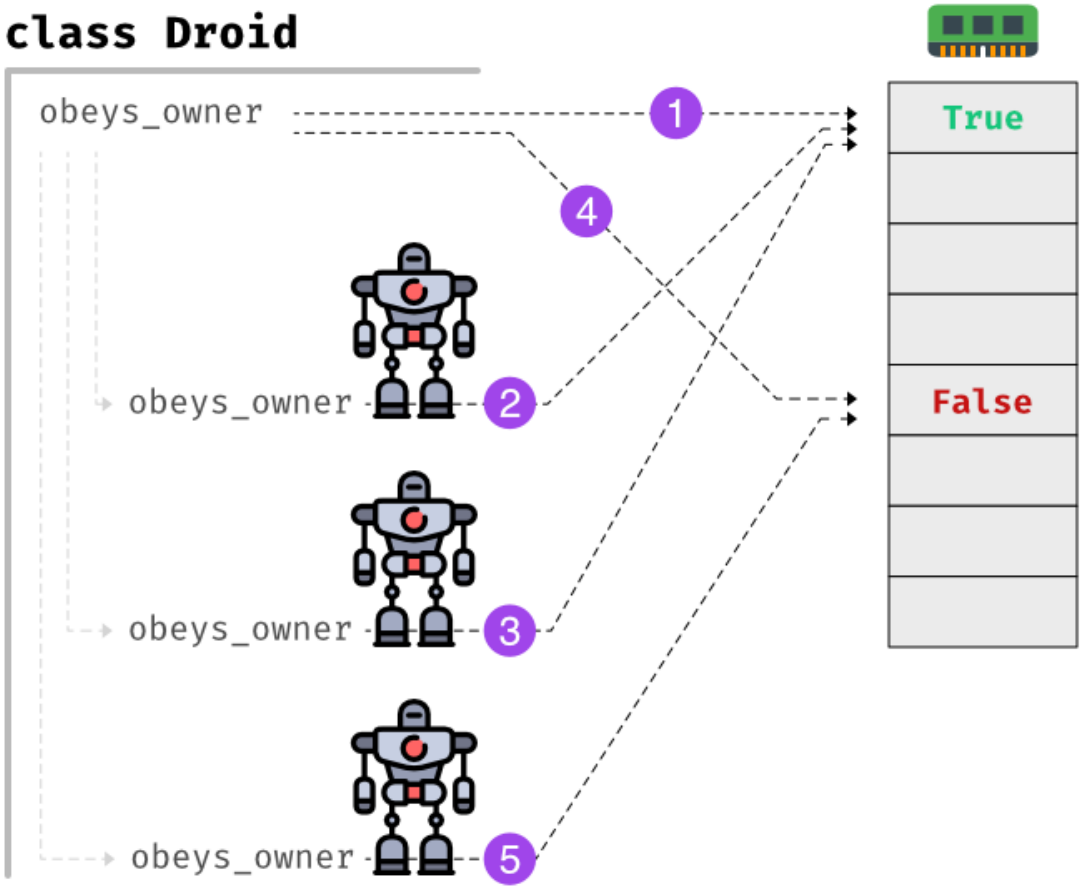


Figura 16: Atributo de clase con asignación de instancia

Propiedades vs Métodos

Es razonable plantearse cuándo usar *propiedades* o cuándo usar *métodos de instancia*.

Si la implementación requiere de parámetros, no hay confusión, necesitamos usar métodos.

Pero más allá de esto, no existe una respuesta clara y concisa a la pregunta. Aunque sí podemos dar algunas «pistas» para saber cuándo usar propiedades o cuándo usar métodos:

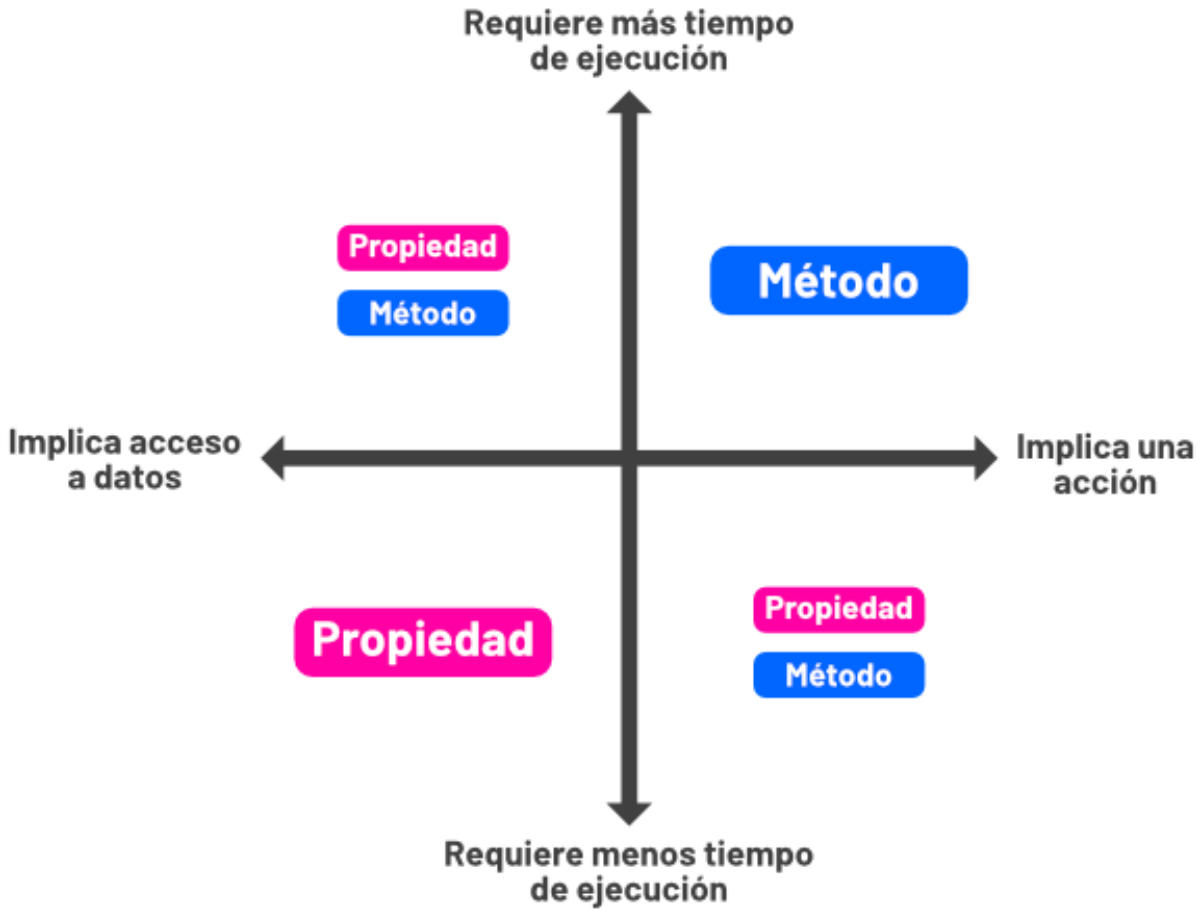


Figura 17: ¿Cuándo usar propiedades vs métodos?

Métodos de clase

Un **método de clase** es un método que modifica o accede al estado de la clase a la que hace referencia. Recibe `cls` como primer parámetro, el cual se convierte en la propia clase sobre la que estamos trabajando. Python envía este argumento de forma transparente. La identificación de estos métodos se completa aplicando el decorador `@classmethod` a la función.

Veamos un ejemplo en el que implementamos un método de clase que **muestra el número de droides creados**:

```
>>> class Droid:
...     count = 0
...
...     def __init__(self):
...         Droid.count += 1
...
...     @classmethod
...     def total_droids(cls) -> None:
...         print(f'{cls.count} droids built so far!')
...

>>> droid1 = Droid()
>>> droid2 = Droid()
>>> droid3 = Droid()

>>> Droid.total_droids()
3 droids built so far!
```

Consejo: El nombre `cls` es sólo una convención. Este parámetro puede llamarse de otra manera, pero seguir el estándar ayuda a la legibilidad.

Métodos estáticos

Un **método estático** es un método que no «debería» modificar el estado del objeto ni de la clase. No recibe ningún parámetro especial. La identificación de estos métodos se completa aplicando el decorador `@staticmethod` a la función.

Veamos un ejemplo en el que creamos un método estático para devolver las categorías de droides que existen en StarWars:

```
>>> class Droid:
...     def __init__(self):
...         pass
...
...     @staticmethod
...     def get_droids_categories() -> tuple[str]:
...         return ('Messeger', 'Astromech', 'Power', 'Protocol')
...

>>> Droid.get_droids_categories()
('Messeger', 'Astromech', 'Power', 'Protocol')
```

Métodos decorados

Es posible que, según el escenario, queramos **decorar ciertos métodos** de nuestra clase. Esto lo conseguiremos siguiendo la misma estructura de *decoradores* que ya hemos visto, pero con ciertos matices.

A continuación veremos un ejemplo en el que creamos un decorador para auditar las acciones de un droide y saber quién ha hecho qué:

```
>>> class Droid:
...     @staticmethod
...     def audit(method):
...         def wrapper(self, *args, **kwargs):
...             print(f'Droid {self.name} running {method.__name__}')
...             return method(self, *args, **kwargs)  # Ojo llamada!
...         return wrapper
...
...     def __init__(self, name: str):
...         self.name = name
...         self.pos = [0, 0]
...
...     @audit
...     def move(self, x: int, y: int):
...         self.pos[0] += x
...         self.pos[1] += y
...
...     @audit
...     def reset(self):
...         self.pos = [0, 0]

>>> droid = Droid('B1')

>>> droid.move(1, 1)
Droid B1 running move

>>> droid.reset()
Droid B1 running reset
```

A tener en cuenta la llamada al método de instancia dentro del decorador:

```
>>> method(self, *args, **kwargs) == self.method(*args, **kwargs)
```

El decorador se puede poner dentro o fuera de la clase. Por una cuestión de encapsulamiento podría tener sentido dejarlo **dentro de la clase como método estático**.

Ver también:

También es posible aplicar esta misma técnica usando *decoradores con parámetros*.

Métodos mágicos

Nivel avanzado

Cuando escribimos `'hello world' * 3` ¿cómo sabe el objeto `'hello world'` lo que debe hacer para multiplicarse con el objeto entero `3`? O dicho de otra forma, ¿cuál es la implementación del operador `*` para «strings» e «int»? En valores numéricos puede parecer evidente (siguiendo los operadores matemáticos), pero no es así para otros objetos. La solución que proporciona Python para estas (y otras) situaciones son los **métodos mágicos**.

Los métodos mágicos empiezan y terminan por doble subguión `__` (es por ello que también se les conoce como «dunder-methods»). Uno de los «dunder-methods» más famosos es el constructor de una clase: `__init__()`.

Importante: Digamos que los métodos mágicos se «disparan» de manera transparente cuando utilizamos ciertas estructuras y expresiones del lenguaje.

Para el caso de los operadores, existe un método mágico asociado (que podemos personalizar). Por ejemplo la comparación de dos objetos se realiza con el método `__eq__()`:

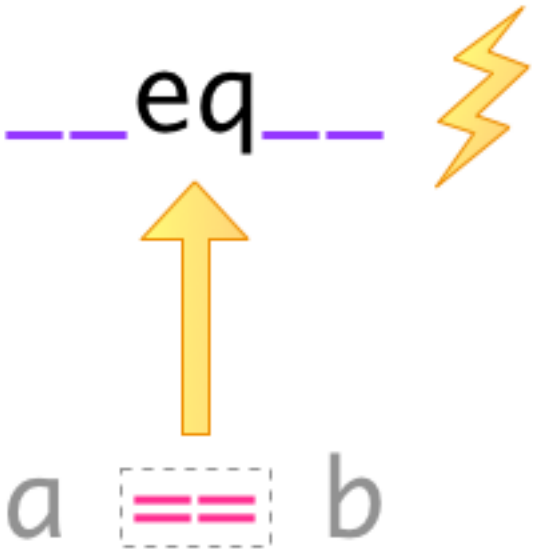


Figura 18: Equivalencia entre operador y método mágico

Extrapolando esta idea a nuestro universo StarWars, podríamos establecer que dos droides son iguales si su nombre es igual, independientemente de que tengan distintos números de serie:

```
>>> class Droid:
...     def __init__(self, name: str, serial_number: int):
```

(continué en la próxima página)